

Godot 游戏系统演化

从原始方案到成熟架构的完整考古记录

共计 132 章, 133 篇正文

2026年6月

序：为什么游戏系统会演化

每一个游戏开发者都曾面对过同一个问题："这个功能到底应该怎么做？"

搜索引擎会给你答案——通常是最新的、最流行的答案。但你有没有想过，这个"最新答案"是怎么来的？在它之前人们用的是是什么？为什么那个方案被淘汰了？它真的是因为"不行"被淘汰的，还是仅仅因为"有了更好的替代品"？

这本书试图回答的，就是这些问题。

它不是一本教程。它不教你"如何用 Godot 做游戏"，而是带你走一遍游戏开发史上那些系统方案的演化之路——从最原始的 `x++`，到 `delta` 时间修正，到 `move_and_slide`，再到 `RigidBody2D` 和 `AnimatableBody2D`。每一个方案的出现，都不是凭空而来的，而是因为前一个方案在某个场景下撑不住了。

这就是"演化"的含义：**不是新旧更替，而是场景倒逼。**

被淘汰的方案，也是老师

在编写本书的过程中，我刻意保留了那些"淘汰方案"的代码。不是因为它们有用——在今天看来它们确实大多没用——而是因为只有理解了它们为什么被淘汰，才能真正理解今天的方案为什么是这么设计的。

举个例子：如果你直接学习 `move_and_slide`，你当然会用。但如果你先写过 `position += velocity * delta`，再自己手写过碰撞响应，你才会明白 `move_and_slide` 到底帮你省了什么。更重要的是，当你未来遇到 `move_and_slide` 搞不定的场景时（比如自定义物理），你才知道怎么退回去手写。

这就是为什么我坚持在每个文档里呈现从简到繁的完整链条。每一个中间态都不是多余的，它是某个开发者在一个深夜为了解决某个具体问题而诞生的。

"文化"视角

技术方案和生物物种一样，有"生存压力"。

- 为什么要有对象池？因为大量 `instantiate + queue_free` 会导致 GC 卡顿。
- 为什么要有状态机？因为到处写 `if is_attacking and not is_jumping and not is_hurt` 迟早会失控。
- 为什么要有 NavigationAgent？因为手动 A* 无法处理动态障碍物。
- 为什么要淘汰 RVO？因为大量单位时性能撑不住。

每一个压力，催生了下一个方案。这就是游戏系统演化的"文化"——不是某个人坐在房间里想出来的，而是被问题逼出来的。

所以这本书的定位不是"权威指南"，而是"考古记录"。它记录的是游戏开发社区在面对一个个具体问题时，想出来的一个个具体解法。有些解法活了，有些死了，但每个都留下了痕迹。

如何使用这本书

本书按游戏系统分类，每个系统独立成章。你可以：

- **从头读**：感受一个系统从原始到成熟的完整演化路径。
- **跳着读**：只读你关心的系统，每个章节都是独立的。
- **对照读**：先看最终方案，再回头看被淘汰的方案，理解"为什么"。
- **当字典查**：当你需要实现某个系统时，找到对应章节，从最终方案入手。

代码使用 Godot 4.x GDScript 编写。虽然语言和引擎会过时，但方案背后的思路不会——无论换到什么引擎，移动算法、状态机、对象池、寻路的核心矛盾都是一样的。

为什么是 Godot

选择 Godot 是因为它足够"裸"。Unity 和 Unreal 把很多东西封装好了，你直接用就行，反而看不清底层在发生什么。Godot 让你更接近问题的本质——你需要自己拼装 `CharacterBody2D`、`AnimationTree`、`NavigationAgent2D`，这种"拼装感"反而更适合展示方案的演化。

此外，Godot 是开源的、免费的，不会因为商业授权问题阻碍知识的传播。

写给谁

- **新手开发者**：你会看到你即将遇到的那些问题的前人解法。
- **有经验的开发者**：你会看到你习以为常的方案背后，曾经有多少人走过弯路。
- **教育者**：这是一份"从第一性原理出发"的教学材料，适合用来讲解为什么系统要这么设计。
- **所有对游戏开发感兴趣的人**：技术不是冷冰冰的代码，它背后有故事。

最后

这 133 个文件只覆盖了通用系统。每个游戏类型——音游、格斗、SLG、卡牌——还有自己的专有系统等待被记录。

但如果这本书能让你在实现下一个功能时，不是直接搜索"Godot + 功能名"，而是先想一句：

"在最终方案出现之前，人们是怎么做的？"

那它就完成了它的使命。

2026年6月

目录

第一卷 · 核心机制

- 第1章 移动算法演化
- 第2章 状态机演化
- 第3章 寻路演化
- 第4章 插值与平滑演化
- 第5章 基础算法演化
- 第6章 物理引擎演化
- 第7章 路径跟随系统演化

第二卷 · 战斗系统

- 第8章 战斗系统演化
- 第9章 防御系统演化
- 第10章 连击系统演化
- 第11章 技能系统演化
- 第12章 天赋系统演化
- 第13章 装备系统演化
- 第14章 强化系统演化
- 第15章 附魔系统演化
- 第16章 继承系统演化
- 第17章 觉醒系统演化
- 第18章 转职系统演化
- 第19章 敌人 AI 演化

第20章 怪物系统演化
第21章 弹幕系统演化
第22章 掉落系统演化
第23章 经验系统演化
第24章 布娃娃系统演化
第25章 召唤系统演化
第26章 副本系统演化
第27章 关卡系统演化
第28章 钓鱼系统演化
第29章 属性系统演化
第30章 仇恨系统演化
第31章 状态效果系统演化
第32章 霸体韧性系统演化
第33章 受击反馈系统演化
第34章 锁定系统演化
第35章 变身系统演化
第36章 元素克制系统演化
第37章 连携技系统演化
第38章 能量系统演化
第39章 镶嵌系统演化

第三卷·经济系统

第40章 货币系统演化
第41章 背包系统演化
第42章 商店系统演化
第43章 交易系统演化
第44章 合成系统演化

第45章 烹饪系统演化
第46章 分解系统演化
第47章 抽卡系统演化
第48章 转盘系统演化
第49章 签到系统演化
第50章 体力系统演化
第51章 首充系统演化
第52章 充值系统演化
第53章 礼包码系统演化
第54章 离线收益系统演化
第55章 战令系统演化
第56章 拍卖行系统演化
第57章 耐久度修理系统演化
第58章 锻造炼金系统演化
第59章 红包竞猜系统演化
第60章 折扣限购系统演化

第四卷 · UI 与显示

第61章 UI 系统演化
第62章 弹窗系统演化
第63章 通知系统演化
第64章 小地图系统演化
第65章 相机系统演化
第66章 特效系统演化
第67章 动画系统演化
第68章 Tween 动画演化
第69章 骨骼动画系统演化

第70章 图鉴系统演化
第71章 时装系统演化
第72章 染色系统演化
第73章 拖拽系统演化
第74章 Tooltip 系统演化
第75章 屏幕震动系统演化
第76章 HUD 系统演化
第77章 加载画面系统演化
第78章 伤害数字系统演化

第五卷·角色与场景

第79章 NPC 系统演化
第80章 对话系统演化
第81章 好感度系统演化
第82章 捏脸系统演化
第83章 场景切换演化
第84章 场景系统演化
第85章 场景交互系统演化
第86章 机关系统演化
第87章 建筑系统演化
第88章 家园系统演化
第89章 昼夜系统演化
第90章 天气系统演化
第91章 采集系统演化
第92章 宠物系统演化
第93章 坐骑系统演化
第94章 载具系统演化

第95章 瓦片系统演化

第96章 传送点系统演化

第97章 大地图系统演化

第98章 游泳潜水系统演化

第99章 攀爬梯子系统演化

第100章 坠落伤害系统演化

第101章 种植畜牧系统演化

第102章 光源系统演化

第六卷 · 架构与数据

第103章 对象池演化

第104章 存档系统演化

第105章 设置系统演化

第106章 联网系统演化

第107章 聊天系统演化

第108章 音频系统演化

第109章 反作弊系统演化

第110章 外挂制作演化

第111章 Mod 制作系统演化

第112章 本地化系统演化

第113章 资源预加载系统演化

第114章 调试控制台系统演化

第七卷 · 社交与内容

第115章 新手引导系统演化

第116章 任务系统演化

第117章 成就系统演化

第118章 称号系统演化
第119章 邮件系统演化
第120章 活动系统演化
第121章 日常系统演化
第122章 公告系统演化
第123章 好友系统演化
第124章 组队系统演化
第125章 公会系统演化
第126章 PVP 系统演化
第127章 排行榜系统演化
第128章 表情系统演化
第129章 举报屏蔽系统演化
第130章 师徒结婚系统演化
第131章 回放录像系统演化
第132章 观战系统演化

第一卷·核心机制

共计 7 章

第1章 移动算法演化

Godot 移动算法演化史

这不是 API 文档, 这是一个故事。一个从像素到物理引擎, 从 `position.x += 1` 到 `move_and_slide()` 的演化史。每个方案的诞生, 都是因为上一个方案"还不够"。

目录

1. [史前时代: position += 1](#)
 2. [发现帧率: delta 的诞生](#)
 3. [方向抽象: -1, 0, 1](#)
 4. [维度扩展: Vector2 与斜向归一化](#)
 5. [物理困境: 穿墙问题](#)
 6. [第一次碰撞: move_and_collide](#)
 7. [滑墙需求: move_and_slide](#)
 8. [手感革命: 加速度与摩擦](#)
 9. [物理引擎介入: RigidBody2D](#)
 10. [平台的移动: AnimatableBody2D](#)
 11. [非实时移动: Tween](#)
 12. [3D 时代的适配](#)
 13. [演化的底层逻辑](#)
-

1. 史前时代: position += 1

时间: 你刚学编程, 或者刚打开 Godot。 **问题:** "我要让这个方块动起来。"

```
# 最简单的心智模型: 每帧把位置+1
func _process(delta):
    position.x += 1 # 每次向右移 1 像素
```

为什么这个方案不够? 你很快发现两个致命问题:

1. **60fps → 60px/s, 但 30fps → 30px/s** – 在不同电脑上速度不一样! 卡顿
时角色会瞬移。
2. **x += 1 只能向右** – 每个方向都得写一行, 代码变得臃肿。

试着改进: 加个速度变量。

```
var speed = 1
if Input.is_action_pressed("ui_right"):
    position.x += speed
elif Input.is_action_pressed("ui_left"):
    position.x -= speed
```

但这只是把问题藏起来了。速度还是依赖帧率。

2. 发现帧率: delta 的诞生

问题: 不同帧率下速度不一致。60fps 和 30fps 的角色不在一个世界。

解决方案: delta – 上一帧的耗时(秒)。

```
# 60fps → delta ≈ 0.0167s, 每帧移动 speed * 0.0167
# 30fps → delta ≈ 0.0333s, 每帧移动 speed * 0.0333
# 1秒总位移 = speed × 1.0 – 与帧率无关!
func _process(delta):
    position.x += speed * delta
```

这个方案解决了: 帧率不一致问题。从此速度单位变成了像素/秒。

但这个方案暴露了新问题: 碰撞检测呢? 如果我有平台, 有个敌人, 我需要知道"碰到"这件事。只改 `position` 相当于"瞬移"--即使是 1 像素的瞬移, 也可能穿模。

此时社区开始寻找"带碰撞的移动"。

3. 方向抽象: -1, 0, 1

问题: 写 `position.x += 1` 只能硬编码方向。左右得写 if-else, 上下又得写一套。

关键洞察: 左右本质上是同一个操作, 只是方向不同。可以用 **-1, 0, 1** 来表示方向。

```
var direction = 0

if Input.is_action_pressed("ui_right"):
    direction = 1
elif Input.is_action_pressed("ui_left"):
    direction = -1
else:
    direction = 0

# 方向 × 速度 就统一了左右
velocity_x = direction * speed
position.x += velocity_x * delta
```

这为什么是革命性的? - 第一次把"输入"和"移动"解耦了 - `direction` 是一个纯数据, 不关心按键具体是什么 - 后续所有移动方案都沿用这个模式: 输入 → 方向 → 速度 → 移动

但此时还是单轴的。如果要上下左右怎么办?

4. 维度扩展: Vector2 与斜向归一化

问题: 两套 direction 变量太笨了。

```
# 笨办法: x 一套, y 一套
var dir_x = 0
var dir_y = 0
```

解决方案: Godot 内置的 `Vector2` – 天生就是“一对轴”。

```
var direction = Vector2.ZERO # (0, 0)

if Input.is_action_pressed("ui_right"): direction.x += 1
if Input.is_action_pressed("ui_left"): direction.x -= 1
if Input.is_action_pressed("ui_down"): direction.y += 1
if Input.is_action_pressed("ui_up"): direction.y -= 1

velocity = direction * speed
position += velocity * delta
```

发现了一个隐藏 Bug: 斜向比直线快!

```
右 (1,0) → 长度 1.0 → 速度 200
右下 (1,1) → 长度 1.414 → 速度 283 – 作弊一样快!
```

修复: `normalized()` 保证方向向量长度永远是 1。

```
if direction.length() > 0:
    direction = direction.normalized()
velocity = direction * speed
```

至此, **方向向量模式** 成熟了。这段代码至今依然是大多数 Godot 移动的起点。

但——还是没有碰撞。

5. 物理困境：穿墙问题

故事：你做了一个 RPG，角色欢快地在场景里跑。然后你加了堵墙。

```
# ❌ 直接改 position 的致命问题
func _physics_process(delta):
    position += velocity * delta
    # 如果 velocity 很大，可能直接穿过墙
    # 即使 velocity 很小，两帧之间也可能穿越薄墙
```

为什么会穿墙？ - `position += velocity * delta` 是瞬移，不是"移动过去" - Godot 的物理引擎基于**离散碰撞检测** - 只检查"起点"和"终点" - 如果墙在两帧的路径中间，角色直接越过，物理引擎看不到

解决办法：1. 减小速度 (但体验差) 2. 增加物理帧率 (性能开销大) 3. 用 Godot 的碰撞移动 API (正解)

6. 第一次碰撞：move_and_collide

问题：我要移动，还要检测碰撞，不想穿墙。

Godot 的方案：`move_and_collide(位移量)` - 把 movement 交给 Godot 物理引擎。

```
func _physics_process(delta):
    velocity = direction * speed
    var collision = move_and_collide(velocity * delta)

    if collision:
        print("碰到：", collision.get_collider())
```

工作原理：1. Godot 从当前位置沿 velocity 方向做射线/扫描检测 2. 如果路径上有碰撞体 → 停在碰撞点前 3. 返回 **KinematicCollision2D** 对象，包含碰撞法线、碰撞点、碰撞对象等

解决了: 穿墙问题。物理引擎会确保角色不会穿透碰撞体。

但又出现了新问题: 沿墙摩擦。

想象你跳向一个平台, 头碰到了平台的底面: - 你期望: 头碰墙 → 落下来 - 实际:

`move_and_collide` 碰墙就停 → 头被"粘"在墙上, 不掉下来

它不会"沿着墙面滑动", 不会把法线方向的速度清零。

你只能自己处理:

```
# 每次碰撞后手动投影到墙面方向
var collision = move_and_collide(velocity * delta)
if collision:
    velocity = velocity.slide(collision.get_normal())
```

这太麻烦了。每个开发者都在写同一套循环。Godot 团队决定: **把这个循环封装进引擎。**

7. 滑墙需求: `move_and_slide`

为什么诞生: 每个 Godot 用户都在手写:

```
for i in range(4):
    var c = move_and_collide(vel * delta)
    if !c: break
    vel = vel.slide(c.get_normal())
```

Godot 团队: "我们帮你做了。"

```
# 一行代替上面所有
move_and_slide()
```

它内部做了什么:

```
move_and_slide() 内部伪逻辑:
```

```
把 velocity 传给物理引擎
重复最多 max_slides 次:
    调用 move_and_collide(velocity * delta)
    如果没碰撞 → break
    如果碰撞了:
        把 velocity 沿碰撞法线投影(即去掉垂直墙面的速度)
        继续下一轮移动(沿墙滑动)
更新 is_on_floor / is_on_wall / is_on_ceiling 等状态标志
```

move_and_slide 就是"带碰撞的智能移动"。它让 Godot 2D 角色移动变得极其简单。

输入输出模型:

```
你设置 velocity → 你调用 move_and_slide() → velocity 被修改(碰撞后的速度)
                                                    ↓
                                                    is_on_floor() 等状态就緒
```

此时 Godot 的"标准移动模式"成形了:

```
func _physics_process(delta):
    # 1. 方向输入
    var direction = Input.get_axis("ui_left", "ui_right")
    # 2. 设置速度
    velocity.x = direction * speed
    # 3. 交给 Godot
    move_and_slide()
```

但 developer 们又发现一个问题: **响应太硬了**。按下立刻全速, 松开立刻归零。

8. 手感革命: 加速度与摩擦

问题: `velocity.x = direction * speed` 是**瞬间变速**。- 按右 → 马上 300px/s - 松右 → 马上 0px/s

这像"冰面上没有惯性的方块"。真正的好游戏(马里奥、蔚蓝、空洞骑士)都有**加速/减速过程**。

为什么需要手感参数? 游戏中的"手感"本质上就是加速度和摩擦力的函数:

```
高加速度 + 高摩擦 = 马里奥 (响应快, 停止也快)
低加速度 + 低摩擦 = 冰面关卡 (滑溜)
中加速度 + 无摩擦 = 太空 (一直滑)
低加速度 + 中摩擦 = 大块头角色 (笨重)
```

Godot 的解决方案: `move_toward()`。

```
var acceleration = 1200.0 # 加速度
var friction = 800.0      # 摩擦力

func _physics_process(delta):
    var direction = Input.get_axis("ui_left", "ui_right")

    if direction:
        # 逐渐加速到目标速度
        velocity.x = move_toward(velocity.x, direction * speed,
            acceleration * delta)
    else:
        # 逐渐减速到 0
        velocity.x = move_toward(velocity.x, 0, friction *
            delta)

    move_and_slide()
```

`move_toward(from, to, step)` – 从 from 向 to 移动 step 步长, 不会超过 to。

这就是为什么你今天看到的 Godot 教程里, 几乎都有一段"带加速度和摩擦力"的版本。这不是炫技, 而是**游戏手感的基本要求**。

进一步, 社区还区分了**地面控制**和**空中控制**:

```
var air_control_multiplier = 0.3 # 空中只能获得 30% 加速度

func _physics_process(delta):
```

```
var accel = acceleration if is_on_floor() else acceleration
* air_control_multiplier
# ...
```

这模仿了现实: 人在空中没有摩擦力可以借力, 无法快速转向。马里奥就是这个感觉。

9. 物理引擎介入: RigidBody2D

问题: "我想做一个弹球/物理谜题/布娃娃, 不想手动算重力、弹力、摩擦力。"

思路转变: 不用 `move_and_slide` 手动控制移动, 而是把控制权交给 Godot 物理引擎。

```
extends RigidBody2D

func _ready():
    gravity_scale = 1.0    # 引擎自动施加重力
    bounce = 0.8          # 弹力
    friction = 0.5         # 摩擦力

func _physics_process(delta):
    var direction = Input.get_axis("ui_left", "ui_right")
    linear_velocity.x = direction * speed * 100 # 物理单位的力
```

区别在哪? - `CharacterBody2D` + `move_and_slide`: 你控制 position/velocity, Godot 只处理碰撞滑动 - `RigidBody2D`: 物理引擎控制 position/velocity, 你只能"施加力"

适用场景分水岭: | 你需要 | 用 ||-----|-----|| 精确的平台跳跃角色 |

`CharacterBody2D` || 会被推走的箱子/弹球 | `RigidBody2D` || 布娃娃/ragdoll
| `RigidBody2D` || 玩家控制的载具 | `RigidBody2D` + `VehicleBody` |

但 `RigidBody2D` 的移动不可预测 – 物理引擎的求解器可能因为帧率、碰撞数量而给出不同结果。所以"需要精确控制"的游戏永远选 `CharacterBody2D`。

10. 平台的移动: AnimatableBody2D

问题: "我做了个移动平台。角色站上去, 平台动了, 但角色原地不动。" -- 因为 `move_and_slide` 只移动 `CharacterBody2D` 自身, 不会跟着父节点走。

这个问题的特殊之处: 不是角色在动, 是场景里的物体在动, 角色只是"站在上面"。

```
extends AnimatableBody2D

func _physics_process(delta):
    # 平台自己做正弦运动
    position.y += sin(Time.get_ticks_msec() / 1000.0) * 2.0
    move_and_collide(Vector2.ZERO) # 通知物理引擎: 我移动了
```

`AnimatableBody2D` (Godot 4, 之前叫 `StaticBody2D` + 特殊处理): - 它是一个静态碰撞体但可以移动 - 当它移动时, 物理引擎会推动上面站着角色 - 角色不需要额外代码来处理"站在移动平台上"

适用场景: 移动平台、电梯、旋转机关、自动门。

11. 非实时移动: Tween

问题: "我不需要物理, 我只要让门在两秒内从 A 平滑移到 B。"

```
# 用 _process 自己插值:
var t = 0.0
func _process(delta):
    t += delta
    $Door.position.x = lerp(0.0, 200.0, t / 2.0)
```

太累赘了。Godot 内置了 **Tween** 系统:

```
var tween = create_tween()
tween.tween_property($Door, "position", Vector2(200, 0), 2.0)
```

Tween 移动和物理移动的区别：

物理移动 (CharacterBody2D)：

- 每帧根据输入和物理规则算速度
- 有碰撞、有滑动、有重力
- 结果不可预测（取决于玩家操作）

Tween 移动：

- 预设好起始/终点/时长/缓动曲线
- 没有碰撞检测（如果半路有墙，直接穿过去）
- 结果是确定的（一定会到终点）

适用场景：UI 动画、过场动画、门的开关、摄像机运镜、任何"我不想管物理，只要从 A 到 B"的移动。

Tween 不抢物理的活，物理也干不了 Tween 的事。它们是两个世界。

12. 3D 时代的适配

问题："2D 的这一套，在 3D 里还管用吗？"

答案：管用。move_and_slide 3D 版几乎一样。

```
extends CharacterBody3D

var speed = 5.0
var gravity = 9.8

func _physics_process(delta):
    if not is_on_floor():
        velocity.y -= gravity * delta

    var input_dir = Input.get_vector("ui_left", "ui_right",
    "ui_forward", "ui_back")
    var direction = (transform.basis * Vector3(input_dir.x, 0,
    input_dir.y)).normalized()

    velocity.x = direction.x * speed
    velocity.z = direction.z * speed
```



```
move_and_slide()
```

但 3D 带来了新问题:

1. 相机朝向影响移动方向
2. 2D: 按右就是往右
3. 3D: 按右是往"相机的右", 相机会转!

所以 3D 角色移动输入要乘以相机的旋转:

```
gdscript var camera_basis = camera.global_transform.basis var
direction = camera_basis * Vector3(input_dir.x, 0, input_dir.y)
```

1. Z 轴是深度
2. $\text{Vector2}(x, y) \rightarrow \text{Vector3}(x, y, z)$
3. 俯视角游戏其实是在 XZ 平面上移动, Y 是上下
4. 斜坡检测
5. `floor_max_angle` 参数控制什么角度算"地板", 超过就滑下来

3D 移动的核心公式仍然是: 输入 $\rightarrow$ 方向 $\rightarrow$ velocity $\rightarrow$ move_and_slide()。

只是方向计算多了相机旋转这一层。

13. 演化的底层逻辑

回头看这整个演化史, 每个阶段解决一个问题:

```
position += 1
    ↓ 帧率不稳定
position += speed * delta
    ↓ 没有方向抽象
direction(-1,0,1)  $\rightarrow$  velocity  $\rightarrow$  position += vel * delta
    ↓ 没有碰撞检测
move_and_collide(vel * delta)
    ↓ 不会沿墙滑动
```

`move_and_slide()` ← 90% 的游戏止步于此

↓ 手感太硬

加速度 + 摩擦力 + 空气控制

↓ 想省掉物理计算

`RigidBody2D` (物理驱动)

↓ 场景物体也需要动

`AnimatableBody2D` (移动平台)

↓ 非物理的过渡动画

`Tween` (UI/过场)

核心公式始终是：输入 → 方向向量 → `velocity` → 移动函数 只是"移动函数"从 `position +=` 一路升级到 `move_and_slide()`。

附：移动方案决策树

你要让角色移动？



附：什么时候用 `move_and_collide`？

当你只需要"碰一次"而不需要滑墙时 (子弹反弹、投掷物)

这个故事告诉我们: - 没有"最好的移动方案", 只有"最适合当前场景的方案"

- 每个方案都是因为上一个方案不够而出现的 - 掌握了这个演化链, 你就理解了任何 Godot 移动代码的来龙去脉 - 面试/社区里如果有人问你"为什么用 `move_and_slide`", 你可以从 `position+=1` 开始讲起

第2章 状态机演化

Godot 状态管理演化史

每个游戏角色体内, 都住着一个状态机。从 boolean 地狱到分层状态机, 这是一场反抗混沌的战争。

目录

1. [原始时代: boolean 地狱](#)
 2. [enum + match: 第一次统一](#)
 3. [状态模式: 每个状态一个类](#)
 4. [状态机封装: 通用的 FSM](#)
 5. [叠加状态: 状态栈](#)
 6. [AnimationTree 状态机](#)
 7. [层级状态机 \(HSM\)](#)
 8. [行为树 \(Behavior Tree\)](#)
 9. [最终方案? 选型指南](#)
-

1. 原始时代: boolean 地狱

问题: "角色有 idle、run、jump、attack 几种状态, 怎么区分?"

最初的想法: 每个状态用一个 boolean。

```
var is_idle = true
var is_running = false
var is_jumping = false
var is_attacking = false
var is_falling = false
var is_hurt = false
```

然后代码变成了这样:

```
func _physics_process(delta):
    # 移动
    if is_idle or is_running:
        var direction = Input.get_axis("ui_left", "ui_right")
        if direction:
            velocity.x = direction * speed
            is_idle = false
            is_running = true
        else:
            velocity.x = move_toward(velocity.x, 0, friction *
delta)
            is_running = false
            is_idle = true

    # 跳跃
    if Input.is_action_just_pressed("jump"):
        if is_idle or is_running:
            velocity.y = jump_velocity
            is_jumping = true
            is_idle = false
            is_running = false

    # 攻击
    if Input.is_action_just_pressed("attack"):
        if is_idle or is_running:
            is_attacking = true
            # 播放攻击动画...

    # 落地
    if is_on_floor() and is_jumping:
        is_jumping = false
        is_idle = true
```

```
move_and_slide()
```

问题很快暴露了:

1. **组合爆炸** – 如果要在"空中攻击"呢? 要加 `is_air_attacking`。如果攻击时不能移动呢? 要在移动代码里加 `and not is_attacking`。
2. **无效状态** – `is_jumping=true` 和 `is_running=true` 同时成立是合法的吗? `is_attacking=true` 和 `is_hurt=true` 同时成立角色该做什么动画?
3. **散落各地的逻辑** – 跳跃逻辑分散在 3 个 if 块里。加了新状态就要改 5 个地方。
4. **你永远不敢删 boolean** – 因为不知道哪里还在用它。

N 个 boolean 有 2^N 种组合。5 个 boolean $\rightarrow$ 32 种状态, 但你的游戏可能只有 6 种合法状态。

2. enum + match: 第一次统一

问题: boolean 太多了, 互相打架。

核心洞察: 角色同一时刻只能处于一个状态。所以应该用一个变量而不是多个。

```
enum State { IDLE, RUN, JUMP, FALL, ATTACK }  
  
var state = State.IDLE
```

所有逻辑统到一个 match:

```
func _physics_process(delta):  
    match state:  
        State.IDLE, State.RUN:  
            var direction = Input.get_axis("ui_left",  
            "ui_right")  
            if direction:
```

```

        velocity.x = direction * speed
        change_state(State.RUN)
    else:
        velocity.x = move_toward(velocity.x, 0, friction
* delta)

    if Input.is_action_just_pressed("jump"):
        velocity.y = jump_velocity
        change_state(State.JUMP)

    if Input.is_action_just_pressed("attack"):
        change_state(State.ATTACK)

    State.JUMP:
        # 空中移动(可控制)
        var direction = Input.get_axis("ui_left",
"ui_right")
        velocity.x = direction * speed

        if velocity.y > 0:
            change_state(State.FALL)

    State.FALL:
        var direction = Input.get_axis("ui_left",
"ui_right")
        velocity.x = direction * speed

        if is_on_floor():
            change_state(State.IDLE)

    State.ATTACK:
        # 攻击时不能移动
        velocity.x = 0
        if not $AnimationPlayer.is_playing():
            change_state(State.IDLE)

    move_and_slide()

func change_state(new_state):
    if new_state == state:
        return
    state = new_state
    # 更新动画
    $AnimationPlayer.play(State.keys()[state].to_lower())

```


进步: - 任何时刻只在一个状态 → 状态互斥, 不可能冲突 - 所有代码按状态组织, 不散的 - 新状态就是加一个 enum 值 + match 分支

但问题还在:

1. **match 越来越胖** – 20 个状态时, `_physics_process` 上千行
 2. **状态的进入/退出/更新混在一起** – 进入 JUMP 时要设一次 `velocity.y`, 但如果从别的状态切到 JUMP 呢?
 3. **状态间的过渡混在逻辑里** – "什么情况下攻击结束切到 idle" 写在 ATTACK 分支里, "什么情况下 idle 切到 attack" 写在 IDLE 分支里。过渡逻辑分散在两端。
-

3. 状态模式: 每个状态一个类

问题: match 太胖了, 每个状态的逻辑互相干扰。需要一个地方只放"这个状态的事"。

引入设计模式: State Pattern – 每个状态一个独立脚本/类。

```
# 状态基类
class_name State
extends Node

func enter(owner: Node, previous_state: String) -> void:
    pass

func exit(owner: Node) -> void:
    pass

func update(owner: Node, delta: float) -> void:
    pass

func physics_update(owner: Node, delta: float) -> void:
    pass
```

```
func handle_input(owner: Node, event: InputEvent) -> void:
    pass
```

```
# IdleState.gd
class_name IdleState
extends State

func enter(owner, previous_state):
    owner.animation_player.play("idle")

func physics_update(owner, delta):
    var direction = Input.get_axis("ui_left", "ui_right")
    if direction:
        owner.change_state("run")
        return

    owner.velocity.x = move_toward(owner.velocity.x, 0,
owner.friction * delta)

    if Input.is_action_just_pressed("jump"):
        owner.velocity.y = owner.jump_velocity
        owner.change_state("jump")
```

```
# 角色脚本
extends CharacterBody2D

var states = {}
var current_state: State

func _ready():
    states["idle"] = $States/IdleState
    states["run"] = $States/RunState
    states["jump"] = $States/JumpState
    states["fall"] = $States/FallState
    change_state("idle")

func change_state(new_state_name):
    if current_state:
        current_state.exit(self)
    current_state = states[new_state_name]
    current_state.enter(self, current_state_name)
    current_state_name = new_state_name
```

```
func _physics_process(delta):
    current_state.physics_update(self, delta)
    move_and_slide()
```

状态文件结构:

```
Player/
├── Player.gd
├── States/
│   ├── IdleState.gd
│   ├── RunState.gd
│   ├── JumpState.gd
│   ├── FallState.gd
│   └── AttackState.gd
```

巨大进步: - 每个状态一个文件 → 修改一个状态不影响其他状态 - enter/exit 钩子 → 进入/退出时可以播放音效、重置变量、切换动画 - 新加状态 → 新建一个脚本, 注册到 states 字典

但代价是: 1. **文件爆炸** - 10 个角色 × 6 个状态 = 60 个文件 2. **状态间通信麻烦** - IdleState 想读取 AttackState 的数据? 要通过 owner 3. **共享逻辑很难复用** - 多个状态都要做的事(比如"空中只能微移")写成函数放在哪? 放在 owner 上, 但 owner 又越来越胖 4. **过渡逻辑仍在两端** - 从哪里能切到这个状态, 这个状态能切到哪——没有集中管理

4. 状态机封装: 通用的 FSM

问题: 每个角色都在重复写 `change_state`、`states` 字典、`current_state`。能不能抽成一个通用模块?

```
# FiniteStateMachine.gd
class_name FiniteStateMachine
extends Node

var states := {}
```

```

var current_state: State
var current_state_name := ""

func add_state(name: String, state: State) -> void:
    states[name] = state

func change_state(name: String) -> bool:
    if not states.has(name):
        return false
    if current_state:
        current_state.exit(owner)
    current_state = states[name]
    current_state_name = name
    current_state.enter(owner, current_state_name)
    return true

func update(delta: float) -> void:
    if current_state:
        current_state.update(owner, delta)

func physics_update(delta: float) -> void:
    if current_state:
        current_state.physics_update(owner, delta)

func handle_input(event: InputEvent) -> void:
    if current_state:
        current_state.handle_input(owner, event)

```

```

# 角色脚本 - 干净多了
extends CharacterBody2D

@onready var fsm := $FiniteStateMachine as FiniteStateMachine

func _ready():
    fsm.add_state("idle", $States/IdleState)
    fsm.add_state("run", $States/RunState)
    fsm.add_state("jump", $States/JumpState)
    fsm.change_state("idle")

func _physics_process(delta):
    fsm.physics_update(delta)
    move_and_slide()

```

现在可以复用 FSM 了:

Player/ FSM	Enemy/ FSM	NPC/ FSM	← 都用同一个 FiniteStateMachine.gd
----------------	---------------	-------------	-------------------------------

但新问题来了: 过渡逻辑还是散在状态里。IdleState 决定什么时候切 RunState, RunState 决定什么时候切 IdleState——过渡的"谁来决定"问题"。有些团队引入过渡表 (Transition Table):

```
# FSM 增加过渡管理
var transitions = {}

func add_transition(from: String, to: String, condition:
Callable):
    transitions[from] = transitions.get(from, [])
    transitions[from].append({to = to, condition = condition})

func _process(delta):
    # 检查当前状态的过渡
    if transitions.has(current_state_name):
        for t in transitions[current_state_name]:
            if t.condition.call():
                change_state(t.to)
                return
    # 没发生过渡, 正常 update
    if current_state:
        current_state.update(owner, delta)
```

所有过渡一目了然:

```
fsm.add_transition("idle", "run", func(): return
Input.get_axis(...) != 0)
fsm.add_transition("run", "idle", func(): return
Input.get_axis(...) == 0)
fsm.add_transition("jump", "fall", func(): return
owner.velocity.y > 0)
```

过渡集中管理了。但 FSM 概念还在进化。

5. 叠加状态: 状态栈

问题: "角色攻击时被打, 应该播放受伤动画, 然后回到攻击状态继续攻击吗?"

传统 FSM 的问题: 无法记住"之前的状态"。

- 你在 RUN → 按攻击 → ATTACK → 被打 → HURT → 回到? IDLE? 但如果只是在攻击中被打, 应该回到 ATTACK 才对。

状态栈 (State Stack): 像一个 undo 系统。

栈底 栈顶
[IDLE] → [IDLE, RUN] → [IDLE, RUN, ATTACK] → 被打
→ [IDLE, RUN] → pop → [IDLE]

```
class_name StateStack
extends Node

var stack := []

func push_state(name: String):
    if stack.size() > 0:
        stack.back().exit(owner)
    stack.append(states[name])
    stack.back().enter(owner, name)

func pop_state():
    if stack.size() == 0:
        return
    stack.back().exit(owner)
    stack.pop_back()
    if stack.size() > 0:
        stack.back().enter(owner, name) # re-enter

func change_state(name: String):
    # 清空栈, 替换为新状态
    while stack.size() > 0:
        stack.back().exit(owner)
        stack.pop_back()
    push_state(name)
```

适用场景: - 暂停菜单 → 推入 Pause 状态, 回到游戏时 pop - 攻击被打断后恢复 → push HURT, pop 回到 ATTACK - 对话系统 → 推入 Dialogue 暂停角色控制, pop 恢复

但大多数游戏用不上栈。2~3 层历史就够了。

6. AnimationTree 状态机

问题: 代码状态机和动画状态机是**两套系统**, 每次都要手动同步。如果动画状态机也在变, 开发者要维护两套过渡逻辑。

```
代码: IDLE → RUN (Input 检测到方向)
动画: "idle" → "run" (匹配代码状态)
```

Godot 团队: "我们把动画状态机做得足够强, 让它直接驱动游戏逻辑。"

```
# AnimationTree + StateMachine
@onready var anim_tree = $AnimationTree
@onready var anim_state = anim_tree.get("parameters/playback")

func _physics_process(delta):
    var direction = Input.get_axis("ui_left", "ui_right")

    # 直接通过 AnimationTree 的参数控制
    anim_tree.set("parameters/move/blend_position", direction)

    # AnimationTree 的 StateMachine 自己管理过渡
    # "idle" → "run" 由条件的 blend_position 决定
```

在 AnimationTree 编辑器中:

```
AnimationStateMachine:
    idle —[条件: blend_position != 0]→ run
    run —[条件: blend_position == 0]→ idle
    idle —[条件: jump]→ jump
    jump —[条件: 动画结束]→ idle
```

进步: - 动画过渡自动处理 (跨淡化, 时间线同步) - 减少代码中的 match, 把过渡逻辑交给可视化编辑器 - 设计师可以自己调, 不需要程序员

但局限: - 只能管动画, 游戏逻辑(伤害判定、无敌帧)还得代码管 - 复杂逻辑(加速度计算、物理)不适合放在动画状态机里 - 多人协作时, 设计师动一下, 程序员不知道

常见折中: 代码管主状态机, 动画管子状态。主状态决定"在做什么", AnimationTree 决定"动画怎么播放"。

7. 层级状态机 (HSM)

问题: 角色在 GROUNDED 下分为 IDLE/RUN, 在 AIRBORNE 下分为 JUMP/FALL。但 IDLE/RUN 共享"可以攻击、可以跳跃", JUMP/FALL 共享"空中控制减半"。



每个父状态可以: - 定义**共享逻辑** (如 GROUNDED 可以攻击、可以跳跃) - 定义**默认子状态** (进入 GROUNDED 时自动走 IDLE) - 子状态可以覆盖父状态的逻辑

```
class_name HierarchicalState

var name: String
var parent: HierarchicalState
var children := {}
var default_child: String
var active_child: HierarchicalState

func enter(owner):
    if default_child:
```



```

        active_child = children[default_child]
        active_child.enter(owner)

func update(owner, delta):
    # 父状态做通用逻辑
    if active_child:
        active_child.update(owner, delta)

func handle_input(owner, event):
    # 父状态可以拦截输入
    # 比如 GROUNDED 统一处理跳跃
    if active_child:
        active_child.handle_input(owner, event)

```

实际场景:

```

Player (HSM)
├── GROUNDED
│   ├── IDLE      ← 默认
│   ├── RUN
│   ├── CROUCH
│   └── ATTACK
│       ├── ATTACK_1
│       ├── ATTACK_2
│       └── ATTACK_3 ← 连招链
└── AIRBORNE
    ├── JUMP
    ├── FALL
    ├── AIR_ATTACK
    └── DOUBLE_JUMP

```

HSM 为什么强大: - GROUNDED 统管"可以跳跃、可以格挡、可以攻击" -
 AIRBORNE 统管"空中控制减半、不能格挡" - 加一个新地面动作 → 只需加在
 GROUNDED 下, 自动拥有"可跳跃、可格挡"

大多数商业游戏引擎 (Unreal、Unity 的 Animancer) 都用 HSM。Godot 不自带 HSM, 但社区有很多实现。

8. 行为树 (Behavior Tree)

问题: 状态机描述的是"角色可能处于什么状态"。但 AI 描述的是"角色应该做什么"。

思维转变: - FSM: 角色在 IDLE 状态 → 检测到敌人 → 转到 CHASE 状态 - BT: Selector(检测敌人? → 追击:巡逻)

节点类型:

行为树节点:

- ├─ Composite (组合节点)
 - ├─ Selector (选择器) - 依次尝试子节点, 成功就停
 - └─ Sequence (序列) - 依次执行子节点, 全成功才成功
- ├─ Decorator (装饰器) - 修饰子节点 (重复、取反、限时)
- └─ Leaf (叶子节点) - 实际动作 (移动、攻击、播放动画)

行为树示例:

- Selector (选择器) ← 优先处理紧急的事
 - ├─ Sequence (序列)
 - ├─ 生命 < 30%? ← 条件
 - └─ 跑向血包 ← 动作
 - ├─ Sequence (序列)
 - ├─ 看到敌人? ← 条件
 - ├─ 距离 < 攻击范围? ← 条件
 - └─ 攻击 ← 动作
 - └─ 巡逻 ← 默认行为

和 FSM 对比:

FSM 思考方式: "我现在是啥状态?"

- └─ 一堆 if(state == IDLE): 检测过渡

BT 思考方式: "我现在该干啥?"

- └─ 从根节点每帧遍历, 找到最高优先级的可行动作

FSM vs BT:

	FSM	行为树
状态数量爆炸	加行为 → 加状态 × 过渡	加子树
可读性	过渡散落, 大项目变蜘蛛网	树形结构, 一目了然
复用	状态复用难	子树随意组合
重入	回到之前状态要特殊处理	自然支持
调试	看到当前状态	看到当前执行的节点路径
实现复杂度	低	高
Godot 支持	手写或社区插件	社区插件 (LimboAI)

实际项目: 角色控制用 FSM/HSM, AI 行为用 BT。不冲突, 分层使用。

9. 最终方案? 选型指南

故事讲完了。到底用哪个?

按项目规模

原型/Game Jam (1-7天):

└ enum + match ← 最快, 状态少于 8 个够用

小游戏 (1-3月):

└ FSM (通用模块) ← 够干净, 够灵活

商业 2D 游戏:

└ HSM + 状态栈 ← 处理复杂角色行为

大型 3D 游戏:

└ FSM 管角色 + BT 管 AI + AnimationTree 管动画

按复杂度

少于 5 个状态，几乎没有共享逻辑：

- └ enum + match，别过度设计

5-15 个状态，有共享逻辑：

- └ FSM + 状态类 + 过渡表

15+ 个状态，有层级结构：

- └ HSM

角色逻辑 + AI 逻辑：

- └ FSM(角色) + BT(AI)

按团队

独立开发者：

- └ enum + match 起步，胖了再重构 FSM

小团队（2-5人）：

- └ FSM + 过渡表，设计师也能看懂

大团队（10+人）：

- └ HSM + 可视化工具（AnimationTree / BT 编辑器）

- └ 程序员管状态，设计师管动画过渡

常见陷阱

❌ 一开始就上 HSM

- 3 个状态搞 30 个文件，改个方向要改 5 个类

- enum + match 就行了

❌ 状态机里混了太多职责

- 状态管"能做什么"，不管"长什么样"

- 动画和特效交给 AnimationPlayer

❌ 状态过渡写得到处都是

- IdleState 里写转到 Run，RunState 里写转到 Idle

- 集中到过渡表，或者统一在角色脚本里检查

- ✗ 状态的 enter/exit 忘了调用父类
 - HSM 常见 Bug: 父状态的 enter 没执行
 - 用 super() 或在 FSM 级别统一处理
- ✗ 状态机里塞了"持续时间"相关逻辑
 - "攻击后 0.5 秒才能再攻击"不是状态机的事
 - Timer 或 cooldown 变量, 不要为此加一个状态

附: 最简单且实用的 FSM 模板

```
# 如果你只需要一个能用的 FSM, 这个就够了:

class_name FSM
extends Node

var states := {}
var current: String = ""
var previous: String = ""

func add(name: String, state: Node) -> void:
    states[name] = state
    state.set_process(false)
    state.set_physics_process(false)

func change(name: String) -> void:
    if not states.has(name): return
    if current: _exit(states[current])
    previous = current
    current = name
    _enter(states[name])

func _enter(s: Node) -> void:
    s.set_process(true)
    s.set_physics_process(true)
    if s.has_method("enter"): s.enter(previous)

func _exit(s: Node) -> void:
    if s.has_method("exit"): s.exit()
    s.set_process(false)
    s.set_physics_process(false)
```

```
# 用 scene tree 的组织来决定父子关系
# 把状态作为 FSM 节点的子节点, FSM 自动找到它们
func _ready():
    for child in get_children():
        add(child.name.to_lower(), child)
```

这个故事告诉我们: - 状态管理没有银弹, 每个方案解决特定的复杂度 - **不要提前上架构** - enum + match 撑到 8 个状态, 胖了再重构 - 状态机的本质是: **把互斥的逻辑分段, 保证同一时刻只有一段代码在控制角色** - 无论是 boolean、enum、类、树, 最终目标都一样 - 让混乱变得有序

第3章 寻路演化

Godot 寻路系统演化史

从一个"走向目标"到能够穿越复杂地形的 AI, 从撞墙、绕圈、卡在墙角, 到 NavigationAgent 优雅地避开所有障碍物。

这是一场"怎么不撞墙地走到目标"的进化战争。

目录

1. [原始时代: 直线走过去](#)
 2. [Waypoint: 路点导航](#)
 3. [A* 手写: 自己造轮子](#)
 4. [Godot AStar2D: 内置 A*](#)
 5. [导航网格: NavigationPolygon](#)
 6. [NavigationAgent: 寻路一条龙](#)
 7. [分层寻路: Navigation Layers](#)
 8. [动态避障: 多角色不撞](#)
 9. [NavigationServer: 底层控制](#)
 10. [最终方案? 选型指南](#)
-

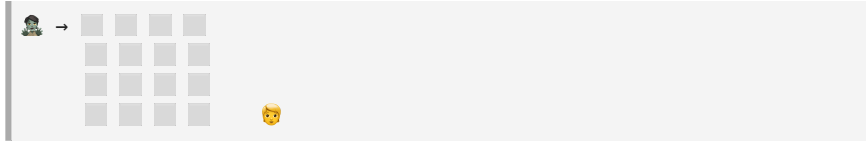
1. 原始时代: 直线走过去

问题: "我要让敌人走向玩家。"

最初的想法: 玩家在 (px, py), 敌人在 (ex, ey), 走过去。

```
func _physics_process(delta):
    # 计算指向玩家的方向
    var dir = (player.position - position).normalized()
    velocity = dir * speed
    move_and_slide()
```

看起来工作正常... 直到:



敌人直接撞墙, 然后停在原地, 因为 `move_and_slide` 碰到墙就把垂直方向速度置零了。敌人只能沿着墙蠕动, 完全绕不过去。

敌人行为: 走向玩家 → 撞墙 → 沿墙滑 → 滑到墙角卡住

问题本质: "朝目标方向走"只适合无障碍场景。有障碍时, 需要找到一条绕过障碍的路径, 而不是直接朝目标走。

2. Waypoint: 路点导航

问题: 敌人撞墙。最简单的解决: 告诉它"不要走直线, 走这些点"。

```
var waypoints = [
    Vector2(100, 200), # 起点旁边
    Vector2(300, 200), # 绕过障碍
    Vector2(500, 400), # 玩家附近
]
var current_waypoint_index = 0

func _physics_process(delta):
    var target = waypoints[current_waypoint_index]
    var dir = (target - position).normalized()
```

```

velocity = dir * speed
move_and_slide()

# 到达路点 → 切到下一个
if position.distance_to(target) < 10:
    current_waypoint_index += 1
    if current_waypoint_index >= waypoints.size():
        # 走完了, 重头或者停
        current_waypoint_index = waypoints.size() - 1

```

进步: 能绕过固定障碍物了。

新问题: 1. **硬编码** – 地图变了就得手改所有路点坐标 2. **玩家在动** – 路点是固定的, 玩家不会站在原地等你走完路径 3. **只会走固定路线** – 无论玩家在哪, 敌人都走同一串路点 4. **无法应对动态障碍** – 路上多了一块石头, 敌人还是会去撞

改良方案: 每次动态计算路点。

```

var waypoints = []

func _update_path():
    # 每 0.5 秒重新计算一串路点
    waypoints = calculate_path_to_player()

func calculate_path_to_player():
    # 简陋的"尝试绕过障碍"
    var path = [player.position]
    # 如果直线路径上有障碍, 在中间插一个绕行点
    if has_obstacle_between(position, player.position):
        var detour = find_detour_point(position,
player.position)
        path.insert(0, detour)
    path.insert(0, position)
    return path

```

但这本质上就是**手写寻路算法**。很快你会发现——你正在重新发明 A*。

3. A* 手写: 自己造轮子

问题: 路点太死板, 需要一种"自动找出避开所有障碍的最短路径"的算法。

A*: 图搜索算法, Dijkstra + 启发式。

```
# 手写 A* (简版)

var grid = {
    # tile_position → is_walkable
}
var grid_size = Vector2(20, 20)
var tile_size = 32

class AStarNode:
    var pos: Vector2
    var g: float = INF # 从起点到此处的代价
    var h: float = 0   # 从此处到目标的估算代价
    var f: float = 0   # g + h
    var parent = null

func astar(start: Vector2, end: Vector2) -> Array:
    var open = []
    var closed = []

    var start_node = AStarNode.new()
    start_node.pos = start
    start_node.g = 0
    start_node.h = heuristic(start, end)
    start_node.f = start_node.h
    open.append(start_node)

    while open.size() > 0:
        # 取 f 值最小的节点
        var current = _pop_lowest_f(open)
        if current.pos == end:
            return _reconstruct_path(current)

        closed.append(current)
        for neighbor in _get_neighbors(current.pos):
            if _in_list(closed, neighbor):
                continue
```

```

        var g = current.g + _cost(current.pos, neighbor)
        var existing = _find_in_list(open, neighbor)
        if existing and g >= existing.g:
            continue
        var node = existing if existing else AStarNode.new()
        node.pos = neighbor
        node.g = g
        node.h = heuristic(neighbor, end)
        node.f = node.g + node.h
        node.parent = current
        if not existing:
            open.append(node)

    return [] # 没找到路径

func heuristic(a: Vector2, b: Vector2) -> float:
    return abs(a.x - b.x) + abs(a.y - b.y) # 曼哈顿距离

func _get_neighbors(pos: Vector2) -> Array:
    var neighbors = []
    for dir in [Vector2(1,0), Vector2(-1,0), Vector2(0,1),
Vector2(0,-1)]:
        var n = pos + dir
        if _is_walkable(n):
            neighbors.append(n)
    return neighbors

```

然后敌人会真正地绕墙:

```

func _update_path_to_player():
    var start_tile = world_to_tile(position)
    var end_tile = world_to_tile(player.position)
    path = astar(start_tile, end_tile)
    path_index = 0

func _physics_process(delta):
    if path_index < path.size():
        var target = tile_to_world(path[path_index])
        var dir = (target - position).normalized()
        velocity = dir * speed
        move_and_slide()
        if position.distance_to(target) < 4:
            path_index += 1

```

```
else:  
    velocity = Vector2.ZERO
```

进步: 自动寻找最短路径, 无需手摆路点。地图啥样, 敌人啥样走。

但问题也来了:

1. **自己维护 grid** – 地图变了, 要同步更新 grid
2. **Tile 必须对齐** – 基于网格的 A* 只能走 4/8 方向, 路径僵硬
3. **性能** – 300 个敌人同时跑 A* 会卡
4. **只适合 TileMap** – 不规则形状的地图(多边形障碍)没法网格化
5. **玩家移动后路径过时** – 每次重跑 A* 负载高
6. **没有避障** – 两个敌人会互相挤在路口, 谁都不让谁

各路开发者开始寻找: "有没有不用手写 A* 的办法?"

4. Godot AStar2D: 内置 A*

问题: 每个人都在写一样的 A*。Godot 说: "我帮你做了。"

```
extends AStar2D  
  
# 自己添加节点 (Point ID)  
func setup():  
    for x in range(20):  
        for y in range(20):  
            if is_walkable(x, y):  
                var id = y * 20 + x  
                add_point(id, Vector2(x, y))  
  
# 连接相邻节点  
for id in get_point_ids():  
    var pos = get_point_position(id)  
    for dir in [Vector2(1,0), Vector2(-1,0), Vector2(0,1),  
Vector2(0,-1)]:  
        var neighbor_pos = pos + dir  
        var neighbor_id = int(neighbor_pos.y * 20 +
```

```

neighbor_pos.x)
    if has_point(neighbor_id):
        connect_points(id, neighbor_id)

# 直接调用
var path_ids = get_id_path(start_id, end_id)
var path_points = get_point_path(start_id, end_id)

```

进步: 不用手写 A* 算法, `get_id_path` 直接返回最短路径。

但问题仍在:

1. 还要手动建图 – `add_point + connect_points`, 地图大了这本身就是个工程
2. AStar2D 不知道你的碰撞体 – 你得自己判断哪些格能走
3. 网格限制 – 路径还是网格化的, 走起来像机器人
4. 没有路径平滑 – 敌人会在网格节点之间直来直去, 每次转角都是 90 度

```

# 路径还是锯齿状的:
# * → * → *
#       ↓
#       * → * ← 不好看

```

社区开始使用路径平滑:

```

# 用 Catmull-Rom 样条或去掉冗余节点
func _smooth_path(points: Array) -> Array:
    var result = [points[0]]
    for i in range(1, points.size() - 1):
        # 如果三点共线, 去掉中间点
        var a = points[i - 1]
        var b = points[i]
        var c = points[i + 1]
        if (b - a).normalized() != (c - b).normalized():
            result.append(b)
    result.append(points[-1])
    return result

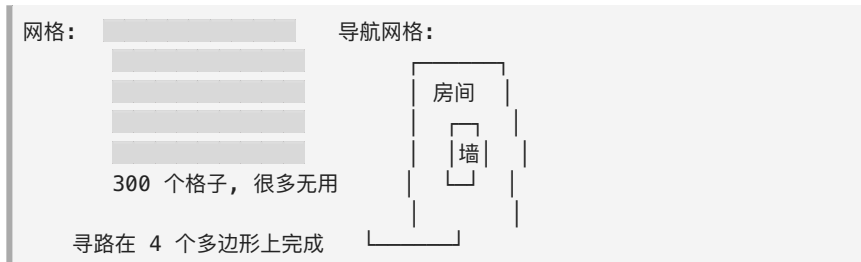
```

但这不是根本方案。根本方案是: 不要让寻路基于网格, 基于实际的可走区域。

5. 导航网格: NavigationPolygon

问题: 基于网格的 A* 路径僵硬、需要手动维护图、不适合不规则地形。

关键洞察: 不是"哪些格子能走", 而是"哪些区域能走"。把地图分成多边形区域, 在这些多边形上做寻路。



Godot 的 NavigationServer 方案:

```
# NavigationRegion2D + NavigationPolygon
# 1. 在编辑器中绘制 NavigationPolygon (标记可走区域)
# 2. Godot 自动生成导航网格
# 3. NavigationServer 在多边形上执行 A*
```

在编辑器里: 用 NavigationRegion2D 节点 + NavigationPolygon, 在可走区域上点几个点, Godot 自动生成导航网格。

代码里:

```
# NavigationServer2D 是自动运行的

var path = NavigationServer2D.map_get_path(
    get_world_2d().navigation_map,
    start_position,          # Vector2 起点
    target_position,         # Vector2 终点
    true                     # 是否优化路径
)
# path 返回一个 Vector2[] - 最终路径点
```

进步: 1. **不需要手动建图** – 编辑器里画可走区域, 或是从 TileMap 自动生成 2. **路径平滑** – 基于多边形边界的路径比网格路径自然得多 3. **支持不规则形状** – 斜墙、圆角、走廊, 都能走 4. **编辑器可视化** – 你画了什么区域一目了然

```
# 最终的用法极其简单
var path = NavigationServer2D.map_get_path(
    navigation_map,
    global_position,
    target.global_position,
    true
)

# 然后沿着 path 走
for point in path:
    # move_to(point) ...
```

但问题还没完: 你要自己沿着 path 走, 每帧找到当前要去哪个点, 到了再切到下一个点。而且如果路径很长, 敌人走到一半玩家跑了, 路径作废, 要重新请求。

6. NavigationAgent: 寻路一条龙

问题: 每次自己追踪路径太麻烦。路径过时了要重新算。玩家一直动, 路径一直作废。

NavigationAgent2D: "帮你把路径追踪也做了。"

```
extends CharacterBody2D

@onready var navigation_agent := $NavigationAgent2D

func _ready():
    # NavigationAgent2D 自动管理路径
    navigation_agent.target_position = player.position

func _physics_process(delta):
```



```

if navigation_agent.is_navigation_finished():
    velocity = Vector2.ZERO
    move_and_slide()
    return

# 🌀 核心: agent 自动算出下一个要去的位置
var next_pos = navigation_agent.get_next_path_position()
var dir = (next_pos - global_position).normalized()
velocity = dir * speed
move_and_slide()

# 如果玩家跑了, 自动重新寻路
# NavigationAgent.update_target_position(player.position) 周期性更新

```

NavigationAgent 内部帮你做了:

NavigationAgent 内部逻辑:

1. 接收 target_position
2. 调用 NavigationServer.map_get_path()
3. 追踪路径: 当前点在哪个线段, 下一个点在哪
4. 走到离下一个点 安全距离 → 自动切到下一个点
5. 如果路径失败/过时 → 自动重新请求
6. 提供 is_navigation_finished() / get_next_path_position() / distance_to_target()

简化成了:

```

# 敌人 AI 完整寻路 (就这么几行)
func _physics_process(delta):
    navigation_agent.target_position = player.position

    if navigation_agent.is_navigation_finished():
        # 到玩家了, 攻击
        return

    var dir = global_position.direction_to(
        navigation_agent.get_next_path_position()
    )
    velocity = dir * speed
    move_and_slide()

```

此时寻路终于从"自己实现"变成了"配置一下就行"。

但: 1. 多个敌人会互相推挤 – 每个 agent 各自寻路, 不关心其他 agent 2. 动态障碍物 (比如箱子被推走了) – NavigationAgent 默认不考虑场景中移动的障碍 3. 性能 – 100 个 agent 同时寻路, 帧率开始掉

7. 分层寻路: Navigation Layers

问题: "我的大怪要走大路, 小怪能钻洞。但寻路网格只有一套。"

现实世界: 不同单位能走的区域不同。 - 步兵: 能走所有路 - 车辆: 只能走大路 - 飞行: 无视所有地面障碍, 走直线 - 潜水: 只能在水域

```
# NavigationRegion2D 支持 layers (类似碰撞层)

# 大路: layer = 1 (所有单位)
# 小路: layer = 2 (只有步兵能进)
# 水域: layer = 4 (只有潜水能进)

navigation_agent.set_navigation_layers(0b001) # 只走大路
navigation_agent.set_navigation_layers(0b011) # 大路 + 小路

# 或者分多个 NavigationRegion2D:
# RegionA (大路): layers = 1
# RegionB (小路): layers = 2
# RegionC (水域): layers = 4
```

在 **NavigationServer** 层面: 每层独立构建导航网格, agent 只能看到自己层级的区域。

```
飞行单位:
是导航:  ← 全部是空地 (无视障碍)
走的路径: 直线

车辆:
是导航:  ← 只有大路
走的路径: 沿路走
```

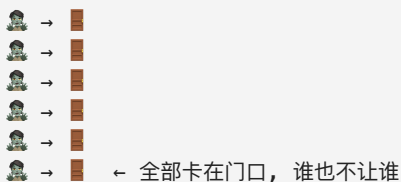
步兵：

是导航：██████████ ← 所有路

走的路径：可以抄小路

8. 动态避障：多角色不撞

问题：10 个敌人都走向玩家，在门口全部挤在一起。



为什么？NavigationAgent 只考虑静态障碍物，不考虑其他 agent。

解决方案：本地避障 (RVO – Reciprocal Velocity Obstacles)。

Godot 的 NavigationAgent 内置 RVO：

```
# 启用 RVO
navigation_agent.pathfinding_algorithm =
NavigationServer2D.PATHFINDING_ALGORITHM_ASTAR
navigation_agent.simplify_path = true

# RVO 相关
navigation_agent.radius = 16.0           # agent 的碰撞半径
navigation_agent.max_speed = speed       # RVO 会用这个做速度预测
navigation_agent.current_velocity = velocity # 告诉 RVO 当前速度

# 在 Update 最后：
navigation_agent.velocity = velocity     # RVO 可能会调整速度来避免碰撞

# 完整 RVO 寻路
func _physics_process(delta):
    navigation_agent.target_position = player.position
    navigation_agent.current_velocity = velocity
```

```

    if navigation_agent.is_navigation_finished():
        velocity = Vector2.ZERO
        move_and_slide()
        return

    var next_pos = navigation_agent.get_next_path_position()
    var dir = global_position.direction_to(next_pos)
    velocity = dir * speed
    move_and_slide()

# 在 RV0 回调中获得调整后的速度:
func _on_navigation_agent_2d_velocity_computed(safe_velocity:
Vector2):
    velocity = safe_velocity # RV0 算出来的"安全速度"

```

RVO 的原理 (一句话):

每个 agent 预测其他 agent 的未来位置，
 如果自己的路径会导致碰撞 → 稍微偏一点方向 → 大家都偏一点 → 自动避开

效果: 门口不会再卡死, agent 会自动排队穿过窄口。

9. NavigationServer: 底层控制

问题: NavigationAgent 很方便, 但当你需要自定义时——比如手动控制路径采样、在异步线程中寻路、修改导航地图——你发现被封装了。

NavigationServer 是 NavigationAgent 底层的实际执行者。

```

# 直接使用 NavigationServer

func get_my_path(from: Vector2, to: Vector2) ->
PackedVector2Array:
    var map = get_world_2d().navigation_map
    return NavigationServer2D.map_get_path(
        map,
        from,
        to,
    )

```

```

        true                                # 优化路径
    )

# 创建自定义导航地图（用于分层世界、镜像世界等）
var custom_map = NavigationServer2D.map_create()
NavigationServer2D.map_set_active(custom_map, true)

# 添加区域
var region = NavigationServer2D.region_create()
NavigationServer2D.region_set_map(region, custom_map)
NavigationServer2D.region_set_navigation_polygon(region,
my_polygon)

# 在这个地图上寻路
NavigationServer2D.map_get_path(custom_map, from, to, true)

```

什么时候需要直接用 **NavigationServer**: - 多张导航地图 (分层世界、大地图分块加载) - 自定义避障算法 (不用 RVO, 用你自己的) - 异步、多线程寻路 - 服务器端寻路 (多人游戏, 只传路径结果给客户端)

但 95% 的开发者不需要这个层次。NavigationAgent 够用。

10. 最终方案? 选型指南

你的角色需要走到目标吗？

- └ 没有障碍物 → `direction_to(target)` → 直线走 ●
(弹幕、追踪子弹、磁铁吸金币)
- └ 有固定障碍，地图不变：
 - └ 基于 `TileMap` → `NavigationRegion2D` + `TileMap` 自动烘焙 ●
(2D RPG、俯视角射击)
 - └ 不规则地图 → 编辑器画 `NavigationPolygon` ●
(平台跳跃、开放地图)
- └ 有动态障碍/移动平台：
 - └ `NavigationAgent2D` + 定期更新 `target_position` ●
(敌人追击、随从)

- └ 多个角色同时寻路，会互相挤：
 - └ NavigationAgent2D + RV0 (radius/velocity 回调) ●
 - (RTS 单位、成群敌人、模拟经营)
- └ 不同单位走不同区域：
 - └ Navigation Layers + set_navigation_layers ●
 - (大怪小怪各有路、水陆两套)
- └ 需要完全自定义控制：
 - └ NavigationServer 直接操作 ●
 - (多人在线、自定义寻路算法)

性能指南：

- 10 个 agent 以内：
 - └ NavigationAgent2D，直接开用
- 10-100 个 agent：
 - └ NavigationAgent2D + RV0
 - └ 减少 target_position 更新频率 (每 0.3s 更新一次)
 - └ 超出玩家视野的 agent 只寻路不避障
- 100-500 个 agent：
 - └ NavigationServer 直接操作
 - └ 自己管理路径缓存
 - └ 分帧寻路：每帧只计算一部分 agent 的路径
 - └ 远处 agent 走简化路径
- 500+ 个 agent (RTS 级别)：
 - └ 流场寻路 (Flow Field)
 - └ Godot 默认不支持，需要自己实现
 - └ 思路：对目标点生成方向场，agent 读场走，0(1) 的寻路

常见陷阱：

- ✗ 每帧重新设置 target_position
 - NavigationAgent 会自动更新，但每帧重建路径浪费
 - 每 0.2~0.5s 更新一次就够了
- ✗ NavigationAgent 和 move_and_slide 打架
 - agent 给你路径点，move_and_slide 处理碰撞
 - 它们不冲突，是"去哪走" vs "怎么走" 的关系

→ direction_to(next_path_point) → velocity → move_and_slide()

✗ 寻路路径走到悬崖边

- NavigationPolygon 如果没有覆盖悬崖, agent 会掉下去
- 确保 NavigationPolygon 只画在有地面的地方

✗ 忘了用 RVO, 以为寻路会自动避开其他角色

- 寻路只避开导航网格里标记的静态障碍
- 要避开其他角色, 必须开 RVO

✗ 路径点到了但会来回抖动

- NavigationAgent2D.path_max_distance 设大一点
- 或者到达判断用 distance < threshold, 不要用 ==

附: NavigationAgent2D 最简模板

```
extends CharacterBody2D

@export var speed := 200.0
@onready var nav_agent := $NavigationAgent2D

func _ready():
    nav_agent.velocity_computed.connect(_on_velocity_computed)

func move_to(target: Vector2):
    nav_agent.target_position = target

func _physics_process(delta):
    if nav_agent.is_navigation_finished():
        velocity = Vector2.ZERO
        move_and_slide()
        return

    var next = nav_agent.get_next_path_position()
    var dir = global_position.direction_to(next)
    var desired = dir * speed

    # RVO 调整速度
    nav_agent.velocity = desired
```

```
func _on_velocity_computed(safe_velocity: Vector2):  
    velocity = safe_velocity  
    move_and_slide()
```

这个故事告诉我们: - 寻路的本质是 "我应该往哪走" 和 "我该怎么走" 两层分离 - 前一层是 NavigationAgent/Server (路径规划), 后一层是 move_and_slide (路径跟随) - 从手写 A → NavigationRegion → NavigationAgent → RVO - 每一步都在解决: "知道目标在哪, 但怎么不撞墙/不撞人地走过去"

第4章 插值与平滑演化

Godot 插值与平滑算法演化

游戏里的一切"手感", 本质都是插值。摄像机跟随、角色移动、UI 动画、颜色过渡、角度旋转——没有插值, 游戏就是一帧一帧的跳变。

从 `=` 到 `lerp` 到弹簧系统, 这是一场"怎么从 A 到 B"的进化。

目录

1. [直接赋值: 最原始的跳变](#)
 2. [匀速移动: `move\_toward`](#)
 3. [Lerp: 指数平滑](#)
 4. [Delta 修正的 Lerp: 帧率无关](#)
 5. [Lerp 的数学本质](#)
 6. [角度插值: `lerp\_angle`](#)
 7. [缓动曲线: `Easing`](#)
 8. [贝塞尔曲线: 路径插值](#)
 9. [弹簧系统: 物理感的平滑](#)
 10. [临界阻尼: 最快不超调](#)
 11. [样条插值: 路过点自动平滑](#)
 12. [选型指南](#)
-

1. 直接赋值: 最原始的跳变

问题: "让摄像机跟着玩家。"

最直接的想法:

```
func _process(delta):  
    position = player.position # 直接等于
```

效果: 摄像机和玩家完全同步, 一帧不差。

问题: 太稳了, 反而没有游戏感。

玩家突然转向 → 摄像机瞬间跳到新位置
玩家跳跃 → 摄像机瞬间上移
像个贴纸贴在玩家身上, 没有"跟拍"的感觉

但直接赋值也不是一无是处, 在特定场景下它是对的:

```
# ✅ 正确的场景: 准星/十字线  
crosshair.position = get_viewport().get_mouse_position()  
# 准星必须和鼠标完全同步, 没有延迟  
  
# ✅ 正确的场景: 拖拽物品  
dragged_item.position = get_global_mouse_position()  
# 拖拽的物品跟随鼠标, 不能有延迟
```

直接赋值的核心属性:

延迟: 0 (无延迟)
超调: 无
平滑度: 0 (不平滑)
适用: 需要绝对同步的 UI 元素

对于"有质感"的物体 (摄像机、角色视角、菜单过渡), 我们需要延迟和平滑。

2. 匀速移动: move_toward

问题: 摄像机直接跳到玩家位置, 太生硬。我要它**匀速**移过去。

```
# move_toward(from, to, step) - 从 from 走向 to, 步长 step
# 不会超过 to

var camera_speed = 200.0 # 像素/秒

func _process(delta):
    position.x = move_toward(position.x, player.position.x,
    camera_speed * delta)
    position.y = move_toward(position.y, player.position.y,
    camera_speed * delta)
```

内部实现:

```
func move_toward(from: float, to: float, delta: float) -> float:
    if abs(to - from) <= delta:
        return to # 到了
    return from + sign(to - from) * delta # 往目标走一步
```

效果: 摄像机以恒定速度移向玩家, 追上了就停。

但有个奇怪的问题: 玩家走, 摄像机在后面追。玩家停, 摄像机追上来也停。整个过程是**匀速**的, 像机器人。

```
玩家移动 → 摄像机开始追
玩家停 → 摄像机匀速追过来 → 突然停
```

"突然停"的感觉不自然。物理世界里, 靠近目标时会**减速**, 而不是撞墙式急停。

move_toward 适合的场景:

```
# ✅ 血条缓慢减少 (HP 从 100 降到 0)
hp_bar.value = move_toward(hp_bar.value, player.hp, 100 * delta)
# 血条匀速减少, 看起来清晰

# ✅ 进度条
progress_bar.value = move_toward(progress_bar.value, target,
```

```

fill_speed * delta)
# 进度条匀速填满

# ✅ 敌人巡逻 (从 A 点到 B 点匀速走)
position = position.move_toward(patrol_target, speed * delta)

```

move_toward 的核心属性:

延迟: 取决于距离/速度
 超调: 无 (到达就停)
 平滑度: 线性 (匀速, 开始和结束都突然)
 适用: 进度条、血条、巡逻

3. Lerp: 指数平滑

问题: 匀速接近的"急停"感不自然。我想要的是: **靠近目标时自动减速。**

Lerp (Linear Interpolation, 线性插值):

```

# lerp(from, to, weight)
# weight = 0 → 返回 from
# weight = 1 → 返回 to
# weight = 0.5 → 返回 from 和 to 的中点

var lerp_weight = 0.05 # 每帧向目标靠近 5%

func _process(delta):
    position = lerp(position, target, lerp_weight)

```

效果:

```

第 1 帧: position = (0, 0), target = (100, 0)
        → lerp((0,0), (100,0), 0.05) = (5, 0)
        跳跃 5 像素

第 2 帧: position = (5, 0), target = (100, 0)
        → lerp((5,0), (100,0), 0.05) = (9.75, 0)
        跳跃 4.75 像素 (比上一帧少了!)

```

第 3 帧: $\rightarrow (13.76, 0)$

跳跃 4.01 像素

...

第 N 帧: 越靠近 target, 每帧移动越少

永远在缩短距离的 5%

这就是"指数平滑": 每次缩短剩余距离的固定比例。

剩余距离: $100 \rightarrow 95 \rightarrow 90.25 \rightarrow 85.74 \rightarrow \dots$

每帧缩短 5%, 永远不会真正到达 0

数学上: $\text{remaining} = \text{remaining} * (1 - \text{weight})^n$

"永远不会真正到达"? 对, 但在游戏里, 当距离小于 0.5px 时, 人眼看不出来。

可以换一种写法:

`position = position.lerp(target, lerp_weight)`

Vector2 的 lerp 方法

Lerp 的"自动减速"特性:

远处的物体 $\rightarrow$ 移动速度快 (缩短 5% 的 $100 = 5$)

近处的物体 $\rightarrow$ 移动速度慢 (缩短 5% 的 $10 = 0.5$)

这恰好是物理世界的感觉: 远处的物体看起来移动快, 近处的慢。

Lerp 的核心属性:

延迟: 永远追不上 (但肉眼看不出来)

超调: 无 (永远在逼近)

平滑度: 自然, 指数平滑

适用: 摄像机跟随、UI 动画、平滑过渡

Lerp 在 Godot 中的位置:

```
lerp(from, to, weight)      # float
lerp(from: Vector2, to, w)  # Vector2
lerp(from: Vector3, to, w)  # Vector3
```

```
lerp(from: Color, to, w)      # Color  
lerp(from: Quaternion, to, w) # 四元数
```

Lerp 的最常见用法 -- 摄像机平滑跟随:

```
func _process(delta):  
    position = position.lerp(target_position, 0.05)
```

但 Lerp 有一个严重陷阱:

4. Delta 修正的 Lerp: 帧率无关

问题: `lerp(position, target, 0.05)` 在不同帧率下速度不同。

```
60fps:  每帧移动剩余距离的 5% → 1 秒移动了  $1 - 0.95^{60} = 95.4\%$   
30fps:  每帧移动剩余距离的 5% → 1 秒移动了  $1 - 0.95^{30} = 78.6\%$   
144fps: 1 秒移动了  $1 - 0.95^{144} = 99.9\%$ 
```

同一行代码, 在不同电脑上效果完全不同! 144fps 的摄像机跑得飞快, 30fps 的慢吞吞。

为什么? lerp 的 weight 是"每帧的比例", 不是"每秒的比例"。帧率越高, 每帧的间隔越短, weight 应该更小。

修正方案:

```
# 标准做法: 把 weight 从"每帧比例"转成"每秒比例"  
  
var lerp_speed = 0.05 # 每帧 5% (等价于这个速度)  
  
# 帧率无关的 lerp:  
func _process(delta):  
    var t = 1.0 - exp(-lerp_speed * delta)  
    position = lerp(position, target, t)
```

为什么这个式子对? 数学推导:

我们想要：每秒移动剩余距离的固定比例，与帧率无关

指数衰减公式： $\text{remaining} = \text{initial} \times (1 - \text{weight_per_second})^{\text{time}}$

Godot 每帧执行：

```
remaining_new = remaining × (1 - weight_per_frame)
```

帧率 60fps: $\text{weight_per_frame} = \text{weight_per_second} / 60$

帧率 30fps: $\text{weight_per_frame} = \text{weight_per_second} / 30$

问题在于 weight_per_frame 不能简单地 $= \text{weight_per_second} \times \text{delta}$

因为 60 帧的累积 $(1 - w/60)^{60} \approx 1 - w + w^2/2 - \dots$ (不等于 $1 - w$)

正确的累积公式： $\text{weight_per_frame} = 1 - \exp(-\text{weight_per_second} \times \text{delta})$

因为 $(1 - (1 - \exp(-w \times 1/60)))^{60} = \exp(-w)$ ← 与帧率无关！

简化封装

```
func smooth_follow(current: Vector2, target: Vector2, speed: float, delta: float) -> Vector2:
```

```
    # speed: "每秒向目标靠近多少比例"
```

```
    # speed = 5.0 表示每秒靠近 99.3% ( $1 - e^{-5}$ )
```

```
    var t = 1.0 - exp(-speed * delta)
```

```
    return lerp(current, target, t)
```

使用：(帧率无关!)

```
position = smooth_follow(position, target_position, 5.0, delta)
```

直觉理解：

speed = 1.0 → 每秒靠近目标的 63%

speed = 3.0 → 每秒靠近目标的 95%

speed = 5.0 → 每秒靠近目标的 99.3%

speed = 10.0 → 每秒靠近目标的 99.995%

不用记公式，Godot 4 提供了现成的：

但是你得自己处理 delta

```
var t = 1.0 - exp(-lerp_speed * delta)
```

```
position = position.lerp(target, t)
```

Lerp + delta 修正是游戏里最常用的平滑方案, 没有之一。

5. Lerp 的数学本质

你以为 lerp 只是"平滑移动"? 它是很多算法的基础。

5.1 Inverse Lerp (反插值)

```
# 已知当前值, 问: 它在 [from, to] 之间的哪个位置?
# 返回 0~1 之间的值

func inverse_lerp(from: float, to: float, value: float) -> float:
    return (value - from) / (to - from)

# 用途: 进度、百分比、归一化
var health_percent = inverse_lerp(0, max_hp, current_hp)
# 血量 75 → 100: → 0.75 (75%)
```

5.2 Remap (映射)

```
# 把一个范围的值映射到另一个范围
# 本质 = inverse_lerp + lerp

func remap(value: float, in_from: float, in_to: float, out_from: float, out_to: float) -> float:
    var t = (value - in_from) / (in_to - in_from) # inverse_lerp
    return out_from + (out_to - out_from) * t      # lerp

# 用途: 音量条、灵敏度映射、颜色映射
# Godot 内置了: remap(value, in_from, in_to, out_from, out_to)
```

5.3 Smoothstep

```
# Lerp 是线性的, Smoothstep 在两端缓入缓出

func smoothstep(edge0: float, edge1: float, x: float) -> float:
    var t = clamp((x - edge0) / (edge1 - edge0), 0.0, 1.0)
    return t * t * (3.0 - 2.0 * t)
```

```
# 曲线形状: 0 → 缓慢加速 → 匀速 → 缓慢减速 → 1
```

```
# 用途: 渐入渐出、摄像机运镜、动画
```

5.4 Lerp 家族关系

```
lerp → 线性插值 (匀速过渡)
├── inverse_lerp → 求"位置" (进度)
├── remap → 范围映射 (音量→进度条)
├── smoothstep → 两端平滑 (渐入渐出)
├── move_toward → 匀速移动 (有界的 lerp)
└── exp 修正 lerp → 帧率无关的指数平滑
```

Lerp 是把"抽象进度"变成"具体数值"的工具。所有"渐变的量"都用 lerp。

6. 角度插值: lerp_angle

问题: 用 lerp 插值角度会出 Bug。

```
# ❌ 错误: 角度用普通 lerp
var current_angle = 350 # 度
var target_angle = 10   # 度
# lerp(350, 10, 0.5) = 180
# 正确路径: 350 → 360 → 10 (只差 20 度)
# lerp 结果: 350 → 180 (绕了 170 度!)
```

角度是循环的: 350° 和 10° 只差 20°, 但普通 lerp 绕了大圈。

解决方案: lerp_angle:

```
# lerp_angle 自动选择最短路径
var current_angle = deg_to_rad(350)
var target_angle = deg_to_rad(10)

var smoothed = lerp_angle(current_angle, target_angle, 0.1)
# 从 350 度 → 355 → 0 → 5 → 10 度 (正确的短路径)
```

内部实现:

```

func lerp_angle(from: float, to: float, weight: float) -> float:
    var difference = to - from
    # 把差值归一化到 [-PI, PI] 区间
    var shortest = fmod(difference, TAU)
    if shortest > PI:
        shortest -= TAU
    elif shortest < -PI:
        shortest += TAU
    return from + shortest * weight

```

3D 中更复杂: 四元数 slerp:

```

# 3D 旋转用 Quaternion.slerp()
# Slerp = Spherical Linear Interpolation
# 在球面上做最短路径插值

var current_rotation: Quaternion
var target_rotation: Quaternion

# slerp (球面线性插值)
current_rotation = current_rotation.slerp(target_rotation, 0.1 *
delta)

```

何时用哪个:

```

# 2D 角度: lerp_angle
rotation = lerp_angle(rotation, target_angle, t)

# 2D 方向向量: 直接 lerp + normalized
direction = direction.lerp(target_dir, t).normalized()

# 3D 旋转 (四元数): slerp
transform.basis = transform.basis.slerp(target_basis, t)

# 3D 旋转 (欧拉角): lerp_angle 每个轴
rotation.x = lerp_angle(rotation.x, target.x, t)
rotation.y = lerp_angle(rotation.y, target.y, t)
rotation.z = lerp_angle(rotation.z, target.z, t)

```

7. 缓动曲线: Easing

问题: `lerp` 是指数平滑, `move_toward` 是匀速。但如果我想要的不是这两种呢?

- **弹跳效果:** 落地时弹两下 (物理感)
- **过冲效果:** 超过目标再弹回来 (有弹性的感觉)
- **缓入:** 慢慢开始, 快速结束 (沉重物体启动)
- **缓出:** 快速开始, 慢慢停止 (轻快物体停下)

Easing 函数: 定义了一个进度 (0→1) 如何映射到另一个进度 (0→1)。

```
# Godot 的 Tween 内置大量缓动曲线

var tween = create_tween()
tween.tween_property(node, "position:x", 500,
1.0).set_trans(Tween.TRANS_BOUNCE)

# TRANS_ 常量:
# LINEAR    → 匀速 (最机械)
# SINE      → 正弦 (自然柔和)
# QUAD      → 二次方
# CUBIC     → 三次方
# QUART     → 四次方
# QUINT     → 五次方
# EXPO      → 指数 (极端的缓入缓出)
# BACK      → 过冲再回弹
# BOUNCE    → 落地弹跳
# ELASTIC   → 橡皮筋振荡
```

```
# 如果你需要自己实现:

# 缓入 (Ease In) - 慢→快, 适合重物启动
func ease_in(t: float) -> float:
    return t * t # 二次缓入

# 缓出 (Ease Out) - 快→慢, 适合轻物停止
func ease_out(t: float) -> float:
    return t * (2 - t) # 二次缓出
```

缓入缓出 (Ease In Out) - 慢→快→慢, 适合运镜

```
func ease_in_out(t: float) -> float:
```

```
    if t < 0.5:
```

```
        return 2 * t * t
```

```
    else:
```

```
        return -1 + (4 - 2 * t) * t
```

弹跳 (Bounce)

```
func ease_bounce(t: float) -> float:
```

```
    # 模拟落地弹跳, 多次衰减反弹
```

```
    if t < 1 / 2.75:
```

```
        return 7.5625 * t * t
```

```
    elif t < 2 / 2.75:
```

```
        t -= 1.5 / 2.75
```

```
        return 7.5625 * t * t + 0.75
```

```
    elif t < 2.5 / 2.75:
```

```
        t -= 2.25 / 2.75
```

```
        return 7.5625 * t * t + 0.9375
```

```
    else:
```

```
        t -= 2.625 / 2.75
```

```
        return 7.5625 * t * t + 0.984375
```

Easing 的本质:

输入 t (0→1, 时间进度)

↓

Easing 函数

↓

输出 t' (0→1, 插值进度)

↓

lerp(from, to, t')

Easing 的效果选择:

菜单出现: Ease Out Bounce (弹入)

菜单消失: Ease In Back (过冲收回)

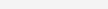
UI 通知滑入: Ease Out Cubic

摄像机运镜: Ease In Out Sine

打击反馈: Ease Out Elastic (抖动)

进度条: Ease In Quad (开始慢, 结尾快, 显得快)

Easing 曲线的可视化:

LIN:		(直线匀速)
EaseIn:		(先慢后快)
EaseOut:		(先快后慢)
InOut:		(慢慢慢)
Bounce:		(弹跳衰减)
Back:		(过冲再回来)
Elastic:		(振荡衰减)

8. 贝塞尔曲线: 路径插值

问题: 想让物体走一条弧线路径, 而不是直线。比如抛射物、菜单弧形动画、摄像机弧线运镜。

贝塞尔曲线: 用控制点定义一条曲线。

```
# 二次贝塞尔: 3 个点 (起点 P0, 控制点 P1, 终点 P2)
func bezier_quad(p0: Vector2, p1: Vector2, p2: Vector2, t:
float) -> Vector2:
    var q0 = lerp(p0, p1, t)
    var q1 = lerp(p1, p2, t)
    return lerp(q0, q1, t)

# 三次贝塞尔: 4 个点 (起点 P0, 控制点1 P1, 控制点2 P2, 终点 P3)
func bezier_cubic(p0: Vector2, p1: Vector2, p2: Vector2, p3:
Vector2, t: float) -> Vector2:
    var q0 = lerp(p0, p1, t)
    var q1 = lerp(p1, p2, t)
    var q2 = lerp(p2, p3, t)
    var r0 = lerp(q0, q1, t)
    var r1 = lerp(q1, q2, t)
    return lerp(r0, r1, t)
```

贝塞尔曲线的几何意义: 对控制点的连线做递归 lerp。

三次贝塞尔:
P0 — P1 — P2 — P3

```
第 1 层: lerp(P0,P1) → lerp(P1,P2) → lerp(P2,P3)
第 2 层: lerp(上面第1层的1, 上面第1层的2) → lerp(上面第1层的2, 上面第1层的3)
第 3 层: lerp(上面第2层的两个结果)

t 从 0 走到 1, 点沿着曲线走
```

Godot 的内置实现:

```
# Godot 有 Curve2D / Curve3D 资源

@export var curve: Curve2D # 在编辑器里画路径

func follow_path(t: float):
    position = curve.sample_baked(t) # t 从 0 到 1

# sample_baked 返回的是"均匀"采样的位置
# 也可以用 sample() 返回参数化(不均匀)采样
```

贝塞尔曲线的应用:

```
# 1. 抛物物弧线
var start = global_position
var end = target.global_position
var control = (start + end) / 2 + Vector2(0, -100) # 弧顶向上

t += delta * speed
bullet.position = bezier_quad(start, control, end, t)
if t >= 1.0:
    bullet.hit_target()

# 2. 鱼竿钓鱼线
# 控制点动态变化, 模拟钓竿弯曲

# 3. UI 菜单弧形排列
for i in range(count):
    var t = float(i) / (count - 1)
    button.position = bezier_quad(start_pos, control_point,
    end_pos, t)
```

贝塞尔的核心属性:

局部性：移动一个控制点只影响附近曲线
直观：控制点拉得越远，曲线越"吸"向它
可导：可以算切线（速度），二阶导（加速度）

9. 弹簧系统：物理感的平滑

问题：lerp 平滑，但没有"物理感"。如果我想要：- 物体超过目标再弹回来 (有弹性) - 被推动后自己摆正 (弹簧门) - 有惯性的摄像机 (拉拽感)

弹簧-阻尼系统 (Spring Damper):

```
# 弹簧系统的核心：
# 位置靠近目标时→弹簧力拉回来
# 位置越过目标时→弹簧力反向拉回
# 阻尼防止无限振荡

var position = 0.0      # 当前位置
var velocity = 0.0      # 当前速度
var stiffness = 10.0    # 弹簧刚度（越大越硬，回弹越快）
var damping = 1.0       # 阻尼（越大越不弹）

func spring_update(target: float, delta: float):
    var force = (target - position) * stiffness # 弹簧力：偏离越远，拉力越大
    velocity += force * delta                  # 力 → 加速度 → 速度
    velocity -= velocity * damping * delta     # 阻尼：抵消速度，防止无限弹
    position += velocity * delta               # 速度 → 位移
```

效果：

```
target 跳到 100:
    position 慢慢拉向 100
    到达 100 时 velocity 不为 0 (惯性) → 冲过 100
    冲过了 → 弹簧力反向 → 拉回来
    反复 → 阻尼让振荡衰减 → 最终停在 100
```


关键参数调感:

```
stiffness = 1, damping = 0.5 → 软绵绵, 像果冻  
stiffness = 10, damping = 1 → 有弹性, 像弹簧  
stiffness = 50, damping = 8 → 硬, 几乎不弹 (接近 lerp)  
stiffness = 100, damping = 0 → 永远振荡, 像永动机
```

完整的摄像机弹簧跟随 (2D)

```
extends Camera2D
```

```
var spring_pos := Vector2.ZERO  
var spring_vel := Vector2.ZERO
```

```
@export var stiffness := 8.0  
@export var damping := 2.0  
@export var max_speed := 500.0
```

```
func _physics_process(delta):  
    var target = get_parent().global_position # 跟随父节点  
  
    # 弹簧力  
    var force = (target - spring_pos) * stiffness  
  
    # 速度更新 (限制最大速度, 防止突然位移太大)  
    spring_vel += force * delta  
    spring_vel = spring_vel.limit_length(max_speed)  
  
    # 阻尼 (与速度方向相反)  
    spring_vel -= spring_vel * damping * delta  
  
    # 位置更新  
    spring_pos += spring_vel * delta  
  
    # 最终位置  
    global_position = spring_pos
```

实际效果:

```
# 玩家突然跳跃:  
# 弹簧摄像机: 玩家跳起来 → 摄像机延迟 0.2s 才跟上 → 然后超过一点 → 弹回来  
→ 稳住
```

```
# 感觉：摄像机有"质量"，被玩家"拉着走"
```

```
# lerp 摄像机：玩家跳起 → 摄像机指数逼近 → 永远追不上，但也没有惯性感
```

弹簧 vs Lerp:

Lerp:	弹簧:
└ 不会超调	└ 会超调再弹回
└ 没有惯性	└ 有惯性（过了才停）
└ 参数少	└ 参数多（stiffness/damping）
└ 永远追不上	└ 最终追上并停住
└ 适合 UI	└ 适合物理感的追踪

10. 临界阻尼：最快不超调

问题：弹簧系统的 oscillations (振荡) 在某些场景下很难看：- 菜单从 A 移到 B，结果冲过头弹两下 - 很丑 - 摄像机在狭窄走廊里弹来弹去 - 很晕

临界阻尼 (Critically Damped)：弹簧系统的一个特殊状态——以最快的速度到达目标，且不超调。

阻尼 < 临界：	欠阻尼 (underdamped)	→ 会弹，但有弹性感
阻尼 = 临界：	临界阻尼 (critically)	→ 最快到达，不弹
阻尼 > 临界：	过阻尼 (overdamped)	→ 慢吞吞到，不弹

```
# 临界阻尼公式（不需要调参，只有一个 speed 参数）
```

```
func critically_damped_follow(  
    current: Vector2,  
    target: Vector2,  
    speed: float,      # 响应速度（越大越快）  
    velocity: Vector2, # 当前速度（引用）  
    delta: float  
)-> Vector2:  
    var diff = target - current  
    var omega = 2.0 * speed      # 角频率  
    var x = omega * diff * delta # 位置修正项  
    var y = velocity * delta     # 速度修正项
```

```

# 更新位置
var new_current = current + x + y

# 更新速度
velocity += (omega * omega * diff + omega * velocity) *
delta * -1.0

return new_current

# 使用:
var vel := Vector2.ZERO

func _process(delta):
    position = critically_damped_follow(position,
target_position, 4.0, vel, delta)

```

临界阻尼的定位:

```

lerp:          最简单, 但帧率相关, 永远追不上
move_toward:   匀速, 急停
弹簧:          有弹性, 可能振荡
临界阻尼:      最快到达, 不弹, 帧率无关

```

实战中最常用的是哪种? —— 看场景。

```

# 2D 摄像机跟随 → 临界阻尼 (稳, 快, 不晕)
# 3D 摄像机跟随 → 弹簧 (有物理感, 电影感)
# UI 动画 → lerp + ease 曲线 (轻量, 可控)
# 血条 → move_toward (匀速减少, 清晰)
# 菜单过渡 → 缓动曲线 (有设计感)

```

11. 样条插值: 路过点自动平滑

问题: 有一个路径点数组, 想让物体"平滑地经过每个点", 而不是在每个点之间走直线。

```

var waypoints = [
    Vector2(0, 0),
    Vector2(100, 200),
    Vector2(300, 50),
    Vector2(500, 300),
]

```

最粗糙的做法: 点之间用 lerp。

```

# 折线路径 - 每个点之间直线, 到了拐角突然转向
func follow_waypoints(t: float):
    var total = waypoints.size() - 1
    var segment = floor(t * total)    # 当前在哪个线段
    var local_t = (t * total) - segment # 在线段内的进度
    return lerp(waypoints[segment], waypoints[segment + 1],
        local_t)

```

效果: 经过每个点时会有急转弯。像机器人。

Catmull-Rom 样条: 自动让曲线平滑经过每个点。

```

# Catmull-Rom: 通过 4 个点 (P0, P1, P2, P3) 插值 P1→P2 这一段
# 曲线会平滑地经过 P1 和 P2

func catmull_rom(p0: Vector2, p1: Vector2, p2: Vector2, p3:
Vector2, t: float) -> Vector2:
    var t2 = t * t
    var t3 = t2 * t

    return 0.5 * (
        (2.0 * p1) +
        (-p0 + p2) * t +
        (2.0 * p0 - 5.0 * p1 + 4.0 * p2 - p3) * t2 +
        (-p0 + 3.0 * p1 - 3.0 * p2 + p3) * t3
    )

# 遍历 waypoints:
func follow_smooth(t: float):
    var total = waypoints.size() - 1
    var i = int(t * total)    # 当前在哪个区间
    var local_t = (t * total) - i

```

```

var p0 = waypoints[max(i - 1, 0)]
var p1 = waypoints[i]
var p2 = waypoints[i + 1]
var p3 = waypoints[min(i + 2, waypoints.size() - 1)]

return catmull_rom(p0, p1, p2, p3, local_t)

```

效果: 路径点之间的过渡自动平滑, 经过每个点时自然转弯。

Godot 内置样条:

```

# Curve2D 内部使用 Catmull-Rom 样条
# 在编辑器里添加点, Godot 自动平滑连接
@export var path: Curve2D

func follow(t: float):
    position = path.sample_baked(t)
    # sample_baked 返回均匀采样的路径位置

```

样条插值的应用场景:

1. 敌人巡逻路径 - 不用在每个路点转向, 自然转弯
2. 摄像机运镜 - 预设路径, 平滑走过每个镜头位
3. 赛道 - 赛车游戏路径
4. 动画曲线 - 关键帧之间的插值

12. 选型指南

按需求选

"平滑跟随目标"	→ Lerp + delta 修正
└ "有物理感, 会超调"	→ 弹簧系统
└ "最快到达, 不超调"	→ 临界阻尼
└ "匀速到达"	→ move_toward
"从 A 到 B 的过渡"	→ Tween + Easing 曲线
└ "弹入"	→ Bounce
└ "有弹性"	→ Elastic / Back

└ "自然柔和"	→ Sine
"沿路径移动"	→ Catmull-Rom 样条
└ "有控制点调形状"	→ 贝塞尔曲线
└ "能编辑"	→ Curve2D 资源
"角度/旋转"	→ lerp_angle
└ "3D 旋转"	→ Quaternion.slerp
"数值映射"	→ remap / inverse_lerp
"渐入渐出"	→ smoothstep
"颜色过渡"	→ Color.lerp

按感觉选

感觉	方案
机械、精准、无延迟	直接赋值
匀速、稳定、清晰	move_toward
柔和、自然、轻量	Lerp
有弹性、惯性、物理感	弹簧系统
快速、稳定、不会超调	临界阻尼
优雅、设计感	缓动曲线
电影感、弧线	贝塞尔/样条

性能对比

直接赋值: 0(1) – 最快
move_toward: 0(1) – 快
Lerp: 0(1) – 快
Easing: 0(1) – 快
贝塞尔: 0(n) – n 是控制点数量
样条: 0(n) – n 是路径点数量
弹簧: 0(1) – 快 (但每帧要多存一个 velocity 变量)

常见陷阱

✗ 角度用普通 lerp → 用 lerp_angle (2D) / Quaternion.slerp (3D)
--

✗ Lerp weight 没乘 delta

→ 不同帧率速度不同

→ $t = 1 - \exp(-\text{speed} * \text{delta})$

✗ move_toward 用在"平滑跟随"

→ 急停感强，用 lerp 更自然

✗ 贝塞尔曲线用 t 均匀采样

→ 控制点密集处运动快，稀疏处运动慢

→ 用 sample_baked 均匀采样

✗ 弹簧参数乱调

→ 先调 stiffness 决定"多快回正"

→ 再调 damping 决定"多快停住"

→ damping 一般 = $2 \times \sqrt{\text{stiffness}}$ 附近（临界阻尼）

✗ 动画每帧手动 lerp 而不是用 Tween

→ Tween 内置 delta 修正 + 缓动曲线 + 序列

→ 除非需要自定义控制，否则优先 Tween

附：最常用的几个平滑模板

—— 1. 帧率无关的 Lerp 跟随（90% 场景够用） ——

```
func smooth_follow(current: Vector2, target: Vector2, speed: float, delta: float) -> Vector2:
    var t = 1.0 - exp(-speed * delta)
    return current.lerp(target, t)
```

使用:

```
position = smooth_follow(position, target, 5.0, delta)
```

—— 2. 角度插值 ——

```
rotation = lerp_angle(rotation, target_angle, 1.0 - exp(-speed * delta))
```

—— 3. 弹簧系统（摄像机、物理感跟随） ——

```
var spring_vel := 0.0

func spring_follow(current: float, target: float, stiffness:
float, damping: float, delta: float) -> float:
    var force = (target - current) * stiffness
    spring_vel += force * delta
    spring_vel -= spring_vel * damping * delta
    return current + spring_vel * delta

# 使用:
position.x = spring_follow(position.x, target.x, 8.0, 2.0,
delta)
```

—— 4. 二维贝塞尔弧线 ——

```
func bezier_quad(p0: Vector2, p1: Vector2, p2: Vector2, t:
float) -> Vector2:
    return lerp(lerp(p0, p1, t), lerp(p1, p2, t), t)
```

—— 5. 最简单的缓动（Ease Out） ——

```
func ease_out(t: float) -> float:
    return t * (2 - t)
```

这个故事告诉我们：- 所有的平滑，底层都是 lerp（线性插值）- lerp 的各种变体（exp修正，lerp_angle，slerp，贝塞尔，样条）解决特定场景的问题 - 弹簧系统和临界阻尼是 lerp 的“物理版本”，多了速度/惯性维度 - 选哪个方案 = 在 延迟 × 超调 × 平滑度 × 性能 之间做权衡

第5章 基础算法演化

Godot 基础算法演化

有时候, 最简单的代码反而是最容易被写错的。一个 `if x > max: x = max` → 你不知道有 `clamp`。一个 `if a == true: a = false; else: a = true` → 你不知道 `= not`。

这里没有复杂的数学。只有"这行代码还能这么写?"

1. 夹紧: 不让值越界

问题: "玩家的位置不能超出屏幕边界。"

最初的做法:

```
# 左边界
if position.x < 0:
    position.x = 0
# 右边界
if position.x > screen_width:
    position.x = screen_width
```

4 个方向 → 8 行代码。每个边界都写一次 if。

进化 1: min + max

```
position.x = max(0, position.x)
position.x = min(screen_width, position.x)

# 合并:
position.x = max(0, min(screen_width, position.x))
```

进化 2: clamp (Godot 内置)

```
# clamp(value, min, max)
position.x = clamp(position.x, 0, screen_width)
position.y = clamp(position.y, 0, screen_height)

# 一行搞定两个轴:
position = position.clamp(Vector2.ZERO, Vector2(screen_width,
screen_height))
```

举一反三:

```
# 血量
hp = clamp(hp + heal_amount, 0, max_hp)

# 速度
speed = clamp(speed + boost, min_speed, max_speed)

# 角度
rotation = clamp(rotation, -PI/4, PI/4)

# 透明度
modulate.a = clamp(modulate.a + delta, 0.0, 1.0)

# 百分比
progress = clamp(progress + delta * rate, 0.0, 1.0)

# 索引 (防越界)
index = clamp(index, 0, array.size() - 1)
```

写法演化:

```
if x < 0: x = 0, if x > max: x = max    4行
→ max(0, min(max, x))                1行
→ clamp(x, 0, max)                   1行, 更直观
```

2. 开关/切换：翻转值

问题：按一下开灯，再按一下关灯。

最初的做法：

```
if light_on == true:
    light_on = false
else:
    light_on = true
```

进化 1: `= not`

```
light_on = not light_on
```

进化 2: 异或 XOR

```
# 对数字的 0/1 切换
state = state ^ 1

# 在 Godot 里bool用 not, int 用 ^1
```

实际场景：

```
# 暂停开关
is_paused = not is_paused

# 显示/隐藏
visible = not visible

# 敌人追踪/巡逻切换
is_chasing = not is_chasing

# 多人游戏回合制
is_my_turn = not is_my_turn
```

写法演化：

```
if a then false else true    3行
→ a = not a                 1行
```

3. 符号判断：正还是负

问题："如果玩家在左边就往右追，在右边就往左追。"

最初的做法：

```
var direction = 0
if player.position.x > enemy.position.x:
    direction = 1
elif player.position.x < enemy.position.x:
    direction = -1
else:
    direction = 0
```

进化：sign / signf

```
# sign(value): 正数 → 1, 负数 → -1, 0 → 0
var direction = sign(player.position.x - enemy.position.x)

# 或者用 Vector2 的 direction_to
var direction = enemy.position.direction_to(player.position)
```

进化对比：

```
# before:
if value > 0: result = 1
elif value < 0: result = -1
else: result = 0

# after:
result = sign(value)
```

sign 的实际用途：

```

# 朝向玩家
sprite.scale.x = sign(direction.x) # 左→flip, 右→正常
if direction.x != 0:
    sprite.scale.x = abs(sprite.scale.x) * sign(direction.x)

# 往玩家方向射击
bullet.direction.x = sign(player.x - position.x)

# 判断同一个方向
if sign(velocity.x) == sign(target_direction.x):
    # 已经在往目标方向移动了

```

写法演化:

if...elif...else	5行
→ sign(value)	1行

4. 取整: 小数变整数

问题: `speed * delta` 出现了小数, 但坐标需要整数 (TileMap 对齐等)。

最初的做法:

```

var float_val = 3.7

# 直接 int() - 截断 (向零取整)
var int_val = int(float_val) # 3 (3.7 → 3)

# 但 int(-3.7) = -3 - 不是 -4!

```

进化的各种方案:

```

# floor - 向下取整 (往小)
floor(3.7)    # 3.0
floor(-3.7)   # -4.0

# ceil - 向上取整 (往大)
ceil(3.7)     # 4.0

```

```

ceil(-3.7)    # -3.0

# round - 四舍五入
round(3.7)    # 4.0
round(3.2)    # 3.0
round(-3.7)   # -4.0

# roundi / floori / ceiling - 直接返回 int
roundi(3.7)   # 4
floori(3.7)   # 3
ceiling(3.7)  # 4

```

Godot 特有的 snap:

```

# snap - 对齐到步长
snapped(13, 5)      # 15 (对齐到 5 的倍数)
snapped(12, 5)      # 10
snapped(0.73, 0.25) # 0.75

# 用途: 网格对齐
position = snapped(position, Vector2(32, 32)) # 对齐到 32px 网格

# 用途: 音量步进
volume = snapped(volume, 0.1)

```

写法演化:

```

int(value) → 截断, 但不直觉
→ floor/ceil/round → 取整方向和策略可选
→ snapped → 对齐到任意步长

```

5. 计时器/倒计时: 等一会儿

问题: "攻击后 0.5 秒才能再攻击。"

最初的做法: 用 `_process` 手算。

```

var attack_cooldown = 0.0

```

```

func _process(delta):
    if attack_cooldown > 0:
        attack_cooldown -= delta

func attack():
    if attack_cooldown > 0:
        return # 冷却中
    # 执行攻击
    attack_cooldown = 0.5

```

进化 1: Timer 节点

```

@onready var attack_timer = $AttackTimer

func attack():
    if not attack_timer.is_stopped():
        return # 冷却中
    # 执行攻击
    attack_timer.start(0.5)

# 在编辑器设 one_shot = true, Timer 自动倒计时

```

进化 2: 用 SceneTreeTimer

```

func attack():
    # get_tree().create_timer(seconds) 返回一个 Timer, 自动删除
    # 配合 await 使用
    get_tree().create_timer(0.5).timeout.connect(_reset_attack)

# 或者直接 await
func attack_sequence():
    play_animation("attack")
    await get_tree().create_timer(0.3).timeout # 等 0.3 秒
    spawn_hitbox()
    await get_tree().create_timer(0.2).timeout # 再等 0.2 秒
    play_animation("idle")

```

进化 3: Tween 代替计时器

```

# Tween 做延时比其他方式更可控
func flash_red():
    var tween = create_tween()

```



```
tween.tween_property($Sprite, "modulate", Color.RED, 0.1)
tween.tween_interval(0.2)
tween.tween_property($Sprite, "modulate", Color.WHITE, 0.1)
```

延时执行

```
func delayed_call(delay: float, callback: Callable):
    var tween = create_tween()
    tween.tween_interval(delay)
    tween.tween_callback(callback)
```

写法演化:

手写 delta 累减	~5行
→ Timer 节点	~3行 (创建+启动+信号)
→ SceneTreeTimer + await	~1行
→ Tween	~3行 (可控、可链)

6. 循环列表/绕圈: 到头了从头来

问题: 敌人走完最后一个巡逻点, 回到第一个重新走。

最初的做法:

```
var waypoints = [pos1, pos2, pos3]
var index = 0

func next_waypoint():
    index += 1
    if index >= waypoints.size():
        index = 0 # 回到开头
    return waypoints[index]
```

进化: 用 % 取余

```
index = (index + 1) % waypoints.size()
```

但 % 在负数时有问题。

```

# 普通 %:
-1 % 3    # -1 (不是 2!)
# 因为 Godot 的 % 是"余数"不是"模"

# 用 posmod / wrapi (Godot 内置的"正模"):
posmod(-1, 3) # 2 ✓
wrapi(index, waypoints.size())
wrapf(value, 0.0, 1.0)

# 前后循环
index = wrapi(index + 1, waypoints.size()) # 下一个
index = wrapi(index - 1, waypoints.size()) # 上一个

# 角度环绕
rotation = wrapf(rotation, -PI, PI)

# 进度条
progress = wrapf(progress + delta, 0.0, 1.0)

```

写法演化:

index++, if >= size: index=0	3行
→ index = (index + 1) % size	1行 (但负数bug)
→ wrapi(index + 1, size)	1行 (安全)

7. 限制变化量: 不让值突变

问题: "血量减少时最大减少 10, 不能一次扣太多。"

最初的做法:

```

var damage = player.attack - enemy.defense
if damage > max_damage_per_tick:
    damage = max_damage_per_tick
hp -= damage

```

进化: `move_toward`

```
# move_toward(from, to, step)
# 从 from 向 to 走, 但每步不超过 step
hp = move_toward(hp, hp - damage, max_damage_per_tick)

# 通用: "不让当前值变化得太快"
current = move_toward(current, target, max_change)

# 实际场景:
# 音量渐变
volume = move_toward(volume, target_volume, 0.1 * delta)

# 摄像机缩放不能突变
zoom = move_toward(zoom, target_zoom, zoom_speed * delta)

# 速度不能突变
velocity.x = move_toward(velocity.x, target_speed.x,
    acceleration * delta)
```

写法演化:

```
if diff > max: diff = max      3行
→ move_toward(from, to, step) 1行
```

8. 在范围内: 判断值是否在区间

问题: "玩家在某个区域内触发事件。"

最初的做法:

```
if x > left and x < right and y > top and y < bottom:
    trigger()
```

进化:

```
# Rect2 的 has_point
var area = Rect2(left, top, right - left, bottom - top)
if area.has_point(position):
    trigger()
```

```

# 或者用 Area2D 节点（更适合）
# 在编辑器里拉个 Area2D + CollisionShape2D
# 用 body_entered / body_exited 信号

# 距离判断也有简写：
if position.distance_to(target) < range:
    # 在范围内

# 但 distance_to 用了 sqrt，性能更好的：
if position.distance_squared_to(target) < range * range:
    # 不用开根号，更快

```

其他区间判断：

```

# 判断值是否在 [a, b] 之间
if value >= a and value <= b:

# 用 clamp 来判断（巧妙）
if clamp(value, a, b) == value:
    # 在区间内

# 百分比判断
if inverse_lerp(min_val, max_val, value) > 0.5:
    # 值超过中点了

```

写法演化：

多条件 && 连接	~4行
→ Rect2.has_point()	1行（矩形）
→ Area2D 信号	0行代码（节点方案）
→ distance_squared_to	1行（去掉sqrt）

9. 延迟删除：过一会儿消失

问题：敌人死亡后，播放死亡动画，然后 2 秒后删除节点。

最初的做法：

```

var death_timer = 0.0

func _process(delta):
    if is_dead:
        death_timer += delta
        if death_timer > 2.0:
            queue_free()

```

进化:

```

# 方案 1: Timer 节点
$DeathTimer.start(2.0)
$DeathTimer.timeout.connect(queue_free)

# 方案 2: SceneTreeTimer
get_tree().create_timer(2.0).timeout.connect(queue_free)

# 方案 3: await
await get_tree().create_timer(2.0).timeout
queue_free()

```

减少重复: 封装成工具函数:

```

func delayed_free(node: Node, delay: float = 0.0):
    if delay <= 0:
        node.queue_free()
    else:
        node.get_tree().create_timer(delay).timeout.connect(node.queue_free)

# 使用:
delayed_free(self, 2.0)

```

写法演化:

手写 delta 累加 + if 判断	~5行
→ create_timer().timeout	1行
→ 封装成工具函数	1行调用

10. 随机选择: 从数组中取一个

问题: "随机播放一个音效。"

最初的做法:

```
var sounds = [hit1, hit2, hit3]
var index = randi() % sounds.size()
sound.play(sounds[index])
```

进化: 用数组工具:

```
# Godot 4 的 .pick_random()
sound.play(sounds.pick_random())

# 从场景组中随机
var enemy = get_tree().get_nodes_in_group("enemy").pick_random()

# 加权随机 (常用)
var items = [sword, shield, potion]
var weights = [10, 30, 60] # 总 100

func weighted_random(items: Array, weights: Array):
    var total = 0
    for w in weights:
        total += w
    var r = randi() % total
    for i in items.size():
        r -= weights[i]
        if r < 0:
            return items[i]
    return items[-1]
```

不重复随机 (洗牌):

```
# Fisher-Yates 洗牌
var cards = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
for i in range(cards.size() - 1, 0, -1):
    var j = randi() % (i + 1)
    var temp = cards[i]
    cards[i] = cards[j]
```

```

        cards[j] = temp

# Godot 有现成的 .shuffle()
cards.shuffle()
# 随机弹出一个
var card = cards.pop_back() # 不重复！抽完为止

```

写法演化：

```

randi() % size + 索引访问      2行
→ .pick_random()              1行
→ .shuffle() + .pop_back()     2行（不重复）

```

11. 简易动画帧：换图

问题："玩家走路时切换脚的位置 (帧动画)。"

最初的做法：

```

var frame = 0
var frame_timer = 0.0

func _process(delta):
    if is_moving:
        frame_timer += delta
        if frame_timer > 0.1:
            frame_timer = 0.0
            frame = (frame + 1) % 4 # 4 帧走路
            $Sprite.frame = frame

```

进化：AnimatedSprite2D

```

# 拖入 AnimatedSprite2D
# 在编辑器里添加 SpriteFrames 资源
# 添加动画 "walk", 拖入 4 帧, 设 fps = 10

$AnimatedSprite2D.play("walk" if is_moving else "idle")
# 自动换帧！

```

再进化: AnimationPlayer

```
# AnimationPlayer 不光能换帧，还能做任何属性动画
# position 位移、modulate 变色、scale 缩放

$AnimationPlayer.play("walk")
$AnimationPlayer.play("idle")
# 帧率、循环、过渡速度都在编辑器里设
```

写法演化:

手算 frame timer + 换帧	~6行
→ AnimatedSprite2D.play()	1行 (只能换图)
→ AnimationPlayer.play()	1行 (任何属性)

12. Ping-Pong 值: 来回弹

问题: "让一个灯光的亮度在 0~1 之间来回渐变。"

最初的做法:

```
var brightness = 0.0
var direction = 1

func _process(delta):
    brightness += direction * delta * 0.5
    if brightness > 1.0:
        brightness = 1.0
        direction = -1
    elif brightness < 0.0:
        brightness = 0.0
        direction = 1
```

进化: ping-pong + sin:

```
# pingpong(value, length) - 在 0~length 之间来回弹
# 输入自增的时间，输出来回弹的值
var t = Time.get_ticks_msec() / 1000.0
```



```
brightness = pingpong(t * 0.5, 1.0)
```

```
# 或者用 sin (更平滑):
```

```
brightness = (sin(t * 2.0) + 1.0) * 0.5 # 0~1 平滑正弦
```

```
# 频繁出现的场景:
```

```
# 浮动效果 (捡到的金币往上飘)
```

```
position.y -= sin(t * 3.0) * delta * 20
```

```
# 呼吸灯
```

```
modulate.a = 0.5 + sin(t * 2.0) * 0.5
```

```
# 摇晃
```

```
rotation = sin(t * 10.0) * 0.1
```

```
# 颜色脉冲
```

```
modulate = Color.WHITE.lerp(Color.RED, (sin(t * 4.0) + 1.0) * 0.5)
```

写法演化:

direction 变量 + 边界 if	~8行
→ pingpong(t, range)	1行
→ sin(t) + remap	1行 (平滑)

13. 延迟初始化: 懒加载

问题: "敌人只在第一次看到玩家时才加载声音/特效资源。"

最初的做法:

```
func _ready():
    hit_sound = preload("res://hit.wav")
    explosion = preload("res://explosion.tscn")
    # 全部提前加载, 即使可能用不上
```

进化: @onready

```
@onready var hit_sound = preload("res://hit.wav")
# 在 _ready 时自动加载，不是场景加载时
```

再进化: 懒加载 (lazy load)

```
var _hit_sound = null

func play_hit():
    if _hit_sound == null:
        _hit_sound = preload("res://hit.wav")
    _hit_sound.play()
```

再进化: 用 get_node 懒加载子节点

```
# 不如 @onready 方便，但场景深时有用
@onready var health_bar = $"%HealthBar"
# 如果场景结构变化了，@onready 报错
# 可以自定义懒加载:
var _health_bar = null
func health_bar():
    if _health_bar == null:
        _health_bar = find_child("HealthBar")
    return _health_bar
```

写法演化:

_ready 里全部预加载	~N行
→ @onready var	1行
→ if null 再加载	3行 (懒加载, 节省内存)

14. 数组最后一个元素

问题: "获取数组最后一个。"

最初的做法:

```
var last = array[array.size() - 1]
```

进化:

```
var last = array.back()

# 删掉最后一个
var last = array.pop_back()

# 前端
var first = array.front()
var first = array.pop_front()
```

15. 数值归零: 接近零就归零

问题: 速度越来越小, 但不会真正归零。UI 上看起来像微动。

最初的做法:

```
velocity *= 0.9 # 每帧衰减
if abs(velocity) < 0.1:
    velocity = 0.0
```

进化: `move_toward` 或 `snap`:

```
# 向 0 靠拢, 步长 friction*delta
velocity.x = move_toward(velocity.x, 0.0, friction * delta)

# 或者用 snapped 归零
if is_zero_approx(velocity.x): # 判断是否≈0
    velocity.x = 0.0
```

16. 安全导航: 避免 null 崩溃

问题: "敌人在 `_ready` 里获取玩家引用, 但玩家可能还没加载。"

最初的做法:

```
func _ready():
    player = get_tree().get_first_node_in_group("player")
    # 如果玩家不存在 → null → 报错
```

进化: 安全访问:

```
func _ready():
    player = get_tree().get_first_node_in_group("player")

func _process(delta):
    if not player: # 等 player 出现
        return
    if not is_instance_valid(player): # 检查 player 是否被删了
        player = null
        return

# 或者用 ? 操作符 (简化 null 判断)
# Godot 4 支持:
player?.take_damage(10)
# 如果 player == null, 不执行, 不报错
```

17. 最简碰撞: 距离判断

问题: "两个圆形的物体碰撞了吗?"

最初的做法:

```
var dx = a.x - b.x
var dy = a.y - b.y
var dist = sqrt(dx * dx + dy * dy)
if dist < a.radius + b.radius:
    # 碰撞了
```

进化: distance_squared_to:

```
# 避免开根号, 性能更好
var dist_sq = a.position.distance_squared_to(b.position)
var radius_sum = a.radius + b.radius
```

```
if dist_sq < radius_sum * radius_sum:
    # 碰撞了
```

最简 AABB 碰撞:

```
# 矩形碰撞
if abs(a.x - b.x) < (a.w + b.w) / 2 and \
    abs(a.y - b.y) < (a.h + b.h) / 2:
    # 碰撞了

# 或用 Rect2
var rect_a = Rect2(a.x, a.y, a.w, a.h)
var rect_b = Rect2(b.x, b.y, b.w, b.h)
if rect_a.intersects(rect_b):
    # 碰撞了
```

选型指南

你要做啥?	最简写法
限制值在范围内	<code>clamp(value, min, max)</code>
反转 bool	<code>a = not a</code>
判断正负	<code>sign(value)</code>
小数取整	<code>roundi / floori / ceil</code>
对齐到网格	<code>snapped(value, grid_size)</code>
计时/冷却	Timer 节点 / <code>create_timer</code>
循环列表索引	<code>wrapi(index, size)</code>
限制变化量	<code>move_toward(from, to, step)</code>
矩形内检测	<code>Rect2.has_point()</code>
删除前等一会儿	<code>create_timer(delay).timeout</code>
随机选元素	<code>.pick_random()</code>
不重复随机	<code>.shuffle() + .pop_front()</code>
来回弹	<code>pingpong(t, range)</code>
数组首尾	<code>.front() / .back()</code>
安全空值	<code>is_instance_valid() / ?.</code>
圆形碰撞	<code>distance_squared_to < r²</code>
矩形碰撞	<code>Rect2.intersects()</code>

这个故事告诉我们: - 最常用的算法往往最简单—clamp, sign, not, wrap - 好的写法不是"更聪明", 是"更直接地表达意图" - `clamp(x, 0, max)` 比 `if x<0: x=0; if x>max: x=max` 更容易读 - 不是代码少了, 而是脑子不用想"这段代码在干嘛"

第6章 物理引擎演化

Godot 物理引擎：从手写碰撞到物理服务器

物理引擎 = 重力、碰撞、反弹、摩擦力、关节。最早的游戏自己算碰撞: if $x_1 < x_2 + w_2$ and $x_1 + w_1 > x_2$... Godot 内置了完整的 2D/3D 物理引擎。物理的进化 = "怎么让物体像真实世界一样运动, 同时性能可控"。

1. 原始：手写碰撞检测

```
# 矩形碰撞，每帧自己算
func check_collision(a: Rect2, b: Rect2) -> bool:
    return a.position.x < b.position.x + b.size.x \
        and a.position.x + a.size.x > b.position.x \
        and a.position.y < b.position.y + b.size.y \
        and a.position.y + a.size.y > b.position.y

# 圆形碰撞
func circle_collision(a: Vector2, ra: float, b: Vector2, rb:
float) -> bool:
    return a.distance_to(b) < ra + rb
```

问题: 每帧手动遍历所有物体, 没有树结构, $O(n^2)$ 。没人手写三角碰撞。

2. Area2D/Area3D: 触发检测

```
# Godot 帮你检测，你只管处理事件
func _on_area_entered(area: Area2D):
    if area.is_in_group("pickup"):
        collect(area)
```



```
func _on_body_entered(body: Node2D):  
    if body.is_in_group("enemy"):  
        take_damage(10)
```

背后: Godot 用空间哈希 / BVH 树加速, 你不用关心。

3. StaticBody2D: 静态碰撞体

```
# 地板、墙壁、不会动的平台  
# 加一个 CollisionShape2D + RectangleShape2D  
# 角色撞到它会自动停下, 不需要写代码
```

static = 不响应物理力, 但其他物体可以和它碰撞。

4. RigidBody2D: 刚体物理

```
# 让一个箱子被推着走:  
# 加 RigidBody2D + CollisionShape2D  
# 重力自动: gravity_scale = 1.0  
# 玩家推它: apply_central_impulse(direction * force)
```

rigid = 受重力、碰撞力、摩擦力影响。完全由物理引擎驱动。

5. CharacterBody2D: 受控移动体

```
# 玩家/敌人: 自己控制移动, 但和世界碰撞  
extends CharacterBody2D  
  
var speed = 200  
  
func _physics_process(delta):  
    var dir = Input.get_axis("left", "right")  
    velocity.x = dir * speed  
    move_and_slide() # 自动处理碰撞响应
```

```
func _on_hit():  
    velocity = Vector2(-500, -300) # 被击退
```

character = 自己设 velocity, Godot 做碰撞 + 滑动。不直接受重力影响(需要自己加)。

6. 物理材质 (PhysicsMaterial)

```
# .tres 文件配置:  
# bounce = 0.5 # 弹性  
# friction = 0.8 # 摩擦力  
# absorbent = true # 吸收碰撞能量  
  
# 给 RigidBody2D 的碰撞体挂上物理材质  
# 不同表面有不同的弹性和摩擦
```

7. 碰撞层级 (Collision Layers & Masks)

```
# 2D: 32 个层级  
# 3D: 32 个层级  
  
# Layer: 你在第几层  
# Mask: 你和哪些层碰撞  
  
# 玩家: layer=1, mask=2 (只和层 2 碰撞)  
# 敌人: layer=2, mask=1 (只和层 1 碰撞)  
# 子弹: layer=4, mask=2 (只和敌人碰撞)  
# 金币: layer=8, mask=0 (不和任何物体碰撞)  
  
# 代码设置:  
player.collision_layer = 1  
player.collision_mask = 2
```

核心: 避免无效碰撞检测, 提升性能。

8. 关节 (Joint)

```
# PinJoint2D: 两个物体钉在一起(链条)
# GrooveJoint2D: 一个物体沿轨道滑动
# DampedSpringJoint2D: 弹簧连接

# 3D 还有:
# Generic6DOFJoint3D: 6 自由度约束
# ConeTwistJoint3D: 圆锥旋转约束(如手臂)
```

关节: 让物体之间有约束关系, 不需要手写。

9. 物理服务器 (PhysicsServer2D/3D)

```
# 绕过 Node 体系, 直接操作物理引擎
# 性能最优, 适合大量物体

var space = PhysicsServer2D.space_create()
var body = PhysicsServer2D.body_create()
PhysicsServer2D.body_set_space(body, space)
PhysicsServer2D.body_set_shape(body, 0, rectangle_shape)

# 检测:
var result =
PhysicsServer2D.space_get_direct_state(space).intersect_point(m
ouse_pos)
```

适用场景: 成千上万的子弹/Boids 群体, Node 开销太大时。

10. 最终方案模板

```
# Godot 物理选择指南:
```

```
#
# | 你需要 | 用哪个 |
# |-----|-----|
```

#	触发器/检测	Area2D / Area3D
#	墙壁/地板	StaticBody2D / StaticBody3D
#	玩家/受控角色	CharacterBody2D / 3D
#	物理模拟(箱子)	RigidBody2D / RigidBody3D
#	移动平台	AnimatableBody2D / 3D
#	大数量物理	PhysicsServer2D / 3D
#	约束/关节	PinJoint / SpringArm ...
#		

碰撞矩阵配置:

在 Project Settings > Layer Names > 2D Physics 里命名

然后在每个节点的 Collision 选项卡勾选层

性能关键:

- 用 collision_mask 减少碰撞对

- RigidBody 用 disabled 模式 + sleeping

- 大量子弹用 PhysicsServer2D 而不是场景

第7章 路径跟随系统演化

Godot 路径跟随系统: 从路点到 PathFollow

路径跟随 = 物体沿预设路径移动。最早是手写路点数组, 每个点算位置。
Godot 提供了 Path2D + PathFollow2D, 一行代码搞定。路径跟随的进化
= "怎么让物体沿曲线走得流畅自然"。

1. 原始: 路点数组

```
var waypoints = [  
    Vector2(100, 100),  
    Vector2(300, 100),  
    Vector2(300, 300),  
    Vector2(100, 300),  
]  
var current = 0  
var speed = 100  
  
func _process(delta):  
    var target = waypoints[current]  
    var dir = (target - position).normalized()  
    position += dir * speed * delta  
    if position.distance_to(target) < 5:  
        current = (current + 1) % waypoints.size()
```

2. Path2D + PathFollow2D

```
# 场景结构:  
# Path2D (画一条曲线)  
#   └─ PathFollow2D (自动沿路径移动)
```

```
# └─ Sprite2D (移动的物体)

# Path2D 的 Curve 定义了路径形状
# PathFollow2D 的 progress 属性控制走到哪了

func _process(delta):
    $Path2D/PathFollow2D.progress += speed * delta
    # progress = 沿路径走的距离
    # progress_ratio = 0~1, 走完整个路径的比例
```

一行搞定: 不用手算方向、距离、路点索引。

3. 循环 / 往返

```
# PathFollow2D 属性:
# - loop = true: 走到终点回到起点 (默认)
# - loop = false: 走到终点停

# 往返模式 (pingpong) 需要手动:
var forward = true

func _process(delta):
    var follow = $Path2D/PathFollow2D
    if forward:
        follow.progress += speed * delta
        if follow.progress_ratio >= 1.0:
            forward = false
    else:
        follow.progress -= speed * delta
        if follow.progress_ratio <= 0.0:
            forward = true
```

4. 方向自动旋转

```
# PathFollow2D 的 rotation 模式:
# - 设置 rotation = true
# - 自动根据路径切线方向旋转
# - 非常适合: 轨道上的车、传送带上的物品、巡逻敌人
```

```
$Path2D/PathFollow2D.rotation_follow = true
# 物体会自动朝向路径前进方向
```

5. 多物体沿同一路径

```
# 多个 PathFollow2D 共享一个 Path2D
# 设置不同的 progress 偏移实现编队

func setup_patrol(count: int):
    for i in count:
        var follow = PathFollow2D.new()
        follow.rotation_follow = true
        follow.progress_ratio = float(i) / count
        $Path2D.add_child(follow)

        var enemy = preload("res://enemy.tscn").instantiate()
        follow.add_child(enemy)

# 每个敌人均匀分布在路径上
# 一起移动, 保持间距
```

6. 最终方案模板

```
# 路径跟随用法:

# 1. 巡逻敌人:
#   Path2D → PathFollow2D(loop=true, rotation=true) → Enemy
#   _process 中 progress += speed * delta

# 2. 轨道车/过山车:
#   同上, 加 Tween 控制速度曲线

# 3. 动画路径 (过场动画):
#   Tween.tween_property(follow, "progress_ratio", 1.0, 5.0)
#   tween.tween_method(progress_func, 0, 1, duration)

# 4. 传送带物品:
```



```
# 物品生成时放到 PathFollow2D 上，沿路径移动到终点消失

func move_along_path(node: Node2D, path: Path2D, duration:
float):
    var follow = PathFollow2D.new()
    path.add_child(follow)
    node.reparent(follow)
    var tween = create_tween()
    tween.tween_property(follow, "progress_ratio", 1.0,
duration)
    await tween.finished
    node.reparent(get_tree().current_scene)
    follow.queue_free()
```

第二卷·战斗系统

共计 32 章

第8章 战斗系统演化

Godot 战斗系统: 从你一刀我一刀到实时战斗

战斗 = 玩家和怪物互相造成伤害的过程。最早是回合制 (你打我一下, 我打你一下)。后来有了实时战斗、连击、闪避、格挡、硬直。战斗系统的进化 = "怎么让打架这件事越来越有操作感和反馈感"。

1. 原始: 碰触伤害

```
# Area2D 重叠即伤害
func _on_body_entered(body):
    if body.is_in_group("player"):
        body.take_damage(10)
```

问题: 每帧都在触发, 玩家碰到怪物瞬间掉好几次血。

2. 冷却伤害 (防重复触发)

```
var can_damage = true

func _on_body_entered(body):
    if body.is_in_group("player") and can_damage:
        body.take_damage(10)
        can_damage = false
        await get_tree().create_timer(0.5).timeout
        can_damage = true
```

3. 攻击动作 + 伤害判定帧

```
# 攻击时启用攻击碰撞体，只在特定帧开启
func attack():
    $AttackHitbox.monitoring = true
    $AnimationPlayer.play("attack")

# 在动画的某个关键帧调用 enable_hitbox()
# 在动画结束调用 disable_hitbox()

func enable_hitbox():
    $AttackHitbox.monitoring = true

func disable_hitbox():
    $AttackHitbox.monitoring = false
```

4. 受伤无敌帧

```
var invincible = false

func take_damage(dmg: int):
    if invincible:
        return
    hp -= dmg
    invincible = true
    modulate = Color(1, 0.5, 0.5, 1) # 闪红
    await get_tree().create_timer(0.5).timeout
    modulate = Color.WHITE
    invincible = false
```

5. 最终方案模板

```
# Combat 核心三要素：
# 1. 攻击碰撞体：只在攻击动画特定帧开启
# 2. 受伤无敌帧：受伤后 0.5 秒内不再受伤
# 3. 伤害公式：attack - defense，最小 1
```

```
func deal_damage(attacker, target, damage: int):
    if target.is_invincible:
        return
    var actual = max(1, damage - target.defense)
    target.hp -= actual
    target.on_hit(actual)
    # 显示伤害数字
    DamageNumber.show(actual, target.global_position)
```

第9章 防御系统演化

Godot 防御系统：从简单扣血到格挡闪避

防御 = 减少或避免受到的伤害。最早是防御力 = 攻击力 - 防御力。后来有了格挡 (减伤)、闪避 (免伤)、招架 (反击)、护盾 (吸收)。防御系统的进化 = "怎么让挨打也有操作感"。

1. 原始：减法公式

```
func take_damage(amount: int):  
    hp -= amount
```

2. 减伤公式

```
func take_damage(attack: int):  
    var actual = max(1, attack - defense)  
    hp -= actual
```

3. 闪避 + 格挡

```
func take_damage(attack: int) -> Dictionary:  
    # 闪避判定  
    if randf() < dodge_rate:  
        return { "type": "dodge", "damage": 0 }  
  
    # 格挡判定  
    var blocked = false  
    if randf() < block_rate:
```



```

        blocked = true

    var actual = max(1, attack - defense)
    if blocked:
        actual = int(actual * 0.3) # 格挡只受 30% 伤害

    hp -= actual
    return { "type": "block" if blocked else "hit", "damage":
actual }

```

4. 护盾系统

```

var shield = 0      # 护盾值
var max_shield = 0

func take_damage(amount: int) -> Dictionary:
    # 护盾吸收伤害
    var shield_dmg = min(shield, amount)
    shield -= shield_dmg
    amount -= shield_dmg

    var actual = max(1, amount - defense)
    hp -= actual
    return { "damage": actual, "shield_absorbed": shield_dmg }

```

5. 无敌帧 + 受击反馈

```

var invincible_timer = 0.0

func _process(delta):
    if invincible_timer > 0:
        invincible_timer -= delta

func take_damage(amount: int) -> Dictionary:
    if invincible_timer > 0:
        return { "type": "invincible", "damage": 0 }

    var result = _calc_damage(amount)

```

```

# 受击反馈
$AnimationPlayer.play("hit")
$Sprite.modulate = Color.RED
create_tween().tween_property($Sprite, "modulate",
Color.WHITE, 0.2)

# 无敌帧（防连续伤害）
invincible_timer = 0.3
return result

```

6. 最终方案模板

```

# DefenseManager.gd（非 Autoload，挂载到角色）
# 防御流程：
# 1. 闪避判定 → 完全免疫
# 2. 护盾吸收 → 先扣盾
# 3. 格挡判定 → 减伤 70%
# 4. 减伤公式 →  $attack - defense$ 
# 5. 无敌帧 → 防止连续受伤

func process_attack(incoming: DamageData) -> DamageResult:
    var result = DamageResult.new()
    if invincible > 0:
        result.type = "miss"
        return result
    if randf() < stats.dodge:
        result.type = "dodge"
        return result
    var dmg = incoming.amount
    dmg = _shield_absorb(dmg, result)
    dmg = _block_check(dmg, result)
    dmg = max(1, dmg - stats.defense)
    stats.hp -= dmg
    result.damage = dmg
    invincible = 0.3
    return result

```

第10章 连击系统演化

Godot 连击系统: 从单次攻击到 Combo Chain

连击 = 连续攻击命中, 次数越多伤害越高。最早是 flat damage, 打一下算一下。后来有了 combo 计数器、连击加成、终结技。连击系统的进化 = "怎么让连续命中比瞎按更有回报"。

1. 原始: 单次伤害

```
# 每下攻击独立计算伤害
# 没有"连续命中"的概念
```

2. Combo 计数器

```
var combo_count = 0
var combo_timer = 0.0
const COMBO_WINDOW = 1.5 # 秒

func on_hit_enemy():
    combo_count += 1
    combo_timer = COMBO_WINDOW
    $ComboLabel.text = "%d COMBO!" % combo_count

func _process(delta):
    if combo_count > 0:
        combo_timer -= delta
        if combo_timer <= 0:
```

```
combo_count = 0
$ComboLabel.hide()
```

3. 连击加成

```
func get_combo_multiplier() -> float:
    if combo_count < 5:    return 1.0
    if combo_count < 10:   return 1.2
    if combo_count < 20:   return 1.5
    if combo_count < 50:   return 2.0
    return 3.0

func deal_combo_damage(base_damage: int, target) -> int:
    var multiplier = get_combo_multiplier()
    var final_damage = int(base_damage * multiplier)
    target.take_damage(final_damage)
    return final_damage
```

4. 连击终结技

```
func on_combo_milestone(combo: int):
    match combo:
        10:
            _unlock_skill("快速连斩")    # 10 连击解锁新技能
        25:
            _trigger_effect("旋风斩")    # 25 连击自动触发范围攻击
        50:
            _activate_buff("狂怒", 5.0) # 50 连击进入狂暴状态

func _on_combo_expire():
    if combo_count >= 30:
        # 高连击断掉时有惩罚或奖励
        _deal_finisher_damage(combo_count * 5) # 终结一击
        combo_count = 0
```

5. 最终方案模板

```
# ComboManager.gd (挂载到角色或 Autoload)
extends Node

signal combo_changed(count: int)
signal combo_milestone(count: int)

var count := 0
var timer := 0.0
var window := 1.5

func add():
    count += 1
    timer = window
    combo_changed.emit(count)
    if count % 10 == 0:
        combo_milestone.emit(count)

func get_damage_multiplier() -> float:
    return 1.0 + count * 0.02 # 每连击 +2%

func _process(delta):
    if count > 0:
        timer -= delta
        if timer <= 0:
            count = 0
            combo_changed.emit(0)
```

第11章 技能系统演化

Godot 技能系统: 从 if-else 到数据驱动

火球术、回血、旋风斩、闪现。每个技能都做同一件事: 玩家按下一个键
→ 发生了点什么。

但"发生了点什么"的写法, 从简单的 if 到数据驱动, 走了很长一段路。

目录

1. [原始: if-else 技能分发](#)
 2. [技能数据化: 数组和字典](#)
 3. [Resource 技能: 编辑器里配数据](#)
 4. [技能状态: 施法前摇后摇](#)
 5. [技能管理器: 冷却与消耗统一管](#)
 6. [Buff 系统: 技能效果持续化](#)
 7. [技能树: 解锁与升级](#)
 8. [最终方案: 最简可用模板](#)
-

1. 原始: if-else 技能分发

问题: "玩家按 1 放火球, 按 2 回血, 按 3 放旋风斩。"

最初的做法:


```

func _input(event):
    if event.is_action_pressed("skill_1"):
        cast_fireball()
    elif event.is_action_pressed("skill_2"):
        cast_heal()
    elif event.is_action_pressed("skill_3"):
        cast_whirlwind()

func cast_fireball():
    if mana < 30:
        return # 蓝不够
    mana -= 30
    var fireball = preload("res://Fireball.tscn").instantiate()
    fireball.global_position = global_position
    fireball.direction = aim_direction
    add_child(fireball)
    cooldown_fireball = 3.0

func cast_heal():
    if mana < 20:
        return
    mana -= 20
    hp = min(hp + 50, max_hp)
    cooldown_heal = 5.0

func cast_whirlwind():
    if mana < 40:
        return
    mana -= 40
    # 播放旋转动画，对周围敌人造成伤害
    # ...
    cooldown_whirlwind = 8.0

```

问题:

1. **重复代码** – 每个技能都写一遍"蓝不够就返回, 扣蓝, 设冷却"。
2. **数据硬编码** – 火球耗蓝 30、伤害 50、冷却 3 秒。全写在代码里。改一个数值要打开脚本。
3. **加一个新技能** = 新建一个函数 + 加一行 elif + 配一套数值。

4. **没有统一的结构** – 火球生成场景, 回血直接改 hp, 旋风斩播放动画。每个技能的"效果"写法完全不一样, 没法统一管理。
-

2. 技能数据化: 数组和字典

问题: 技能逻辑和技能数据混在一起。改数值要改代码。

想法: 把技能的数值抽出来, 存到数据结构里。

```
# 技能数据字典
var skill_data = {
  "fireball": {
    "name": "火球术",
    "mana_cost": 30,
    "cooldown": 3.0,
    "damage": 50,
    "scene": preload("res://Fireball.tscn"),
    "type": "projectile"
  },
  "heal": {
    "name": "治疗术",
    "mana_cost": 20,
    "cooldown": 5.0,
    "heal_amount": 50,
    "type": "self_heal"
  },
  "whirlwind": {
    "name": "旋风斩",
    "mana_cost": 40,
    "cooldown": 8.0,
    "damage": 30,
    "radius": 100.0,
    "type": "aoe"
  }
}

# 统一的分发
func use_skill(skill_name: String):
  var data = skill_data.get(skill_name)
```

```

if not data:
    return

# 统一检查：冷却
if cooldowns.has(skill_name) and cooldowns[skill_name] > 0:
    return # 冷却中

# 统一检查：蓝耗
if mana < data.mana_cost:
    return

# 扣蓝
mana -= data.mana_cost

# 设冷却
cooldowns[skill_name] = data.cooldown

# 根据类型执行不同的效果
match data.type:
    "projectile":
        var projectile = data.scene.instantiate()
        projectile.damage = data.damage
        projectile.global_position = global_position
        projectile.direction = aim_direction
        add_child(projectile)

    "self_heal":
        hp = min(hp + data.heal_amount, max_hp)

    "aoe":
        # 对周围敌人造成伤害
        for enemy in get_tree().get_nodes_in_group("enemy"):
            if
global_position.distance_to(enemy.global_position) <
data.radius:
                enemy.take_damage(data.damage)

```

进步： - 技能数据集中在一个字典里，改数值不用改逻辑 - 蓝耗检查、冷却检查
 统一到一个地方 - 加新技能 = 在字典加一条 + 加一个 match 分支 (或不用加，
 如果效果类型已存在)

问题: 1. 字典写在脚本里 – 策划不能改, 每次改数值要程序员打开脚本。2. **match** 分支还是臃肿 – 新效果类型还要加 **match** 分支。3. 场景里配数值 – **Fireball.tscn** 自身的伤害和 **skill_data** 里的伤害是两个来源, 容易不一致。

3. Resource 技能: 编辑器里配数据

问题: 数据在代码里, 策划改不了。而且一个技能拆成 "字典里的数值" + "场景文件" 太分散。

Resource: Godot 的核心概念 – 可以看作"存在文件里的数据对象"。在编辑器里创建、编辑, 代码里引用。

```
# SkillResource.gd
class_name SkillResource
extends Resource

@export var skill_name: String = "火球术"
@export var icon: Texture2D
@export var description: String = "发射一颗火球"

@export var mana_cost: int = 30
@export var cooldown: float = 3.0
@export var cast_time: float = 0.5

@export var effect_type: String = "projectile"
@export var damage: int = 50
@export var scene: PackedScene
@export var radius: float = 0.0
@export var heal_amount: int = 0
@export var duration: float = 0.0
```

在编辑器里: 右键 → 新建资源 → SkillResource → 填数值 → 保存为 **Fireball.tres**。

```
res://skills/
├─ Fireball.tres          (Resource 文件, 编辑器里填数值)
```

```
└─ Heal.tres
└─ Whirlwind.tres
└─ Teleport.tres
```

```
# 使用:
@export var fireball_skill: SkillResource # 在编辑器里拖入
Fireball.tres

func use_skill(skill: SkillResource):
    if mana < skill.mana_cost:
        return
    if cooldown_manager.is_on_cooldown(skill):
        return

    mana -= skill.mana_cost
    cooldown_manager.start_cooldown(skill)

    match skill.effect_type:
        "projectile":
            var proj = skill.scene.instantiate()
            proj.damage = skill.damage
            proj.global_position = global_position
            add_child(proj)

        "self_heal":
            hp = min(hp + skill.heal_amount, max_hp)

        "aoe":
            for enemy in get_tree().get_nodes_in_group("enemy"):
                if
                    global_position.distance_to(enemy.global_position) <
                    skill.radius:
                        enemy.take_damage(skill.damage)
```

Resource 的进步:

1. 策划可以直接改 – 打开 Fireball.tres, 改 damage 从 50 到 60, 保存, 生效。不用碰代码。
2. 复用 – 火球术和新技能"冰箭术"共用一个 PackedScene? 拖入不同的 scene 就行。

3. 版本控制友好 – .tres 是文本文件, Git 能 diff。
4. 编辑器可视化 – 所有数值在 Inspector 里显示, 不用背变量名。

但问题还在:

1. 效果逻辑还在 **match** 里 – 加"召唤"效果又要改 use_skill 函数。
2. 一个 **Resource** 既要存 **projectile** 参数 (**damage**), 又要存 **heal** 参数 (**heal_amount**), 又要存 **aoe** 参数 (**radius**) – 不用的字段空着, 难看且容易配错。
3. 技能没有"状态" – 火球术需要 0.5 秒吟唱, 吟唱时不能移动。这个"状态切换" **Resource** 管不了。

4. 技能状态: 施法前摇后摇

问题: 技能不只是"扣蓝 + 出效果"。还有: - 前摇 (吟唱 0.5 秒, 不能动) - 生效 (火球飞出去) - 后摇 (收招 0.2 秒, 不能动)

技能需要自己的状态机:

```
# 技能状态
enum SkillState { READY, CASTING, ACTIVE, COOLDOWN }

# PlayerSkillController.gd
var current_skill: SkillResource
var skill_state = SkillState.READY
var cast_timer = 0.0

func try_cast_skill(skill: SkillResource):
    if skill_state != SkillState.READY:
        return
    if mana < skill.mana_cost:
        return
    if cooldown_manager.is_on_cooldown(skill):
        return

    # 开始施法
```

```

current_skill = skill
skill_state = SkillState.CASTING
cast_timer = skill.cast_time
mana -= skill.mana_cost

# 播放施法动画
animation_player.play("cast_" + skill.skill_name)

func _process(delta):
    match skill_state:
        SkillState.CASTING:
            cast_timer -= delta
            if cast_timer <= 0:
                # 施法完成, 释放技能
                _activate_skill(current_skill)
                skill_state = SkillState.ACTIVE

        SkillState.ACTIVE:
            # 等待技能效果结束 (火球飞行中/动画播放中)
            # 由技能自身通知完成
            pass

        SkillState.COOLDOWN:
            # 冷却管理器自动处理
            pass

func _activate_skill(skill: SkillResource):
    match skill.effect_type:
        "projectile":
            var proj = skill.scene.instantiate()
            proj.damage = skill.damage
            proj.source = self
            add_child(proj)
            # 火球飞行中, 等它撞到东西或飞出边界
            # 然后在 proj 的 _on_destroy 里调用 _finish_skill()

        "self_heal":
            hp = min(hp + skill.heal_amount, max_hp)
            _finish_skill() # 瞬间完成

func _finish_skill():
    skill_state = SkillState.COOLDOWN
    cooldown_manager.start_cooldown(current_skill)

```

```
current_skill = null
animation_player.play("idle")
```

技能状态图:

```
READY —(按技能键 + 有蓝 + 冷却好)→ CASTING
CASTING —(cast_timer 到 0)→ ACTIVE
ACTIVE —(效果完成)→ COOLDOWN
COOLDOWN —(冷却时间到)→ READY
```

5. 技能管理器: 冷却与消耗统一管

问题: 每个技能都要管冷却, 代码散落在各个地方。

CooldownManager: 统一管理所有技能的冷却。

```
# CooldownManager.gd (可以挂载到玩家节点或 Autoload)

class_name CooldownManager
extends Node

var _cooldowns := {}
var _timers := {}

func start_cooldown(skill: SkillResource, multiplier: float = 1.0):
    var key = skill.resource_path
    _cooldowns[key] = skill.cooldown * multiplier
    # 可选: 用 Timer 节点
    var timer = Timer.new()
    timer.wait_time = skill.cooldown * multiplier
    timer.one_shot = true
    timer.timeout.connect(_on_cooldown_end.bind(key))
    add_child(timer)
    timer.start()
    _timers[key] = timer

func is_on_cooldown(skill: SkillResource) -> bool:
    var key = skill.resource_path
```



```

        return _timers.has(key) and not _timers[key].is_stopped()

func get_remaining(skill: SkillResource) -> float:
    var key = skill.resource_path
    if _timers.has(key):
        return _timers[key].time_left
    return 0.0

func get_progress(skill: SkillResource) -> float:
    var key = skill.resource_path
    if _timers.has(key):
        return 1.0 - (_timers[key].time_left / skill.cooldown)
    return 1.0

func _on_cooldown_end(key: String):
    _timers.erase(key)

# 使用:
# cooldown_manager.start_cooldown(fireball_skill)
# if cooldown_manager.is_on_cooldown(fireball_skill):

```

CooldownManager 进步: 冷却逻辑统一在一个地方。UI 可以随时查询冷却进度显示倒计时。

ResourceManager: 统一管理消耗:

```

func can_afford(skill: SkillResource) -> bool:
    return mana >= skill.mana_cost and \
           stamina >= skill.stamina_cost and \
           not cooldown_manager.is_on_cooldown(skill)

func spend_cost(skill: SkillResource):
    mana -= skill.mana_cost
    stamina -= skill.stamina_cost

```

6. Buff 系统: 技能效果持续化

问题: "火球术命中敌人后, 附加灼烧效果, 每秒掉 5 血, 持续 3 秒。"

这个"持续效果"不能放在技能释放的瞬间处理。它需要一个独立的 Buff 系统。

```
# BuffResource.gd
class_name BuffResource
extends Resource

@export var buff_name: String = "灼烧"
@export var duration: float = 3.0
@export var tick_interval: float = 1.0
@export var tick_damage: int = 5
@export var stackable: bool = true
@export var max_stacks: int = 3
@export var icon: Texture2D
```

```
# BuffManager.gd (挂载到每个角色)
class_name BuffManager
extends Node

var active_buffs := []

func apply_buff(buff: BuffResource, source = null):
    if buff.stackable:
        # 找到已有的同类型 buff, 刷新持续时间或叠加层数
        for existing in active_buffs:
            if existing.resource == buff:
                existing.remaining = buff.duration
                existing.stacks = min(existing.stacks + 1,
buff.max_stacks)
            return

        # 新 buff
        var instance = BuffInstance.new()
        instance.resource = buff
        instance.remaining = buff.duration
        instance.stacks = 1
        instance.source = source
        active_buffs.append(instance)

        # 启动 tick 计时器
        if buff.tick_interval > 0:
            instance.tick_timer = buff.tick_interval

        buff_applied.emit(buff)
```

```

func _process(delta):
    var to_remove := []
    for buff in active_buffs:
        buff.remaining -= delta

        # tick
        if buff.resource.tick_interval > 0:
            buff.tick_timer -= delta
            if buff.tick_timer <= 0:
                buff.tick_timer = buff.resource.tick_interval
                _apply_tick(buff)

        # 到期移除
        if buff.remaining <= 0:
            to_remove.append(buff)

    for buff in to_remove:
        active_buffs.erase(buff)
        buff_expired.emit(buff.resource)

func _apply_tick(buff):
    if buff.resource.tick_damage > 0:
        owner.take_damage(buff.resource.tick_damage *
buff.stacks)

class BuffInstance:
    var resource: BuffResource
    var remaining: float
    var stacks: int
    var source
    var tick_timer: float

```

完整链条:

火球术命中敌人

- 对敌人造成 50 伤害
- 附加 Buff: "灼烧"
 - BuffManager 接管
 - 每秒 tick: 敌人掉 5 血
 - 3 秒后到期, 移除

Buff 系统进化:

没有 buff → 技能效果都是瞬发的
简单 timer → 伤害靠子弹撞到的那一下
buff 数组 → 持续效果独立管理, 支持叠加
Resource buff → 策划配灼烧/冰冻/中毒, 不需要代码

7. 技能树: 解锁与升级

问题: "角色升到 5 级解锁火球术, 10 级可以升级成大火球。"

```
# SkillTreeResource.gd
class_name SkillTreeResource
extends Resource

@export var skill_name: String
@export var required_level: int = 1
@export var required_skill: SkillResource # 前置技能
@export var skill: SkillResource         # 解锁的技能
@export var upgrades: Array[SkillUpgrade] # 升级选项

class SkillUpgrade:
    var name: String
    var description: String
    var required_level: int
    var modifiers: Dictionary # {"damage": +20, "mana_cost":
-5}
```

```
# SkillTreeManager.gd
extends Node

var unlocked := []
var learned := {}

func can_unlock(node: SkillTreeResource) -> bool:
    if node.skill_name in unlocked:
        return false
    if player.level < node.required_level:
        return false
```

```

        if node.required_skill and node.required_skill.resource_path
not in learned:
            return false
        return true

func unlock(node: SkillTreeResource):
    unlocked.append(node.skill_name)
    learned[node.skill.resource_path] = 1 # 等级 1

func upgrade(skill: SkillResource, upgrade_node: SkillUpgrade):
    var current_level = learned.get(skill.resource_path, 0)
    learned[skill.resource_path] = current_level + 1

# 应用修改器
if upgrade_node.modifiers.has("damage"):
    skill.damage += upgrade_node.modifiers["damage"]
if upgrade_node.modifiers.has("mana_cost"):
    skill.mana_cost += upgrade_node.modifiers["mana_cost"]
if upgrade_node.modifiers.has("cooldown"):
    skill.cooldown += upgrade_node.modifiers["cooldown"]

```

8. 最终方案: 最简可用模板

```

# 从最简单的技能系统开始，够用就好：

# —— SkillResource.gd ——
class_name SkillResource
extends Resource

@export var skill_name: String = ""
@export var icon: Texture2D
@export var description: String = ""

@export var mana_cost: int = 0
@export var cooldown: float = 0.0
@export var cast_time: float = 0.0

@export var damage: int = 0
@export var heal: int = 0
@export var scene: PackedScene

```

```

@export var buff: BuffResource

# —— SkillController.gd (挂在玩家身上) ——
class_name SkillController
extends Node

@export var skills: Array[SkillResource] = []
var cooldowns := {}

func _ready():
    for s in skills:
        cooldowns[s.resource_path] = 0.0

func _process(delta):
    # 每帧减少所有冷却
    for key in cooldowns.keys():
        if cooldowns[key] > 0:
            cooldowns[key] -= delta

func use(index: int) -> bool:
    if index < 0 or index >= skills.size():
        return false
    return _cast(skills[index])

func _cast(skill: SkillResource) -> bool:
    var key = skill.resource_path

    # 检查冷却
    if cooldowns[key] > 0:
        return false

    # 检查蓝耗 (假设 owner 有 mana)
    if owner.has_method("spend_mana"):
        if not owner.spend_mana(skill.mana_cost):
            return false

    # 设冷却
    cooldowns[key] = skill.cooldown

    # 释放效果
    if skill.scene:
        var instance = skill.scene.instantiate()
        owner.add_child(instance)

```

```

        instance.global_position = owner.global_position
        if instance.has_method("set_damage"):
            instance.set_damage(skill.damage)

    if skill.heal > 0 and owner.has_method("heal"):
        owner.heal(skill.heal)

    if skill.buff:
        var buff_manager = owner.get_node_or_null("BuffManager")
        if buff_manager:
            buff_manager.apply_buff(skill.buff)

    return true

# 使用:
# 把 Fireball.tres 拖入技能数组的 slot 0
# $SkillController.use(0) # 放火球
# $SkillController.use(1) # 回血

```

这个故事告诉我们：- 技能系统从 if-else 到 Resource, 核心是**把数据从代码中剥离** - 每个进化阶段解决一个问题: 重复代码 → 技能字典, 策划改数值 → Resource, 施法状态 → 状态机, 持续效果 → Buff 系统, 升级解锁 → 技能树 - 最简可用的技能系统 = SkillResource + CooldownManager + 效果分发, 三样就够了

第12章 天赋系统演化

Godot 天赋系统：从固定成长到自定义流派

天赋 = 玩家可以自主选择的被动能力。最早升级就是加攻击力/防御力, 没得选。后来有了天赋树: 你可以选择走暴击流还是攻速流。天赋系统的进化 = "怎么让每个玩家 Build 不一样"。

1. 原始: 固定升级

```
func level_up():
    attack += 5
    defense += 3
    hp += 20
# 玩家没有选择权
```

2. 天赋点 + 天赋列表

```
var talent_points = 0
var talents = {
    "crit_rate": { "name": "暴击率", "max": 5, "cost": 1,
"value_per_level": 0.02 },
    "attack_up": { "name": "攻击力", "max": 5, "cost": 1,
"value_per_level": 5 },
    "hp_regen": { "name": "生命恢复", "max": 3, "cost": 2,
"value_per_level": 0.5 },
}

var learned = {} # talent_id → level

func learn_talent(talent_id: String):
```

```

var t = talents[talent_id]
var current = learned.get(talent_id, 0)
if current >= t.max: return false
if talent_points < t.cost: return false
talent_points -= t.cost
learned[talent_id] = current + 1
_apply(talent_id, learned[talent_id])
return true

```

3. 天赋树 (前置要求)

```

var talent_tree = {
  "crit_rate": {
    "name": "暴击率",
    "max": 5,
    "prerequisites": [],          # 无前置
    "column": 0, "row": 0,
  },
  "crit_damage": {
    "name": "暴击伤害",
    "max": 5,
    "prerequisites": ["crit_rate"], # 需要先学暴击率
    "column": 1, "row": 0,
  },
  "crit_heal": {
    "name": "暴击回血",
    "max": 3,
    "prerequisites": ["crit_damage"], # 需要先学暴击伤害
    "column": 2, "row": 0,
  },
}

func can_learn(talent_id: String) -> bool:
  var t = talent_tree[talent_id]
  for pre in t.prerequisites:
    if learned.get(pre, 0) <= 0:
      return false
  return true

```

4. 多天赋页/流派

```
# 玩家可以保存多套天赋配置
class TalentBuild:
    var name: String
    var points: Dictionary # talent_id → level

# 战斗/副本前切换
func switch_build(build_id: String):
    _clear_all()
    var build = _builds[build_id]
    for talent_id in build.points:
        var level = build.points[talent_id]
        for i in level:
            _apply(talent_id, i + 1)
```

5. 最终方案模板

```
# TalentManager.gd (Autoload)
# 天赋数据结构:
# - 天赋点 (每级获得)
# - 天赋定义: id, name, max_level, cost_per_level, prerequisites
# - 已学天赋: learned[id] = level
# - 天赋效果: 通过 signal/stat_recalc 应用到角色

func get_stat_modifier(stat: String) -> float:
    var mod = 0.0
    for tid in learned:
        var t = _definitions[tid]
        var lv = learned[tid]
        if t.stat == stat:
            mod += t.value_per_level * lv
    return mod
```

第13章 装备系统演化

Godot 装备系统：从硬编码到数据驱动

"捡到一把剑, 攻击力 +10。" 这是最简单的装备。但当你有头盔、胸甲、戒指、项链, 每件装备有 5 条随机属性、3 种品质、套装效果、强化等级、宝石孔——装备系统就从一个变量变成了一整套数据架构。

目录

1. [原始: 硬编码装备](#)
 2. [装备槽位: 穿在哪](#)
 3. [装备 Resource: 数据驱动](#)
 4. [属性汇总: 脱下加上的计算](#)
 5. [随机词缀: 每件装备不一样](#)
 6. [品质/稀有度: 白蓝紫橙](#)
 7. [套装效果: 穿一套有加成](#)
 8. [最终方案: 最简可用模板](#)
-

1. 原始: 硬编码装备

问题: "角色捡到一把剑, 攻击力提升。"

最初的做法:

```

var has_sword = false
var has_helmet = false

func equip_sword():
    has_sword = true
    attack += 10

func unequip_sword():
    has_sword = false
    attack -= 10

```

然后有了更多装备:

```

var weapon = null      # "sword" / "axe" / "bow"
var helmet = null
var armor = null
var ring = null

func equip(item_name: String, slot: String):
    # 先脱掉当前装备
    unequip(slot)

    # 穿上新装备
    match slot:
        "weapon":
            weapon = item_name
            match item_name:
                "sword":
                    attack += 10
                "axe":
                    attack += 15
                    speed -= 0.1
                "bow":
                    attack += 8
                    range += 50

        "helmet":
            helmet = item_name
            match item_name:
                "iron_helmet":
                    defense += 5
                "leather_helmet":
                    defense += 2

```

```

        speed += 0.05

func unequip(slot: String):
    # 先移除当前装备的属性
    match slot:
        "weapon":
            match weapon:
                "sword": attack -= 10
                "axe": attack -= 15; speed += 0.1
                "bow": attack -= 8; range -= 50
            weapon = null
        "helmet":
            match helmet:
                "iron_helmet": defense -= 5
                "leather_helmet": defense -= 2; speed -= 0.05
            helmet = null

```

问题:

1. 每个装备的数值写在 **match** 里 – 加一把新武器要加两个 match (equip + unequip), 改数值要改代码。
 2. 属性加减靠手动维护 – 穿上 +10, 脱下 -10。如果忘了写脱下, 装备脱了属性还在。
 3. 装备没有独立数据 – "铁剑"的攻击力是 10 还是 15? 只能靠名字约定, 没地方存。
 4. 没有属性汇总 – 想知道总攻击力, 要遍历所有 slot 的数值。但当前是散落在各个变量里的。
-

2. 装备槽位: 穿在哪

问题: 装备放在不同槽位, 但每个槽位的添加/移除逻辑都是手写的。

用字典统一槽位:

```
enum Slot { WEAPON, HELMET, ARMOR, RING, AMULET, BOOTS }
```

```

var equipment = {}
var stats = {
    "attack": 10,    # 基础攻击
    "defense": 5,    # 基础防御
    "speed": 100,    # 基础速度
}

func _init():
    for slot in Slot.values():
        equipment[slot] = null

func equip(item, slot: int):
    if equipment[slot] != null:
        unequip(slot)
    equipment[slot] = item
    _apply_stats(item, 1)    # 1 = 加上

func unequip(slot: int):
    var item = equipment[slot]
    if item:
        _apply_stats(item, -1)    # -1 = 减去
        equipment[slot] = null

func _apply_stats(item, sign: int):
    # sign = 1 穿上, sign = -1 脱下
    stats["attack"] += item.attack * sign
    stats["defense"] += item.defense * sign
    stats["speed"] += item.speed * sign

```

进步: - 穿上/脱下统一了逻辑, 不再每个槽位手写加减 - 加新槽位 = 在 enum 加一项

但 item 是什么类型? 目前假设 item 有 attack/defense/speed 属性。但这个 item 对象是什么? 怎么来的?

3. 装备 Resource: 数据驱动

问题: 装备的数据 (攻击力、名称、描述) 写在代码里, 策划改不了。

装备 Resource:

```
# EquipmentResource.gd
class_name EquipmentResource
extends Resource

@export var item_name: String = ""
@export var icon: Texture2D
@export var description: String = ""
@export var slot: int = 0 # Slot enum 的值

@export var attack: int = 0
@export var defense: int = 0
@export var hp: int = 0
@export var speed: int = 0
@export var crit_rate: float = 0.0
@export var mana: int = 0
```

在编辑器里创建装备:

右键 → 新建资源 → EquipmentResource

铁剑.tres:

```
item_name: "铁剑"
slot: WEAPON
attack: 10
```

铁头盔.tres:

```
item_name: "铁头盔"
slot: HELMET
defense: 5
```

速度之靴.tres:

```
item_name: "速度之靴"
slot: BOOTS
speed: 25
```

使用 Resource 装备

```
@export var iron_sword: EquipmentResource
@export var iron_helmet: EquipmentResource
```

```
func _ready():
```

```
equip(iron_sword, Slot.WEAPON)
equip(iron_helmet, Slot.HELMET)
```

进步: - 策划在编辑器里改铁剑的攻击力, 不需要碰代码 - 每个装备是一个独立文件, 方便管理和复用 - 商店出售、怪物掉落、宝箱奖励, 都可以引用同一个 Resource

4. 属性汇总: 脱下加上的计算

问题: "sign = 1 加上, sign = -1 减去" 这种方式不安全。 - 脱下时如果忘传 -1, 属性就永久保留了 - 基础属性 (裸体时的攻击力) 和装备属性混在一个 stats 字典里

更好的方案: 实时汇总所有装备的属性:

```
# EquipmentManager.gd
class_name EquipmentManager
extends Node

enum Slot { WEAPON, HELMET, ARMOR, RING, AMULET, BOOTS }

var _equipment := {}
var _base_stats := {
    "attack": 10,
    "defense": 5,
    "hp": 100,
    "speed": 100,
    "crit_rate": 0.05,
}

func _init():
    for s in Slot.values():
        _equipment[s] = null

func equip(item: EquipmentResource, slot: int) -> bool:
    if _equipment[slot] != null:
        return false # 槽位已占用, 先卸下
    _equipment[slot] = item
```

```

        return true

func unequip(slot: int) -> EquipmentResource:
    var item = _equipment[slot]
    _equipment[slot] = null
    return item

func get_total_stats() -> Dictionary:
    var total = _base_stats.duplicate()

    for slot in Slot.values():
        var item = _equipment[slot]
        if item:
            total["attack"] += item.attack
            total["defense"] += item.defense
            total["hp"] += item.hp
            total["speed"] += item.speed
            total["crit_rate"] += item.crit_rate

    return total

func get_equipped_list() -> Array:
    var list = []
    for slot in Slot.values():
        if _equipment[slot]:
            list.append(_equipment[slot])
    return list

```

get_total_stats 每次调用都遍历所有装备, 实时汇总:

```

# 使用:
func _process(delta):
    var stats = $EquipmentManager.get_total_stats()
    attack_label.text = "攻击: %d" % stats["attack"]
    defense_label.text = "防御: %d" % stats["defense"]

```

不需要手动加減了: - 穿上装备 → 放进 `_equipment` 字典 → `get_total_stats` 自动包含它 - 脱下装备 → 从 `_equipment` 移除 → 自动消失 - 基础属性和装备属性分离, 互不干扰

优化: 缓存 (如果 `get_total_stats` 每帧调用太多次):

```

var _stats_dirty = true
var _cached_stats = {}

func _mark_dirty():
    _stats_dirty = true

func equip(item, slot):
    _equipment[slot] = item
    _mark_dirty()

func get_total_stats() -> Dictionary:
    if _stats_dirty:
        _cached_stats = _calculate_stats()
        _stats_dirty = false
    return _cached_stats

```

5. 随机词缀：每件装备不一样

问题："每把铁剑攻击力都是 10", 太单调了。玩家想要"铁剑 + 3", "锋利的铁剑"。

词缀系统：装备有基础属性 + 随机词缀。

```

# AffixResource.gd
class_name AffixResource
extends Resource

@export var affix_name: String = "锋利的"
@export var stat: String = "attack"      # 影响哪个属性
@export var value: int = 5                # 加多少
@export var tier: int = 1                  # 词缀等级

```

```

# 装备生成时随机附加词缀
func generate_weapon(base: EquipmentResource, level: int) -> EquipmentResource:
    var new_item = base.duplicate() # 复制基础装备
    new_item.item_name = base.item_name

    # 根据等级决定词缀数量

```

```

var affix_count = 0
if level >= 10: affix_count = 1
if level >= 20: affix_count = 2
if level >= 35: affix_count = 3

for i in affix_count:
    var affix = _random_affix(level)
    new_item.affixes.append(affix)
    new_item.item_name = affix.affix_name +
new_item.item_name

return new_item

func _get_display_stats(item: EquipmentResource) -> Dictionary:
    var stats = {
        "attack": item.attack,
        "defense": item.defense,
        "hp": item.hp,
    }
    # 叠加词缀
    for affix in item.affixes:
        if stats.has(affix.stat):
            stats[affix.stat] += affix.value
    return stats

```

效果:

```

铁剑: 攻击力 10
锋利的铁剑: 攻击力 15 (锋利 +5)
锋利的铁剑 of 力量: 攻击力 20 (锋利 +5, 力量 +5)

```

6. 品质/稀有度: 白蓝紫橙

问题: "所有的装备都是白色的, 没有稀有感。"

品质系统: 品质决定词缀数量和数值范围。

```

enum Rarity { COMMON, MAGIC, RARE, EPIC, LEGENDARY }

```

```

var rarity_colors = {
  Rarity.COMMON:    Color.WHITE,
  Rarity.MAGIC:     Color.BLUE,
  Rarity.RARE:      Color.YELLOW,
  Rarity.EPIC:      Color.PURPLE,
  Rarity.LEGENDARY: Color.ORANGE,
}

var rarity_config = {
  Rarity.COMMON:    { "affix_min": 0, "affix_max": 0,
"multiplier": 1.0 },
  Rarity.MAGIC:     { "affix_min": 1, "affix_max": 1,
"multiplier": 1.2 },
  Rarity.RARE:      { "affix_min": 2, "affix_max": 2,
"multiplier": 1.5 },
  Rarity.EPIC:      { "affix_min": 3, "affix_max": 3,
"multiplier": 2.0 },
  Rarity.LEGENDARY: { "affix_min": 4, "affix_max": 4,
"multiplier": 3.0 },
}

func generate_equipment(base: EquipmentResource, rarity: Rarity)
-> EquipmentResource:
  var item = base.duplicate()
  var config = rarity_config[rarity]

  # 品质倍数: 基础属性乘以品质系数
  item.attack = roundi(item.attack * config.multiplier)
  item.defense = roundi(item.defense * config.multiplier)

  # 随机词缀
  var count = randi_range(config.affix_min, config.affix_max)
  for i in count:
    item.affixes.append(_random_affix(item.slot))

  item.rarity = rarity
  return item

```

掉落时随机品质:

```

func roll_rarity() -> Rarity:
  var roll = randf()
  if roll < 0.5:    return Rarity.COMMON    # 50%

```

```

if roll < 0.8:    return Rarity.MAGIC        # 30%
if roll < 0.94:   return Rarity.RARE         # 14%
if roll < 0.99:   return Rarity.EPIC         # 5%
return Rarity.LEGENDARY                        # 1%

```

7. 套装效果：穿一套有加成

问题："穿 2 件火焰套 → 攻击附带灼烧。穿 4 件 → 免疫火焰。"

```

# SetBonusResource.gd
class_name SetBonusResource
extends Resource

@export var set_name: String = "火焰套装"
@export var pieces: Array[EquipmentResource] # 属于该套装的装备
@export var bonuses: Dictionary = {
    2: { "effect": "附加灼烧", "damage": 5 },
    4: { "effect": "火焰免疫", "immune": "fire" },
}

```

在 EquipmentManager 中检查套装：

```

func _check_set_bonuses():
    # 获取当前装备的所有套装的件数
    var set_counts := {}

    for slot in Slot.values():
        var item = _equipment[slot]
        if item and item.set_bonus:
            var set_name = item.set_bonus.set_name
            set_counts[set_name] = set_counts.get(set_name, 0) + 1

    # 检查每个套装触发了几档效果
    for set_name in set_counts:
        var count = set_counts[set_name]
        var set_bonus = _get_set_resource(set_name)
        if set_bonus:
            for threshold in set_bonus.bonuses.keys():
                if count >= threshold and _activated[set_name] <

```

```

threshold:

_activate_set_bonus(set_bonus.bonuses[threshold])
    elif count < threshold and _activated[set_name]
>= threshold:

_deactivate_set_bonus(set_bonus.bonuses[threshold])

```

8. 最终方案: 最简可用模板

```

# 一个够用的装备系统, 不需要多余的功能:

# —— EquipmentResource.gd ——
class_name EquipmentResource
extends Resource

@export var item_name: String = ""
@export var icon: Texture2D
@export var slot: int = 0

@export var attack: int = 0
@export var defense: int = 0
@export var hp: int = 0
@export var speed: int = 0

# —— EquipmentManager.gd (挂在玩家身上) ——
class_name EquipmentManager
extends Node

enum Slot { WEAPON, HELMET, ARMOR, RING, AMULET, BOOTS }

var _slots := {}

func _init():
    for s in Slot.values():
        _slots[s] = null

func equip(item: EquipmentResource, slot: int) -> bool:
    if _slots[slot] != null:

```



```

        return false
    _slots[slot] = item
    return true

func unequip(slot: int):
    _slots[slot] = null

func get_stats() -> Dictionary:
    var total = { "attack": 0, "defense": 0, "hp": 0, "speed":
0 }
    for slot in _slots:
        var item = _slots[slot]
        if item:
            total["attack"] += item.attack
            total["defense"] += item.defense
            total["hp"] += item.hp
            total["speed"] += item.speed
    return total

func get_slot(slot: int) -> EquipmentResource:
    return _slots[slot]

# 使用:
# $EquipmentManager.equip(iron_sword,
EquipmentManager.Slot.WEAPON)
# var stats = $EquipmentManager.get_stats()
# attack = 10 + stats["attack"]

```

这个故事告诉我们：- 装备系统从手写加减属性开始，每一步都在解决"数据从哪里来，怎么算，怎么改" - Resource 让装备数据脱离代码，任何人都能编辑 - 实时汇总 (get_total_stats) 比手动加减更安全 - 词缀 + 品质 + 套装是装备系统最常用的三层扩展

第14章 强化系统演化

Godot 强化系统: 从 +1 到概率与保底

强化 = 让已有的装备变得更强大。+1, +2, +3... 越高级概率越低, 失败了可能碎掉。强化系统的进化 = "怎么让玩家又爱又恨但停不下来"。

1. 原始: 固定强化

```
func enhance(weapon):  
    weapon.attack += 5  
    weapon.level += 1
```

问题: 没有风险, 都可以强化满。内容消耗快。

2. 概率强化 (失败不掉级)

```
func enhance(weapon) -> bool:  
    var rate = get_success_rate(weapon.level)  
    if randf() < rate:  
        weapon.attack += 5  
        weapon.level += 1  
        return true  
    return false # 失败, 但没惩罚  
  
func get_success_rate(level: int) -> float:  
    return max(0.1, 1.0 - level * 0.1)  
    # +0: 100%, +1: 90%, +2: 80% ... +9: 10%
```

3. 失败掉级/碎掉

```
func enhance(weapon) -> bool:
    var rate = get_success_rate(weapon.level)
    if randf() < rate:
        weapon.attack += 5
        weapon.level += 1
        return true
    else:
        if weapon.level >= 7:
            weapon.queue_free() # 碎了!
        else:
            weapon.level = max(0, weapon.level - 1)
        return false
```

问题: 概率太随机, 有人 10 次全成, 有人 50 次全碎。

4. 保底/概率递增

```
var fail_count = 0

func enhance(weapon) -> bool:
    var base_rate = get_success_rate(weapon.level)
    var actual_rate = base_rate + fail_count * 0.1
    if randf() < actual_rate:
        weapon.level += 1
        fail_count = 0
        return true
    else:
        fail_count += 1
        return false
```

5. 最终方案模板

```
func try_enhance(item: EquipmentResource) -> Dictionary:
    var level = item.enhance_level
    var rate = _success_rate(level)
```

```

var cost = _enhance_cost(level)

if not CurrencyManager.spend("gold", cost):
    return { "success": false, "reason": "金币不足" }

if randf() < rate:
    item.enhance_level += 1
    item.attack += _stat_gain(level)
    return { "success": true, "new_level":
item.enhance_level }
else:
    if level >= _protect_level():
        return { "success": false, "broken": true }
    item.enhance_level = max(0, level - 1)
    return { "success": false, "broken": false }

func _success_rate(level: int) -> float:
    var rates = [1.0, 0.9, 0.8, 0.7, 0.5, 0.3, 0.15, 0.08, 0.03,
0.01]
    return rates[clamp(level, 0, rates.size() - 1)]

```

第15章 附魔系统演化

Godot 附魔系统：从固定属性到随机词缀

附魔 = 给装备额外加一条属性。强化是 +1 +2 +3 (数值增长)。附魔是额外加"火焰伤害""吸血""冰冻" (新增效果)。附魔系统的进化 = "怎么让同一件装备有不同可能性"。

1. 原始：固定附魔

```
func enchant_fire(weapon):  
    weapon.fire_damage = 10  
# 每把剑的附魔都一样
```

2. 附魔石 + 随机效果

```
class EnchantStone:  
    var id: String  
    var name: String  
    var possible_effects: Array[EnchantEffect]  
    var success_rate: float = 1.0  
  
class EnchantEffect:  
    var stat: String          # "fire_damage" / "life_steal" /  
    "attack_speed"  
    var min_value: float  
    var max_value: float
```

```
func enchant(item: EquipmentResource, stone: EnchantStone) ->  
bool:  
    if randf() > stone.success_rate:
```

```

        return false # 附魔失败

        var available = stone.possible_effects.filter(func(e):
            return not item.enchant_effects.any(func(ie): return
ie.stat == e.stat)
        )
        if available.is_empty(): return false

        var chosen = available.pick_random()
        var value = randf_range(chosen.min_value, chosen.max_value)
        value = snapped(value, 0.1)

        item.enchant_effects.append({
            "stat": chosen.stat,
            "value": value,
            "source": stone.id,
        })
        item.recalc_stats()
        return true

```

3. 附魔数量限制 + 覆盖

```

const MAX_ENCHANT = 3

func can_enchant(item: EquipmentResource) -> bool:
    return item.enchant_effects.size() < MAX_ENCHANT

func replace_enchant(item: EquipmentResource, index: int, stone:
EnchantStone) -> bool:
    # 用新附魔覆盖旧附魔
    if index >= item.enchant_effects.size(): return false
    item.enchant_effects.remove_at(index)
    return enchant(item, stone)

```

4. 最终方案模板

```

# EnchantManager.gd (Autoload)

```



```
# 附魔流程：
# 1. 选择装备 + 附魔石
# 2. 检查装备可附魔槽位是否已满
# 3. 从附魔石的效果池中随机选一条
# 4. 数值在范围内随机
# 5. 写入装备.enchant_effects
# 6. 装备重新计算总属性

func apply(item: EquipmentResource, stone: EnchantStone) ->
Dictionary:
    if item.enchant_effects.size() >= 3:
        return { "success": false, "reason": "附魔已满" }
    # ... 附魔逻辑
    return { "success": true, "effect": chosen }
```

第16章 继承系统演化

Godot 继承系统：从重新培养到转移

继承 = 把一件装备的强化等级/附魔转移到另一件上。最早每件装备各自强化, 换了装备从头再来。后来有了继承: 旧装备的强化等级转到新装备上。继承系统的进化 = "怎么让玩家换装备不心疼"。

1. 原始：重新强化

```
# 换了新武器，从 +0 开始重新强化
# 旧武器卖店，强化石不返还
# 玩家体验：不敢换装备
```

2. 强化转移

```
func inherit(source: EquipmentResource, target:
EquipmentResource) -> bool:
    # 同类型才能继承（剑→剑）
    if source.type != target.type: return false
    # 目标等级不能低于一定范围
    if target.level_required - source.level_required > 10:
return false

    # 交换强化等级
    var temp = target.enhance_level
    target.enhance_level = source.enhance_level
    source.enhance_level = temp # 旧装备变回 +0

    # 消耗
    CurrencyManager.spend("gold",
```

```

_inherit_cost(source.enhance_level))

    target.recalc_stats()
    source.recalc_stats()
    return true

```

3. 附魔继承 + 损耗

```

func inherit_with_enchant(source: EquipmentResource, target:
EquipmentResource) -> Dictionary:
    if source.type != target.type: return { "success": false,
"reason": "类型不同" }

    var result = { "success": true }

    # 强化等级继承 (无损)
    target.enhance_level = source.enhance_level
    source.enhance_level = 0

    # 附魔继承 (部分随机丢失)
    var kept = []
    for ench in source.enchant_effects:
        if randf() < 0.7: # 30% 概率丢失
            kept.append(ench)
    target.enchant_effects = kept
    result.transferred_enchant = kept.size()
    result.lost_enchant = source.enchant_effects.size() -
kept.size()

    source.enchant_effects.clear()
    CurrencyManager.spend("gold", _cost(source.enhance_level))
    target.recalc_stats()
    return result

```

4. 最终方案模板

```

# InheritManager.gd (Autoload)
# 继承规则:

```

```

# 1. 同类型装备 (剑→剑)
# 2. 目标等级 ≤ 源等级 + 10
# 3. 消耗金币 (根据强化等级递增)
# 4. 附魔随机保留 (或无损)

func transfer(source, target) -> Dictionary:
    if source.type != target.type:
        return { "ok": false, "msg": "类型不同" }
    var cost = _calc_cost(source.enhance_level)
    if not CurrencyManager.spend("gold", cost):
        return { "ok": false, "msg": "金币不足" }
    target.enhance_level = source.enhance_level
    source.enhance_level = 0
    target.enchant_effects = source.enchant_effects.duplicate()
    source.enchant_effects.clear()
    target.recalc_stats()
    source.recalc_stats()
    return { "ok": true, "cost": cost }

```

第17章 觉醒系统演化

Godot 觉醒系统：从数值提升到质变

觉醒 = 角色/装备达到某种条件后发生质变。强化是 +1+2+3 (量变)。觉醒是从 3 星变 4 星, 外观变化, 获得新技能 (质变)。觉醒系统的进化 = "怎么让养成有突破感"。

1. 原始：无觉醒

```
# 只有等级和强化，没有"质变"概念
```

2. 星级觉醒

```
class AwakenConfig:
    var current_star: int
    var next_star: int
    var cost_materials: Dictionary    # { item_id: count }
    var cost_gold: int
    var stat_multiplier: float       # 属性倍率
    var unlock_skill: String         # 觉醒解锁的技能
    var level_required: int
    var success_rate: float          # 部分游戏有成功率

func awaken(item: EquipmentResource) -> bool:
    var cfg = AwakenDB.get(item.id, item.star)
    if item.star >= 6: return false    # 满星
    if item.level < cfg.level_required: return false
    if not _has_materials(cfg.cost_materials): return false
    if not CurrencyManager.spend("gold", cfg.cost_gold): return false
    return true
```

```

    if randf() > cfg.success_rate: return false

    # 扣材料
    for mid in cfg.cost_materials:
        InventoryManager.remove(mid, cfg.cost_materials[mid])

    # 升星
    item.star += 1
    item.stat_multiplier = cfg.stat_multiplier

    # 解锁技能
    if cfg.unlock_skill:
        SkillManager.unlock(cfg.unlock_skill)

    return true

```

3. 觉醒特效

```

func play_awaken_effect(item: EquipmentResource):
    # 光柱特效
    var effect = preload("res://effects/
awaken_beam.tscn").instantiate()
    add_child(effect)

    # 屏幕闪烁
    $ScreenFlash.play()

    # 新外观 (换 Sprite)
    $Sprite.texture = item.awaken_texture

    # 弹窗展示新属性
    AwakenPopup.show(item)

```

4. 最终方案模板

```

# AwakenManager.gd (Autoload)

func can_awaken(item) -> bool:

```



```
var cfg = _cfg(item)
if not cfg: return false
return item.level >= cfg.level_required \
    and item.star < cfg.max_star \
    and _has_materials(cfg.cost) \
    and CurrencyManager.has("gold", cfg.gold)

func awaken(item) -> bool:
    if not can_awaken(item): return false
    _consume_costs(item)
    item.star += 1
    item.recalc_stats()
    AwakenEffect.play(item)
    return true
```

第18章 转职系统演化

Godot 转职系统：从单一路线到职业分化

转职 = 角色达到一定条件后变成更强的职业。最早没有职业, 所有人都一样。后来有了战士/法师/弓箭手, 各职业技能不同。转职系统的进化 = "怎么让玩家选择自己的道路"。

1. 原始：无职业

所有角色都一样，技能全都能学

2. 初始职业选择

```
enum Job { WARRIOR, MAGE, ARCHER }

func choose_job(job: Job):
    match job:
        Job.WARRIOR:
            base_attack = 15
            base_defense = 10
            base_hp = 120
            starting_skills = ["slash", "shield_block"]
        Job.MAGE:
            base_attack = 8
            base_defense = 4
            base_hp = 70
            starting_skills = ["fireball", "frost_nova"]
        Job.ARCHER:
            base_attack = 12
            base_defense = 5
```

```
base_hp = 85
starting_skills = ["shoot", "dodge"]
```

3. 一转/二转

```
class JobTree:
    var id: String
    var name: String
    var level_required: int
    var previous_job: String      # 前置职业
    var stat_growth: Dictionary  # { "attack": 2, "defense":
1 }
    var unlock_skills: Array[String]
    var quest_required: String    # 转职任务

var job_tree = {
    "warrior":    { "next": ["knight", "berserker"], "level":
10 },
    "knight":     { "next": ["paladin", "dark_knight"], "level":
30 },
    "berserker":  { "next": ["warlord"], "level": 30 },
    "mage":       { "next": ["wizard", "warlock"], "level":
10 },
    "archer":     { "next": ["sniper", "ranger"], "level": 10 },
}
```

```
func can_change_job(target_job: String) -> bool:
    var data = JobDB.get(target_job)
    if not data: return false
    if PlayerData.level < data.level_required: return false
    if data.previous_job and PlayerData.job !=
data.previous_job: return false
    if data.quest_required and not
QuestManager.is_completed(data.quest_required): return false
    return true

func change_job(target_job: String):
    if not can_change_job(target_job): return
    var data = JobDB.get(target_job)
    PlayerData.job = target_job
    # 应用属性成长
```

```
for stat in data.stat_growth:
    PlayerData[stat] += data.stat_growth[stat]
# 解锁技能
for skill_id in data.unlock_skills:
    SkillManager.unlock(skill_id)
```

4. 最终方案模板

```
# JobManager.gd (Autoload)
# 转职核心：前置职业 + 等级要求 + 任务要求
# 转职后：属性成长 + 解锁技能 + 可能解锁新装备

func get_available_jobs() -> Array:
    return JobDB.all().filter(func(j): return
can_change_job(j.id))

func get_job_name(job_id: String) -> String:
    return JobDB.get(job_id).name
```

第19章 敌人 AI 演化

Godot AI 敌人演化史

从"看到你就追"到"包围你"。从 if-else 到 Utility AI。敌人的智商进化, 就是游戏设计深度进化的缩影。

目录

1. [反应式 AI: 看到就追](#)
 2. [巡逻 AI: 没事走两步](#)
 3. [有限状态机 AI: 一套完整的脑子](#)
 4. [感知系统: 视觉 + 听觉 + 距离](#)
 5. [AI 子弹与弹幕](#)
 6. [行为树 AI: 节点化决策](#)
 7. [群体 AI: 协同作战](#)
 8. [Utility AI: 数值化决策](#)
 9. [最终方案? 选型指南](#)
-

1. 反应式 AI: 看到就追

问题: "我要做一个敌人, 玩家靠近就追上去。"

最初的想法: 每帧检测距离, 在范围内就去追。

```

extends CharacterBody2D

@export var speed = 100.0
@export var chase_range = 200.0

var player: Node2D

func _ready():
    player = get_tree().get_first_node_in_group("player")

func _physics_process(delta):
    if not player:
        return

    var dist =
global_position.distance_to(player.global_position)

    if dist < chase_range:
        # 追!
        var dir = (player.global_position -
global_position).normalized()
        velocity = dir * speed
    else:
        velocity = Vector2.ZERO

    move_and_slide()

```

这算 AI 吗? 算。这是最原始的 AI——**反应式 (Reactive)**。

工作原理:

```

每帧:
    如果 玩家在范围内 → 追
    否则 → 站着不动

```

效果: 敌人像磁铁一样吸向玩家。

问题: 1. **追到了然后呢?** – 只会贴脸, 不会攻击 2. **出了范围就失忆** – 完全不管你了, 即使你就在它面前 3. **永远在追** – 没有"我是不是该停下来攻击了"的判断 4. **所有敌人一样** – 都是"范围内→追"

改进: 加个攻击距离。

```
@export var attack_range = 30.0

func _physics_process(delta):
    var dist =
    global_position.distance_to(player.global_position)

    if dist < attack_range:
        # 攻击!
        attack()
        velocity = Vector2.ZERO
    elif dist < chase_range:
        # 追!
        var dir = (player.global_position -
        global_position).normalized()
        velocity = dir * speed
    else:
        velocity = Vector2.ZERO

    move_and_slide()
```

状态变成了三个:

```
静止 —(玩家进入范围)→ 追击 —(贴脸)→ 攻击
    ↑
    |—————(玩家跑远)—————|
```

但这就是**手写的状态机**。当敌人的行为更复杂时, if-else 会爆炸。

2. 巡逻 AI: 没事走两步

问题: 敌人站在那一动不动, 像个傻子。玩家还没靠近, 就能看到一尊雕像。

需求: 没事的时候走一走, 看起来像在"活着"。

```
extends CharacterBody2D

enum State { PATROL, CHASE, ATTACK }
```

```

var state = State.PATROL

# 巡逻
var patrol_points: Array[Vector2] = []
var patrol_index = 0
var wait_timer = 0.0

func _ready():
    # 自动生成一些巡逻点, 或者从编辑器配置
    patrol_points = [Vector2(100, 100), Vector2(300, 100),
Vector2(300, 300)]

func _physics_process(delta):
    match state:
        State.PATROL:
            _do_patrol(delta)
        State.CHASE:
            _do_chase(delta)
        State.ATTACK:
            _do_attack(delta)

func _do_patrol(delta):
    if patrol_points.size() == 0:
        return

    var target = patrol_points[patrol_index]
    var dist = global_position.distance_to(target)

    if dist < 10.0:
        # 到了, 等一会儿
        wait_timer += delta
        velocity = Vector2.ZERO
        if wait_timer > 2.0:
            patrol_index = (patrol_index + 1) %
patrol_points.size()
            wait_timer = 0.0
    else:
        var dir = (target - global_position).normalized()
        velocity = dir * speed * 0.5 # 巡逻走得慢

    move_and_slide()

# 检测玩家
if player and

```

```
global_position.distance_to(player.global_position) <
chase_range:
    state = State.CHASE
```

进步: - 敌人不再像雕像了, 它在世界里"活着" - 玩家可以观察到敌人的巡逻路线, 制定策略 (潜行游戏的核心)

问题: 1. 巡逻路线是固定的 - 玩家背后后可以绕过 2. 只有巡逻 → 追击, 不能追击 → 巡逻 - 玩家跑出范围后, 敌人还在追 3. "放弃追击"的逻辑缺失 - 追多远才放弃? 追多久才放弃?

加回退逻辑:

```
var chase_timer = 0.0
var max_chase_time = 5.0    # 追 5 秒追不到就放弃
var max_chase_distance = 400.0 # 离开出生点太远也放弃
var spawn_position: Vector2

func _do_chase(delta):
    chase_timer += delta

    # 放弃条件
    if chase_timer > max_chase_time:
        state = State.PATROL
        chase_timer = 0.0
        return

    if spawn_position.distance_to(global_position) >
max_chase_distance:
        state = State.PATROL
        chase_timer = 0.0
        return

    # 继续追...
```

这已经是一个完整的"巡逻-追击-攻击-放弃"环了。但代码量开始在单文件里膨胀。

3. 有限状态机 AI: 一套完整的脑子

问题: 敌人的行为越来越复杂: - 巡逻时听到声音要去调查 - 追击过程中玩家躲起来了要搜索 - 受伤了要后撤 - 残血了要逃跑 - 队友死了要狂暴

match state: 的每个分支越来越长, 而且状态之间的**过渡条件**散落在各个分支里。

解决方案: 第二章的状态机模式 - 每个状态一个函数/脚本。

```
# 一个 Enemy AI 的完整 FSM (单文件版)

extends CharacterBody2D

enum State {
    IDLE, PATROL, INVESTIGATE, CHASE,
    ATTACK, HURT, FLEE, DEAD
}
var state = State.IDLE

# 感知数据
var player: Node2D
var last_known_position: Vector2
var suspicious_noise_position: Vector2
var health = 100

func _physics_process(delta):
    _update_perception(delta)    # 先感知
    _execute_state(delta)       # 再行动

func _update_perception(delta):
    if not player:
        return
    var dist =
global_position.distance_to(player.global_position)

    # 视觉检测
    if dist < vision_range and _has_line_of_sight(player):
        last_known_position = player.global_position
        if state in [State.IDLE, State.PATROL]:
            _change_state(State.CHASE)
```

```

# 听觉检测（外部触发，比如枪声）
# suspicious_noise_position 由别的脚本设置

func _execute_state(delta):
    match state:
        State.IDLE:
            _state_idle(delta)
        State.PATROL:
            _state_patrol(delta)
        State.INVESTIGATE:
            _state_investigate(delta)
        State.CHASE:
            _state_chase(delta)
        State.ATTACK:
            _state_attack(delta)
        State.HURT:
            _state_hurt(delta)
        State.FLEE:
            _state_flee(delta)
        State.DEAD:
            _state_dead(delta)

func _change_state(new_state):
    # 退出当前状态
    match state:
        State.CHASE:
            pass # 重置追击计时器等

    state = new_state

    # 进入新状态
    match state:
        State.CHASE:
            chase_timer = 0.0
        State.HURT:
            velocity = -velocity * 3 # 击退
            $AnimationPlayer.play("hurt")

```

这样设计的优点: - 每个状态是一个 `_state_xxx` 函数, 逻辑隔离 -

`_update_perception` 统一处理"看到/听到玩家", 不散在各个状态里 -

`_change_state` 统一处理进入/退出的钩子

但规模上来后, 单文件的问题: - 这个文件很快变成 500+ 行 - 要理解一个敌人的全部行为, 要滚动 10 屏 - 不同敌人 (近战/远程/Boss) 共用逻辑没法抽

于是回到第三章的状态类方案:

```
# ChaseState.gd
class_name ChaseState
extends AiState

func enter(owner: Enemy):
    owner.animation_player.play("run")

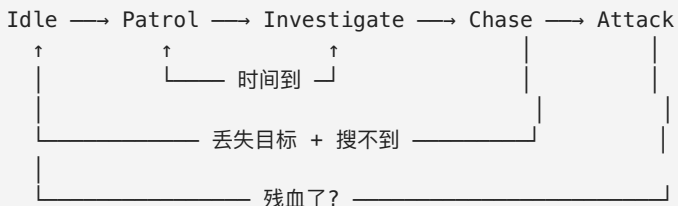
func update(owner: Enemy, delta):
    if not owner.last_known_position:
        owner.change_state("patrol")
        return

    var dir =
owner.global_position.direction_to(owner.last_known_position)
    owner.velocity = dir * owner.speed
    owner.move_and_slide()

    # 看到玩家了 → 追击升级
    if owner.can_see_player():
        owner.last_known_position = owner.player.global_position

    # 贴脸了 → 攻击
    if
owner.global_position.distance_to(owner.player.global_position)
< owner.attack_range:
        owner.change_state("attack")
```

完整的状态集:



↓
Flee → (跑出视野/回到出生点) → Idle

这是一套完整的 AI 大脑, 比之前的 if-else 强大太多。但它的决策是硬编码的:

- "看到玩家就追" - 总会追 - "血量 < 30% 就逃跑" - 总会逃

如果我想让敌人有一定随机性, 或者根据不同的武器/环境/玩家状态做不同决策, 这种硬编码的状态机会变得臃肿。

4. 感知系统: 视觉 + 听觉 + 距离

问题: 之前的 AI 用 `distance_to()` 来判断"看到玩家"。但距离 $\neq$ 看到: 玩家在墙后面, 距离再近也看不到; 玩家在视野外, 距离再近也看不到。

敌人应该有三个通道来感知世界:

视觉 (Vision) - 锥形检测 + 射线检测遮挡
听觉 (Hearing) - 距离衰减, 隔墙减半
触觉 (Touch) - 玩家贴脸了, 还能感觉不到吗?

```
# 视觉检测: 锥形视野
func can_see_player(player: Node2D) -> bool:
    # 1. 距离检测
    var dir_to_player = player.global_position - global_position
    var dist = dir_to_player.length()
    if dist > vision_range:
        return false

    # 2. 角度检测 (锥形视野)
    var angle_to_player = dir_to_player.angle()
    var facing_angle = -rotation if is_instance_of(self,
CharacterBody2D) else 0

    # 假设面朝前, 视野  $\pm 60^\circ$ 
    var diff = wrapf(angle_to_player - facing_angle, -PI, PI)
    if abs(diff) > deg_to_rad(60):
        return false
```

```

# 3. 遮挡检测
var space_state = get_world_2d().direct_space_state
var query = PhysicsRayQueryParameters2D.create(
    global_position,
    player.global_position
)
query.exclude = [self]
var result = space_state.intersect_ray(query)
if result:
    # 射线碰到了东西，但不是玩家 - 被挡住了
    if result.collider != player:
        return false

return true

# 听觉检测
func can_hear(sound_position: Vector2, sound_intensity: float) -
> bool:
    var dist = global_position.distance_to(sound_position)
    var heard_volume = sound_intensity - dist * 0.1 # 距离衰减
    return heard_volume > hearing_threshold

# 触觉检测（其实是碰撞信号）
# 在 Area2D 上:
func _on_DetectionArea_body_entered(body):
    if body.is_in_group("player"):
        # 贴脸了，直接攻击
        change_state("attack")

```

完整感知循环:

```

每帧更新感知:
├─ 视觉: 锥形 + 射线 → 能看到吗? → 更新 last_known_position
├─ 听觉: 事件触发 (有人开枪/跑步) → 设置 investigate 点
└─ 触觉: Area2D 信号 → 直接触发攻击

```

最新感知到的信息 → 输入给决策系统 (FSM/BT)

感知系统的意义: 它让 AI 的行为变得真实。 - 玩家蹲在草丛里 → 不在视野锥内, 距离再近也看不到 - 玩家在墙后走路 → 看不到, 但听得到 → AI 会去调查 - 玩家绕后 → AI 视野有限, 可以被背刺

这也是**潜行游戏**的基础。没有感知系统, 潜行不存在。

5. AI 子弹与弹幕

问题: "敌人开枪, 子弹怎么走?"

最简单的: 子弹直接飞向玩家。

```
# 发射子弹
func shoot():
    var bullet = bullet_scene.instantiate()
    bullet.global_position = $Muzzle.global_position
    bullet.direction = (player.global_position -
global_position).normalized()
    bullet.speed = bullet_speed
    get_parent().add_child(bullet)
```

```
# Bullet.gd
extends CharacterBody2D

var direction: Vector2
var speed: float

func _physics_process(delta):
    velocity = direction * speed
    move_and_slide()
```

但"永远打中玩家"的子弹是无聊的。玩家不需要躲, 要么被打中要么格挡。

AI 子弹的演化:

```
第 1 代: 直线追踪子弹
├─ 永远朝玩家当前位置飞 → 必中 (除非玩家跑得比子弹快)
├─ 适合: 基础敌人、教程敌人
```

第 2 代：预判子弹 (Lead Shot)

- └ 计算玩家未来的位置，朝那个方向打
- └ 玩家如果不变向，必然被打中
- └ 玩家变向 → 子弹落空 → 有交互感

第 3 代：弹幕模式

- └ 扇形射击、螺旋弹幕、延迟弹幕
- └ 不是一发子弹，而是一个"弹幕模式"
- └ 适合：Boss 战

第 4 代：可控子弹

- └ 激光（瞬间命中，需要蓄力提示）
- └ 制导导弹（慢慢转向玩家，可以绕柱躲）
- └ 弹跳子弹（撞墙反弹，封走位）

第 2 代：预判射击

```
func shoot_with_prediction():
    var bullet = bullet_scene.instantiate()
    bullet.global_position = $Muzzle.global_position

    # 预估玩家未来的位置
    var player_vel = player.velocity
    var time_to_hit =
global_position.distance_to(player.global_position) /
bullet_speed
    var predicted_pos = player.global_position + player_vel *
time_to_hit

    bullet.direction = (predicted_pos -
global_position).normalized()
    get_parent().add_child(bullet)

# 第 3 代：扇形 3 发
func shoot_spread():
    for i in range(-1, 2):
        var bullet = bullet_scene.instantiate()
        bullet.global_position = $Muzzle.global_position
        var base_dir = (player.global_position -
global_position).normalized()
        bullet.direction = base_dir.rotated(deg_to_rad(15 * i))
        get_parent().add_child(bullet)
```

```
# 第 4 代：制导导弹
# 在 Bullet.gd 中：
var homing_strength = 2.0 # 转向强度

func _physics_process(delta):
    if player and is_instance_valid(player):
        var desired = (player.global_position -
            global_position).normalized()
        direction = direction.lerp(desired, homing_strength *
            delta).normalized()
        velocity = direction * speed
        move_and_slide()
```

子弹 + 感知 + 状态机 构成了一个完整的敌人 AI 框架。

6. 行为树 AI: 节点化决策

问题: FSM 的过渡是硬编码的。"看到玩家就去追"——没有优先级的概念。如果同时"看到玩家"和"血量低", 哪个更重要?

行为树 (Behavior Tree) 的思维: - 不是 "我处于什么状态", 而是 "当前什么最重要" - 从根节点开始, 每帧遍历整棵树, 找到最高优先级的动作执行

```
# 用代码模拟行为树 (简化版)

enum BtStatus { SUCCESS, FAILURE, RUNNING }

class BtNode:
    func tick(owner: Enemy, delta: float) -> BtStatus:
        return BtStatus.SUCCESS

class Selector extends BtNode:
    var children: Array[BtNode]

    func tick(owner: Enemy, delta: float) -> BtStatus:
        for child in children:
            var status = child.tick(owner, delta)
```

```

        if status == BtStatus.SUCCESS or status ==
BtStatus.RUNNING:
            return status
        return BtStatus.FAILURE

class Sequence extends BtNode:
    var children: Array[BtNode]

    func tick(owner: Enemy, delta: float) -> BtStatus:
        for child in children:
            var status = child.tick(owner, delta)
            if status != BtStatus.SUCCESS:
                return status
        return BtStatus.SUCCESS

# 叶子节点
class ConditionLowHealth extends BtNode:
    func tick(owner: Enemy, delta: float) -> BtStatus:
        return BtStatus.SUCCESS if owner.health < 30 else
BtStatus.FAILURE

class ActionFlee extends BtNode:
    func tick(owner: Enemy, delta: float) -> BtStatus:
        # 逃跑逻辑...
        return BtStatus.RUNNING

class ActionChase extends BtNode:
    func tick(owner: Enemy, delta: float) -> BtStatus:
        # 追击逻辑...
        return BtStatus.RUNNING

```

行为树结构:

```

Selector (根节点 - 选最高优先级的)
├── Sequence (残血 → 逃跑)
│   ├── ConditionLowHealth (血量 < 30%)
│   └── ActionFlee (跑向血包/跑回老家)
├── Sequence (看到玩家 → 战斗)
│   ├── ConditionCanSeePlayer
│   ├── Sequence (距离 > 攻击范围 → 追)
│   │   ├── ConditionOutOfRange
│   │   └── ActionChase
└──

```

```
| └─ ActionAttack (在攻击范围 → 攻击)
└─ ActionPatrol (默认行为: 巡逻)
```

每帧执行过程:

从根 Selector 开始:

```
└─ 尝试 Sequence "残血逃跑":
    │ └─ ConditionLowHealth: 血量 80% → FAILURE
    │   (短路, 整个 Sequence FAILURE)
└─ 尝试 Sequence "战斗":
    │ └─ ConditionCanSeePlayer: 没看到 → FAILURE
    │   (短路)
└─ ActionPatrol: 执行巡逻 → SUCCESS / RUNNING
```

如果玩家出现, 血量 80%:

根 Selector:

```
└─ 残血逃跑: FAILURE (血量 >= 30)
└─ 战斗:
    │ └─ ConditionCanSeePlayer: SUCCESS (看到)
    │   └─ Sequence (子序列):
    │       │ └─ ConditionOutOfRange: SUCCESS (距离远)
    │       │ └─ ActionChase: 追击 → RUNNING (每帧返回 RUNNING)
```

FSM vs BT 的核心区别:

FSM: "我现在在 CHASE 状态, 如果被打了就转到 HURT"

- 状态的切换由开发者硬编码
- 适合: 状态数量有限、切换逻辑明确的场景

BT: "从根开始, 残血?→跑. 看到人?→追. 都不满足?→巡逻"

- 每帧做一次"优先级决策"
- 适合: 行为多、优先级经常变化、需要"最紧急的事优先"

Godot 的行为树实现: 社区插件 LimboAI 是 Godot 4 的行为树方案。

```
# 用 LimboAI 的 BTPlayer
@onready var bt_player = $BTPlayer

func _ready():
    bt_player.blackboard.set_var("player", player)
```

```
bt_player.start()
```

```
# 不需要在 _physics_process 里写任何状态逻辑
# 行为树自己控制
```

FSM + BT 的组合:

角色基础控制: FSM (Idle/Run/Jump/Fall)

AI 决策: BT (选择"现在做什么")

↓

AI 把决策结果传给 FSM 的状态

BT 说"追击" → FSM 切换到 CHASE → move_and_slide 追

7. 群体 AI: 协同作战

问题: 单个敌人再聪明, 也打不过一群。但一群**各自为战**的敌人, 也不如一群**协同**的敌人。

各自为战:



😬 ← 三个敌人从三个方向独立追击, 互相挡路

协同:



← 一个正面吸引, 一个绕后, 一个包抄

群体 AI 的几种模式:

1. 阵型系统 (Formation)

敌人之间保持一定相对位置

```
var formation_offset = Vector2(50, 0) # 相对于队长
```

```
func _physics_process(delta):
```

```
    var leader_pos = leader.global_position if leader else
    global_position
```

```

var target = leader_pos + formation_offset

var dir = (target - global_position).normalized()
velocity = dir * speed
move_and_slide()

# 2. 包抄 (Flanking)
# 部分敌人正面攻击，部分从侧面/后面绕

func decide_flank():
    var angle_to_player =
global_position.angle_to_point(player.global_position)

    if role == "flanker":
        # 绕到玩家侧面
        var flank_target = player.global_position + \
            Vector2(100, 0).rotated(angle_to_player + PI / 2)
        move_to(flank_target)
    else:
        # 正面压制
        move_to(player.global_position)

# 3. 信息共享 (Shared Awareness)
# 一个敌人看到玩家 → 所有敌人都知道

signal player_spotted(position: Vector2, time: float)

func _ready():
    # 加入群体网络
    swarm_network.connect_to_group("enemies", self)

func on_player_spotted(pos, time):
    last_known_position = pos
    spot_time = time
    if state == State.PATROL:
        change_state(State.CHASE)

func _on_saw_player():
    player_spotted.emit(player.global_position,
        Time.get_ticks_msec())

```

```

# 4. 简单战术 (Tactical States)
# 敌人之间分配角色

enum SwarmRole { ATTACKER, SUPPORTER, FLANKER }

func assign_roles():
    var enemies = get_tree().get_nodes_in_group("enemies")
    var count = enemies.size()

    for i in count:
        if i == 0:
            enemies[i].role = SwarmRole.ATTACKER      # 正面
        elif i == 1 and count > 2:
            enemies[i].role = SwarmRole.FLANKER       # 左绕
            enemies[i].flank_direction = -1
        elif i == 2 and count > 3:
            enemies[i].role = SwarmRole.FLANKER       # 右绕
            enemies[i].flank_direction = 1
        else:
            enemies[i].role = SwarmRole.SUPPORTER     # 火力支援/远
程

```

群体 AI 的关键原则:

1. 不要每个敌人独立决策 - 选一个队长 (Leader), 其他跟随
2. 信息共享 - 一个看到 = 全部知道
3. 角色分工 - 不是所有敌人都做同样的事
4. 避免路径重叠 - 每个敌人略微偏移目标点, 不然会挤

8. Utility AI: 数值化决策

问题: 行为树的优先级是静态的--"残血逃跑"永远比"追击"优先级高。但如果残血 29%, 但玩家也残血了, 而且我有远程武器呢?

FSM 和 BT 的共同缺点: 决策是二元的是/否, 缺乏连续数值。

Utility AI (实用度 AI): 每个可能的动作计算一个分数, 选分数最高的做。

Utility AI 核心

```
class Action:
    var name: String
    func get_score(owner: Enemy) -> float:
        return 0.0
    func execute(owner: Enemy) -> void:
        pass

class AttackAction extends Action:
    func get_score(owner: Enemy) -> float:
        if not owner.can_see_player():
            return 0.0

        var dist =
owner.global_position.distance_to(owner.player.global_position)

        # 距离越近, 攻击的欲望越高 (0~1)
        var distance_score = 1.0 - (dist / owner.attack_range)
        distance_score = clamp(distance_score, 0.0, 1.0)

        # 血量越低, 越不敢攻击
        var health_score = owner.health / 100.0

        return distance_score * health_score * 0.8

class FleeAction extends Action:
    func get_score(owner: Enemy) -> float:
        var health_ratio = owner.health / 100.0

        # 血量越低, 越想跑
        var health_score = 1.0 - health_ratio

        # 离血包越远, 越想跑
        var health_pack_dist = INF
        if owner.nearest_health_pack:
            health_pack_dist =
owner.global_position.distance_to(
                owner.nearest_health_pack.global_position
            )
        var distance_score = 1.0 - (health_pack_dist / 500.0)
        distance_score = clamp(distance_score, 0.0, 1.0)
```

```

        return (health_score + distance_score) * 1.5 # 逃跑权重高

class PatrolAction extends Action:
    func get_score(owner: Enemy) -> float:
        # 默认行为的分数恒定低
        return 0.2

```

```

# AI 决策：选分数最高的
var actions: Array[Action]

func _ready():
    actions = [
        AttackAction.new(),
        FleeAction.new(),
        PatrolAction.new(),
    ]

func _decide():
    var best_score = -INF
    var best_action = null

    for action in actions:
        var score = action.get_score(self)
        if score > best_score:
            best_score = score
            best_action = action

    if best_action:
        current_action = best_action
        best_action.execute(self)

```

Utility AI 的优势:

不是"血量 < 30% → 逃跑", 而是:

攻击分数 = 距离 × 血量 (越近越高, 越高越高)

逃跑分数 = (1 - 血量) (越低越高)

最终: 哪个分数高做哪个

结果: 血量 80% 时攻击分数 > 逃跑分数 → 追击

血量 20% 时逃跑分数 > 攻击分数 → 逃跑

血量 30%, 但玩家就在脸上 → 攻击分数可能还比逃跑高 → 回头干你!

决策是连续的，不是二元跳跃的

Utility AI 的曲线调整：

```
# 可以用曲线让决策更自然
@export var attack_curve: Curve # 在编辑器里拉曲线

func get_score(owner: Enemy) -> float:
    var health_ratio = owner.health / 100.0
    # 曲线：血量 50% 以上时攻击欲平缓，50% 以下时急剧下降
    return owner.health > 50 \
        ? attack_curve.sample(health_ratio) \
        : attack_curve.sample(health_ratio) * 0.5
```

三个 AI 范式对比：

FSM: "状态切换" - 适合简单、确定的行为
BT: "优先级" - 适合行为多，需要灵活组合
Utility: "数值决策" - 适合需要细腻、连续变化的决策

实际项目中：- 小怪用 FSM (够用) - 精英怪用 BT (行为多，可组合) - Boss 用 Utility (根据战况动态调整策略)

9. 最终方案? 选型指南

敌人 AI 复杂度

- |
- | — 只需要"看到就追":
 - | — 反应式 AI (distance_to + direction_to) ●
 - | — 一行代码，适合炮灰
- |
- | — 有巡逻、追击、攻击三种行为：
 - | — 简易 FSM (enum + match) ●
 - | — (3-5 个状态，手写够用)
- |
- | — 行为 > 5 种，有多个敌人类型：
 - | — 状态类 FSM (每个状态一个脚本) ●

(可复用、可测试)

— 需要"残血逃跑""听到声音去调查""多个优先级":

└ 行为树 (LimboAI 或手写) ●
(决策灵活, 可视化)

— Boss 战, 需要根据玩家行为动态变化:

└ Utility AI ●
(连续决策, 不会重复)

— 成群结队的敌人:

└ 群体 AI (Leader + 阵型 + 信息共享) ●
(一个 AI 策略 -> 全员执行)

— 所有以上:

└ FSM + BT + 感知 + 群体 = 混合架构 ●
(商业游戏标配)

常见陷阱:

✗ 所有敌人都用同一套 AI

→ 炮灰用 FSM, 精英用 BT, Boss 用 Utility
→ 按复杂度分层, 不强行统一

✗ 感知和决策混在一起

→ 应该: 感知系统收集信息 → 决策系统做选择 → 行动系统执行
→ 不要在决策代码里写 `distance_to`

✗ AI 每帧都做完整决策

→ 追一个直线跑的人, 每帧的决策结果都一样
→ 决策频率降下来: 巡逻 1s 一次, 追击 0.2s 一次
→ 性能翻倍

✗ 有了 BT 就放弃 FSM

→ BT 决策选"追谁", 但"追"这个动作的状态 (开始追/追上了/追丢了) 是 FSM
→ BT 和 FSM 是互补, 不是替代

✗ AI 完美得不像人类

→ 完美预判、完美索敌 → 玩家觉得你作弊
→ 给 AI 加延迟、加误差、加随机性
→ NPC 的"笨"也是游戏性的一部分

附：敌人 AI 的最简实用模板

```
# 90% 的敌人的够用模板：
# 巡逻 + 追击 + 攻击 + 放弃

extends CharacterBody2D

enum State { PATROL, CHASE, ATTACK, RETURN }
var state = State.PATROL

@export var speed = 80.0
@export var chase_range = 200.0
@export var attack_range = 30.0
@export var max_chase_distance = 300.0

var player: Node2D
var spawn_pos: Vector2
var patrol_target: Vector2

func _ready():
    spawn_pos = global_position
    player = get_tree().get_first_node_in_group("player")
    _pick_patrol_target()

func _physics_process(delta):
    match state:
        State.PATROL:
            var dir =
global_position.direction_to(patrol_target)
            velocity = dir * speed * 0.5
            move_and_slide()
            if global_position.distance_to(patrol_target) < 10:
                _pick_patrol_target()
            if _player_in_range():
                state = State.CHASE

        State.CHASE:
            var dir =
global_position.direction_to(player.global_position)
            velocity = dir * speed
            move_and_slide()
            if
```

```

global_position.distance_to(player.global_position) <
attack_range:
    state = State.ATTACK
    if global_position.distance_to(spawn_pos) >
max_chase_distance:
    state = State.RETURN

State.ATTACK:
    velocity = Vector2.ZERO
    move_and_slide()
    # 攻击动画里通过信号/回调触发伤害
    if
global_position.distance_to(player.global_position) >
attack_range * 1.5:
    state = State.CHASE

State.RETURN:
    var dir = global_position.direction_to(spawn_pos)
    velocity = dir * speed
    move_and_slide()
    if global_position.distance_to(spawn_pos) < 20:
        state = State.PATROL
    if _player_in_range():
        state = State.CHASE

func _pick_patrol_target():
    patrol_target = spawn_pos + Vector2(
        randf_range(-100, 100),
        randf_range(-100, 100)
    )

func _player_in_range() -> bool:
    return player and \
        global_position.distance_to(player.global_position) <
chase_range

```

这个故事告诉我们：- 敌人 AI 的进化是**从本能到智力**的过程 - 反应式 AI = 本能 (看到就追) - FSM = 基本智力 (巡逻/追击/攻击) - 感知系统 = 五感 (视觉/听觉/触觉) - BT = 决策能力 (什么最重要先做什么) - Utility = 直觉

(根据形势判断, 不是死规则) - 群体 AI = 社交能力 (协作、信息共享) - 游戏的"敌人体验" = 这些层次的叠加大于各自之和

第20章 怪物系统演化

Godot 怪物系统：从硬编码到数据驱动

从属性写在脚本里的史莱姆, 到随机生成、属性随等级缩放、携带战利品表的精英怪物。怪物系统的进化 = 怎么把"一个敌人"变成"一套敌人生产流水线"。

目录

1. [原始: 场景里手摆怪物](#)
 2. [怪物 Resource: 数据驱动](#)
 3. [怪物生成器: 刷怪点](#)
 4. [属性随等级缩放](#)
 5. [掉落表: 死亡后掉什么](#)
 6. [怪物波次: 一波一波来](#)
 7. [精英/稀有怪物变异](#)
 8. [最终方案: 最简可用模板](#)
-

1. 原始: 场景里手摆怪物

问题: "在关卡里放几个史莱姆, 玩家打死后消失。"

最初的做法:

```
# Slime.gd
extends CharacterBody2D

var hp = 20
var attack = 5
var speed = 30

func _ready():
    $Label.text = "HP: %d" % hp

func take_damage(dmg: int):
    hp -= dmg
    if hp <= 0:
        die()

func die():
    queue_free()
```

在场景编辑器里：手动拖入 10 个 Slime.tscn, 摆在不同位置。

问题：1. 每个怪物的属性写死在脚本里 – 20 血, 5 攻。想换数值要改脚本。2. 绿色史莱姆和红色史莱姆 – 要复制场景, 改脚本里的 hp 和 attack? 两个场景文件。3. 每个关卡都手摆 – 100 个关卡 × 10 个怪物 = 1000 次手拖。4. 没有掉落 – 打死就消失, 什么都不给。

2. 怪物 Resource: 数据驱动

问题：怪物类型多了以后 ("史莱姆"、"哥布林"、"蝙蝠"), 每个都要一个场景文件, 属性散落在不同脚本里。

Resource 统一数据：

```
# MonsterResource.gd
class_name MonsterResource
extends Resource

@export var name: String = "史莱姆"
```

```

@export var scene: PackedScene          # 怪物场景（外观、动画）
@export var level: int = 1
@export var hp: int = 20
@export var attack: int = 5
@export var defense: int = 2
@export var speed: int = 30
@export var experience: int = 10
@export var drop_table: DropTableResource # 掉落表
@export var color: Color = Color.GREEN

```

在编辑器里创建怪物数据:

```

res://monsters/
├─ Slime.tres          (hp:20, attack:5, speed:30, 绿色)
├─ RedSlime.tres       (hp:30, attack:8, speed:35, 红色)
├─ Goblin.tres         (hp:25, attack:10, speed:40)
├─ Bat.tres            (hp:10, attack:3, speed:80)
└─ Boss.tres          (hp:500, attack:25, speed:20)

```

```

# Monster.gd (通用怪物脚本)
extends CharacterBody2D

var data: MonsterResource
var current_hp: int
var current_level: int

func setup(monster_data: MonsterResource):
    data = monster_data
    current_hp = monster_data.hp
    current_level = monster_data.level

    $Sprite.modulate = monster_data.color
    $Label.text = monster_data.name

func take_damage(dmg: int):
    current_hp -= max(1, dmg - data.defense)
    if current_hp <= 0:
        die()

func die():
    if data.drop_table:
        data.drop_table.spawn(global_position)
    # 经验

```

```
if has_node("/root/GameState"):
    GameState.add_exp(data.experience)
queue_free()
```

现在区分怪物类型 = 不同的 .tres 文件, 不需要复制场景:

```
# 生成怪物:
var slime_data = preload("res://monsters/Slime.tres")
var monster = slime_data.scene.instantiate()
monster.setup(slime_data)
monster.position = spawn_point
add_child(monster)
```

3. 怪物生成器: 刷怪点

问题: 还是要手摆怪物。而且怪物死了就没了, 不会刷新。

Spawner: 在场景里放 Spawner 节点, 它负责生成怪物。

```
# Spawner.gd
extends Node2D

@export var monster_data: MonsterResource
@export var spawn_count: int = 1
@export var spawn_radius: float = 50.0
@export var respawn_time: float = 0.0      # 0 = 不刷新
@export var max_alive: int = 3

var _alive := []

func _ready():
    spawn()

func spawn():
    # 清除已死亡的对象引用
    _alive = _alive.filter(func(m): return is_instance_valid(m))

    var to_spawn = mini(spawn_count - _alive.size(), max_alive -
        _alive.size())
```

```

    for i in to_spawn:
        _spawn_one()

func _spawn_one():
    var monster = monster_data.scene.instantiate()
    monster.setup(monster_data)
    monster.position = global_position + Vector2(
        randf_range(-spawn_radius, spawn_radius),
        randf_range(-spawn_radius, spawn_radius)
    )
    monster.died.connect(_on_monster_died)
    add_child(monster)
    _alive.append(monster)

func _on_monster_died(monster):
    _alive.erase(monster)
    if respawn_time > 0:
        await get_tree().create_timer(respawn_time).timeout
        _spawn_one()

```

Spawner 配置示例:

```

Spawner (Slime):
  怪物: Slime.tres
  数量: 5
  半径: 80
  刷新: 10 秒
  最大存活: 3

```

→ 这个区域永远有 1~3 只史莱姆, 打死 10 秒后刷新

4. 属性随等级缩放

问题: 玩家 10 级了, 打的史莱姆还是 20 血。一刀秒杀, 毫无挑战。

属性缩放: 怪物的属性根据玩家等级或区域等级动态调整。

```

# 在 MonsterResource 里定义每级成长
@export var hp_per_level: int = 5

```

```

@export var attack_per_level: int = 2
@export var defense_per_level: int = 1
@export var exp_per_level: int = 3

func get_scaled_stats(level: int) -> Dictionary:
    return {
        "hp": hp + hp_per_level * (level - 1),
        "attack": attack + attack_per_level * (level - 1),
        "defense": defense + defense_per_level * (level - 1),
        "experience": experience + exp_per_level * (level - 1),
    }

# 或者用倍率缩放:
func get_scaled_by_multiplier(multiplier: float) -> Dictionary:
    return {
        "hp": roundi(hp * multiplier),
        "attack": roundi(attack * multiplier),
        "experience": roundi(experience * multiplier),
    }

```

区域等级: 每个区域设定一个等级

"森林": 1-3 级, "山洞": 5-8 级, "火山": 10-15 级

```

func spawn_monster_in_zone(monster_data: MonsterResource,
    zone_level: int):
    var level = zone_level + randi_range(-1, 1)
    var stats = monster_data.get_scaled_stats(max(1, level))

    var monster = monster_data.scene.instantiate()
    monster.setup(monster_data)
    monster.current_hp = stats["hp"]
    monster.attack = stats["attack"]
    monster.defense = stats["defense"]
    # ...

```

效果: 同一只史莱姆, 在森林里 20 血, 在山洞里 40 血, 在火山里 80 血。玩家等级越高, 遇到的怪物越强。

5. 掉落表: 死亡后掉什么

问题: 怪物死了, 掉什么? 掉几个? 概率多少?

掉落表 Resource:

```
# DropTableResource.gd
class_name DropTableResource
extends Resource

@export var drops: Array[DropEntry]

class DropEntry:
    @export var item: ItemResource
    @export var chance: float = 1.0      # 0~1 概率
    @export var min_count: int = 1
    @export var max_count: int = 1
    @export var guaranteed: bool = false # 必定掉落

# 掉落计算
func roll_drops() -> Array:
    var result = []
    for entry in drops:
        if entry.guaranteed or randf() < entry.chance:
            var count = randi_range(entry.min_count,
entry.max_count)
            if count > 0:
                result.append({ "item": entry.item, "count":
count })
    return result

func spawn(position: Vector2):
    var items = roll_drops()
    for drop in items:
        var pickup = drop.item.scene.instantiate()
        pickup.position = position + Vector2(randf_range(-10,
10), randf_range(-10, 10))
        pickup.setup(drop.item, drop.count)
        get_tree().current_scene.add_child(pickup)
```

Slime.tres 的掉落表:

必定掉落: 粘液 ×1
50% 掉: 小型药水 ×1
10% 掉: 史莱姆王冠 ×1 (稀有)

6. 怪物波次: 一波一波来

问题: "进入房间 → 锁门 → 刷 3 波怪物 → 门开。"

```
# WaveSpawner.gd
extends Node2D

@export var waves: Array[Wave]

func start():
    for wave in waves:
        await _spawn_wave(wave)
        if wave.break_time > 0:
            # 显示"下一波即将到来"
            await
get_tree().create_timer(wave.break_time).timeout

    # 所有波次完成
    $Door.open()

func _spawn_wave(wave: Wave):
    var alive_count = 0

    for entry in wave.monsters:
        for i in entry.count:
            var monster = entry.monster_data.scene.instantiate()
            monster.setup(entry.monster_data)
            monster.position = _get_random_spawn_point()
            monster.died.connect(func(): alive_count -= 1)
            add_child(monster)
            alive_count += 1
            await
get_tree().create_timer(entry.spawn_interval).timeout

    # 等待这一波的所有怪物被消灭
    while alive_count > 0:
```



```

        await get_tree().process_frame

class Wave:
    var monsters: Array[WaveEntry]
    var break_time: float = 3.0 # 波间间隔

class WaveEntry:
    var monster_data: MonsterResource
    var count: int = 1
    var spawn_interval: float = 0.5

```

7. 精英/稀有怪物变异

问题: 所有史莱姆都长一样, 没有"精英怪"和"普通怪"的区别。

变异系统: 普通怪物有概率变异成精英/稀有版本。

```

enum MonsterRank { NORMAL, ELITE, BOSS }

var rank_multipliers = {
    MonsterRank.NORMAL: { "hp": 1.0, "attack": 1.0, "exp": 1.0,
        "size": 1.0, "color": null },
    MonsterRank.ELITE: { "hp": 3.0, "attack": 1.5, "exp": 3.0,
        "size": 1.3, "color": Color.YELLOW },
    MonsterRank.BOSS: { "hp": 8.0, "attack": 2.5, "exp": 10.0,
        "size": 2.0, "color": Color.RED },
}

func apply_rank(monster, rank: MonsterRank):
    var config = rank_multipliers[rank]
    monster.current_hp = roundi(monster.current_hp * config.hp)
    monster.max_hp = monster.current_hp
    monster.attack = roundi(monster.attack * config.attack)
    monster.exp_multiplier = config.exp
    monster.scale = Vector2(config.size, config.size)

    if config.color:
        monster.modulate = config.color

# 精英名字前加前缀

```

```
if rank == MonsterRank.ELITE:
    monster.data.name = "精英 " + monster.data.name
elif rank == MonsterRank.BOSS:
    monster.data.name = "BOSS " + monster.data.name
```

生成时有概率出现精英：

```
func spawn_monster(data: MonsterResource, pos: Vector2):
    var monster = data.scene.instantiate()
    monster.setup(data)
    monster.position = pos

    # 5% 概率变成精英
    if randf() < 0.05:
        apply_rank(monster, MonsterRank.ELITE)
        # 精英额外掉落
        monster.drop_table = elite_drop_table

    add_child(monster)
```

8. 最终方案：最简可用模板

``gdscript

———— **MonsterResource.gd** ————

```
class_name MonsterResource extends Resource
```

```
@export var name: String = "" @export var scene: PackedScene @export
```

```
var hp: int = 20 @export var attack: int = 5 @export var defense: int = 0
```

```
@export var speed: int = 30 @export var exp: int = 10 @export var color:
```

```
Color = Color.WHITE
```

———— **Monster.gd** ————

```
extends CharacterBody2D

var data: MonsterResource var current_hp: int

func setup(d: MonsterResource): data = d current_hp = d.hp
$Sprite.modulate = d.color

func take_damage(dmg: int): current_hp -= max(1, dmg -
data.defense) if current_hp <= 0: die()

func die(): queue_free()
```

———— Spawner.gd ————

```
extends Node2D
```

```
@export var monster_data: MonsterResource @export var count: int = 1
```

```
@export var radius: float = 50
```

```
func _ready(): for i in count: var m = monster_data.scene.instantiate()
```

```
m.setup(monster_data) m.position = global_position +
```

```
Vector2(randf_range(-radius, radius), randf_range(-radius, radius))
```

```
add_child(m)
```

第21章 弹幕系统演化

Godot 弹幕系统：从单发到子弹地狱

弹幕 = 敌人发射的大量子弹, 玩家需要躲避。最早敌人只会射直线单发子弹。后来有了扇形弹、螺旋弹、追踪弹、弹幕 Pattern。弹幕系统的进化 = "怎么让躲避子弹本身成为玩法"。

1. 原始：直线单发

```
func shoot():
    var bullet = preload("res://bullet.tscn").instantiate()
    add_child(bullet)
    bullet.global_position = $Muzzle.global_position
    bullet.direction = Vector2.DOWN
    bullet.speed = 200
```

2. 扇形弹幕

```
func fan_shot(count: int, spread: float):
    var base = Vector2.DOWN
    var step = spread / (count - 1)
    var start = base.rotated(-spread / 2)

    for i in count:
        var bullet = _spawn_bullet()
        bullet.direction = start.rotated(step * i)
        bullet.speed = 200
```

3. 弹幕 Pattern

```
class BulletPattern:
    var name: String
    var type: String          # "circle" / "spiral" / "wave" /
    "aimed"
    var bullet_count: int
    var waves: int            # 重复几轮
    var interval: float       # 每轮间隔
    var speed: float
    var spread: float
```

```
func execute_pattern(pattern: BulletPattern, elapsed: float):
    match pattern.type:
        "circle":
            var step = TAU / pattern.bullet_count
            for i in pattern.bullet_count:
                var dir = Vector2.RIGHT.rotated(step * i)
                _fire(dir, pattern.speed)

        "spiral":
            var base_angle = elapsed * 2.0
            var step = TAU / pattern.bullet_count
            for i in pattern.bullet_count:
                var dir = Vector2.RIGHT.rotated(base_angle +
step * i)
                _fire(dir, pattern.speed)

        "aimed":
            var dir = (player_pos -
$Muzzle.position).normalized()
            for i in pattern.bullet_count:
                var offset = pattern.spread * (i -
pattern.bullet_count / 2) / pattern.bullet_count
                _fire(dir.rotated(offset), pattern.speed)
```


4. 子弹池

```
# 大量子弹用对象池
var bullet_pool = ObjectPool.new(preload("res://bullet.tscn"),
50)

func _fire(direction: Vector2, speed: float):
    var bullet = bullet_pool.get()
    if not bullet: return
    bullet.global_position = $Muzzle.global_position
    bullet.init(direction, speed)
```

5. 最终方案模板

```
# BulletManager.gd (挂载到 BOSS)
# 核心: Pattern → 多波发射 → 对象池复用

var patterns := []
var wave := 0
var pool := ObjectPool.new(bullet_scene, 100)

func _process(delta):
    _timer += delta
    var pattern = patterns[wave % patterns.size()]
    if _timer >= pattern.interval:
        execute_pattern(pattern, _timer)
        wave += 1
        _timer = 0.0

func _fire(dir: Vector2, speed: float):
    var b = pool.get()
    if b:
        b.global_position = $Muzzle.position
        b.fire(dir, speed)
```

第22章 掉落系统演化

Godot 掉落系统: 从随机到战利品表

怪物死了, 掉出东西。这是游戏给玩家的即时反馈。从"随机掉一个"到完整的战利品表体系, 每一步都跟"这个掉落物有没有价值"相关。

1. 原始: 代码硬编码

```
func die():
    if randf() < 0.3:
        spawn_item("hp_potion")
    if randf() < 0.1:
        spawn_item("sword")
    queue_free()
```

改掉落概率要改代码, 每个怪物的概率散落在不同脚本。

2. 掉落表 (Drop Table)

```
class DropEntry:
    var item_id: String
    var weight: int          # 权重 (越高概率越大)
    var min_count: int = 1
    var max_count: int = 1
    var rarity: String = "common"

var drop_table = [
    DropEntry.new("hp_potion", 50, 1, 2),
    DropEntry.new("mp_potion", 30, 1, 1),
    DropEntry.new("iron_sword", 10),
```

```

    DropEntry.new("rare_ring", 5),
    DropEntry.new("legendary_helm", 1),
]

```

3. 分层掉落

```

# 必定掉落 + 概率掉落 + 稀有掉落
func get_drops() -> Array:
    var result = []

    # 必定掉落
    result.append({ "item": "bone", "count": 1 })

    # 概率掉落 (抽 N 次)
    for entry in drop_table:
        if randf() < entry.chance:
            result.append({ "item": entry, "count":
randi_range(entry.min, entry.max) })

    # 稀有掉落 (权重抽选)
    result.append(weighted_pick(rare_drops))

    return result

```

4. 最终方案模板

```

# DropTableResource.gd
class_name DropTableResource
extends Resource

@export var guaranteed_drops: Array[GuaranteedEntry]
@export var random_drops: Array[RandomEntry]

class GuaranteedEntry:
    @export var item: ItemResource
    @export var count: int = 1

class RandomEntry:

```

```

@export var item: ItemResource
@export var weight: float = 1.0
@export var min_count: int = 1
@export var max_count: int = 1

func roll() -> Array:
    var result = []

    # 必定掉落
    for entry in guaranteed_drops:
        result.append({ "item": entry.item, "count":
entry.count })

    # 随机掉落 (全部独立概率)
    var total_weight = 0
    for entry in random_drops:
        total_weight += entry.weight

    for entry in random_drops:
        if randf() < entry.weight / total_weight:
            result.append({
                "item": entry.item,
                "count": randi_range(entry.min_count,
entry.max_count)
            })

    return result

```

第23章 经验系统演化

Godot 经验系统：从打怪升级到曲线成长

经验值 = 玩家投入时间的度量衡。攒够经验 → 升级 → 变强 → 打更难的怪 → 获得更多经验。这个循环是 RPG 的核心。怎么让这个循环不卡住、不无聊、有期待感,全靠经验曲线。

1. 原始：线性升级

```
var level = 1
var exp = 0
var exp_to_next = 100

func add_exp(amount):
    exp += amount
    if exp >= exp_to_next:
        level += 1
        exp -= exp_to_next
        exp_to_next = 100 # 每级固定 100
```

问题: 每级需求一样。低级玩家打低级怪 3 只升级, 高级玩家打高级怪也是 3 只升级。没有"越升级越难"的成长感。

2. 经验曲线

```
# 每级需要的经验递增
func get_exp_for_level(lv: int) -> int:
    return 100 + (lv - 1) * 50
# 1级: 100, 2级: 150, 3级: 200, 10级: 550...
```

```
# 常用曲线：二次函数
func get_exp_quadratic(lv: int) -> int:
    return 100 + lv * lv * 10
    # 1级: 110, 5级: 350, 10级: 1100, 20级: 4100

# 指数曲线（前期快后期极慢）
func get_exp_exponential(lv: int) -> int:
    return roundi(100 * pow(1.2, lv - 1))
    # 1级: 100, 10级: 516, 20级: 3194
```

3. 动态等级：怪物随玩家等级

```
# 怪物属性根据玩家等级调整
func spawn_monster(zone_level: int):
    var player_lv = GameState.level
    var effective_lv = max(zone_level, player_lv - 2)
    monster.setup(monster_data, effective_lv)
```

4. 最终方案模板

```
# ExpSystem.gd
extends Node

signal leveled_up(new_level: int)
signal exp_changed(current: int, needed: int)

var level := 1
var exp := 0

func get_exp_needed(lv: int) -> int:
    return roundi(100 * pow(1.15, lv - 1))

func add_exp(amount: int):
    exp += amount
    var needed = get_exp_needed(level)
    while exp >= needed:
        exp -= needed
```



```
        level += 1
        needed = get_exp_needed(level)
        leveled_up.emit(level)
        exp_changed.emit(exp, needed)

func get_level_progress() -> float:
    return float(exp) / get_exp_needed(level)
```

第24章 布娃娃系统演化

Godot 布娃娃系统: 从固定动画到物理死亡

布娃娃 = 角色死亡时用物理模拟代替固定动画。最早死亡就是播放一个"倒地"动画。后来有了布娃娃: 尸体受重力影响, 被击中时关节物理模拟。布娃娃系统的进化 = "怎么让死亡看起来更真实"。

1. 原始: 固定死亡动画

```
func die():
    $AnimationPlayer.play("death")
    await $AnimationPlayer.animation_finished
    queue_free()
```

2. 简易布娃娃 (RigidBody 替代)

```
func die():
    # 隐藏本体
    hide()
    $CollisionShape2D.disabled = true

    # 生成 RigidBody 碎片
    var parts = [
        $SpriteBody, $SpriteHead, $SpriteArmL, $SpriteArmR
    ]
    for part in parts:
        var rb = RigidBody2D.new()
        rb.gravity_scale = 1.0
        var sprite = Sprite2D.new()
        sprite.texture = part.texture
```

```

        rb.add_child(sprite)
        rb.global_position = part.global_position
        get_parent().add_child(rb)

        # 随机飞出去
        var force = Vector2(randf_range(-200, 200),
randf_range(-300, -100))
        rb.apply_impulse(force)

        # 几秒后清理
        await get_tree().create_timer(5.0).timeout
        for rb in get_tree().get_nodes_in_group("ragdoll"):
            rb.queue_free()

```

3. PinJoint 连接 (2D)

```

# 把身体各部分用 PinJoint 连起来
func create_ragdoll():
    var head = _create_part("head")
    var body = _create_part("body")
    var arm_l = _create_part("arm_l")
    var arm_r = _create_part("arm_r")

    # 头 → 身体
    var j1 = PinJoint2D.new()
    j1.global_position = $Neck.global_position
    j1.node_a = head.get_path()
    j1.node_b = body.get_path()
    add_child(j1)

    # 手臂 → 身体
    var j2 = PinJoint2D.new()
    j2.global_position = $ShoulderL.global_position
    j2.node_a = arm_l.get_path()
    j2.node_b = body.get_path()
    add_child(j2)
    # ... 以此类推

```

4. 最终方案模板

```
# RagdollManager.gd
# 2D 简易方案: 角色死亡 → 隐藏 → 生成多个 RigidBody2D 碎片
# 3D 方案: PhysicalBone + SkeletonIK

func spawn_ragdoll(character: Node2D):
    var container = Node2D.new()
    get_tree().current_scene.add_child(container)
    for slot in character.get_ragdoll_slots():
        var rb = RigidBody2D.new()
        rb.gravity_scale = 1.0
        var tex = Sprite2D.new()
        tex.texture = slot.texture
        rb.add_child(tex)
        rb.global_position = slot.global_position
        rb.apply_impulse(Vector2(randf_range(-150, 150),
randf_range(-200, -50)))
        container.add_child(rb)
    await get_tree().create_timer(4.0).timeout
    container.queue_free()
```

第25章 召唤系统演化

Godot 召唤系统：从帮手到独立作战单位

召唤 = 战斗中召唤额外的单位协助作战。宠物是长期跟随, 召唤是临时出现。最早召唤是"放个技能出个火球"。后来有了召唤物AI、持续时间、数量限制、献祭机制。召唤系统的进化 = "怎么让一个人的战斗变成一支军队"。

1. 原始：技能特效

```
# 召唤术 = 一个火球飞出去  
# 没有"召唤物"的概念
```

2. 临时单位

```
func summon(scene: PackedScene, position: Vector2, duration:  
float):  
    var unit = scene.instantiate()  
    add_child(unit)  
    unit.global_position = position  
    unit.init(owner_level)  
  
    # 持续时间结束后消失  
    var tween = create_tween()  
    tween.tween_interval(duration)  
    tween.tween_callback(unit.queue_free)
```

3. 召唤物AI + 数量限制

```
class SummonData:
    var scene: PackedScene
    var max_count: int = 3          # 最多同时存在几个
    var duration: float = 10.0
    var ai_type: String = "nearest" # "nearest" / "taunt" /
    "explode"
    var stats_scale: float = 0.6    # 继承主人 60% 属性

func summon(summon_id: String, position: Vector2):
    var data = SummonDB.get(summon_id)
    # 检查当前召唤物数量
    var active = _get_active_summon().filter(func(s): return
    s.id == summon_id)
    if active.size() >= data.max_count:
        return # 达到上限, 不能再召

    var unit = data.scene.instantiate()
    add_child(unit)
    unit.global_position = position
    unit.stats = _calc_summon_stats(data)
    unit.ai_type = data.ai_type

    # 超时自动消失
    _summon_timers[unit] = data.duration

func _process(delta):
    for unit in _summon_timers.keys():
        _summon_timers[unit] -= delta
        if _summon_timers[unit] <= 0:
            unit.queue_free()
            _summon_timers.erase(unit)
```

4. 最终方案模板

```
# SummonManager.gd (Autoload 或挂载到角色)

var active := []
```



```

func summon(id: String, pos: Vector2) -> Node2D:
    var cfg = SummonDB.get(id)
    if active_count(id) >= cfg.max: return null
    var unit = cfg.scene.instantiate()
    get_tree().current_scene.add_child(unit)
    unit.global_position = pos
    unit.setup(_scaled_stats(cfg))
    active.append({ "node": unit, "expire":
Time.get_ticks_msec() + int(cfg.duration * 1000) })
    return unit

func active_count(id: String) -> int:
    return active.filter(func(a): return a.node.id == id and
is_instance_valid(a.node)).size()

func cleanup():
    var now = Time.get_ticks_msec()
    active = active.filter(func(a):
        if not is_instance_valid(a.node): return false
        if now > a.expire: a.node.queue_free(); return false
        return true
    )

```

第26章 副本系统演化

Godot 副本系统：从固定关卡到动态生成

副本 = 玩家进入一个独立的战斗场景。最早副本就是"关卡 1, 关卡 2, 关卡 3"。后来有了随机地图、难度选择、次数限制、扫荡。副本系统的进化 = "怎么让每次进副本都有新鲜感"。

1. 原始：固定关卡

```
# 场景 1：野外  
# 场景 2：山洞  
# 场景 3：Boss 房间  
# 玩家按顺序打，打完就没了
```

2. 副本入口 + 实例化

```
# 在村里有个传送门，进入后加载副本场景  
func enter_dungeon(dungeon_id: String):  
    var scene = load("res://dungeons/" + dungeon_id + ".tscn")  
    var instance = scene.instantiate()  
    get_tree().current_scene.queue_free()  
    get_tree().root.add_child(instance)  
    get_tree().current_scene = instance
```

3. 副本数据结构

```
class Dungeon:  
    var id: String
```

```

var name: String
var min_level: int
var max_players: int = 1
var difficulty_levels: Array[DifficultyConfig]
var entry_cost: Dictionary      # { "stamina": 10 }
var daily_limit: int = 0        # 0 = 不限
var rewards: Array[DungeonReward]

class DifficultyConfig:
    var name: String          # "普通" / "困难" / "地狱"
    var enemy_scale: float    # 1.0 / 1.5 / 2.5
    var drop_rate: float      # 1.0 / 1.2 / 1.5
    var recommend_level: int

```

4. 随机生成 (简易 Roguelite)

```

func generate_dungeon(dungeon: Dungeon, seed_value: int) ->
Node2D:
    seed(seed_value)
    var map = Node2D.new()
    var rooms = _generate_rooms(randi_range(5, 10))
    for i in rooms.size():
        var room = _create_room(rooms[i])
        map.add_child(room)
        if i > 0:
            var corridor = _connect_rooms(rooms[i-1], rooms[i])
            map.add_child(corridor)
    # 放怪物
    var enemy_count = randi_range(3, 6)
    for i in enemy_count:
        var enemy = _spawn_enemy(dungeon.difficulty)
        map.add_child(enemy)
    # 放宝箱
    map.add_child(_spawn_chest())
    return map

```

5. 扫荡功能

```
func sweep(dungeon_id: String, times: int) -> Array:
    var dungeon = DungeonDB.get(dungeon_id)
    var cost = dungeon.entry_cost
    var total_stamina = cost.get("stamina", 0) * times
    if not StaminaManager.spend(total_stamina):
        return []

    var results = []
    for i in times:
        var reward = _simulate_drop(dungeon)
        if reward:
            InventoryManager.add(reward.item_id, reward.count)
            results.append(reward)
    return results
```

6. 最终方案模板

```
# DungeonManager.gd (Autoload)
# 核心流程：进入 → 生成 → 战斗 → 结算

func enter(dungeon_id: String, difficulty: int = 0) -> bool:
    var dungeon = DungeonDB.get(dungeon_id)
    if PlayerData.level < dungeon.min_level:
        return false
    if dungeon.daily_limit > 0 and _today_count(dungeon_id) >=
dungeon.daily_limit:
        return false
    # 扣体力
    if not
StaminaManager.spend(dungeon.entry_cost.get("stamina", 0)):
        return false
    # 加载场景
    _load_dungeon_scene(dungeon, difficulty)
    return true

func on_dungeon_clear(rewards: Array):
    for r in rewards:
```

```
        InventoryManager.add(r.item_id, r.count)
    _record_clear(dungeon_id)
    # 回城
    SceneManager.switch("town")
```

第27章 关卡系统演化

Godot 关卡/章节系统: 从线性到世界地图

关卡 = 游戏的进度结构。最早是 1-1, 1-2, 1-3... 线性推进。后来有了世界地图、章节、难度选择、星级评价。关卡系统的进化 = "怎么让游戏进度有结构感"。

1. 原始: 线性关卡

```
# 场景1 → 场景2 → 场景3  
# 打完上一关解锁下一关
```

2. 章节 + 关卡结构

```
class Chapter:  
    var id: String  
    var name: String  
    var stage_ids: Array[String]  
    var unlock_condition: String # "clear_prev_chapter" /  
    "player_level_10"  
    var reward_on_complete: Dictionary  
  
class Stage:  
    var id: String  
    var name: String  
    var scene_path: String  
    var recommended_level: int  
    var stamina_cost: int = 10  
    var stars: Array[StageCondition] # [通关, 限时, 无死亡]
```



```
var drop_table: DropTableResource
var first_clear_reward: Dictionary
```

```
# 关卡选择（世界地图）
func get_available_stages(chapter_id: String) -> Array:
    var chapter = ChapterDB.get(chapter_id)
    var stages = []
    for sid in chapter.stage_ids:
        var stage = StageDB.get(sid)
        if _is_stage_unlocked(sid):
            stages.append(stage)
    return stages

func _is_stage_unlocked(stage_id: String) -> bool:
    var stage = StageDB.get(stage_id)
    for cond in stage.unlock_conditions:
        if not _check_condition(cond):
            return false
    return true
```

3. 星级评价 + 扫荡限制

```
func on_stage_clear(stage_id: String, time: float, deaths: int):
    var stars = 1 # 通关 = 1 星
    if time < StageDB.get(stage_id).time_limit:
        stars += 1 # 限时完成 = 2 星
    if deaths == 0:
        stars += 1 # 无死亡 = 3 星

    var prev = _save.get_best(stage_id)
    if stars > prev.stars:
        _save.set_best(stage_id, stars)
        if stars >= 3:
            _unlock_sweep(stage_id) # 三星解锁扫荡

func can_sweep(stage_id: String) -> bool:
    return _save.get_best(stage_id) >= 3
```

4. 最终方案模板

```
# StageManager.gd (Autoload)
# 数据结构: Chapter → Stage
# 关卡节点: 世界地图上每个 Stage 显示锁/可打/已完成

func get_progress() -> Dictionary:
    return { "chapter": _save.current_chapter, "stage":
_save.current_stage }

func get_star_total() -> int:
    var total = 0
    for sid in _save.stars:
        total += _save.stars[sid]
    return total

# 关卡流程:
# enter_stage(id) → 扣体力 → 加载场景 → 战斗 → 结算 → 保存进度
```

第28章 钓鱼系统演化

Godot 钓鱼系统: 从概率到 mini-game

钓鱼 = 游戏里的休闲小游戏。最早是"点一下, 概率出鱼"。后来有了抛竿、等待、拉杆 QTE、不同鱼种、鱼饵系统。钓鱼系统的进化 = "怎么让钓鱼本身就好玩"。

1. 原始: 概率出鱼

```
func fish() -> ItemResource:
    if randf() < 0.3:
        return ItemDB.get("common_fish")
    return null
```

2. 等待 + 按键收杆

```
extends Node2D

var is_fishing = false
var has_bite = false

func start_fishing():
    is_fishing = true
    $Bobber.visible = true
    $Bobber.position = _get_water_pos()
    # 等待随机时间鱼上钩
    await get_tree().create_timer(randf_range(2, 8)).timeout
    if is_fishing:
        has_bite = true
        $Bobber.play("splash") # 浮漂动
```

```

func _input(event):
    if event.is_action_pressed("interact") and has_bite:
        _catch_fish()
    elif event.is_action_pressed("interact") and is_fishing:
        stop_fishing() # 提前收竿

func _catch_fish():
    var fish = _random_fish()
    InventoryManager.add(fish.id, 1)
    is_fishing = false
    has_bite = false
    $Bobber.visible = false

```

3. 拉杆 QTE

```

var progress = 0.0
var fish_position = 0.0
var bar_width = 200
var fish_width = 40

func _process(delta):
    if not is_reeling: return

    # 鱼随机左右移动
    fish_position += randf_range(-100, 100) * delta
    fish_position = clamp(fish_position, 0, bar_width -
fish_width)

    # 玩家按住按钮进度条向右，松开向左
    if Input.is_action_pressed("action"):
        progress += 150 * delta
    else:
        progress -= 50 * delta
    progress = clamp(progress, 0, bar_width - fish_width)

    # 判定：绿条和鱼重叠 → 进度增加
    var overlap = abs(progress - fish_position) < fish_width
    if overlap:
        reel_progress += 20 * delta
    else:

```

```
reel_progress -= 10 * delta

reel_progress = clamp(reel_progress, 0, 100)
if reel_progress >= 100:
    _catch_fish()
elif reel_progress <= 0:
    _fish_escaped()
```

4. 最终方案模板

```
# 钓鱼系统核心三阶段：
# 1. 抛竿：选择钓点，消耗鱼饵
# 2. 等待：随机时间后鱼咬钩（浮漂动画提示）
# 3. 拉杆：QTE 保持绿条对齐鱼的位置，进度满则成功

# 鱼种由以下因素决定：
# - 钓点（海边/河边/湖）
# - 鱼饵类型
# - 时间（白天/夜晚）
# - 天气

func _determine_fish() -> FishResource:
    var pool = FishDB.get_for_location(location)
    pool = pool.filter(func(f): return f.bait == current_bait)
    if TimeManager.is_night():
        pool = pool.filter(func(f): return f.active_at_night)
    return pool.pick_random()
```

第29章 属性系统演化

Godot 属性系统：从单数值到多维度派生

属性 = 角色的各项能力值。最早只有 HP 和攻击力。后来有了力量/敏捷/智力、暴击/闪避/穿透、百分比加成、buff 叠加。属性系统的进化 = "怎么用公式支撑起整个战斗系统"。

1. 原始：硬编码属性

```
var hp = 100
var attack = 10
var defense = 5
```

2. 基础属性 + 派生属性

```
class Stats:
    # 基础属性（由等级/装备决定）
    var strength: int = 10    # 力量
    var agility: int = 10     # 敏捷
    var intellect: int = 10   # 智力

    # 派生属性（由基础属性计算）
    var max_hp: int:          get: return 100 + strength * 10
    var attack: int:          get: return strength * 2 + agility
    * 0.5
    var defense: int:         get: return strength * 1 + agility
    * 0.3
    var crit_rate: float:     get: return agility * 0.002
    var crit_damage: float:   get: return 1.5 + strength * 0.005
    var attack_speed: float:  get: return 1.0 + agility * 0.01
```


3. 百分比加成

```
class StatSystem:
    var base: Dictionary = {}      # 基础值
    var flat_bonus: Dictionary = {} # 固定加成
    var percent_bonus: Dictionary = {} # 百分比加成

    func get_final(stat: String) -> float:
        var value = base.get(stat, 0.0)
        value += flat_bonus.get(stat, 0.0)
        value *= 1.0 + percent_bonus.get(stat, 0.0)
        return value

    func add_flat(stat: String, amount: float):
        flat_bonus[stat] = flat_bonus.get(stat, 0.0) + amount

    func add_percent(stat: String, amount: float):
        percent_bonus[stat] = percent_bonus.get(stat, 0.0) +
amount

    func remove_flat(stat: String, amount: float):
        flat_bonus[stat] = flat_bonus.get(stat, 0.0) - amount

    func remove_percent(stat: String, amount: float):
        percent_bonus[stat] = percent_bonus.get(stat, 0.0) -
amount
```

4. Buff 叠加/覆盖

```
class StatModifier:
    var source: String      # "skill_bash" /
"equipment_sword" / "buff_potion"
    var stat: String
    var value: float
    var type: String        # "flat" / "percent"
    var duration: float = 0 # 0 = 永久

    func apply(system: StatSystem):
        if type == "flat":
```

```

        system.add_flat(stat, value)
    else:
        system.add_percent(stat, value)

func remove(system: StatSystem):
    if type == "flat":
        system.remove_flat(stat, value)
    else:
        system.remove_percent(stat, value)

```

```

# 同源加成不叠加（取最高）：
func add_modifier(mod: StatModifier):
    # 检查是否有同源同属性的旧 modifier
    var existing = _modifiers.filter(func(m):
        return m.source == mod.source and m.stat == mod.stat
    )
    if existing.size() > 0:
        # 覆盖：移除旧的，添加新的
        existing[0].remove(self)
        _modifiers.erase(existing[0])
    mod.apply(self)
    _modifiers.append(mod)

```

5. 最终方案模板

```

# StatSystem.gd（可复用组件）
# 核心：base → flat_bonus → percent_bonus → final

class StatSystem:
    var _base := {}
    var _flat := {}
    var _pct := {}
    var _mods := []
    var _dirty := true
    var _cache := {}

    func get(stat: String) -> float:
        if _dirty: _recalc()
        return _cache.get(stat, 0.0)

    func _recalc():

```

```

        _cache.clear()
    for stat in _base:
        var v = _base[stat]
        v += _flat.get(stat, 0.0)
        v *= 1.0 + _pct.get(stat, 0.0)
        _cache[stat] = v
    _dirty = false

func set_base(stat: String, value: float):
    _base[stat] = value
    _dirty = true

func add_modifier(mod: StatModifier):
    _mods.append(mod)
    if mod.type == "flat":
        _flat[mod.stat] = _flat.get(mod.stat, 0.0) +
mod.value
    else:
        _pct[mod.stat] = _pct.get(mod.stat, 0.0) + mod.value
    _dirty = true

```

第30章 仇恨系统演化

Godot 仇恨系统：从就近攻击到仇恨值

仇恨 = 怪物决定打谁。最早怪物打最近的人。后来有了仇恨值、坦克拉怪、OT（仇恨失控）、仇恨列表。仇恨系统的进化 = "怎么让肉盾能拉住怪, 输出不会死"。

1. 原始：就近攻击

```
func _find_target() -> Node2D:
    var nearest = null
    var min_dist = INF
    for player in get_tree().get_nodes_in_group("player"):
        var d =
            global_position.distance_squared_to(player.global_position)
        if d < min_dist:
            min_dist = d
            nearest = player
    return nearest
```

2. 仇恨值

```
class ThreatEntry:
    var target: Node2D
    var threat: float = 0

var threat_list := []

func add_threat(target: Node2D, amount: float):
    for entry in threat_list:
```

```

        if entry.target == target:
            entry.threat += amount
            return
    threat_list.append(ThreatEntry.new(target, amount))

func get_primary_target() -> Node2D:
    if threat_list.is_empty(): return null
    threat_list.sort_custom(func(a, b): return a.threat >
b.threat)
    return threat_list[0].target

# 技能产生仇恨:
# 战士: 嘲讽技能 +500 仇恨
# 治疗: 治疗产生 50% 仇恨
# 输出: 伤害产生 100% 仇恨

```

3. 仇恨修正

```

class ThreatManager:
    var threat_list := []
    var threat_multiplier := {} # 对特定角色的仇恨倍率

    func add_threat(source: Node2D, base: float, type: String =
"damage"):
        var mult = threat_multiplier.get(source, 1.0)
        match type:
            "damage":    mult *= 1.0    # 伤害: 100%
            "heal":      mult *= 0.5    # 治疗: 50%
            "taunt":     mult *= 5.0    # 嘲讽: 500%
        var actual = base * mult
        _add(source, actual)

    func get_primary_target() -> Node2D:
        threat_list.sort_custom(func(a, b): return a.threat >
b.threat)
        return threat_list[0].target if threat_list.size() > 0
        else null

    func reset():
        threat_list.clear()

```

OT 机制：当输出的仇恨超过坦克 110% 时，BOSS 转头打输出

4. 最终方案模板

```
# ThreatManager.gd (挂载到怪物)
# 核心：仇恨值列表 → 取最高 → 每帧切换目标

var _threat := {}

func add(target: Node2D, amount: float, type: String =
"damage"):
    var mult = _get_multiplier(type)
    _threat[target] = _threat.get(target, 0.0) + amount * mult

func get_target() -> Node2D:
    var best: Node2D = null
    var most: float = 0.0
    for t in _threat:
        if _threat[t] > most and is_instance_valid(t):
            most = _threat[t]
            best = t
    return best

func check_ot(tank: Node2D) -> Node2D:
    var tank_threat = _threat.get(tank, 0.0)
    for t in _threat:
        if t != tank and _threat[t] > tank_threat * 1.1:
            return t # OT! 目标转向
    return null

func _get_multiplier(type: String) -> float:
    match type:
        "damage": return 1.0
        "heal": return 0.5
        "taunt": return 5.0
    return 1.0
```

第31章 状态效果系统演化

Godot 异常状态系统：从硬编码到可堆叠

状态 = 中毒/灼烧/冰冻/眩晕/诅咒等临时效果。最早写在战斗代码里 if branch。后来有了状态数据类、持续时间、堆叠规则、驱散。状态的进化 = "怎么让 debuff 能灵活组合"。

1. 原始: if-else

```
func apply_poison(target):
    target.hp -= 5
    await get_tree().create_timer(1.0).timeout
    target.hp -= 5 # 持续掉血写死
```

2. StatusEffect 数据类

```
class StatusEffect:
    var id: String          # "poison" / "burn" / "stun"
    var duration: float     # 总持续时间
    var tick_interval: float # 每多少秒触发一次
    var tick_damage: int    # 每次触发伤害
    var remaining: float    # 剩余时间
    var timer: float = 0    # 计时器

    func apply(target):
        match id:
            "stun": target.can_act = false
            "slow": target.speed *= 0.5

    func tick(target, delta):
```

```

        timer += delta
    if timer >= tick_interval:
        timer = 0
        if tick_damage > 0:
            target.take_damage(tick_damage)

    func expire(target):
        match id:
            "stun": target.can_act = true
            "slow": target.speed = target.base_speed

```

3. 状态管理器

```

class StatusManager:
    var _effects: Array[StatusEffect] = []

    func add(effect: StatusEffect):
        # 同类刷新持续时间
        var existing = _effects.filter(func(e): return e.id ==
effect.id)
        if existing.size() > 0:
            existing[0].remaining = effect.duration # 刷新
            return
        effect.apply(owner)
        _effects.append(effect)

    func remove(id: String):
        var effect = _effects.filter(func(e): return e.id == id)
        if effect.size() > 0:
            effect[0].expire(owner)
            _effects.erase(effect[0])

    func process(delta):
        for e in _effects.duplicate():
            e.tick(owner, delta)
            e.remaining -= delta
            if e.remaining <= 0:
                e.expire(owner)
                _effects.erase(e)

```

4. 堆叠规则

```
class StatusEffect:
    enum StackRule { NONE, REFRESH, INTENSIFY, ACCUMULATE }

    var stack_rule: StackRule = StackRule.NONE
    var stacks: int = 1
    var max_stacks: int = 5

    func apply_stacks(target):
        match stack_rule:
            StackRule.NONE:
                # 同名不叠加
                pass
            StackRule.REFRESH:
                # 刷新持续时间
                pass
            StackRule.INTENSIFY:
                # 每层增强效果 (伤害 * stacks)
                tick_damage = base_damage * stacks
            StackRule.ACCUMULATE:
                # 最大层数触发特殊效果 (5层引爆)
                if stacks >= max_stacks:
                    target.take_damage(big_explosion_damage)
                stacks = 0
```

5. 最终方案模板

```
# StatusEffect.gd (Resource)
class_name StatusEffect extends Resource

@export var id: String
@export var duration: float = 3.0
@export var tick_interval: float = 1.0
@export var tick_damage: int = 0
@export var stack_rule: String = "refresh" # none/refresh/
intensify/accumulate

# StatusManager.gd
```

```
var _effects := {}

func apply(effect: StatusEffect, target):
    var existing = _effects.get(effect.id)
    if existing:
        existing.remaining = effect.duration
        existing.stacks = mini(existing.stacks + 1, 5)
        return
    effect.apply(target)
    _effects[effect.id] = effect.duplicate()

func clear(target):
    for e in _effects.values(): e.expire(target)
    _effects.clear()
```

第32章 霸体韧性系统演化

Godot 霸体/韧性系统：从一碰就断到钢铁之躯

霸体 = 受击时不被中断动作。最早任何攻击都能打断玩家。后来有了霸体状态、韧性值、破霸体阈值。霸体的进化 = "怎么区分轻击和重击的打断效果"。

1. 原始：一碰就断

```
# 任何伤害都打断当前动作
func take_damage(amount):
    if is_attacking:
        interrupt_attack() # 攻击被打断
    hp -= amount
```

2. 霸体状态

```
# 某些动作期间不可被打断
enum SuperArmor { NONE, LIGHT, HEAVY, IMMUNE }

var super_armor: SuperArmor = SuperArmor.NONE

func take_damage(amount, hit_force: int):
    if super_armor == SuperArmor.IMMUNE:
        return # 完全不被打断
    if super_armor == SuperArmor.HEAVY and hit_force < 3:
        return # 轻击不中断
    if super_armor == SuperArmor.LIGHT and hit_force < 1:
        return # 只防最轻的攻击
    interrupt_action()
```

3. 韧性值 (Poise)

```
# 类似耐力条的韧性系统
var max_poise := 100
var current_poise := 100
var poise_recovery := 10 # 每秒恢复

func take_damage(amount, poise_damage: int):
    current_poise -= poise_damage
    if current_poise <= 0:
        stagger() # 破韧 → 被打断 + 硬直
        current_poise = 0

func _process(delta):
    current_poise = min(current_poise + poise_recovery * delta,
max_poise)

# 轻型攻击: poise_damage = 10 (7次破韧)
# 重型攻击: poise_damage = 40 (3次破韧)
# 蓄力重击: poise_damage = 100 (一击破韧)
```

4. 最终方案模板

```
# PoiseSystem.gd
extends Node

@export var max_poise := 100.0
@export var recovery_rate := 15.0
@export var broken_time := 2.0

var poise := 100.0
var is_broken := false
var broken_timer := 0.0

func hit(poise_damage: float) -> bool:
    """返回是否被打断"""
    if is_broken:
        return true
    poise -= poise_damage
```

```

if poise <= 0:
    is_broken = True
    broken_timer = broken_time
    poise = 0
    return True # 破韧 → 硬直
return False # 没破韧 → 继续动作

func _process(delta):
    if is_broken:
        broken_timer -= delta
        if broken_timer <= 0:
            is_broken = False
    poise = min(poise + recovery_rate * delta, max_poise)

```


第33章 受击反馈系统演化

Godot 受击反馈系统：从血条闪烁到击飞

受击反馈 = 角色被击中时的表现。最早只是血条变一下。后来有了闪白、后仰、击退、击飞、受击状态机。受击反馈的进化 = "怎么让每一次命中都有打击感"。

1. 原始：血条闪烁

```
func take_damage(amount):
    hp -= amount
    $HPBar.modulate = Color.RED # 变红
    await get_tree().create_timer(0.1).timeout
    $HPBar.modulate = Color.WHITE # 恢复
```

2. 受击动画

```
# 播放受击动画帧
func take_damage(amount, hit_direction: Vector2):
    hp -= amount
    if hp > 0:
        $AnimationPlayer.play("hit_front" if hit_direction.x > 0
        else "hit_back")
    else:
        $AnimationPlayer.play("die")

# 受击动画期间进入 hurt 状态
# 状态机：IDLE → HURT → IDLE
```

3. 击退 + 击飞

```
func apply_hit_force(hit_direction: Vector2, force: float):  
    velocity = hit_direction * force  
    move_and_slide()
```

```
# 轻击: force = 50, 小退一步  
# 重击: force = 300, 击飞撞墙  
# 浮空: force 含向上分量, 落地后弹起
```

```
# KnockbackData  
var knockback_data = {  
    "light": { "force": 50, "duration": 0.1, "stun": 0.2 },  
    "heavy": { "force": 300, "duration": 0.3, "stun": 0.8 },  
    "launch": { "force": 400, "duration": 0.5, "stun": 1.2 },  
    "drag": { "force": 200, "duration": 0.5, "stun": 0.3 },  
}
```

4. 最终方案模板

```
# HitReaction.gd  
extends Node  
  
@export var flash_color := Color.WHITE  
@export var flash_duration := 0.05  
  
func on_hit(data: Dictionary):  
    # 闪白  
    _flash()  
    # 播放受击动画  
    $AnimationPlayer.play("hit")  
    # 击退  
    _apply_knockback(data.direction, data.force)  
    # 屏幕震动  
    ShakeManager.shake(data.shake_intensity)  
  
func _flash():  
    var mat = $Sprite.material as ShaderMaterial  
    mat.set_shader_parameter("flash", flash_color)
```

```
await get_tree().create_timer(flash_duration).timeout  
mat.set_shader_parameter("flash", Color.TRANSPARENT)
```

第34章 锁定系统演化

Godot 锁定系统: 从自瞄到智能索敌

锁定 = 自动面向目标, 攻击自动追踪。最早全靠手动瞄准。后来有了近战锁定、远程自瞄、锁敌切换、脱锁规则。锁定系统的进化 = "怎么让玩家不用对着像素点打架"。

1. 原始: 手动瞄准

```
# 玩家自己控制攻击方向
# 面向鼠标方向射击/挥砍
func attack():
    var dir = (get_global_mouse_position() -
global_position).normalized()
    shoot(dir)
```

2. 最近目标锁定

```
var locked_target: Node2D = null

func find_nearest_enemy():
    var nearest = null
    var min_dist = INF
    for enemy in get_tree().get_nodes_in_group("enemy"):
        var d =
global_position.distance_squared_to(enemy.global_position)
        if d < min_dist:
            min_dist = d
            nearest = enemy
    locked_target = nearest
```

```

func attack():
    if locked_target:
        var dir = (locked_target.global_position -
global_position).normalized()
        shoot(dir)

```

3. 锁定 UI + 切换

```

func _process(delta):
    if locked_target and is_instance_valid(locked_target):
        # 锁定指示器跟随目标
        $LockOnIcon.global_position =
locked_target.global_position
        # 自动面向
        face_target(delta)
    else:
        $LockOnIcon.visible = false

func _input(event):
    if event.is_action_pressed("lock_on"):
        find_nearest_enemy()
    if event.is_action_pressed("switch_target"):
        cycle_target()

func cycle_target():
    var enemies = get_tree().get_nodes_in_group("enemy")
    if enemies.is_empty(): return
    var idx = enemies.find(locked_target)
    locked_target = enemies[(idx + 1) % enemies.size()]

```

4. 脱锁规则

```

func _process(delta):
    if not locked_target or not
is_instance_valid(locked_target):
        locked_target = null
        return

```

```

        var dist =
global_position.distance_squared_to(locked_target.global_positi
on)
        var max_dist = lock_range * lock_range

        if dist > max_dist:
            locked_target = null          # 超出范围自动脱锁
            Notification.show(tr("TARGET_LOST"))
            return

        if not _is_in_los(locked_target):
            locked_target = null          # 被障碍物挡住, 脱锁

func _is_in_los(target):
    var space = get_world_2d().direct_space_state
    var query =
PhysicsRayQueryParameters2D.create(global_position,
target.global_position)
    query.exclude = [self]
    return space.intersect_ray(query).is_empty()

```

5. 最终方案模板

```

# LockOnSystem.gd
extends Node

@export var range := 500.0
@export var lock_angle := 90.0

var target: Node2D = null
var icon: Sprite2D

func find():
    target = _best_target()

func release():
    target = null

func _best_target() -> Node2D:
    var best = null

```



```

var best_score = -INF
for e in get_tree().get_nodes_in_group("enemy"):
    if not _in_range(e) or not _has_los(e): continue
    var score = _score(e) # 距离近 + 角度小 = 高分
    if score > best_score:
        best_score = score
        best = e
return best

func face_target(delta: float, node: Node2D, speed: float):
    if not target: return
    var dir = (target.global_position -
node.global_position).normalized()
    node.rotation = lerp_angle(node.rotation, dir.angle(), speed
* delta)

```

第35章 变身系统演化

Godot 变身系统：从换贴图到独立战斗形态

变身 = 角色临时变成另一种形态。最早只是换张图片。后来有了独立性、独立技能组、变身条、变身 CD。变身的进化 = "怎么让变身不只是换皮肤"。

1. 原始：换图

```
func transform():
    $Sprite.texture = preload("res://sprites/super_form.png")
    attack_power *= 2
    await get_tree().create_timer(10.0).timeout
    revert()
```

2. 变身状态机

```
enum Form { NORMAL, SUPER, ULTIMATE }
var current_form: Form = Form.NORMAL

var form_stats = {
    Form.NORMAL: { "hp": 100, "atk": 10, "speed": 200 },
    Form.SUPER: { "hp": 150, "atk": 25, "speed": 300 },
    Form.ULTIMATE: { "hp": 200, "atk": 50, "speed": 400 },
}

func change_form(form: Form):
    if form == current_form: return
    current_form = form
    var s = form_stats[form]
```

```

StatSystem.set_base("hp", s.hp)
StatSystem.set_base("atk", s.atk)
StatSystem.set_base("speed", s.speed)
$AnimationTree.travel("form_" + str(form))

```

3. 变身条 + 持续时间

```

var transform_gauge := 0.0
var max_gauge := 100.0
var gauge_decay := 5.0 # 每秒减少

func add_gauge(amount: float):
    transform_gauge = min(transform_gauge + amount, max_gauge)
    if transform_gauge >= max_gauge:
        activate_transform()

func activate_transform():
    change_form(Form.SUPER)
    # 变身期间持续消耗能量条
    var tween = create_tween()
    tween.tween_property(self, "transform_gauge", 0.0, 10.0)

func _process(delta):
    if current_form != Form.NORMAL:
        transform_gauge -= gauge_decay * delta
        if transform_gauge <= 0:
            change_form(Form.NORMAL)

```

4. 最终方案模板

```

# TransformSystem.gd
extends Node

@export var forms: Array[FormData]

var active_form: int = 0 # 0 = normal
var gauge := 0.0

```

```

func transform(idx: int):
    if idx == active_form: return
    _exit_form(active_form)
    active_form = idx
    _enter_form(idx)

func _enter_form(idx: int):
    var f = forms[idx]
    StatSystem.apply_mods(f.stat_mods)
    SkillManager.swap_skills(f.skill_set)
    $AnimationPlayer.play("enter_" + f.id)
    TimerSystem.start("transform_duration", f.duration,
        _on_expire)

func _exit_form(idx: int):
    var f = forms[idx]
    StatSystem.remove_mods(f.stat_mods)
    SkillManager.restore_skills()
    $AnimationPlayer.play("exit_" + f.id)

```

第36章 元素克制系统演化

Godot 元素克制系统: 从 if-else 到循环克制表

元素克制 = 火克草、草克水、水克火。最早写在无数个 if 里。后来有了克制表、克制系数、抗性、穿透。元素克制的进化 = "怎么让属性关系可配置"。

1. 原始: if-else

```
func calculate_damage(attack_element, defense_element,
base_damage):
    if attack_element == "fire" and defense_element == "grass":
        return base_damage * 2.0
    elif attack_element == "fire" and defense_element ==
"water":
        return base_damage * 0.5
    elif attack_element == "grass" and defense_element ==
"water":
        return base_damage * 2.0
    # ... 每加一个元素, if 的数量指数增长
```

2. 克制表

```
# 二维数组: 攻击方 × 防御方
var advantage_table = {
    "fire": { "fire": 1.0, "water": 0.5, "grass": 2.0,
"earth": 1.0, "wind": 1.0 },
```

```

    "water": { "fire": 2.0, "water": 1.0, "grass": 0.5,
"earth": 1.5, "wind": 1.0 },
    "grass": { "fire": 0.5, "water": 2.0, "grass": 1.0,
"earth": 1.0, "wind": 1.5 },
    "earth": { "fire": 1.5, "water": 1.0, "grass": 1.0,
"earth": 1.0, "wind": 0.5 },
    "wind": { "fire": 1.0, "water": 1.0, "grass": 0.5,
"earth": 2.0, "wind": 1.0 },
}

func get_multiplier(attack: String, defense: String) -> float:
    return advantage_table.get(attack, {}).get(defense, 1.0)

```

3. 抗性 + 穿透

```

class ElementResistance:
    var fire: float = 0.0    # 0.0 ~ 1.0 (0 = 无抗性, 1 = 免疫)
    var water: float = 0.0
    var grass: float = 0.0
    var earth: float = 0.0
    var wind: float = 0.0

    func get_resistance(element: String) -> float:
        return get(element) # 返回 0.0 ~ 1.0

# 穿透: 降低目标的抗性
func apply(element: String, penetration: float) -> float:
    var res = target.resistance.get_resistance(element)
    res = max(res - penetration, 0.0) # 穿透最多减到 0
    return 1.0 - res # 实际承伤比例

```

4. 最终方案模板

```

# ElementSystem.gd (Autoload)
# 核心: 克制表 + 抗性 + 穿透

var table := {}

```



```

func load_from_csv(path: String):
    # 从 CSV 加载克制系数, 可配置
    var file = FileAccess.open(path, FileAccess.READ)
    var headers = file.get_csv_line()
    while not file.eof_reached():
        var row = file.get_csv_line()
        table[row[0]] = {}
        for i in range(1, row.size()):
            table[row[0]][headers[i]] = float(row[i])

func calc(atk_elem: String, def_elem: String) -> float:
    return table.get(atk_elem, {}).get(def_elem, 1.0)

func final_damage(atk_elem: String, def_elem: String, base: int,
pen: float) -> int:
    var mult = calc(atk_elem, def_elem)
    var res = target.get_resistance(def_elem)
    res = max(res - pen, 0.0)
    return int(base * mult * (1.0 - res))

```

第37章 连携技系统演化

Godot 连携技系统: 从各打各的到组合大招

连携技 = 两个或多个玩家/角色一起释放的复合技能。最早每人放自己的技能, 互不干涉。后来有了合体技、QTE 连携、组合按键。连携技的进化 = "怎么让多人配合产生 1+1>2 的效果"。

1. 原始: 各打各的

```
# 玩家 A 放火球, 玩家 B 放冰箭  
# 两个技能各自独立, 没有互动
```

2. 触发式合体技

```
# 玩家 A 的技能命中时, 检测范围内有玩家 B 的技能  
# 触发组合效果  
  
# Skill_A.gd  
func on_hit(target):  
    for area in $Hitbox.get_overlapping_areas():  
        var skill_b = area.owner  
        if skill_b.has_method("is_skill_b"):  
            _trigger_combo(target, skill_b)  
  
func _trigger_combo(target, skill_b):  
    skill_b.cancel()  
    var combo = preload("res://skills/  
combo_fire_ice.tscn").instantiate()  
    combo.position = target.global_position
```

```
get_parent().add_child(combo)
combo.activate()
```

3. 连携配置表

```
# combo_table.gd
var combos = {
    "fireball+ice_arrow": {
        "name": "冰火两重天",
        "result": "combo_fire_ice",
        "damage_mult": 3.0,
        "effect": "explosion",
    },
    "lightning+water_ball": {
        "name": "雷水导电",
        "result": "combo_thunder_wave",
        "damage_mult": 2.5,
        "effect": "chain_lightning",
    },
    "wind_blade+fireball": {
        "name": "风助火势",
        "result": "combo_fire_storm",
        "damage_mult": 2.0,
        "effect": "aoe_burn",
    },
}

func find_combo(skill_a_id: String, skill_b_id: String) ->
Dictionary:
    return combos.get(skill_a_id + "+" + skill_b_id)
```

4. 最终方案模板

```
# ComboSkillManager.gd (Autoload)
# 核心: 技能队列 + 组合检测 + 触发

var _recent_skills := [] # [{player_id, skill_id, time}]
const COMBO_WINDOW := 0.5
```

```

func register_use(player_id: String, skill_id: String):
    var now = Time.get_ticks_msec() / 1000.0
    _recent_skills.append({ player=player_id, skill=skill_id,
time=now })
    _recent_skills = _recent_skills.filter(func(e): return now -
e.time < COMBO_WINDOW)
    _check_combo()

func _check_combo():
    if _recent_skills.size() < 2: return
    var a = _recent_skills[-2]
    var b = _recent_skills[-1]
    var key = a.skill + "+" + b.skill
    var reversed_key = b.skill + "+" + a.skill
    var combo = combo_table.get(key) or
combo_table.get(reversed_key)
    if combo and a.player != b.player:
        _trigger(combo, a.player, b.player)
        _recent_skills.clear()

```

第38章 能量系统演化

Godot 能量系统：从无限施放到资源管理

能量 = 释放技能需要消耗的资源。最早所有技能随便放。后来有了 MP、怒气、能量条、连击点、法力宝石。能量系统的进化 = "怎么限制玩家不能无脑放技能"。

1. 原始：无消耗

```
func cast_skill():  
    # 随便放，没有消耗  
    spawn_fireball()
```

2. MP 法力值

```
var mp := 100  
var max_mp := 100  
var mp_regen := 5 # 每秒回复  
  
func cast_skill(cost: int) -> bool:  
    if mp < cost: return false  
    mp -= cost  
    spawn_fireball()  
    return true  
  
func _process(delta):  
    mp = min(mp + mp_regen * delta, max_mp)
```

3. 多种能量类型

```
# 不同职业使用不同能量
class EnergySystem:
    var type: String # "mp" / "rage" / "energy" / "combo_point"
    var current: float
    var max_value: float

    func generate(amount: float):
        match type:
            "mp":      current = min(current + amount * 0.1,
max_value) # 缓慢恢复
            "rage":    current = min(current + amount,
max_value)    # 受伤/攻击增长
            "energy":  current = min(current + amount *
regen_rate, max_value)
            "combo":   current = min(current + 1,
max_value)      # 连击点

    func can_spend(amount: float) -> bool:
        return current >= amount

    func spend(amount: float):
        current -= amount

# 法师: type = "mp",      生成慢, 消耗大
# 战士: type = "rage",    受伤加怒气, 怒气技能
# 盗贼: type = "energy",  恢复快, 技能消耗小
# 武僧: type = "combo",   普攻攒点, 终结技消耗
```

4. 最终方案模板

```
# EnergyComponent.gd
extends Node

@export var energy_type: String = "mp"
@export var max_value: float = 100.0
@export var regen_rate: float = 5.0
```



```

var current: float = 100.0
var last_action_time: float = 0.0

func can_afford(cost: float) -> bool:
    return current >= cost

func spend(cost: float):
    current -= cost
    current = max(current, 0.0)

func on_take_damage(amount: int):
    if energy_type == "rage":
        current = min(current + amount * 0.5, max_value)

func on_hit_enemy():
    if energy_type == "combo":
        current = min(current + 1, max_value)

func on_kill():
    if energy_type == "energy":
        current = max_value # 击杀回满能量

func _process(delta):
    if energy_type == "mp":
        current = min(current + regen_rate * delta, max_value)

```

第39章 镶嵌系统演化

Godot 镶嵌系统：从固定属性到自由搭配

镶嵌 = 在装备上打孔, 嵌入宝石获得额外属性。最早装备属性是固定的。后来有了打孔、宝石合成、套装激活、颜色匹配。镶嵌的进化 = "怎么让玩家自定义装备的超凡属性"。

1. 原始：固定属性

```
var sword = { "name": "火焰剑", "atk": 20, "element": "fire" }  
# 不能改, 不能升级
```

2. 插槽 + 宝石

```
class Equipment:  
    var socket_count: int = 3  
    var sockets: Array = [] # 每个槽: null 或 宝石ID  
  
class Gem:  
    var id: String  
    var name: String  
    var stats: Dictionary # { "atk": 5, "crit": 0.02 }  
  
func apply_gems(equip: Equipment) -> Dictionary:  
    var total = equip.base_stats.duplicate()  
    for gem_id in equip.sockets:  
        if gem_id:  
            var gem = GemDB.get(gem_id)  
            for stat in gem.stats:  
                total[stat] = total.get(stat, 0) +
```

```
gem.stats[stat]
    return total
```

3. 颜色匹配 + 套装效果

```
enum GemColor { RED, BLUE, GREEN, YELLOW, PURPLE }

class Gem:
    var color: GemColor
    var stats: Dictionary

# 插槽也有颜色:
# EquipmentSlot:
#   var color: GemColor # RED / BLUE / GREEN / ANY

func calc_bonus(equip: Equipment) -> Dictionary:
    var total = {}
    var color_counts = {}
    for i in equip.sockets.size():
        var gem = equip.sockets[i]
        if not gem: continue
        var slot_color = equip.socket_colors[i]
        # 同色: 属性 × 1.25
        var mult = 1.25 if gem.color == slot_color else 1.0
        for stat in gem.stats:
            total[stat] = total.get(stat, 0) + gem.stats[stat] *
mult
        color_counts[gem.color] = color_counts.get(gem.color, 0)
+ 1

    # 同色宝石 ≥ 3 激活额外套装属性
    for color in color_counts:
        if color_counts[color] >= 3:
            var bonus = GemSetDB.get_set_bonus(color,
color_counts[color])
            for stat in bonus:
                total[stat] = total.get(stat, 0) + bonus[stat]
    return total
```

4. 最终方案模板

```
# GemSystem.gd (Autoload)
# 核心: 插槽 + 宝石 + 颜色匹配 + 套装

func calc(equip):
    var total = equip.base_stats.duplicate()
    var colors = {}
    for i in equip.sockets.size():
        var gem = GemDB.get(equip.sockets[i])
        if not gem: continue
        var mult = 1.25 if gem.color == equip.socket_colors[i]
    else 1.0
        for k, v in gem.stats:
            total[k] = total.get(k, 0) + v * mult
        colors[gem.color] = colors.get(gem.color, 0) + 1
    for color, count in colors:
        if count >= 3:
            for k, v in GemSetDB.get_bonus(color, count):
                total[k] = total.get(k, 0) + v
    return total
```

第三卷·经济系统

共计 21 章

第40章 货币系统演化

Godot 货币系统: 从金币到多货币

最早的游戏只有"金币"。后来有了"钻石"、"绑定金币"、"荣誉点数"、"体力"。为什么? 因为单一货币管不住玩家的行为。每种新货币的诞生, 都是为了解决一个"玩家可以用钱绕过什么限制"的问题。

1. 原始: 单一货币

```
var gold = 0

func buy_item(price):
    if gold >= price:
        gold -= price
        return true
    return false
```

问题: 金币能买所有东西。玩家攒够了就全买了, 内容消耗太快。而且金币能无限刷, 没有限制手段。

2. 双货币: 金币 + 钻石

```
var gold = 0          # 游戏内可刷
var gems = 0          # 充值获得

func buy_item(item, currency_type):
    match currency_type:
        "gold":
            if gold >= item.gold_price:
```



```

        gold -= item.gold_price
        return true
    "gems":
        if gems >= item.gem_price:
            gems -= item.gem_price
            return true
    return false

```

金币: 游戏内产出 (打怪、任务)。**钻石:** 充值获得, 买稀有物品、加速、复活。

3. 多货币体系

```

class CurrencySystem:
    var wallet = {
        "gold": 0,          # 通用货币, 可交易
        "gems": 0,          # 充值货币
        "bound_gold": 0,    # 绑定金币, 不可交易
        "honor": 0,         # PVP 货币
        "guild_coin": 0,    # 公会货币
        "arena_ticket": 0,  # 竞技场门票 (每日重置)
        "stamina": 120,     # 体力 (随时间恢复)
    }

    func add(currency: String, amount: int):
        if not wallet.has(currency): return
        wallet[currency] += amount

    func spend(currency: String, amount: int) -> bool:
        if wallet.get(currency, 0) < amount:
            return false
        wallet[currency] -= amount
        return true

```

每种货币解决一个设计问题:

金币:	基础流通
钻石:	充值变现
绑定金币:	限制交易, 防止工作室
荣誉:	激励 PVP 玩家

公会币： 激励公会参与
体力： 限制玩家"一天玩多久"

4. 最终方案模板

```
# CurrencyManager.gd (Autoload)
extends Node

signal changed(currency: String, new_amount: int)

var _wallet := {}

func _ready():
    _wallet = {
        "gold": 100,
        "gems": 0,
        "stamina": 120,
        "honor": 0,
    }

func has(currency: String, amount: int) -> bool:
    return _wallet.get(currency, 0) >= amount

func add(currency: String, amount: int):
    _wallet[currency] = _wallet.get(currency, 0) + amount
    changed.emit(currency, _wallet[currency])

func spend(currency: String, amount: int) -> bool:
    if not has(currency, amount):
        return false
    _wallet[currency] = _wallet[currency] - amount
    changed.emit(currency, _wallet[currency])
    return true

func get_amount(currency: String) -> int:
    return _wallet.get(currency, 0)
```

第41章 背包系统演化

Godot 背包系统: 从数组到格子管理

捡到一瓶药水, 放到背包里。这个"放到背包里"比想象中复杂。能堆叠吗? 格子够吗? 同类物品要合并还是分开? 背包不只是"存东西的地方", 是"怎么存、怎么取、怎么显示"的一套系统。

目录

1. [原始: 数组背包](#)
 2. [堆叠: 同类物品合并](#)
 3. [格子背包: 固定大小](#)
 4. [分类管理: 标签和筛选](#)
 5. [背包 UI 与数据分离](#)
 6. [排序与整理](#)
 7. [最终方案: 最简可用模板](#)
-

1. 原始: 数组背包

问题: "玩家捡起一个物品, 放到背包里。"

最初的做法:

```
var inventory := [] # 物品列表

func pick_up(item_name: String):
```

```

        inventory.append(item_name)

func use_item(index: int):
    if index < inventory.size():
        var item = inventory[index]
        if item == "hp_potion":
            heal(20)
        elif item == "mp_potion":
            restore_mana(20)
        inventory.remove_at(index)

```

问题: 1. 没有数量 – 捡了 5 瓶药水就要存 5 次 "hp_potion" 2. 物品名是字符串 – "hp_potion" 和 "HP Potion" 是两回事, 容易写错 3. 没有物品数据 – 药水回多少血? 写在 use_item 的 match 里。加一个新物品要改代码 4. 没有限制 – 可以捡无限个, 没有"背包满了"的概念

2. 堆叠: 同类物品合并

问题: 捡了 5 瓶药水, 数组里有 5 个 "hp_potion"。用的时候一个一个删。

用 Dictionary 存数量:

```

var inventory := {} # { item_id: count }

func pick_up(item_id: String, count: int = 1):
    if inventory.has(item_id):
        inventory[item_id] += count
    else:
        inventory[item_id] = count

func use_item(item_id: String, count: int = 1):
    if not inventory.has(item_id):
        return
    if inventory[item_id] < count:
        return
    inventory[item_id] -= count
    if inventory[item_id] <= 0:
        inventory.erase(item_id)

```

```

func has_item(item_id: String) -> bool:
    return inventory.has(item_id) and inventory[item_id] > 0

# 使用:
pick_up("hp_potion", 5)    # 捡 5 瓶
use_item("hp_potion")      # 用 1 瓶, 剩 4
use_item("hp_potion", 3)   # 用 3 瓶, 剩 1

```

但物品的最大堆叠量呢? 药水可以堆 99 个, 武器只能堆 1 个。

```

# 加上最大堆叠限制
func pick_up(item_id: String, count: int = 1) -> int:
    var max_stack = get_max_stack(item_id)
    var current = inventory.get(item_id, 0)

    var can_add = max_stack - current
    var actual_add = min(count, can_add)

    inventory[item_id] = current + actual_add
    return count - actual_add # 没装下的部分

# 返回 0 表示全部装下, 返回 >0 表示有剩余

```

3. 格子背包: 固定大小

问题: 没有容量限制, 玩家可以带无限物品。

格子系统: 背包有 N 个格子, 每个格子放一种物品。

```

class InventorySlot:
    var item_id: String = ""
    var count: int = 0
    var max_stack: int = 99

class InventoryGrid:
    var slots: Array[InventorySlot]
    var max_slots: int = 20

```

```

func _init(size: int):
    max_slots = size
    slots = []
    for i in size:
        slots.append(InventorySlot.new())

func add_item(item_id: String, count: int) -> int:
    var item_data = ItemDB.get(item_id)
    var max_stack = item_data.max_stack

    # 先找已有堆叠
    for slot in slots:
        if slot.item_id == item_id and slot.count <
max_stack:
            var space = max_stack - slot.count
            var to_add = min(space, count)
            slot.count += to_add
            count -= to_add
            if count <= 0:
                return 0

    # 再找空格
    for slot in slots:
        if slot.item_id == "":
            slot.item_id = item_id
            var to_add = min(max_stack, count)
            slot.count = to_add
            count -= to_add
            if count <= 0:
                return 0

    return count # 没装下的部分

func remove_item(item_id: String, count: int) -> bool:
    var total = get_item_count(item_id)
    if total < count:
        return false

    for slot in slots:
        if slot.item_id == item_id:
            var to_remove = min(slot.count, count)
            slot.count -= to_remove
            count -= to_remove
            if slot.count <= 0:

```

```

        slot.item_id = ""
        if count <= 0:
            return true
        return true

func get_item_count(item_id: String) -> int:
    var total = 0
    for slot in slots:
        if slot.item_id == item_id:
            total += slot.count
    return total

```

4. 分类管理：标签和筛选

问题：背包里 50 件物品混在一起，找一瓶药水要翻半天。

分类：物品有类型，背包可以按类型筛选。

```

# ItemResource.gd
class_name ItemResource
extends Resource

@export var item_id: String = ""
@export var item_name: String = ""
@export var icon: Texture2D
@export var description: String = ""
@export var item_type: String = "consumable" # weapon/armor/
consumable/material/quest
@export var max_stack: int = 99
@export var buy_price: int = 0
@export var sell_price: int = 0
@export var use_effect: String = "" # "heal_20",
"restore_mana_30"

```

```

# 筛选
func get_filtered_items(item_type: String) -> Array:
    var result = []
    for slot in slots:
        if slot.item_id == "":
            continue

```



```

        var data = ItemDB.get(slot.item_id)
        if data.item_type == item_type:
            result.append(slot)
    return result

# 分类槽（弹药/药水快捷栏）
func get_quick_slots() -> Dictionary:
    return {
        "consumable": get_filtered_items("consumable"),
        "ammo": get_filtered_items("ammo"),
    }

```

5. 背包 UI 与数据分离

问题: 背包数据和显示 UI 混在一起。改了数据 UI 不刷新, 或者 UI 逻辑里混着数据修改。

核心原则: 数据存 `InventoryManager`, UI 只负责显示。

```

# InventoryManager.gd (Autoload)
extends Node

signal inventory_changed

var _slots: Array[InventorySlot] = []
var _max_slots: int = 20

func _ready():
    for i in _max_slots:
        _slots.append(InventorySlot.new())

func add_item(item_id: String, count: int) -> int:
    # ... (格子背包的 add_item 逻辑)
    inventory_changed.emit()
    return remaining

func remove_item(item_id: String, count: int) -> bool:
    # ...
    inventory_changed.emit()

```

```

        return true

func get_slots() -> Array:
    return _slots

# InventoryUI.gd
extends Control

@onready var grid = $GridContainer

func _ready():
    InventoryManager.inventory_changed.connect(_refresh)

func open():
    show()
    _refresh()

func _refresh():
    # 清空旧的格子 UI
    for child in grid.get_children():
        child.queue_free()

    # 根据数据重新生成格子
    for slot in InventoryManager.get_slots():
        var slot_ui = preload("res://ui/
InventorySlot.tscn").instantiate()
        slot_ui.setup(slot)
        grid.add_child(slot_ui)

```

UI 从来不直接修改数据:

```

玩家点击"使用药水"
→ UI 发出信号: use_item.emit(slot_index)
→ InventoryManager.use_item(slot_index)
→ 数据修改
→ inventory_changed.emit()
→ UI 刷新

```

6. 排序与整理

问题: 背包乱, 自动整理。

```
func sort_by_type():
    _slots.sort_custom(func(a, b):
        if a.item_id == "": return false
        if b.item_id == "": return true
        var a_data = ItemDB.get(a.item_id)
        var b_data = ItemDB.get(b.item_id)
        if a_data.item_type != b_data.item_type:
            return a_data.item_type < b_data.item_type
        return a.item_id < b.item_id
    )
    inventory_changed.emit()

func merge_stacks():
    # 合并同类型的堆叠
    for i in _slots.size():
        for j in _slots.size():
            if i == j: continue
            if _slots[i].item_id == "" or _slots[j].item_id ==
"":
                continue
            if _slots[i].item_id != _slots[j].item_id:
                continue
            var data = ItemDB.get(_slots[i].item_id)
            var space = data.max_stack - _slots[i].count
            if space <= 0: continue
            var to_move = min(space, _slots[j].count)
            _slots[i].count += to_move
            _slots[j].count -= to_move
            if _slots[j].count <= 0:
                _slots[j].item_id = ""
    inventory_changed.emit()
```

7. 最终方案: 最简可用模板

``gdscript

———— ItemResource.gd ————

```
class_name ItemResource extends Resource
```

```
@export var item_id: String = "" @export var item_name: String = ""
```

```
@export var icon: Texture2D @export var max_stack: int = 99
```

———— InventoryManager.gd (Autoload)

extends Node

signal changed

var slots: Array = [] var max_slots := 20

func _ready(): for i in max_slots: slots.append({ "id": "", "count": 0 })

func add(item_id: String, count: int) -> int: var data = _db(item_id) var
max_s = data.max_stack if data else 99

```
for slot in slots:
    if slot.id == item_id and slot.count < max_s:
        var space = max_s - slot.count
        var add = mini(space, count)
        slot.count += add
        count -= add
        if count <= 0:
            changed.emit()
            return 0

for slot in slots:
    if slot.id == "":
        var add = mini(max_s, count)
        slot.id = item_id
        slot.count = add
        count -= add
        if count <= 0:
            changed.emit()
            return 0
```

```
changed.emit()  
return count
```

```
func remove(item_id: String, count: int) -> bool: if count_of(item_id) <  
count: return false for slot in slots: if slot.id == item_id: var remove =  
mini(slot.count, count) slot.count -= remove count -= remove if  
slot.count <= 0: slot.id = "" if count <= 0: changed.emit() return true  
changed.emit() return true
```

```
func count_of(item_id: String) -> int: var total = 0 for s in slots: if s.id ==  
item_id: total += s.count return total
```

```
func _db(id: String) -> ItemResource: return load("res://items/%s.tres"  
% id)
```

第42章 商店系统演化

Godot 商店系统：从数组买卖到动态定价

商店 = 用货币换物品。最早是固定列表、固定价格。后来有了库存限制、折扣系统、限时商品、回购。商店系统的进化 = "怎么让花钱这件事有选择感"。

1. 原始：固定商品列表

```
var shop_items = ["hp_potion", "mp_potion", "antidote"]
var shop_prices = [50, 50, 30]

func buy(index: int):
    if gold >= shop_prices[index]:
        gold -= shop_prices[index]
        give_item(shop_items[index])
```

2. 商店 Resource

```
class_name ShopResource
extends Resource

@export var shop_name: String
@export var items: Array[ShopItemEntry]

class ShopItemEntry:
    @export var item: ItemResource
    @export var price: int = 0          # 0 = 使用 item 默认价格
    @export var stock: int = -1        # -1 = 无限
    @export var discount: float = 0.0 # 0~1
```

```
func get_price(entry: ShopItemEntry) -> int:
    var base = entry.price if entry.price > 0 else
    entry.item.buy_price
    return roundi(base * (1.0 - entry.discount))
```

3. 限时/刷新商品

```
func refresh_shop():
    for i in items.size():
        if randf() < 0.3: # 30% 概率更换商品
            items[i] = _random_item_for_level(player.level)

func restock():
    for entry in items:
        if entry.stock != -1:
            entry.stock = entry.max_stock
```

4. 回收/出售给商店

```
func sell(item_id: String, count: int) -> int:
    var item_data = ItemDB.get(item_id)
    var price_per = roundi(item_data.buy_price * 0.5) # 半价回收
    var total = price_per * count
    if InventoryManager.remove(item_id, count):
        CurrencyManager.add("gold", total)
    return total
return 0
```

5. 最终方案模板

```
# 商店系统核心：
# 1. ShopResource 定义商品列表
# 2. 购买：检查金币 → 扣金币 → 加物品到背包
# 3. 出售：检查背包 → 扣物品 → 加金币
# 4. 刷新：定时重置库存和商品
```

```
func buy(shop: ShopResource, index: int) -> bool:
    var entry = shop.items[index]
    var price = _get_price(entry)
    if not CurrencyManager.spend("gold", price):
        return false
    if entry.stock > 0:
        entry.stock -= 1
    InventoryManager.add(entry.item.item_id, 1)
    return true
```

第43章 交易系统演化

Godot 交易系统：从邮件发到玩家面对面

交易 = 玩家之间交换物品。最早没有交易, 东西只能自己用。后来有了面对面交易、摆摊、拍卖行、上架费、交易税。交易系统的进化 = "怎么让玩家之间的经济流动起来"。

1. 原始：无交易

物品不可交易，只能自己用

2. 面对面交易

```
# TradeManager.gd (本地双人)
extends Node

signal trade_completed

var my_offers := []
var other_offers := []
var confirmed := { "me": false, "other": false }

func offer(item_uid: String, count: int):
    if confirmed.me: return
    my_offers.append({ "uid": item_uid, "count": count })

func confirm():
    confirmed.me = true
    if confirmed.me and confirmed.other:
        _execute()
```

```

func _execute():
    for offer in my_offers:
        InventoryManager.remove(offer.uid, offer.count)
        # 给对面
        OtherPlayer.inventory.add(offer.uid, offer.count)
    for offer in other_offers:
        OtherPlayer.inventory.remove(offer.uid, offer.count)
        InventoryManager.add(offer.uid, offer.count)
    trade_completed.emit()

```

3. 摆摊系统

```

class StallItem:
    var item_uid: String
    var item_id: String
    var count: int
    var price: int          # 单价
    var currency: String    # "gold" / "gems"

```

```

class PlayerStall:
    var owner_id: String
    var items: Array[StallItem]
    var is_open: bool = false
    var stall_name: String
    var position: Vector2

```

```

func open_stall(items: Array[StallItem]):
    var stall = PlayerStall.new()
    stall.items = items
    _stalls[PlayerData.id] = stall
    # 在场景中创建摊位节点

func buy_from_stall(stall_id: String, item_index: int, count:
int) -> bool:
    var stall = _stalls[stall_id]
    var si = stall.items[item_index]
    var total = si.price * count

    if not CurrencyManager.spend(si.currency, total): return
    false

```

```

    if si.count < count: return false

    si.count -= count
    InventoryManager.add(si.item_id, count)
    # 给摊主发邮件（含金币）
    MailManager.send(stall_id, "售出", "你的 %s 已售出" %
si.item_id)
    return true

```

4. 交易税

```

func calc_tax(price: int) -> int:
    return int(price * 0.05) # 5% 交易税

func execute_trade(buyer, seller, item, price):
    var tax = calc_tax(price)
    CurrencyManager.spend("gold", price, buyer)
    CurrencyManager.add("gold", price - tax, seller)
    CurrencyManager.add("gold", tax, "system") # 系统回收
    InventoryManager.transfer(item, buyer)

```

5. 最终方案模板

```

# TradeManager.gd (Autoload)
# 三人称：
# 1. 面对面：实时协商，双方确认后执行
# 2. 摆摊：离线可购买，收益发邮件
# 3. 拍卖行：竞价/一口价，定时结束

var tax_rate := 0.05

func trade(from: String, to: String, items: Array, gold: int):
    var tax = int(gold * tax_rate)
    CurrencyManager.transfer("gold", from, to, gold - tax)
    CurrencyManager.add("gold", tax, "system")
    for item in items:
        InventoryManager.transfer(item.uid, from, to)

```

第44章 合成系统演化

Godot 合成系统: 从 if 配方到数据驱动

合成 = 把 A + B 变成 C。最早写在 if 里。后来用字典配配方表。合成系统的进化 = "怎么让玩家有东西可以合, 合了有用"。

1. 原始: if 配方

```
func craft(item_a: String, item_b: String):
    if item_a == "木头" and item_b == "铁":
        give_item("木铁剑")
    elif item_a == "药草" and item_b == "瓶子":
        give_item("药水")
```

加配方 = 加 elif。改配方 = 改代码。

2. 配方表 (Dictionary)

```
var recipes = {
    "wooden_sword": {
        "materials": { "wood": 3, "stone": 1 },
        "result": "wooden_sword",
        "result_count": 1,
    },
    "health_potion": {
        "materials": { "herb": 2, "bottle": 1 },
        "result": "health_potion",
        "result_count": 1,
    },
}
```

```

func can_craft(recipe_id: String) -> bool:
    var recipe = recipes[recipe_id]
    for mat_id in recipe.materials:
        if InventoryManager.count_of(mat_id) <
recipe.materials[mat_id]:
            return false
    return true

func craft(recipe_id: String) -> bool:
    if not can_craft(recipe_id):
        return false
    var recipe = recipes[recipe_id]
    for mat_id in recipe.materials:
        InventoryManager.remove(mat_id,
recipe.materials[mat_id])
    InventoryManager.add(recipe.result, recipe.result_count)
    return true

```

3. 配方 Resource

```

class_name RecipeResource
extends Resource

@export var recipe_name: String
@export var materials: Array[MaterialEntry]
@export var result: ItemResource
@export var result_count: int = 1
@export var craft_time: float = 0.0
@export var required_level: int = 1
@export var category: String = "武器"

class MaterialEntry:
    @export var item: ItemResource
    @export var count: int = 1

```

在编辑器里配配方：拖入素材 ItemResource、结果 ItemResource, 填数量。

4. 最终方案模板

```
# CraftingManager.gd
extends Node

var _recipes := []

func _ready():
    # 加载所有配方文件
    var dir = DirAccess.open("res://recipes/")
    for file in dir.get_files():
        if file.ends_with(".tres"):
            _recipes.append(load("res://recipes/" + file))

func get_available() -> Array:
    return _recipes.filter(func(r): return _can_craft(r))

func craft(recipe: RecipeResource) -> bool:
    if not _can_craft(recipe):
        return false
    for mat in recipe.materials:
        InventoryManager.remove(mat.item.item_id, mat.count)
    InventoryManager.add(recipe.result.item_id,
recipe.result_count)
    return true

func _can_craft(recipe: RecipeResource) -> bool:
    for mat in recipe.materials:
        if InventoryManager.count_of(mat.item.item_id) <
mat.count:
            return false
    return true
```

第45章 烹饪系统演化

Godot 烹饪系统: 从药品到美食

烹饪 = 把食材变成食物, 食物提供 Buff。最早没有烹饪, 只有红瓶蓝瓶。
后来有了食材收集、食谱解锁、不同 Buff 的食物、食物过期。烹饪系统的进化 = "怎么让吃东西不仅回血还有策略选择"。

1. 原始: 直接回血道具

```
# 红药水: HP +50  
# 蓝药水: MP +30  
# 不需要烹饪, 直接买
```

2. 食材 + 配方

```
class Recipe:  
    var id: String  
    var name: String  
    var ingredients: Array[Ingredient]  
    var result: CookedFood  
    var cooking_time: float = 3.0  
  
class Ingredient:  
    var item_id: String  
    var count: int  
  
class CookedFood:  
    var item_id: String  
    var effects: Array[FoodEffect]  
    var duration: float = 300.0 # 持续秒数
```

```
class FoodEffect:
    var stat: String      # "hp_regen" / "attack" / "defense" /
    "speed"
    var value: float
```

```
func cook(recipe_id: String) -> bool:
    var recipe = RecipeDB.get(recipe_id)
    # 检查食材是否足够
    for ing in recipe.ingredients:
        if not InventoryManager.has(ing.item_id, ing.count):
            return false
    # 扣食材
    for ing in recipe.ingredients:
        InventoryManager.remove(ing.item_id, ing.count)
    # 产出食物
    InventoryManager.add(recipe.result.item_id, 1)
    return true
```

3. 烹饪小游戏 (QTE)

```
var progress = 0.0
var perfect_zone = 0.8

func _process(delta):
    if not is_cooking: return
    # 进度条自动来回摆动
    progress += sin(Time.get_ticks_msec() * 0.005) * delta
    progress = clamp(progress, 0, 1)

func on_press():
    # 在 Perfect Zone 按 = 高品质
    var quality = "normal"
    if progress > 0.75 and progress < 0.85:
        quality = "perfect" # 属性 x1.5
    elif progress > 0.6 and progress < 0.9:
        quality = "good"    # 属性 x1.2
    else:
        quality = "ok"
    _finish_cooking(quality)
```

4. 最终方案模板

```
# CookingManager.gd (Autoload)
# 简单版：有食材+配方 → 直接出食物

func cook(recipe_id: String) -> bool:
    var recipe = RecipeDB.get(recipe_id)
    if not _has_ingredients(recipe): return false
    _consume_ingredients(recipe)
    InventoryManager.add(recipe.result_id, 1)
    return true

# 食物 Buff 通过 FoodBuffManager 管理：
func eat(food_id: String):
    var food = FoodDB.get(food_id)
    for effect in food.effects:
        BuffManager.add(effect.stat, effect.value,
            food.duration)
```

第46章 分解系统演化

Godot 分解系统：从扔商店到拆材料

分解 = 把不要的装备变成材料。最早不要的装备直接卖商店。后来有了分解：拆成强化石/附魔材料/稀有碎片。分解系统的进化 = "怎么让垃圾装备也有价值"。

1. 原始：卖商店

```
func sell(item: EquipmentResource):
    CurrencyManager.add("gold", item.sell_price)
    InventoryManager.remove(item.id, 1)
```

问题：装备直接变成金币，缺乏深度。

2. 分解产出

```
class DismantleResult:
    var guaranteed: Array[ItemStack]    # 必出
    var random: Array[ChancedItem]      # 概率出
    var item: EquipmentResource

class ChancedItem:
    var item_id: String
    var chance: float    # 0~1
    var min_count: int
    var max_count: int
```

```
func dismantle(item: EquipmentResource) -> Array:
    var results = []
```

```

var config = DismantleDB.get(item.rarity)

# 必出材料
for g in config.guaranteed:
    results.append({ "id": g.item_id, "count": g.count })

# 概率材料
for r in config.random:
    if randf() < r.chance:
        var count = randi_range(r.min_count, r.max_count)
        results.append({ "id": r.item_id, "count": count })

# 特殊: 如果是强化过的装备, 返还部分强化石
if item.enhance_level > 0:
    var refund = item.enhance_level * 2
    results.append({ "id": "enhance_stone", "count":
refund })

InventoryManager.remove(item.instance_id, 1)
for res in results:
    InventoryManager.add(res.id, res.count)
return results

```

3. 分解确认 + 预览

```

func preview_dismantle(item: EquipmentResource) -> Array:
    # 显示会得到什么, 让玩家确认
    return _simulate_dismantle(item, seed=Time.get_ticks_msec())

func confirm_dismantle(item: EquipmentResource):
    var rewards = dismantle(item)
    for r in rewards:
        ToastManager.notify("获得 %s x%d" %
[ItemDB.get(r.id).name, r.count])

```

4. 最终方案模板

```
# DismantleManager.gd (Autoload)

func dismantle(item: EquipmentResource) -> Array:
    var results = []
    var cfg = _config_for(item.rarity)
    for entry in cfg.guaranteed:
        results.append({ "id": entry.id, "count": entry.count })
    for entry in cfg.random:
        if randf() < entry.chance:
            results.append({ "id": entry.id, "count":
randi_range(entry.min, entry.max) })
    if item.enhance_level > 0:
        results.append({ "id": "enhance_stone", "count":
item.enhance_level })
    InventoryManager.remove(item.uid, 1)
    for r in results:
        InventoryManager.add(r.id, r.count)
    return results
```

第47章 抽卡系统演化

Godot 抽卡系统：从随机到保底

抽卡 = 花货币 → 获得随机物品。但"随机"不是真随机。保底、概率递增、软保底、硬保底——每个机制的诞生都是为了解决"玩家抽了 100 次啥都没有"的愤怒。

1. 原始：纯随机

最初的做法：

```
func pull():
    var roll = randf()
    if roll < 0.01:      return "传说"   # 1%
    elif roll < 0.10:   return "史诗"   # 9%
    elif roll < 0.40:   return "稀有"   # 30%
    else:               return "普通"   # 60%
```

问题：纯随机意味着有人抽 100 次可能一张传说都没有。概率是 1% 不等于"100 次必出 1 次"。数学上, 100 次不出传说的概率是 $0.99^{100} \approx 36.6\%$ 。

2. 硬保底：第 N 次必出

```
var pity_counter = 0

func pull() -> String:
    pity_counter += 1
    if pity_counter >= 90:
        pity_counter = 0
```

```

        return "传说"

    var roll = randf()
    if roll < 0.01:
        pity_counter = 0
        return "传说"
    elif roll < 0.10:
        return "史诗"
    ...

```

硬保底的意义: 保证最差情况。1% 概率 + 90 次保底 = 最差 90 次必出。玩家知道"最多抽 90 次就有了", 愿意氪。

3. 软保底: 概率递增

问题: 硬保底太死板, 第 89 次和第一次概率一样。体验上"前面的 89 次都是陪跑"。

```

var pull_count = 0

func get_probability(base_rate: float, count: int) -> float:
    # 每抽一次, 概率增加
    return base_rate + count * 0.005 # 每次 +0.5%

func pull():
    pull_count += 1
    var rate = get_probability(0.01, pull_count)
    if randf() < rate:
        pull_count = 0
        return "传说"
    ...

```

4. 最终方案模板

```

# GachaSystem.gd
extends Node

```

```

signal pull_result(items: Array)

var pity_4star = 0
var pity_5star = 0
const HARD_PITY_4 = 10
const HARD_PITY_5 = 90

func do_ten_pull() -> Array:
    var results = []
    for i in 10:
        results.append(do_single_pull())
    pull_result.emit(results)
    return results

func do_single_pull() -> String:
    pity_4star += 1
    pity_5star += 1

    # 硬保底 5★
    if pity_5star >= HARD_PITY_5:
        pity_5star = 0
        pity_4star = 0
        return "5star"

    # 软保底: 从 74 抽开始概率递增
    var five_star_rate = 0.006
    if pity_5star >= 74:
        five_star_rate += (pity_5star - 73) * 0.06

    if randf() < five_star_rate:
        pity_5star = 0
        pity_4star = 0
        return "5star"

    # 4★: 保底 10 抽
    if pity_4star >= HARD_PITY_4:
        pity_4star = 0
        return "4star"

    if randf() < 0.051:
        pity_4star = 0
        return "4star"

    return "3star"

```

第48章 转盘系统演化

Godot 转盘系统：从抽奖到每日免费转盘

转盘 = 指针旋转, 指到哪格拿哪个奖励。抽卡是用货币抽, 转盘通常是每日免费/任务获得的抽奖机会。最早的转盘是纯随机。后来有了保底格、概率可视化、转盘动画。转盘系统的进化 = "怎么让抽奖看起来是转盘决定的"。

1. 原始：纯随机

```
func spin() -> String:
    var items = ["gold_100", "hp_potion", "gems_10", "nothing",
"scroll", "rare_box"]
    return items.pick_random()
```

2. 权重转盘

```
class WheelSlot:
    var item_id: String
    var weight: float      # 权重, 越大概率越高
    var count: int = 1

func spin(wheel_id: String) -> Dictionary:
    var slots = _wheels[wheel_id]
    var total = slots.reduce(func(sum, s): return sum +
s.weight, 0.0)
    var roll = randf() * total
    var acc = 0.0
    for slot in slots:
        acc += slot.weight
```

```

        if roll < acc:
            InventoryManager.add(slot.item_id, slot.count)
            return { "item": slot.item_id, "count": slot.count }
    return {}

```

3. 转盘动画 (先定结果, 再转指针)

```

func spin_with_animation(wheel_id: String):
    # 1. 先算结果
    var result = _calc_result(wheel_id)

    # 2. 计算指针停在哪个角度
    var slot_index = _get_slot_index(result.item_id)
    var slot_angle = slot_index * (360.0 / slots.size())
    var extra_spins = 3 * 360 # 至少转 3 圈
    var target_angle = extra_spins + slot_angle

    # 3. 动画旋转
    var tween = create_tween()
    tween.tween_method(_set_pointer_angle, 0, target_angle, 2.0)
    \
        .set_ease(Tween.EASE_OUT) \
        .set_trans(Tween.TRANS_CUBIC)

    # 4. 动画结束后发奖励
    await tween.finished
    _give_reward(result)

```

4. 保底机制

```

class LuckyWheel:
    var slots: Array[WheelSlot]
    var free_daily: int = 1
    var cost_per_spin: Dictionary = { "gems": 50 }
    var guarantee_count: int = 10 # 10 次必出稀有
    var guarantee_item: String = "rare_box"
    var spin_count: int = 0 # 累计转了多少次

```

```

func spin(wheel: LuckyWheel) -> Dictionary:
    wheel.spin_count += 1
    # 保底: 第 10 次必定给稀有
    if wheel.spin_count >= wheel.guarantee_count:
        wheel.spin_count = 0
        InventoryManager.add(wheel.guarantee_item, 1)
        return { "item": wheel.guarantee_item, "guaranteed":
true }
    # 正常权重抽奖
    return _normal_spin(wheel)

```

5. 最终方案模板

```

# WheelManager.gd (Autoload)
# 每日免费转盘:
# 1. 每天 1 次免费
# 2. 额外次数消耗钻石
# 3. 转盘动画 2 秒
# 4. 10 次保底

func spin() -> Dictionary:
    if _save.free_used_today >= _wheel.free_daily:
        return { "ok": false, "msg": "免费次数已用完" }
    _save.free_used_today += 1
    var result = _calc_result()
    AnimationManager.play_wheel(result)
    await AnimationManager.wheel_finished
    _give(result)
    return { "ok": true, "result": result }

```

第49章 签到系统演化

Godot 签到系统：从每日奖励到签到日历

签到 = 每天登录游戏领点东西。最早是"今天登录了, 给 100 金币"。后来有了连续签到奖励、补签、月卡、累计签到天数。签到系统的进化 = "怎么让玩家每天想打开游戏"。

1. 原始：每日登录检测

```
func _ready():
    var last_login = GameState.get("last_login_date", 0)
    var today = Time.get_unix_time_from_system() / 86400

    if today != last_login:
        CurrencyManager.add("gold", 100)
        GameState.last_login_date = today
```

2. 签到日历

```
class CheckInSystem:
    var day_rewards = {
        1: { "gold": 100 },
        2: { "gold": 150 },
        3: { "gold": 200, "item": "hp_potion" },
        4: { "gold": 250 },
        5: { "gold": 300 },
        6: { "gold": 350, "item": "scroll" },
        7: { "gold": 500, "item": "rare_chest" },
    }
```

```

func check_in() -> Dictionary:
    var today = _get_today()
    if _last_check_in() == today:
        return { "success": false, "reason":
"already_checked" }

    var streak = _get_streak()
    if _yesterday_check_in() != today - 1:
        streak = 0 # 断签

    streak += 1
    var day = ((streak - 1) % 7) + 1 # 7 天循环
    var reward = day_rewards[day]

    _give_rewards(reward)
    _save_check_in(today, streak)

    return { "success": true, "day": day, "rewards":
reward }

```

3. 补签

```

func make_up_missed():
    if CurrencyManager.spend("gems", 50):
        _save_check_in(_get_today(), _get_streak() + 1)
        return true
    return false

```

4. 月卡

```

class MonthlyCard:
    var active: bool = false
    var expire_date: int = 0

    func activate(days: int = 30):
        active = true
        expire_date = _get_today() + days

```

```

func get_daily_bonus() -> Dictionary:
    if not active or _get_today() > expire_date:
        active = false
        return {}
    return { "gems": 100, "stamina": 60 }

```

5. 最终方案模板

```

# CheckInManager.gd (Autoload)
extends Node

signal checked_in(day: int, rewards: Dictionary)

var _save := {}

func check_in() -> bool:
    var today = _day()
    if _save.get("last_day") == today:
        return false

    var streak = _save.get("streak", 0)
    if _save.get("last_day") == today - 1:
        streak += 1
    else:
        streak = 1

    var day = ((streak - 1) % 7) + 1
    var rewards = _rewards_for(day)
    _give(rewards)
    _save["last_day"] = today
    _save["streak"] = streak
    checked_in.emit(day, rewards)
    return true

func _day() -> int:
    return Time.get_unix_time_from_system() / 86400

func _rewards_for(day: int) -> Dictionary:
    return [{"gold": 100}, {"gold": 150}, {"gold": 200}, {"gold": 250}, {"gold": 300}, {"gold": 350}, {"gold": 500, "chest": true}][day - 1]

```

第50章 体力系统演化

Godot 体力系统：从无限玩到有限资源

体力 = 限制玩家"一天能玩多久"的数值。最早的游戏没有体力, 玩家可以一直玩。后来手游发明了体力: 打一局消耗 5 点, 5 分钟恢复 1 点。体力系统的进化 = "怎么让玩家每天想上线把体力用完"。

1. 原始：无体力限制

```
# 玩家可以无限打副本
# 问题：内容消耗太快，玩家一天通关
```

2. 固定体力

```
var stamina = 100
var max_stamina = 100

func enter_dungeon(cost: int = 10) -> bool:
    if stamina < cost:
        return false
    stamina -= cost
    return true
```

3. 随时间恢复

```
var stamina = 100
var max_stamina = 100
var recover_rate = 1 # 每 5 分钟恢复 1 点
```

```
func _process(delta):
    if stamina < max_stamina:
        stamina += delta / 300 * recover_rate # 300秒=5分钟
        stamina = min(stamina, max_stamina)
```

4. 体力购买 + 每日重置

```
func buy_stamina(amount: int) -> bool:
    var cost = _get_buy_cost()
    if CurrencyManager.spend("gems", cost):
        stamina += amount
        return true
    return false

func _get_buy_cost() -> int:
    # 每日购买次数越多越贵
    var today_buys = _save.get("daily_buys", 0)
    return 50 + today_buys * 10

func daily_reset():
    stamina = max_stamina
    _save["daily_buys"] = 0
```

5. 最终方案模板

```
# StaminaManager.gd (Autoload)
extends Node

signal changed(current: int, max: int)

var _max := 100
var _current := 100
var _recover_per_sec := 1.0 / 300.0 # 每 5 分钟 1 点

func _process(delta):
    if _current < _max:
        _current = min(_current + _recover_per_sec * delta,
```

```
_max)
    changed.emit(_current, _max)

func spend(amount: int) -> bool:
    if _current < amount:
        return false
    _current -= amount
    changed.emit(_current, _max)
    return true

func refill():
    _current = _max
    changed.emit(_current, _max)

func get_current() -> int:
    return int(_current)
```

第51章 首充系统演化

Godot 首充系统: 从无到首充特惠

首充 = 玩家第一次充值时给额外奖励。最早充值就是给对应的货币数量。后来有了"首充双倍"、"首充礼包"、"累计充值奖励"。首充系统的进化 = "怎么让玩家第一次付费的性价比看起来极高"。

1. 原始: 无首充

充 6 元 = 60 钻石, 没有额外奖励

2. 首充双倍

```
class FirstChargeConfig:
    var double: bool = true
    var first_reward: Dictionary # 额外送的东西

func process_purchase(product_id: String):
    var product = ShopDB.get(product_id)
    var gems = product.gems

    if not _has_charged_before():
        # 首充双倍
        gems *= 2
        # 首充礼包
        for item_id in first_reward:
            InventoryManager.add(item_id, 1)
        _mark_charged()

    CurrencyManager.add("gems", gems)
```

3. 首充奖励 (累计档位)

```
class FirstChargeReward:
    var id: String
    var required_amount: int    # 累计充值金额
    var rewards: Dictionary    # { item_id: count }
    var claimed: bool = false

    var first_charge_rewards = [
        { "amount": 6,    "rewards": { "rare_sword": 1 } },
        { "amount": 30,   "rewards": { "gems": 300, "scroll": 5 } },
        { "amount": 98,   "rewards": { "legend_armor": 1 } },
        { "amount": 198,  "rewards": { "mount": 1 } },
        { "amount": 328,  "rewards": { "gems": 1000, "fashion":
1 } } },
    ]
```

```
func on_charged(amount: int):
    _total_charged += amount
    for reward in _charge_rewards:
        if _total_charged >= reward.required_amount and not
reward.claimed:
            reward.claimed = true
            for item_id in reward.rewards:
                InventoryManager.add(item_id,
reward.rewards[item_id])
            ToastManager.notify("累计充值奖励已发放!")
```

4. 最终方案模板

```
# FirstChargeManager.gd (Autoload)

var has_charged := false
var total_charged := 0

func on_purchase(amount_gems: int):
    if not has_charged:
        has_charged = true
        CurrencyManager.add("gems", amount_gems)           # 首充
```

```

双倍
    InventoryManager.add("first_weapon", 1) # 首
充礼包
    else:
        CurrencyManager.add("gems", amount_gems)

        total_charged += amount_gems
        _check_milestones()

func _check_milestones():
    for tier in _milestones:
        if total_charged >= tier.amount and not tier.claimed:
            tier.claimed = true
            for item_id in tier.rewards:
                InventoryManager.add(item_id,
tier.rewards[item_id])

```

第52章 充值系统演化

Godot 充值/内购系统: 从扣钱到 SDK

充值 = 玩家花真钱买游戏虚拟物品。最早是"发短信扣话费"。后来有了 App Store / Google Play / 支付宝 / 微信支付。充值系统的进化 = "怎么让花钱的流程最快、最安全、最合规"。

1. 原始: 无充值

```
# 单机买断，一次性付费
# 没有"内购"的概念
```

2. 商品列表 + 扣钱

```
var shop = {
    "小礼包": { "price": 6, "gems": 60 },
    "大礼包": { "price": 30, "gems": 330 },
    "月卡":    { "price": 30, "gems": 300, "daily": 100 },
}
```

3. SDK 接入

```
# 简单封装，实际接入各平台 SDK

func purchase(product_id: String):
    # iOS: StoreKit
    # Android: Google Play Billing
    # 国内: 支付宝 / 微信支付
```

```

    PaymentService.buy(product_id)

func _on_purchase_success(product_id: String):
    var product = _products[product_id]
    CurrencyManager.add("gems", product.gems)
    if product.daily:
        MonthlyCard.activate(30)

func _on_purchase_fail(error: String):
    # 支付失败 / 取消
    pass

```

4. 最终方案模板

```

# PaymentManager.gd (Autoload)
# 包装底层 SDK，对外提供统一接口

signal purchase_success(product_id: String)
signal purchase_fail(product_id: String, reason: String)

func buy(product_id: String):
    # 1. 检查是否已拥有（防重复购买）
    # 2. 调 SDK
    # 3. 回调成功后发货
    # 4. 失败给提示
    PaymentService.purchase(product_id)

# 收货:
func _deliver(product_id: String):
    var reward = _product_rewards[product_id]
    for key in reward:
        CurrencyManager.add(key, reward[key])
    purchase_success.emit(product_id)

```

第53章 礼包码系统演化

Godot 礼包码系统: 从手工发到兑换码

礼包码 = 输入一串字符领取奖励。"VIP888" "HAPPY2024"。最早是运营手动给玩家发邮件。后来有了兑换码生成、批量生成、有效期、每人限一次。礼包码系统的进化 = "怎么让运营活动可以一键发放"。

1. 原始: 手动发

```
# 运营在后台给指定玩家发邮件  
# 或者直接修改数据库
```

2. 本地兑换码 (硬编码)

```
var codes = {  
    "TEST123": { "items": { "gold": 1000 }, "used": [] },  
    "VIP888": { "items": { "gems": 100, "scroll": 1 }, "used":  
    [] },  
}  
  
func redeem(code: String) -> Dictionary:  
    code = code.to_upper().strip_edges()  
    if not codes.has(code):  
        return { "success": false, "reason": "无效兑换码" }  
  
    var data = codes[code]  
    if data.used.has(PlayerData.id):  
        return { "success": false, "reason": "已使用过" }  
  
    for item_id in data.items:
```

```
InventoryManager.add(item_id, data.items[item_id])
data.used.append(PlayerData.id)
return { "success": true, "rewards": data.items }
```

3. 服务端验证 (生产环境)

```
# 客户端只负责输入和显示结果
# 验证逻辑在服务端：
# 1. CDK 是否存在
# 2. 是否在有效期内
# 3. 是否已被使用
# 4. 是否已被该玩家使用
# 5. 发放奖励

func redeem(code: String):
    # 发送到服务器验证
    var result = await ServerAPI.redeem_code(code)
    if result.success:
        # 服务器直接发到背包了
        ToastManager.notify("兑换成功!", "gold")
    else:
        ToastManager.notify(result.reason, "error")
```

4. 最终方案模板

```
# RedeemManager.gd (Autoload)
# 单机版：硬编码兑换码，用文件记录已使用

func redeem(code: String) -> Dictionary:
    code = code.strip_edges()
    var list = _load_codes()
    if not list.has(code):
        return { "ok": false, "msg": "无效兑换码" }
    var entry = list[code]
    if entry.expiry and Time.get_unix_time_from_system() >
entry.expiry:
        return { "ok": false, "msg": "已过期" }
    if _is_used(code):
```

```
        return { "ok": false, "msg": "已使用" }
    _mark_used(code)
    for item_id in entry.items:
        InventoryManager.add(item_id, entry.items[item_id])
    return { "ok": true, "rewards": entry.items }
```

第54章 离线收益系统演化

Godot 离线收益系统：从在线到离线挂机

离线收益 = 玩家不在线时也在产出资源。最早不在线就什么都没有。后来有了离线收益：挂机自动产金币，离线最多累积 8 小时。离线收益系统的进化 = "怎么让玩家不上线也不掉队"。

1. 原始：无离线收益

```
# 下线 = 完全停止  
# 什么都没有
```

2. 离线时间计算

```
func calculate_offline_reward() -> Dictionary:  
    var last = _load_last_login()  
    var now = Time.get_unix_time_from_system()  
    var elapsed = now - last # 离线秒数  
  
    # 最多累积 8 小时  
    var max_seconds = 8 * 3600  
    var valid = min(elapsed, max_seconds)  
  
    var gold_per_hour = 100  
    var gold = int(valid / 3600.0 * gold_per_hour)  
  
    return { "gold": gold, "hours": valid / 3600.0 }
```

```
func on_player_login():  
    var reward = calculate_offline_reward()
```



```

if reward.hours > 0.5: # 离线超过 30 分钟才弹窗
    OfflineRewardPopup.show(reward)
    InventoryManager.add("gold", reward.gold)
_save_login_time()

```

3. 离线效率 + 加成

```

class OfflineConfig:
    var base_gold_per_hour: int = 100
    var base_exp_per_hour: int = 50
    var max_accumulate: float = 12.0 # 最多累积 12 小时

    var efficiency: float = 1.0      # 离线效率 (vip/月卡加成)

func get_offline_efficiency(player) -> float:
    var eff = 1.0
    if player.has_vip:
        eff += 0.5 # VIP 离线效率 +50%
    if player.has_monthly_card:
        eff += 0.3 # 月卡 +30%
    return eff

```

4. 最终方案模板

```

# OfflineManager.gd (Autoload)

func get_reward() -> Dictionary:
    var hours = _offline_hours()
    hours = min(hours, 12.0) # 上限 12 小时
    var rate = _calc_rate()
    return {
        "gold": int(hours * rate.gold_per_hour),
        "exp": int(hours * rate.exp_per_hour),
        "hours": hours,
    }

func claim():
    var r = get_reward()

```

```
CurrencyManager.add("gold", r.gold)
PlayerData.add_exp(r.exp)
return r

func _offline_hours() -> float:
    var elapsed = Time.get_unix_time_from_system() -
_save.last_login
    return elapsed / 3600.0
```

第55章 战令系统演化

Godot 战令系统：从免费到付费通行证

战令 = 通过完成任务提升等级, 每级拿奖励。免费路线给基础奖励, 付费路线给稀有奖励。最早的战令就是"签到升级"。后来有了每日任务、每周任务、赛季重置、等级上限。战令的进化 = "怎么让玩家愿意每天上线做任务"。

1. 原始：免费等级奖励

```
# 玩家升级给奖励
# 没有"付费额外奖励"的概念
```

2. 战令结构

```
class BattlePass:
    var season_id: String
    var season_name: String
    var start_time: int
    var end_time: int
    var tiers: Array[BattlePassTier]
    var price_gems: int = 680

class BattlePassTier:
    var level: int
    var exp_required: int          # 升到此级所需经验
    var free_reward: Dictionary    # 免费路线奖励
    var premium_reward: Dictionary # 付费路线奖励
```

```

var battle_pass = {
  "tiers": [
    { "level": 1, "exp": 0, "free": { "gold":
1000 }, "premium": { "skin": "summer_sword" } },
    { "level": 2, "exp": 100, "free": { "scroll":
2 }, "premium": { "gems": 50 } },
    { "level": 3, "exp": 200, "free": { "gold":
2000 }, "premium": { } },
    # ...
    { "level": 50, "exp": 5000, "free": { "gems":
200 }, "premium": { "legend_skin": "dragon_armor" } },
  ],
  "total_levels": 50,
  "premium_unlock": 680,
}

```

3. 经验获取

```

func add_exp(amount: int):
  _current_exp += amount
  # 检查是否升级
  while _current_exp >= _get_next_level_exp():
    _current_exp -= _get_next_level_exp()
    _current_level += 1
    _unlock_tier(_current_level)

func _get_daily_exp() -> int:
  var exp = 0
  if _did_daily_task("kill_monsters"): exp += 100
  if _did_daily_task("collect_items"): exp += 80
  if _did_daily_task("login"): exp += 50
  return exp

func _get_weekly_exp() -> int:
  var exp = 0
  if _did_weekly_task("clear_dungeon_10"): exp += 500
  if _did_weekly_task("pvp_win_5"): exp += 400
  return exp

```

4. 付费解锁

```
func unlock_premium() -> bool:
    if _is_premium: return false
    if CurrencyManager.spend("gems", battle_pass.price_gems):
        _is_premium = true
        # 补领已解锁等级的付费奖励
        for i in range(1, _current_level + 1):
            var tier = battle_pass.tiers[i - 1]
            if tier.premium_reward:
                _claim_reward(tier.premium_reward)
        return true
    return false
```

5. 最终方案模板

```
# BattlePassManager.gd (Autoload)
# 核心: 等级 + 经验 + 免费/付费双路线

var level := 1
var exp := 0
var is_premium := false

func add_exp(amount: int):
    exp += amount
    while level < 50 and exp >= _need():
        exp -= _need()
        level += 1
        _on_tier_unlock(level)

func claim(level: int) -> Dictionary:
    var tier = _tiers[level - 1]
    var rewards = {}
    if tier.free: _give(tier.free);    rewards["free"] = tier.free
    if is_premium and tier.premium:
        _give(tier.premium)
        rewards["premium"] = tier.premium
    return rewards
```

第56章 拍卖行系统演化

Godot 拍卖行系统：从面对面到全服交易

拍卖行 = 玩家之间通过市场买卖物品。最早只能面对面交易。后来有了公示板、竞拍、一口价、上架费、交易税。拍卖行的进化 = "怎么让玩家之间的交易不需要同时在线"。

1. 原始：面对面交易

```
# 两个玩家站在一起
# 打开交易窗口，放上物品
# 双方确认后交换

func trade(player_a, player_b, item_a, item_b):
    player_a.inventory.remove(item_a)
    player_b.inventory.remove(item_b)
    player_a.inventory.add(item_b)
    player_b.inventory.add(item_a)
```

2. 摆摊

```
# 玩家在原地放置一个摊位
# 其他玩家可以查看和购买

class Stall:
    var owner_id: String
    var items: Array[StallItem]
    var position: Vector2

class StallItem:
```



```

var item_id: String
var price: int
var quantity: int

func buy_from_stall(buyer, stall_id, item_idx, amount):
    var stall = stalls[stall_id]
    var item = stall.items[item_idx]
    if buyer.gold >= item.price * amount:
        buyer.gold -= item.price * amount
        stall.owner.gold += item.price * amount
        buyer.inventory.add(item.item_id, amount)
        item.quantity -= amount

```

3. 拍卖行服务端

```

# 服务端统一管理
class AuctionHouse:
    var listings := []

    func list_item(seller_id, item_id, price, quantity):
        listings.append({
            seller = seller_id,
            item = item_id,
            price = price,
            qty = quantity,
            time = Time.get_unix_time(),
        })

    func buy(buyer_id, listing_idx, amount):
        var listing = listings[listing_idx]
        var total = listing.price * amount
        if buyer.gold >= total:
            buyer.gold -= total
            listing.seller.gold += int(total * 0.95) # 5% 税
            buyer.inventory.add(listing.item, amount)
            listing.qty -= amount
            if listing.qty <= 0:
                listings.erase(listing)

    func search(query: String) -> Array:
        return listings.filter(func(l): return query.to_lower()

```

```

in l.item.to_lower())

func get_price_history(item_id: String) -> Array:
    # 返回近期的成交价格，用来做价格参考
    return history.filter(func(h): return h.item_id ==
item_id)

```

4. 最终方案模板

```

# AuctionManager.gd (联网用)
# 核心: 上架 → 搜索 → 购买 → 邮件发货

func list(item: String, price: int, duration_hours: int = 24):
    ServerAPI.send("auction_list", {
        item = item, price = price,
        duration = duration_hours * 3600,
    })

func search(keyword: String, sort: String = "price_asc"):
    ServerAPI.send("auction_search", {
        keyword = keyword, sort = sort,
        page = _current_page,
    })

func buy(listing_id: String):
    ServerAPI.send("auction_buy", {
        listing_id = listing_id,
    })
    # 购买成功后，物品通过邮件发送

# 服务端负责:
# - 持久化所有上架记录
# - 到期自动下架
# - 交易抽税 (5%)
# - 价格趋势记录

```

第57章 耐久度修理系统演化

Godot 耐久度/修理系统: 从永不磨损到经济消耗

耐久度 = 装备有使用次数, 用完需要修理。最早装备买一次用一辈子。后来有了耐久度、修理费、成功率、熔炉分解。耐久度的进化 = "怎么让玩家持续有金币消耗, 防止通货膨胀"。

1. 原始: 永不磨损

```
# 装备买一次, 永远不会坏
```

2. 固定耐久度

```
class Equipment:
    var durability: int = 100
    var max_durability: int = 100

    func use():
        durability -= 1
        if durability <= 0:
            # 装备失效, 但保留
            is_broken = true

    func repair():
        durability = max_durability
        # 收取固定修理费
        CurrencyManager.spend("gold", repair_cost)
```

3. 修理费随次数递增

```
class DurabilitySystem:
    var durability: int
    var max_durability: int
    var repair_count: int = 0 # 修理次数

    func get_repair_cost() -> int:
        var base_cost = 100
        var mult = 1.0 + repair_count * 0.2 # 每次修理涨价 20%
        return int(base_cost * mult)

    func repair():
        if durability >= max_durability: return
        var cost = get_repair_cost()
        if CurrencyManager.spend("gold", cost):
            durability = max_durability
            repair_count += 1

    func use():
        durability -= 1
        if durability <= 0:
            is_broken = true
            stats_multiplier = 0.5 # 损坏后属性减半
```

4. 最终方案模板

```
# DurabilityComponent.gd
extends Node

@export var max_durability: int = 100
@export var repair_cost_per_point: int = 5

var current: int = 100
var repair_times: int = 0
var is_broken: bool = false

func consume(amount: int = 1):
    current = max(current - amount, 0)
```

```

    if current <= 0:
        is_broken = true
        on_broken.emit()

func get_repair_cost() -> int:
    return int((max_durability - current) *
repair_cost_per_point * (1 + repair_times * 0.15))

func repair():
    if current >= max_durability: return
    var cost = get_repair_cost()
    if not CurrencyManager.spend("gold", cost): return
    current = max_durability
    is_broken = false
    repair_times += 1
    on_repair.emit()

```

第58章 锻造炼金系统演化

Godot 锻造/炼金系统：从商店买到亲手打造

锻造/炼金 = 用材料自己制造装备/药水。最早装备从商店买, 药水靠掉落。后来有了配方指定、随机词缀、品质浮动、熟练度等级。锻造的进化 = "怎么让制造本身成为一个可玩的系统"。

1. 原始：商店购买

```
# 所有物品在商店买
func buy_sword():
    if gold >= 500:
        gold -= 500
        inventory.add("iron_sword")
```

2. 固定配方

```
var recipes = {
    "iron_sword": {
        "materials": { "iron_ore": 5, "wood": 2 },
        "result": "iron_sword",
    },
    "health_potion": {
        "materials": { "herb": 3, "water": 1 },
        "result": "health_potion",
    },
}

func craft(recipe_id: String) -> bool:
    var recipe = recipes[recipe_id]
```



```

for mat in recipe.materials:
    if not inventory.has(mat, recipe.materials[mat]):
        return false # 材料不足
for mat in recipe.materials:
    inventory.remove(mat, recipe.materials[mat])
inventory.add(recipe.result)
return true

```

3. 品质 + 随机词缀

```

class CraftResult:
    var item_id: String
    var quality: String # normal / magic / rare / legendary
    var extra_stats: Dictionary = {}

func craft(recipe_id: String) -> CraftResult:
    var recipe = RecipeDB.get(recipe_id)
    if not has_materials(recipe): return null
    consume_materials(recipe)

    var result = CraftResult.new()
    result.item_id = recipe.result_item

    # 品质随机
    var roll = randf()
    if roll < 0.6:      result.quality = "normal"
    elif roll < 0.85:  result.quality = "magic"
    elif roll < 0.97:  result.quality = "rare"
    else:              result.quality = "legendary"

    # 高品质额外词缀
    if result.quality == "magic":
        result.extra_stats = _random_prefix(1)
    elif result.quality == "rare":
        result.extra_stats = _random_prefix(2)
    elif result.quality == "legendary":
        result.extra_stats = _random_prefix(3) +
        _random_suffix(1)

    return result

```

4. 熟练度

```
class CraftingSkill:
    var level: int = 1
    var exp: int = 0
    var max_level: int = 100

    func get_exp_for_level(lv: int) -> int:
        return lv * 100

    func add_exp(amount: int):
        exp += amount
        while exp >= get_exp_for_level(level) and level <
max_level:
            exp -= get_exp_for_level(level)
            level += 1
            # 升级解锁新配方

    func get_quality_bonus() -> float:
        return 1.0 + level * 0.005 # 每级 +0.5% 高品质概率

    func craft(recipe, crafter):
        var bonus = crafter.crafting_skill.get_quality_bonus()
        var roll = randf() / bonus
        # 结果: 熟练度高 → 高品质概率高
        return _roll_quality(roll)
```

5. 最终方案模板

```
# CraftingSystem.gd (Autoload)
# 核心: 配方 → 材料消耗 → 品质判定 → 产出

func craft(recipe_id: String, crafter) -> CraftResult:
    var recipe = RecipeDB.get(recipe_id)
    if not _has_materials(crafter, recipe): return null
    _consume_materials(crafter, recipe)
    var quality = _roll_quality(crafter.crafting_level)
    var result = _create_item(recipe.result_id, quality)
```

```
crafter.crafting_skill.add_exp(recipe.exp_reward)
return result
```

第59章 红包竞猜系统演化

Godot 红包/竞猜系统: 从打赏到概率博弈

红包/竞猜 = 玩家消耗货币换取随机奖励或参与竞猜。最早是简单打赏。后来有了拼手气红包、竞猜池、赔率、开奖动画。红包的进化 = "怎么让随机性带来社交乐趣而不是赌博感"。

1. 原始: 打赏

```
func send_gift(target_id: String, amount: int):  
    if my_gold >= amount:  
        my_gold -= amount  
        target.gold += amount
```

2. 拼手气红包

```
class RedPacket:  
    var total_amount: int  
    var count: int  
    var remaining_amount: int  
    var remaining_count: int  
    var sender_id: String  
  
    func grab() -> int:  
        if remaining_count <= 0: return 0  
        if remaining_count == 1:  
            var amount = remaining_amount  
            remaining_amount = 0  
            remaining_count = 0  
            return amount
```

```

        # 随机分配: 最少 1, 最多剩余平均值的 2 倍
        var max_amount = max(1, remaining_amount /
remaining_count * 2)
        var amount = randi_range(1, int(max_amount))
        amount = min(amount, remaining_amount - remaining_count
+ 1)
        remaining_amount -= amount
        remaining_count -= 1
        return amount

```

3. 竞猜系统

```

class BettingPool:
    var pool_id: String
    var options: Array[BetOption] # [{id, label, odds,
total_bet}]
    var total_pool: int = 0
    var end_time: int

    func place_bet(player_id: String, option_id: String, amount:
int):
        if CurrencyManager.spend(player_id, "gold", amount):
            bets.append({ player = player_id, option =
option_id, amount = amount })
            for opt in options:
                if opt.id == option_id:
                    opt.total_bet += amount
                    total_pool += amount
                    _update_odds(opt)

    func settle(winner_option_id: String):
        var winner = null
        for opt in options:
            if opt.id == winner_option_id:
                winner = opt

        if not winner or winner.total_bet == 0: return
        var pool_after_tax = int(total_pool * 0.95) # 5% 抽水
        for bet in bets:
            if bet.option == winner_option_id:
                var share = int(pool_after_tax * bet.amount /

```

```
winner.total_bet)
    CurrencyManager.add(bet.player, "gold", share)

func _update_odds(option):
    option.odds = total_pool / max(option.total_bet, 1)
```

4. 最终方案模板

```
# RedPacketManager.gd / BettingManager.gd
# 核心：发红包 → 抢红包 / 下注 → 结算

func send_redpacket(amount: int, count: int):
    if CurrencyManager.spend("gold", amount):
        ServerAPI.send("redpacket_create", { amount = amount,
count = count })
        # 全服通知
        Chat.broadcast(tr("REDPACKET_SENT") % [player.name,
amount])

func grab_redpacket(redpacket_id: String):
    ServerAPI.send("redpacket_grab", { id = redpacket_id })
    # 服务器分配合并结果
    # 客户端等待开启动画

# 竞猜:
func place_bet(match_id: String, option: String, amount: int):
    ServerAPI.send("bet_place", { match = match_id, option =
option, amount = amount })

func on_settle(match_id: String, winner: String):
    # 开奖动画 + 奖励发放
    var result = await ServerAPI.wait("bet_result", { match =
match_id })
    if result.win:
        Notification.show(tr("BET_WON") % result.reward)
```

第60章 折扣限购系统演化

Godot 折扣/限购系统：从统一定价到运营策略

折扣/限购 = 商品打折、每人限购数量、限时特卖。最早所有商品价格固定, 不限量。后来有了折扣率、限购次数、倒计时、VIP 额外折扣。折扣的进化 = "怎么用价格杠杆调控经济"。

1. 原始：固定价格

```
# 所有商品固定价格
var shop_items = {
    "potion": { "price": 50, "stock": INF },
    "sword": { "price": 500, "stock": INF },
}
```

2. 限购

```
class ShopItem:
    var item_id: String
    var price: int
    var daily_limit: int = 0    # 0 = 不限购
    var total_limit: int = 0

class PurchaseRecord:
    var player_id: String
    var item_id: String
    var daily_count: int = 0
    var total_count: int = 0

func can_purchase(item: ShopItem, record: PurchaseRecord) ->
```

```

bool:
    if item.daily_limit > 0 and record.daily_count >=
item.daily_limit:
    return false
    if item.total_limit > 0 and record.total_count >=
item.total_limit:
    return false
    return true

```

3. 折扣活动

```

class DiscountEvent:
    var name: String
    var item_ids: Array[String]
    var discount_rate: float # 0.8 = 八折
    var start_time: int
    var end_time: int
    var vip_only: bool = false

func get_current_price(item_id: String) -> int:
    var base_price = ShopDB.get_price(item_id)
    var discount = _get_active_discount(item_id)
    if discount:
        return int(base_price * discount.discount_rate)
    return base_price

func _get_active_discount(item_id: String) -> DiscountEvent:
    var now = Time.get_unix_time()
    for event in active_events:
        if item_id in event.item_ids and now >= event.start_time
and now < event.end_time:
            if event.vip_only and not player.is_vip: continue
            return event
    return null

```

4. 最终方案模板

```
# DiscountManager.gd (Autoload)
# 核心: 基础价格 → 折扣活动 → VIP 加成 → 最终价

func get_final_price(item_id: String, player) -> int:
    var base = ShopDB.get(item_id).price
    var discount = 1.0
    for event in _active_discounts:
        if item_id in event.items:
            discount = min(discount, event.rate)
    if player.is_vip:
        discount *= 0.9 # VIP 额外九折
    return int(base * discount)

func can_buy(item_id: String, player) -> bool:
    var record = _records.get(player.id, {}).get(item_id)
    var limit = ShopDB.get(item_id)
    if limit.daily > 0 and record and record.daily >=
limit.daily: return false
    if limit.total > 0 and record and record.total >=
limit.total: return false
    return true
```

第四卷 · UI 与显示

共计 18 章

第61章 UI 系统演化

Godot UI 系统演化史

"UI 没有算法"——这是最大的误解。

从坐标硬编码到自适应布局, 从手写 `set_text` 到数据驱动, 从千篇一律到主题系统, UI 的进化史就是"怎么让界面自动干活"的历史。

目录

1. [布局: 从像素坐标到 Container](#)
 2. [主题: 从手写颜色到 Theme 系统](#)
 3. [数据绑定: 从手动刷新到观察者模式](#)
 4. [UI 动画: 从僵硬到流畅](#)
 5. [层级管理: UI 栈与模态](#)
 6. [UI 与场景分离: UI Manager](#)
 7. [响应式 UI: 不同屏幕不同布局](#)
 8. [最终方案? 选型指南](#)
-

1. 布局: 从像素坐标到 Container

问题: "我要在屏幕左上角放一个血量文字。"

1.0 原始时代: 硬编码坐标

```
# Label 节点
func _ready():
    position = Vector2(20, 20) # 离左上角 20px
```

看着挺对。直到你换了一个分辨率。

```
1280×720: label 在 (20, 20) → 看起来在左上角
1920×1080: label 还在 (20, 20) → 左上角, 但位置比例不对
800×600: label 还在 (20, 20) → 也还行
iPhone 比例: 同样是 20px, 但 SafeArea 切掉了!
```

问题本质: 屏幕尺寸和比例不是固定的, 但坐标是固定的。

1.1 Anchor (锚点) 的诞生

Godot 的解决方案: 锚点 – 告诉控件"你相对于哪条边定位"。

```
Anchor:
    left = 0, top = 0 → 相对于左上角
    left = 1, top = 1 → 相对于右下角
    left = 0.5 → 相对于水平中线
```

```
# 血量文字: 固定在左上角, 无论屏幕多大
$Label.anchor_left = 0.0
$Label.anchor_top = 0.0
$Label.anchor_right = 0.0
$Label.anchor_bottom = 0.0
# offset 控制距离边的像素偏移
$Label.offset_left = 20
$Label.offset_top = 20
```

更好的做法: 用编辑器 Preset:

```
Anchor Preset:
    ■ Top Left    → (0, 0)
    ■ Top Right   → (1, 0)
    ■ Bottom Left → (0, 1)
```

- Center → (0.5, 0.5)
- Full Rect → (0, 0, 1, 1) ← 铺满全屏

但锚点只能控制"位置", 如果要"控件随窗口大小变化而自动排列"呢?

1.2 Container (容器) 系统

Container 是一类特殊的 Control 节点, 它自动管理子节点的布局。

```
# VBoxContainer - 垂直排列
VBoxContainer          ← 自动从上到下排列
├─ Label ("血量")      ← 不需要设置 position
├─ Label ("蓝量")
├─ Label ("体力")
└─ Button ("背包")

# HBoxContainer - 水平排列
HBoxContainer
├─ Button ("确认")
├─ Button ("取消")
└─ Button ("应用")

# GridContainer - 网格排列
GridContainer (columns = 2)
├─ Label ("姓名:")      Label ("张三")
├─ Label ("等级:")      Label ("Lv.10")
└─ Label ("职业:")      Label ("战士")
```

Container 的嵌套: 90% 的 UI 布局靠这个完成:

```
MarginContainer (外间距)
├─ VBoxContainer (纵向主布局)
│   └─ HBoxContainer (顶部栏)
│       ├── TextureRect (游戏 Logo)
│       ├── Stretch (占位弹簧)
│       └─ HBoxContainer
│           ├── Button ("设置")
│           └─ Button ("退出")
├─ CenterContainer (居中)
│   └─ Button ("开始游戏") ← 自动居中
```



```
└─ PanelContainer (底部信息栏)
   └─ Label ("版本 1.0")
```

Container 的核心机制 – size_flags:

```
# size_flags 决定子节点如何分配空间
# size_flags_vertical = SIZE_EXPAND → 纵向填充剩余空间
# size_flags_horizontal = SIZE_EXPAND → 横向填充

var panel = PanelContainer.new()
panel.size_flags_vertical = Control.SIZE_EXPAND # 吃掉所有剩余高度
panel.size_flags_horizontal = Control.SIZE_EXPAND # 吃掉所有剩余宽度
```

Container 系统和锚点的区别:

锚点: 告诉子节点"你相对于父节点的哪条边"

- └ 适合: HUD、固定位置的 UI 元素

Container: 父节点自动排列子节点

- └ 适合: 面板、菜单、列表、设置界面

Container 的进化, 解决了 UI 最大的痛点: 不再手算坐标。加一个按钮, 自动排到下面。窗口调大, 所有控件自动重新排列。

但很多新手会犯一个错:

```
# ❌ 在 Container 里手动设 position (Container 会覆盖)
$Button.position = Vector2(100, 100) # Container 下一帧就重排了

# ✅ 用容器控制布局
$HBoxContainer.add_child($Button) # 自动排好
```

2. 主题: 从手写颜色到 Theme 系统

2.0 原始: 每个节点单独配置

```
func _ready():
    $Button.modulate = Color.RED          # 按钮颜色
    $Button.get("theme_override_colors/font_color") =
    Color.WHITE
    $Label.add_theme_font_size_override("font_size", 24)
    $Label.add_theme_color_override("font_color", Color.YELLOW)
```

20 个按钮就要写 20 遍。改一个主色调, 改 20 处。

2.1 Theme 主题

```
# Theme.tres - 一个资源文件, 全局生效
# 在项目设置中设置: 项目 → 主题 → 主主题

# 或者给某个节点设置:
$Panel.theme = preload("res://themes/game_theme.tres")
```

Theme 里可以定义什么:

```
Theme:
├─ 默认字体 (default_font)
├─ 颜色
│   ├── font_color (文字颜色)
│   ├── font_hover_color (悬停文字颜色)
│   └── font_pressed_color (按下文字颜色)
├─ 样式 (StyleBox)
│   ├── normal (正常状态背景)
│   ├── hover (悬停状态)
│   ├── pressed (按下状态)
│   └── disabled (禁用状态)
├─ 图标
│   ├── close (关闭按钮图标)
│   └── arrow (下拉箭头)
└─ 常量
```

```
|— h_separation (水平间距)
|— v_separation (垂直间距)
```

Theme 的优先级:

节点 theme_override > 节点 theme > 主题资源 > 默认主题

2.2 Theme Variant (变体) – Godot 4.2+

```
# 同一主题的不同变体
"default" → 普通模式
"dark"    → 暗黑模式
"high_contrast" → 高对比度模式

# 在代码里切换
ThemeDB.set_project_theme(dark_theme)
```

Theme 系统的进化意义:

手写颜色 → Theme 资源 → Theme Variant

每个阶段解决的问题:

手写颜色: 改一个按钮颜色要改 20 次 ❌

Theme: 改一个 Theme, 所有按钮都变 ✅

Theme Variant: 整个 UI 一键切换白天/黑夜 ✅

3. 数据绑定: 从手动刷新到观察者模式

3.0 原始: 手动 set_text

```
func _process(delta):
    $HP.text = "HP: %d" % player.hp
    $MP.text = "MP: %d" % player.mp
    $Level.text = "Lv.%d" % player.level
    $Exp.text = "EXP: %d/%d" % [player.exp, player.max_exp]
```

```
$Gold.text = "Gold: %d" % player.gold
# ... 还有 10 个 UI 元素
```

问题: 1. 每帧刷新 – player.hp 又没变, 你刷它干嘛? 2. 逻辑散落 – 改 UI 的地方在 _process 里, 和 UI 逻辑隔了十万八千里 3. 数据源改了但 UI 不知道 – 如果 player.hp 变了, 但 UI 还在显示旧值

3.1 Signal 驱动刷新

```
# Player.gd
signal hp_changed(new_hp)

func take_damage(amount):
    hp -= amount
    hp_changed.emit(hp)

# UI.gd
func _ready():
    player.hp_changed.connect(_on_hp_changed)

func _on_hp_changed(new_hp):
    $HP.text = "HP: %d" % new_hp
```

进步: 只在数据变化时更新 UI, 不再每帧轮询。

3.2 Setter 自动通知

```
# Player.gd
var hp: int:
    set(value):
        hp = value
        hp_changed.emit(hp)

var mp: int:
    set(value):
        mp = value
        mp_changed.emit(mp)
```

进步: 赋值即通知, 不需要在每个修改的地方手动 emit。

3.3 Resource 作为数据模型

```
# PlayerData.gd
class_name PlayerData
extends Resource

@export var hp: int = 100
@export var max_hp: int = 100
@export var mp: int = 50
@export var gold: int = 0

# UI.gd
@export var player_data: PlayerData

func _ready():
    # Resource 是引用类型，修改会反映到所有持有者
    $HP.text = "HP: %d" % player_data.hp

func _process(delta):
    # 不需要，因为 Resource 没有内置变更通知...
    # 除非用 Resource.changed 信号
    player_data.changed.connect(_refresh_all)

func _refresh_all():
    $HP.text = "HP: %d / %d" % [player_data.hp,
    player_data.max_hp]
    $MP.text = "MP: %d" % player_data.mp
    $Gold.text = "Gold: %d" % player_data.gold
```

3.4 观察者模式 (Observer) / 响应式

```
# 一个简单的数据绑定系统

class Observable:
    var _value
    var _observers = []

    func set_value(val):
        if _value != val:
            _value = val
            _notify()
```

```

func get_value():
    return _value

func observe(callback: Callable):
    _observers.append(callback)

func _notify():
    for obs in _observers:
        obs.call(_value)

# 使用:
var hp = Observable.new()
hp.set_value(100)
hp.observe(func(val): $HP.text = "HP: %d" % val)

hp.set_value(80) # ← 自动更新 UI, 不需要手动调用

```

3.5 UI 数据绑定的进化意义

每帧 set_text (浪费性能)

- Signal (只在变化时更新)
- Setter 自动 emit (散落的赋值统一了)
- Resource 模型 (数据和 UI 解耦)
- 观察者/绑定 (真正响应式, 改数据 = 改 UI)

4. UI 动画: 从僵硬到流畅

4.0 原始: 直接 show/hide

```

$Panel.show()      # 啪, 出现了
$Panel.hide()      # 啪, 消失了

```

像石头一样生硬。没有任何过渡, 用户体验极差。

4.1 用 Tween 做过渡

```
# 淡入
func show_panel():
    $Panel.modulate = Color(1, 1, 1, 0) # 透明
    $Panel.show()
    var tween = create_tween()
    tween.tween_property($Panel, "modulate", Color.WHITE, 0.3)

# 滑入
func slide_in():
    $Panel.position.x = -200
    var tween = create_tween()
    tween.tween_property($Panel, "position:x", 0,
0.4).set_trans(Tween.TRANS_BOUNCE)

# 缩放弹出
func pop_in():
    $Panel.scale = Vector2.ZERO
    var tween = create_tween()
    tween.tween_property($Panel, "scale", Vector2.ONE,
0.3).set_trans(Tween.TRANS_BACK)
```

4.2 缓动曲线 (Easing)

```
# 不同的缓动效果
# 物理世界没有"匀速运动", UI 也不应该有

Tween.TRANS_LINEAR → 匀速 (最机械)
Tween.TRANS_SINE → 正弦缓动
Tween.TRANS_QUAD → 二次
Tween.TRANS_CUBIC → 三次
Tween.TRANS_BOUNCE → 弹跳效果 (像物理)
Tween.TRANS_BACK → 过冲再回弹
Tween.TRANS_ELASTIC → 橡皮筋
```

4.3 序列动画

```
# 复杂的 UI 动画序列
var tween = create_tween()
```

```

tween.set_parallel(true) # 同时执行
tween.tween_property($Panel, "modulate:a", 1.0, 0.3) # 淡入
tween.tween_property($Panel, "position:y", 0,
0.4).set_trans(Tween.TRANS_BOUNCE) # 弹入
tween.tween_property($Icon, "scale", Vector2.ONE,
0.5).set_trans(Tween.TRANS_BACK)
tween.chain() # 上面都完成后, 接下面的
tween.tween_callback($Content.show) # 显示内容
tween.tween_property($Content, "modulate:a", 1.0, 0.2) # 内容淡入

```

好的 UI 动画原则:

Duration: 0.2s ~ 0.5s (太快看不清, 太慢等不及)
 Easing: 进入用 ease_out, 退出用 ease_in
 一致性: 同类元素用相同的动画参数
 物理感: 面板像有质量, 弹窗像弹簧

但 Tween 多了有个问题: 代码里散布着各种 tween 调用, UI 过渡逻辑分散在各处。

4.4 统一动画管理器

```

# 一个集中管理 UI 动画的工具
class UIAnimator:
    static func fade_in(node: Control, duration: float = 0.3):
        node.modulate = Color(1, 1, 1, 0)
        node.show()
        var t = node.create_tween()
        t.tween_property(node, "modulate", Color.WHITE,
duration)

    static func slide_in_from_left(node: Control, duration:
float = 0.4):
        node.position.x = -node.size.x
        var t = node.create_tween()
        t.tween_property(node, "position:x",
node.get_parent_control().size.x, duration) \
        .set_trans(Tween.TRANS_BOUNCE)

    static func pop(node: Control, duration: float = 0.3):
        node.scale = Vector2.ZERO

```



```

        node.show()
        var t = node.create_tween()
        t.tween_property(node, "scale", Vector2.ONE,
duration).set_trans(Tween.TRANS_BACK)

# 使用:
UIAnimator.fade_in($SettingsPanel)
UIAnimator.pop($Notification)

```

5. 层级管理: UI 栈与模态

问题: 打开背包 → 打开装备详情 → 打开确认对话框 → 关闭对话框 → 关闭详情 → 关闭背包。

如果只是 `show/hide`, 你会在关闭某个界面后忘记显示之前的界面。

5.0 原始: 手动管理

```

func _on_close_equip_detail():
    $EquipDetail.hide()
    $Inventory.show() # 手动切回去

```

但如果界面不是 `Inventory` 打开的, 而是 `Shop` 打开的? 你得记住"上一个界面是谁"。

5.1 UI 栈 (UI Stack)

```

# UI 管理器 + 栈
class UIManager:
    var _stack = []

    func push(screen: Control):
        if _stack.size() > 0:
            _stack.back().hide()
        _stack.append(screen)
        screen.show()

```

```

func pop():
    if _stack.size() == 0:
        return
    _stack.back().hide()
    _stack.pop_back()
    if _stack.size() > 0:
        _stack.back().show()

func clear():
    while _stack.size() > 0:
        _stack.back().hide()
        _stack.pop_back()

```

```

# 使用:
ui_manager.push($MainMenu)    # MainMenu 显示
ui_manager.push($Settings)    # Settings 显示, MainMenu 隐藏
ui_manager.pop()              # Settings 关闭, MainMenu 恢复
ui_manager.pop()              # MainMenu 关闭

```

5.2 模态对话框

```

# 模态: 弹出一个对话框, 底层不能操作
func show_modal(dialog: Control):
    # 1. 推入栈
    _stack.back().hide()
    _stack.append(dialog)
    dialog.show()

    # 2. 暗色遮罩
    $ModalOverlay.show()
    $ModalOverlay.modulate = Color(0, 0, 0, 0.5)

    # 3. 禁用底层交互
    _stack[_stack.size() - 2].mouse_filter =
Control.MOUSE_FILTER_IGNORE

func close_modal():
    $ModalOverlay.hide()
    pop()

```

5.3 层 (Layer) 系统

```
# 游戏 UI 分层：
# Layer 0: 游戏世界 (3D/2D)
# Layer 1: HUD (血量、子弹数、小地图)
# Layer 2: 菜单 (背包、设置)
# Layer 3: 对话框 (确认弹窗)
# Layer 4: 过场动画

# 每个层独立管理，互不影响
class UILayer:
    var screens = []
    var visible = true

    func show():
        visible = true
        for screen in screens:
            screen.show()

    func hide():
        visible = false
        for screen in screens:
            screen.hide()

# 使用：
UILayer.get("hud").show()          # 显示 HUD
UILayer.get("menu").push(inventory) # 背包在菜单层
UILayer.get("overlay").push(dialog) # 对话框在覆盖层
```

6. UI 与场景分离: UI Manager

问题: 大部分新手把所有 UI 塞在玩家场景里。

```
Player.tscn
├─ Sprite2D
├─ CharacterBody2D
├─ Label ("HP")           ← UI 混在游戏物体里
├─ Label ("子弹数")       ← UI 混在游戏物体里
```

```
├─ TextureProgressBar ← UI 混在游戏物体里
├─ ...
```

问题: 1. **UI 和游戏逻辑耦合** – 改 UI 布局要打开玩家场景 2. **复用困难** – 另一个角色也想显示血条? 再拖一套 UI? 3. **坐标参考混乱** – UI 坐标基于玩家位置? 那玩家移动 UI 也跟着动?

6.1 CanvasLayer: UI 和世界分离

```
# CanvasLayer 是一个"永远在屏幕上固定位置"的层
# 不管摄像机怎么动, CanvasLayer 的子节点都在屏幕位置

CanvasLayer (HUD)
├─ Label ("HP: 100")      ← 屏幕左上角, 不随摄像机移动
├─ TextureProgressBar      ← 血条在屏幕底部
├─ Crosshair              ← 准星在屏幕中央
├─ ...
```

6.2 独立的 UI 场景

UI 场景完全独立, 通过 **Autoload** 或 **signal** 和游戏逻辑通信:

```
res://ui/
├─ HUD.tscn          → 血量、弹药、小地图
├─ MainMenu.tscn     → 主菜单
├─ Inventory.tscn     → 背包
├─ Settings.tscn     → 设置
├─ PauseMenu.tscn    → 暂停
├─ DialogueBox.tscn  → 对话
├─ DamageNumber.tscn → 伤害数字
```

```
# 一个集中管理 UI 的 Autoload
# ui_manager.gd (Autoload)

var scenes = {}
var active_screens = {}

func open(screen_name: String, data = null) -> Control:
    if not scenes.has(screen_name):
```

```

        scenes[screen_name] = preload("res://ui/%s.tscn" %
screen_name)

    var screen = scenes[screen_name].instantiate()
    add_child(screen)

    if data:
        if screen.has_method("set_data"):
            screen.set_data(data)

    active_screens[screen_name] = screen
    return screen

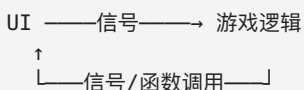
func close(screen_name: String):
    if active_screens.has(screen_name):
        active_screens[screen_name].queue_free()
        active_screens.erase(screen_name)

func is_open(screen_name: String) -> bool:
    return active_screens.has(screen_name)

# 使用:
UIManager.open("inventory", {"player": player})
UIManager.close("inventory")

```

6.3 UI 和游戏逻辑的通信方式



最佳实践:

1. UI 从来不直接修改游戏数据
2. UI 发出信号 (button_pressed, item_dropped)
3. 游戏逻辑响应信号, 修改数据
4. 数据修改后通过信号通知 UI 刷新

举例:

背包 UI 点"使用药水" → 发出 use_potion 信号
 玩家脚本收到信号 → player.heal(20)
 玩家发出 hp_changed 信号
 HUD 收到信号 → 更新血条显示 

6.4 UI 管理器的进化意义

```
UI 散落在各个场景
→ CanvasLayer 分离世界/屏幕坐标
→ 独立 UI 场景（代码清晰，复用）
→ UI Manager（集中管理生命周期）
→ UI 和逻辑通过信号通信（松耦合）
```

7. 响应式 UI: 不同屏幕不同布局

问题: 手机竖屏 → 按钮在底部; 电脑横屏 → 按钮在右侧; 平板 → 按钮在左下。

Godot 的方案:

7.1 多分辨率策略

```
# 项目设置 → 窗口 → 拉伸
# Mode:                canvas_items / viewport / disabled
# Aspect:               keep / keep_width / keep_height / expand
# Scale:                1.0 （或根据自定义倍数）
```

四种模式:

```
keep:                保持原始比例，留黑边
keep_width:          宽度撑满，高度可裁剪
keep_height:         高度撑满，宽度可裁剪
expand:              完全撑满，会拉伸
```

7.2 用 Container 做响应式

```
# 导航栏: 手机端在底部, PC 端在左侧
func _ready():
    if DisplayServer.window_get_size().x < 768:
        # 手机模式: 底部导航
        $NavigationContainer.direction =
```

```

Container.DIRECTION_VERTICAL
    $NavigationContainer.anchor_bottom = 1.0
    $NavigationContainer.anchor_top = 0.9
else:
    # PC 模式：左侧导航
    $NavigationContainer.direction =
Container.DIRECTION_HORIZONTAL
    $NavigationContainer.anchor_right = 0.2
    $NavigationContainer.anchor_left = 0.0

```

7.3 用代码生成 UI 做动态布局

```

# 根据道具数量动态生成格子（背包）
func refresh_inventory(items: Array):
    for child in $GridContainer.get_children():
        child.queue_free() # 清空旧的

    for item in items:
        var slot = preload("res://ui/
ItemSlot.tscn").instantiate()
        slot.setup(item) # 传入道具数据
        $GridContainer.add_child(slot)
# GridContainer 自动排列

```

7.4 响应式的进化

```

固定分辨率（1280×720 only）
→ 拉伸模式（所有屏幕比例）
→ Anchor + Container（自动适应）
→ 代码检测屏幕宽度切换布局（响应式）

```

8. 最终方案? 选型指南

布局方案

界面类型	推荐方案
------	------

HUD（血量、准星）	Anchor（固定到某个角落）
菜单、面板、弹窗	Container 嵌套
列表、物品格	Container + 动态生成
全屏 UI（暂停菜单）	Anchor 铺满 + Container
聊天框/日志	ScrollContainer + VBoxContainer

UI 架构

项目规模	推荐方案
Game Jam / 原型	在场景里直接放 UI 节点
小项目（1-3月）	独立 UI 场景 + CanvasLayer
中等项目	UI Manager + UI 栈 + Theme
大型项目	UI Manager + UI 栈 + Theme Variant + 数据绑定

性能注意事项

1. 不要每帧更新 UI 文字
→ 用 Signal 或观察者驱动
2. 动态生成 UI 用对象池
→ 伤害数字、掉落列表频繁创建/销毁
3. 复杂的 Container 嵌套有性能开销
→ 深度超过 10 层考虑合并
4. Theme 的样式越复杂，渲染越慢
→ 大量按钮的九宫格 StyleBox 比纯色贵

常见陷阱

- ✗ UI 节点混在游戏场景里，没有 CanvasLayer
→ UI 会跟着摄像机抖
- ✗ 通过 _process 每帧更新 UI
→ 玩家又没掉血，你刷什么血条？
- ✗ 在 Container 里手动设 position
→ Container 下一帧就给你重排了

- ✗ 用 Tween 动画时忘了处理节点被删
 - Tween 还在跑，节点已经 free 了 → 报错
 - 或: `tween.kill()` 在节点退出时
- ✗ 所有 UI 在一个场景文件里
 - 加载慢，改一处要重新导入整个场景
 - 拆成独立场景，按需加载
- ✗ 英文文本在 UI 里正常，换中文就溢出
 - `Label.autowrap_mode` 和 `Container` 的 `min_size`
 - 中文两字顶英文四字

附: UI Manager 最简模板

```
# UIManager.gd (设为 Autoload)

extends Node

@onready var ui_layer := $UI          # CanvasLayer 节点
@onready var overlay := $Overlay      # 遮罩层

var _stack := []
var _cache := {}

func open(path: String) -> Control:
    # 从缓存或文件加载
    if not _cache.has(path):
        _cache[path] = load(path)
    var screen = _cache[path].instantiate()

    # 入栈
    if _stack.size() > 0:
        _stack.back().hide()
    ui_layer.add_child(screen)
    _stack.append(screen)
    return screen

func close():
    if _stack.size() == 0:
        return
```

```

        _stack.back().queue_free()
        _stack.pop_back()
    if _stack.size() > 0:
        _stack.back().show()

func close_all():
    while _stack.size() > 0:
        _stack.back().queue_free()
        _stack.pop_back()

func is_open() -> bool:
    return _stack.size() > 0

# 使用:
# UIManager.open("res://ui/Inventory.tscn")
# UIManager.close()

```

这个故事告诉我们: - UI 的进化是**从手算到自动**的过程 - 坐标硬编码 → Anchor (自动定位) - Anchor → Container (自动排列) - 手动刷新 → 数据绑定 (自动更新) - 散落 UI → UI Manager (自动生命周期) - UI 不是"没有算法", 而是"布局算法 + 数据流算法 + 动画算法"的集合 - 好的 UI 系统设计 = 改数据 UI 自动变, 不用写一行刷新代码

第62章 弹窗系统演化

Godot 弹窗系统: 从 show() 到队列管理

弹窗 = 任何突然出现在屏幕上的面板。最早是 show() / hide() 直接控制。后来有了遮罩层、点外面关闭、层级管理、弹窗队列。弹窗系统的进化 = "怎么让弹窗不打架"。

1. 原始: show/hide

```
$ConfirmDialog.show()
$ConfirmDialog.hide()
```

2. 模态弹窗 (带遮罩)

```
func show_popup(panel: Control):
    $Overlay.show()          # 半透明遮罩
    panel.show()             # 弹窗在最上面

func close_popup(panel: Control):
    $Overlay.hide()
    panel.hide()
```

3. 弹栈管理

```
# PopupManager.gd (Autoload)
extends Node

var _stack := []
```

```

func push(panel: Control):
    if _stack.size() > 0:
        _stack.back().hide() # 隐藏前一个
    panel.show()
    _stack.append(panel)

func pop():
    if _stack.size() == 0:
        return
    _stack.pop_back().hide()
    if _stack.size() > 0:
        _stack.back().show() # 显示前一个

func clear():
    for p in _stack:
        p.hide()
    _stack.clear()

# 使用:
# PopupManager.push($ShopPanel) # 打开商店
# PopupManager.push($ConfirmDialog) # 再弹确认框
# PopupManager.pop() # 关闭确认框, 回到商店
# PopupManager.pop() # 关闭商店

```

4. 最终方案模板

```

# PopupManager 核心:
# 1. 栈式管理: push/pop
# 2. 遮罩自动跟随
# 3. 点击遮罩关闭: overlay.gui_input → pop()
# 4. Esc 键关闭: _unhandled_input → pop()

func push(panel: Control):
    if not panel.has_meta("_in_stack"):
        panel.set_meta("_in_stack", true)
        panel.tree_exiting.connect(_on_panel_closed.bind(panel))
    $Overlay.show()
    panel.show()
    _stack.append(panel)

```

```
func pop():
    if _stack.is_empty(): return
    var panel = _stack.pop_back()
    panel.hide()
    $Overlay.visible = not _stack.is_empty()
    if _stack.size() > 0:
        _stack.back().show()
```

第63章 通知系统演化

Godot 通知/Toast 系统: 从 print 到飘字

通知 = 屏幕上的短暂文字提示。"获得金币 +100" "任务完成" "网络连接断开"。最早是 `print()` 打印在控制台。后来有了飘字、渐变消失、队列显示、多种颜色/图标。通知系统的进化 = "怎么让玩家第一时间知道发生了什么"。

1. 原始: print

```
print("获得 100 金币")
# 玩家看不到, 只有开发者能看到
```

2. Label 弹出

```
func show_message(text: String):
    $ToastLabel.text = text
    $ToastLabel.modulate.a = 1
    var tween = create_tween()
    tween.tween_interval(2.0)
    tween.tween_property($ToastLabel, "modulate:a", 0, 0.5)

# 问题: 如果连续触发, 文字会立刻覆盖
```


3. 队列通知

```
# ToastManager.gd (Autoload)
extends CanvasLayer

var _queue := []

func notify(text: String, type: String = "info"):
    _queue.append({ "text": text, "type": type })
    if _queue.size() == 1:
        _show_next()

func _show_next():
    if _queue.is_empty():
        return
    var msg = _queue[0]
    var label = _create_label(msg.text, msg.type)
    add_child(label)
    label.global_position = Vector2(400, 50)

    var tween = create_tween()
    tween.tween_interval(2.0)
    tween.tween_property(label, "position:y", label.position.y -
50, 0.5)
    tween.tween_callback(func():
        label.queue_free()
        _queue.pop_front()
        _show_next()
    )

func _create_label(text: String, type: String) -> Label:
    var label = Label.new()
    label.text = text
    match type:
        "info": label.modulate = Color.WHITE
        "gold": label.modulate = Color.YELLOW
        "item": label.modulate = Color.CYAN
        "error": label.modulate = Color.RED
    return label
```

4. 最终方案模板

```
# 最简单的通知：
# 一行调用，自动显示 2 秒后消失

# ToastManager.notify("金币 +100", "gold")
# ToastManager.notify("获得铁剑", "item")
# ToastManager.notify("体力不足!", "error")
```

第64章 小地图系统演化

Godot 小地图系统：从截图到动态标记

小地图告诉玩家"我在哪, 附近有什么"。从一张静态截图到实时标记的探索地图, 每个阶段的进化都在解决"怎么把世界变小但信息不变少"。

目录

1. [原始: 静态截图小地图](#)
 2. [摄像机渲染小地图](#)
 3. [动态标记: 显示敌人和任务点](#)
 4. [探索迷雾: 未探索区域隐藏](#)
 5. [大地图: 缩放和平移](#)
 6. [最终方案: 最简可用模板](#)
-

1. 原始: 静态截图小地图

问题: "玩家需要一个地图, 知道自己在哪。"

最初的做法: 放一张截图在角落。

```
func _ready():
    $Minimap.texture = preload("res://assets/
map_screenshot.png")
    $PlayerDot.position = Vector2(100, 200) # 硬编码玩家位置
```

问题: - 地图不会动, 玩家走到截图外面, 小地图没更新 - 每次改关卡都要重新截图 - 没有动态信息 (敌人在哪? 任务点在哪?)

2. 摄像机渲染小地图

问题: 地图要跟随玩家, 显示实时地形。

用第二个摄像机渲染小地图:

```
# 在场景里加一个 Camera2D, 只渲染到 Viewport
# Viewport → 小地图的纹理

# MinimapCamera.gd
func _ready():
    # 这个摄像机跟随玩家, 但只渲染到小地图 Viewport
    position = player.position

func _process(delta):
    position = player.position # 跟随玩家

# 或者用 SubViewport:
# SubViewportContainer → SubViewport → Camera2D
# Camera2D 的 target 设为玩家
```

更简单的: 用代码画小地图 (不用摄像机, 省性能):

```
# Minimap.gd
extends Control

@export var player: Node2D
@export var world_size: Vector2 = Vector2(2000, 2000)
@export var minimap_size: Vector2 = Vector2(200, 200)

var markers := [] # { "position": Vector2, "color": Color, "icon": Texture2D }

func _draw():
    # 清空
    draw_rect(Rect2(Vector2.ZERO, minimap_size), Color(0, 0, 0,
```

```

0.5))

    if not player:
        return

    # 比例：世界坐标 → 小地图坐标
    var scale_x = minimap_size.x / world_size.x
    var scale_y = minimap_size.y / world_size.y

    # 玩家在世界的偏移（小地图居中于玩家）
    var offset_x = player.position.x * scale_x -
minimap_size.x / 2
    var offset_y = player.position.y * scale_y -
minimap_size.y / 2

    # 绘制每个标记
    for marker in markers:
        var mx = marker.position.x * scale_x - offset_x
        var my = marker.position.y * scale_y - offset_y

        # 只画在小地图范围内的
        if mx >= 0 and mx <= minimap_size.x and my >= 0 and my
<= minimap_size.y:
            draw_circle(Vector2(mx, my), 3, marker.color)

    # 绘制玩家（居中）
    draw_circle(minimap_size / 2, 4, Color.GREEN)

func add_marker(node: Node2D, color: Color):
    markers.append({ "position": node, "color": color })

func _process(delta):
    queue_redraw() # 每帧重绘

```

比例计算：

世界大小 2000×2000，小地图 200×200
 比例 = 200 / 2000 = 0.1

玩家在 (500, 300)：
 小地图位置 = (500 × 0.1, 300 × 0.1) = (50, 30)

```
世界有个敌人在 (800, 600):  
小地图位置 = (800 × 0.1, 600 × 0.1) = (80, 60)
```

3. 动态标记: 显示敌人和任务点

问题: 小地图上只有玩家, 看不到敌人、宝箱、任务目标。

标记系统: 标记 = 世界中的节点 + 在小地图上的显示。

```
# 标记类型  
enum MarkerType { PLAYER, ENEMY, QUEST, TREASURE, NPC, DANGER }  
  
class MinimapMarker:  
    var node: Node2D          # 对应的场景节点  
    var type: MarkerType  
    var color: Color  
    var icon: Texture2D  
    var visible: bool = true
```

```
# MinimapManager.gd  
extends Node  
  
var markers := []  
var minimap: Control  
  
func register(node: Node2D, type: MarkerType):  
    var marker = MinimapMarker.new()  
    marker.node = node  
    marker.type = type  
    marker.color = _get_color(type)  
    markers.append(marker)  
  
func unregister(node: Node2D):  
    markers = markers.filter(func(m): return m.node != node)  
  
func _get_color(type: MarkerType) -> Color:  
    match type:  
        MarkerType.PLAYER:    return Color.GREEN  
        MarkerType.ENEMY:     return Color.RED  
        MarkerType.QUEST:     return Color.YELLOW
```

```

        MarkerType.TREASURE: return Color.CYAN
        MarkerType.NPC:      return Color.WHITE
        MarkerType.DANGER:   return Color.ORANGE

# 在敌人/宝箱/NPC 生成时:
# MinimapManager.register(self, MinimapManager.MarkerType.ENEMY)

```

标记更新条件:

```

func _draw_markers():
    for marker in markers:
        if not is_instance_valid(marker.node):
            continue # 节点被销毁了, 标记自动消失
        if not marker.visible:
            continue
        if _is_out_of_range(marker.node.global_position,
max_range):
            continue # 太远了不显示, 减少杂乱

        var screen_pos =
_world_to_minimap(marker.node.global_position)
        draw_circle(screen_pos, 3, marker.color)

```

4. 探索迷雾: 未探索区域隐藏

问题: 小地图把整个地图都显示了。玩家还没去过的地方也看到了, 没有探索感。

迷雾系统: 只有玩家探索过的区域才显示在地图上。

```

# 迷雾地图: 用一张纹理记录探索过的区域

class FogOfWar:
    var fog_image: Image
    var fog_texture: ImageTexture
    var map_width: int      # 迷雾图宽度 (像素)
    var map_height: int
    var world_to_fog: float # 世界坐标 → 迷雾图坐标的比例

```



```

func _init(world_w: float, world_h: float, fog_resolution:
int = 256):
    map_width = fog_resolution
    map_height = int(fog_resolution * (world_h / world_w))
    world_to_fog = map_width / world_w

    fog_image = Image.create(map_width, map_height, false,
Image.FORMAT_RGBA8)
    fog_image.fill(Color.BLACK) # 全黑: 未探索
    fog_texture = ImageTexture.create_from_image(fog_image)

func explore(position: Vector2, radius: float):
    var cx = position.x * world_to_fog
    var cy = position.y * world_to_fog
    var r = radius * world_to_fog

    for y in range(cy - r, cy + r):
        for x in range(cx - r, cx + r):
            if x < 0 or x >= map_width or y < 0 or y >=
map_height:
                continue
            if Vector2(x - cx, y - cy).length() <= r:
                # 探索过的区域设为透明 (可见)
                fog_image.set_pixel(x, y, Color(0, 0, 0, 0))

    fog_texture = ImageTexture.create_from_image(fog_image)

func reveal_all():
    fog_image.fill(Color(0, 0, 0, 0))
    fog_texture = ImageTexture.create_from_image(fog_image)

```

每帧更新迷雾:

```

func _process(delta):
    var fog = fog_of_war.explore(player.position, 300.0)
    $MinimapFog.texture = fog_of_war.fog_texture

```

迷雾叠加: 小地图底色显示探索过的地形, 迷雾层覆盖未探索区域, 只露出玩家周围一圈。

5. 大地图：缩放和平移

问题：小地图只能看附近。玩家想看整个世界的全局地图。

大地图：全屏地图，支持缩放和拖动。

```
# WorldMap.gd
extends Control

var zoom_level: float = 1.0
var offset: Vector2 = Vector2.ZERO
var is_dragging: bool = false
var drag_start: Vector2

# 地图标记（同小地图，但在大地图上全部显示）
var markers := []

func _input(event):
    # 滚轮缩放
    if event is InputEventMouseButton and visible:
        if event.button_index == MOUSE_BUTTON_WHEEL_UP:
            zoom_level *= 1.1
        elif event.button_index == MOUSE_BUTTON_WHEEL_DOWN:
            zoom_level /= 1.1
        zoom_level = clamp(zoom_level, 0.2, 5.0)
        queue_redraw()

    # 拖动
    if event is InputEventMouseButton and event.button_index ==
MOUSE_BUTTON_LEFT:
        if event.pressed:
            is_dragging = true
            drag_start = event.position
        else:
            is_dragging = false

    if event is InputEventMouseMotion and is_dragging:
        offset += event.relative / zoom_level
        queue_redraw()

func _draw():
    # 绘制地图背景
```

```
if map_texture:
    var rect = Rect2(offset, size * zoom_level)
    draw_texture_rect(map_texture, rect, false)

# 绘制所有标记（不裁剪范围，全部显示）
for marker in markers:
    if not is_instance_valid(marker.node):
        continue
    var pos = (marker.node.global_position * zoom_level) +
offset
    draw_circle(pos, 4 / zoom_level, marker.color)
    if zoom_level > 1.0:
        draw_string(font, pos + Vector2(6, 0), marker.label)
```

6. 最终方案: 最简可用模板

``gdscript

———— Minimap.gd (挂载到 CanvasLayer)

extends Control

@export var player: Node2D @export var world_size := Vector2(2000, 2000)

var markers := []

func _draw(): if not player: return var s = size var scale = s / world_size
var offset = player.position * scale - s / 2

```
# 浅色背景
draw_rect(Rect2(Vector2.ZERO, s), Color(0.1, 0.1, 0.15, 0.8))

# 标记
for m in markers:
    if not is_instance_valid(m.node):
        continue
    var p = m.node.global_position * scale - offset
    if p.x >= 0 and p.x <= s.x and p.y >= 0 and p.y <= s.y:
        draw_circle(p, 3, m.color)

# 玩家（居中）
draw_circle(s / 2, 4, Color.GREEN)
```

func add_marker(node: Node2D, color: Color = Color.RED):

markers.append({ "node": node, "color": color })

func _process(delta): queue_redraw()

使用：

在玩家或管理器脚本里：

\$Minimap.player = self

**\$Minimap.add_marker(enemy,
Color.RED)**

**\$Minimap.add_marker(quest_target,
Color.YELLOW)**

第65章 相机系统演化

Godot 相机系统：从固定视角到智能跟随

相机 = 玩家的眼睛。最早的相机固定在某个位置不会动。后来它会跟随玩家、平滑追踪、边界限制、缩放。相机系统进化 = "怎么让玩家看得舒服"。

1. 固定相机

```
# Camera2D 放在场景里，不跟随任何东西  
# 玩家走出屏幕 → 看不见了
```

2. 跟随玩家

```
# Camera2D 的 target 设为玩家  
# 或每帧设置位置  
func _process(delta):  
    position = player.position
```

3. 平滑跟随 (Lerp)

```
func _process(delta):  
    var t = 1.0 - exp(-5.0 * delta)  
    position = position.lerp(player.position, t)
```

4. 边界限制

```
# Camera2D 的 limit_left/top/right/bottom
# 玩家走到边界时，相机停住，不会露出关卡外的空白

# 代码设置：
$Camera2D.limit_left = 0
$Camera2D.limit_top = 0
$Camera2D.limit_right = 3200
$Camera2D.limit_bottom = 2400
```

5. 缩放与视口适配

```
# 滚轮缩放
func _input(event):
    if event is InputEventMouseButton:
        if event.button_index == MOUSE_BUTTON_WHEEL_UP:
            $Camera2D.zoom *= 1.1
        elif event.button_index == MOUSE_BUTTON_WHEEL_DOWN:
            $Camera2D.zoom /= 1.1
        $Camera2D.zoom = $Camera2D.zoom.clamp(Vector2(0.5, 0.5),
        Vector2(3, 3))
```

6. 最终方案模板

```
# Camera2D 平滑跟随 + 边界 + 缩放
extends Camera2D

@export var target: Node2D
@export var smooth_speed: float = 5.0
@export var zoom_speed: float = 0.1

func _process(delta):
    if not target:
        return
    var t = 1.0 - exp(-smooth_speed * delta)
    global_position =
```

```
global_position.lerp(target.global_position, t)

func _unhandled_input(event):
    if event is InputEventMouseButton:
        if event.button_index == MOUSE_BUTTON_WHEEL_UP:
            zoom *= 1.1
        elif event.button_index == MOUSE_BUTTON_WHEEL_DOWN:
            zoom /= 1.1
        zoom = zoom.clamp(Vector2(0.5, 0.5), Vector2(3, 3))
```

第66章 特效系统演化

Godot 特效系统: 从 Sprite 叠放到 GPU 粒子

特效 = "看起来很酷但没啥实际作用的东西"。但没有特效, 火球术就是一团黄色圆形, 升级就是一串文字跳出来。特效是游戏给玩家的情感反馈: - 暴击 → 屏幕震动 + 大数字 + 闪光 - 升级 → 金光环绕 + 粒子升空 - 捡到好东西 → 星形闪烁 + 叮的声音

1. 原始: Sprite 闪烁

```
# 最简单的"特效": 改变颜色
func on_hit():
    modulate = Color.RED
    await get_tree().create_timer(0.1).timeout
    modulate = Color.WHITE
```

```
# 闪烁动画
func flash():
    visible = false
    await get_tree().create_timer(0.05).timeout
    visible = true
    await get_tree().create_timer(0.05).timeout
    visible = false
    await get_tree().create_timer(0.05).timeout
    visible = true
```

2. 动画帧: AnimatedSprite 特效

创建独立特效场景, 播完自动销毁:

```
# Explosion.tscn - AnimatedSprite2D, 播完 queue_free

func _ready():
    $AnimatedSprite2D.play("explode")
    $AnimatedSprite2D.animation_finished.connect(queue_free)
```

```
# 使用:
func spawn_explosion(pos: Vector2):
    var exp = preload("res://effects/
Explosion.tscn").instantiate()
    exp.position = pos
    get_parent().add_child(exp)
```

3. Tween 动画特效

```
# 伤害数字飘出
func show_damage_number(dmg: int, pos: Vector2):
    var label = Label.new()
    label.text = str(dmg)
    label.position = pos
    label.modulate = Color.RED
    add_child(label)

    var tween = create_tween()
    tween.set_parallel(true)
    tween.tween_property(label, "position:y", pos.y - 40, 0.5)
    tween.tween_property(label, "modulate:a", 0.0, 0.5)
    tween.tween_callback(label.queue_free)
```

4. GPUParticles2D: 粒子系统

```
# 火焰、烟雾、爆炸、星光、拖尾
# 在编辑器里配好粒子参数(数量/速度/生命周期/颜色/大小)
# 代码里只需:

func play_fire():
    $GPUParticles2D.emitting = true
```



```
func stop_fire():
    $GPUParticles2D.emitting = false
```

粒子系统的进化: 从 CPU 粒子(Particles2D)到 GPU 粒子(GPUParticles2D), 从几千粒子卡顿到几十万粒子流畅。

5. 最终方案模板

```
# 三类特效:
# 1. 瞬态场景 (爆炸/烟雾): 独立场景, 播完自动销毁
# 2. 附着特效 (火焰/光环): GPUParticles2D, 挂在角色上
# 3. 位移特效 (伤害数字/拾取文字): Tween 驱动

# 特效管理器 (对象池):
class EffectManager:
    static func play(path: String, pos: Vector2):
        var effect = pool.get(path)
        effect.global_position = pos
        effect.play()
        await effect.finished
        pool.return(effect)

# 使用:
# EffectManager.play("res://effects/Explosion.tscn",
enemy.position)
```

第67章 动画系统演化

Godot 动画系统: 从逐帧换图到动画树

让一个角色"动起来", 最早是快速切换图片——视觉暂留效应。后来有了关键帧插值、动画混合、状态机驱动。动画系统的进化 = "怎么让动的东西越来越像活的"。

1. 逐帧动画: 手算 timer 换图

```
var frame = 0
var timer = 0.0

func _process(delta):
    timer += delta
    if timer > 0.1:
        timer = 0.0
        frame = (frame + 1) % 4
        $Sprite.texture = frame_textures[frame]
```

问题: 帧率控制靠手写 timer, 多动画要写多套逻辑, 改速度要改代码。

2. AnimatedSprite2D: 配置化 SpriteFrames

```
$AnimatedSprite2D.play("walk")
$AnimatedSprite2D.play("idle")
$AnimatedSprite2D.animation_finished.connect(_on_anim_end)
```

在编辑器里配好 SpriteFrames, 每个动画的帧、帧率、循环方式可视化配置。不需要手写 timer。

局限: 只能换纹理, 不能控制位置/旋转/缩放/颜色。

3. AnimationPlayer: 属性动画

```
$AnimationPlayer.play("door_open")
# door_open 动画可以同时控制:
#   $Door.position.x: 0 → 200 (门滑开)
#   $Door.rotation: 0 → PI/2 (门旋转)
#   $AudioPlayer: play() at frame 10 (音效回调)
```

AnimationPlayer 可以动画任何属性: position、rotation、scale、modulate、shader_param。

局限: 多个动画之间的过渡要手写, 没有混合。

4. AnimationTree: 混合与状态机

```
@onready var anim_tree = $AnimationTree
@onready var anim_state = anim_tree.get("parameters/playback")

anim_tree.set("parameters/move/blend_position", direction.x)
anim_state.travel("jump")
```

AnimationTree 支持: - **BlendSpace2D**: 根据方向混合多个动画 - **状态机**:

idle → run → jump 自动过渡 - **混合**: 动画之间 crossfade

5. 最终方案模板

```
# 简单动画用 AnimatedSprite2D + play()
# 复杂动画用 AnimationPlayer + 回调
# 角色动画用 AnimationTree + 状态机

# 最常用方案 (AnimationPlayer):
func play_animation(name: String):
```

```
        if $AnimationPlayer.current_animation == name:
            return
        $AnimationPlayer.play(name)

# 带过渡:
func play_with_transition(name: String, crossfade: float = 0.1):
    if $AnimationPlayer.current_animation == name:
        return
    $AnimationPlayer.play(name)
    # crossfade 通过 AnimationTree 实现
```

第68章 Tween 动画演化

Godot Tween 动画: 从手动 lerp 到链式动画

AnimationPlayer 做复杂的关键帧动画。Tween 做代码里"从 A 到 B"的即时动画。

最早的 Tween 是每帧手写 lerp。后来 Godot 有了 Tween 节点、链式调用、并行/序列、缓动曲线。Tween 的进化 = "怎么用一行代码完成过去十行代码的动画"。

1. 原始: 手写 lerp

```
var t = 0.0
var start_pos = Vector2.ZERO
var end_pos = Vector2(200, 0)

func _process(delta):
    t += delta
    if t < 1.0:
        position = start_pos.lerp(end_pos, t)
    else:
        position = end_pos
```

2. Tween 节点

```
var tween = create_tween()
tween.tween_property(self, "position", Vector2(200, 0), 1.0)
```

一行代替了上面所有代码。Godot 自动管理: 插值、时间、结束。

3. 链式调用

```
# 并行：同时移动和淡出
var tween = create_tween()
tween.set_parallel(true)
tween.tween_property($Panel, "position:y", 0, 0.5)
tween.tween_property($Panel, "modulate:a", 1.0, 0.5)

# 序列：先淡入，再移动，再回调
tween = create_tween()
tween.tween_property(self, "modulate:a", 1.0, 0.3)
tween.tween_property(self, "position", target, 0.5)
tween.tween_callback(queue_free)
```

4. 缓动曲线

```
tween.tween_property($Ball, "position:y", 500, 1.0) \
    .set_trans(Tween.TRANS_BOUNCE) \
    .set_ease(Tween.EASE_OUT)

# TRANS: LINEAR / SINE / QUAD / CUBIC / QUART / QUINT / EXPO /
BACK / BOUNCE / ELASTIC
# EASE:  IN / OUT / IN_OUT / OUT_IN
```

5. 最终方案模板

```
# Tween 三种主要用法：

# 1. 属性插值（90% 场景）
create_tween().tween_property(node, "position", target, 0.5)

# 2. 延时执行
create_tween().tween_interval(1.0).tween_callback(func():
print("done"))

# 3. 重复动画
var t = create_tween()
```



```
t.set_loops()  
t.tween_property($Sprite, "modulate:a", 0.3, 0.5)  
t.tween_property($Sprite, "modulate:a", 1.0, 0.5)
```

第69章 骨骼动画系统演化

Godot 骨骼动画系统：从逐帧到骨骼变形

骨骼动画 = 用骨骼驱动网格变形, 而不是逐帧换图。逐帧动画每帧一张图, 骨骼动画只需要一套骨骼 + 关键帧姿势。早期游戏全是逐帧 sprite。后来有了 Spine、DragonBones, Godot 内置了 Skeleton2D/Bone2D。骨骼动画的进化 = "怎么让动画更流畅、文件更小、动作更丰富"。

1. 原始: 逐帧 Sprite

```
# 每帧一张贴图
# walk_1.png, walk_2.png, walk_3.png ...
# 帧数越多越流畅, 但贴图量巨大

$AnimatedSprite2D.play("walk")
# 一个 10 帧的走路动画 = 10 张贴图
```

问题: 每增加一个动作方向就要多一套贴图。8 方向 × 10 动作 × 10 帧 = 800 张贴图。

2. 骨骼 + 蒙皮

```
# 角色拆成多个 Sprite, 用 Bone2D 控制
# 身体:
#   Bone "root" (根骨骼)
#   Bone "spine" (脊柱)
#   Bone "head" (头部)
#   Bone "arm_l" / "arm_r" (手臂)
#   Bone "leg_l" / "leg_r" (腿)
```

```
# 把 Sprite 绑定到对应 Bone 上
# 动画就是记录每个 Bone 在不同时间点的 rotation/position/scale
```

```
# Skeleton2D 结构:
# Skeleton2D
#   └─ Bone2D "root"
#       └─ Bone2D "spine"
#           └─ Bone2D "head"           ← Sprite "head" 挂在这里
#           └─ Bone2D "arm_l"         ← Sprite "arm_l" 挂在这里
#           └─ Bone2D "arm_r"         ← Sprite "arm_r" 挂在这里
#           └─ Bone2D "hip"
#               └─ Bone2D "leg_l"      ← Sprite "leg_l" 挂在这里
#               └─ Bone2D "leg_r"      ← Sprite "leg_r" 挂在这里
```

3. 反向动力学 (IK)

```
# 正向动力学 (FK): root → 脚尖, 每级 bone 自己转
# 反向动力学 (IK): 脚尖位置 → 自动算 root → 膝盖的角度
# 踩地、扶墙、手抓东西 → IK 自动适配
```

```
# Godot Skeleton2D 没有内置 IK, 需要手写或插件:
```

```
func solve_ik(bone_chain: Array[Bone2D], target: Vector2):
    # CCD (Cyclic Coordinate Descent) IK
    for _iter in 10:
        for i in range(bone_chain.size() - 1, -1, -1):
            var bone = bone_chain[i]
            var tip = bone_chain[-1].global_position
            var root_pos = bone.global_position
            var to_tip = (tip - root_pos).normalized()
            var to_target = (target - root_pos).normalized()
            var angle = to_tip.angle_to(to_target)
            bone.global_rotation += angle * 0.5 # 步长
```

4. 物理骨骼 (PhysicalBone)

```
# 把骨骼变成物理模拟：
# - 头发/衣服/尾巴可以自然摆动
# - 受击时肢体物理反应
# - 死亡时布娃娃

# Godot 3D: PhysicalBone + SkeletonIK
# Godot 2D: 用 PinJoint 连 Bone2D + RigidBody
```

5. 最终方案模板

```
# 骨骼动画 vs 逐帧动画 选型:

#
# |          | 逐帧 Sprite | 骨骼动画 |
# |-----|-----|-----|
# | 贴图量   | 大（每帧一张） | 小（部件贴图） |
# | 动作丰富度 | 固定，不可组合 | 可混合/叠加 |
# | 实现难度   | 简单 | 复杂 |
# | 流畅度     | 取决于帧数 | 自动插值 |
# | 换装       | 整套换 | 部件换 |
# | Godot 支持 | AnimatedSprite2D | Skeleton2D + |
# |           |           | AnimationPlayer |
# |-----|-----|-----|

# 推荐：
# - 简单角色（2D）：AnimatedSprite2D
# - 复杂角色（2D/3D）：Skeleton2D/Skeleton3D
# - 物理衣摆/头发：PhysicalBone
```

第70章 图鉴系统演化

Godot 图鉴系统：从收集到完成度

图鉴 = 玩家见过的怪物/物品/角色的记录。最早是"你见过这种怪物吗? 见过就亮了。" 后来有了收集奖励、完成度百分比、收集排行榜。图鉴系统的进化 = "怎么让玩家想把所有东西都看一遍"。

1. 原始：见过就记

```
var seen_monsters = []

func on_see_monster(monster_id: String):
    if not seen_monsters.has(monster_id):
        seen_monsters.append(monster_id)
```

2. 图鉴数据结构

```
class CodexEntry:
    var id: String
    var name: String
    var description: String
    var icon: Texture2D
    var discovered: bool = false
    var count: int = 0          # 见过/击杀次数
    var category: String       # "monster" / "item" / "npc" /
    "location"
```

3. 图鉴完成度 + 奖励

```
func get_completion() -> float:
    var total = _entries.size()
    var discovered = _entries.filter(func(e): return
e.discovered).size()
    return float(discovered) / total

func get_category_completion(category: String) -> float:
    var cat = _entries.filter(func(e): return e.category ==
category)
    var discovered = cat.filter(func(e): return
e.discovered).size()
    return float(discovered) / cat.size()

# 完成某个类别 → 给奖励
func check_milestones():
    var pct = get_completion()
    if pct >= 1.0:
        # 全收集奖励
        CurrencyManager.add("gems", 1000)
```

4. 最终方案模板

```
# CodexManager.gd (Autoload)
extends Node

signal discovered(entry_id: String)

var _entries := {}

func _ready():
    _load_all()

func discover(id: String):
    var entry = _entries.get(id)
    if not entry or entry.discovered:
        return
    entry.discovered = true
```



```

        discovered.emit(id)

func get_progress() -> Dictionary:
    var total = _entries.size()
    var found = _entries.values().filter(func(e): return
e.discovered).size()
    return { "found": found, "total": total, "percent":
float(found) / total }

func _load_all():
    var dir = DirAccess.open("res://codex/")
    for file in dir.get_files():
        if file.ends_with(".tres"):
            var entry = load("res://codex/" + file)
            _entries[entry.id] = entry

# 在任何地方:
# CodexManager.discover("slime") # 看到史莱姆时触发

```

第71章 时装系统演化

Godot 时装系统：从换装备到外观自由

时装 = 不影响属性只改变外观的装备。最早外观 = 装备, 穿什么甲看起来就什么样。后来有时装: 穿好看的衣服但不影响战斗力。时装系统的进化 = "怎么让玩家为好看付费而不是为变强"。

1. 原始：装备决定外观

```
# 穿铁甲 → 角色显示铁甲
# 穿布衣 → 角色显示布衣
# 外观和属性绑定，没得选
```

2. 外观槽 + 属性槽分离

```
# 两个独立的槽位：
# [头盔]          [外观头盔]
# [胸甲]          [外观胸甲]
# [武器]          [外观武器]

# 属性来自左边的装备
# 外观来自右边的外观槽

func get_appearance(slot: String) -> String:
    # 优先显示时装，没有时装显示装备
    return _fashion_slots.get(slot) \
        or _equipment_slots.get(slot) \
        or "default"
```

3. 时装 Resource

```
class_name FashionResource
extends Resource

@export var id: String
@export var name: String
@export var description: String
@export var icon: Texture2D
@export var model_scene: PackedScene      # 角色外观
@export var weapon_effect: PackedScene    # 武器特效
@export var color_variants: Array[Color]  # 染色方案
@export var rarity: String                 # "common" / "rare" /
"legendary"
```

```
func equip_fashion(slot: String, fashion_id: String):
    var fashion = FashionDB.get(fashion_id)
    if not fashion: return
    _fashion_slots[slot] = fashion_id
    _update_appearance(slot, fashion)

func on_fashion_changed():
    # 更换角色 Sprite/Animation
    # 更换武器特效 (GPUParticles2D)
    # 更换行走动画 (如果时装改变了走路姿势)
    pass
```

4. 最终方案模板

```
# FashionManager.gd (Autoload)
# 核心：属性装备和外观装备分离

# 时装来源：
# - 商城购买（充值系统）
# - 活动奖励（活动系统）
# - 签到累计（签到系统）
# - 成就解锁（成就系统）
# - 合成制作（合成系统）
```

```
func preview(fashion_id: String) -> Dictionary:  
    var f = FashionDB.get(fashion_id)  
    return { "name": f.name, "slots": f.slots, "effect":  
f.description }
```

第72章 染色系统演化

Godot 染色系统：从固定颜色到自由配色

染色 = 改变装备/时装的颜色。最早装备颜色是固定的。后来有了染色剂、调色盘、分区染色。染色系统的进化 = "怎么让玩家定制自己的配色方案"。

1. 原始：固定颜色

```
# 红色衣服就是红色，改不了
$Sprite.texture = preload("red_clothes.png")
```

2. modulate 改色

```
# 最简单：整体改色
$Sprite.modulate = Color.RED
$Sprite.modulate = Color(1, 0.5, 0.5, 1) # 粉色
```

3. 分区染色 (multi-texture)

```
# 角色 Sprite 拆成多个图层：
# - 身体 (base): 不可以染色
# - 上衣 (top): 可以染色
# - 裤子 (pants): 可以染色
# - 头发 (hair): 可以染色
# - 配饰 (acc): 可以染色

class DyeSlot:
```

```

var slot_name: String      # "top" / "pants" / "hair"
var current_color: Color
var locked: bool = false   # 有些染色需要解锁

```

```

func apply_dye(slot: String, color: Color):
    match slot:
        "top":
            $SpriteTop.modulate = color
        "pants":
            $SpritePants.modulate = color
        "hair":
            $SpriteHair.modulate = color

func preview_dye(slot: String, color: Color):
    # 在染色界面预览，不实际应用
    apply_dye(slot, color)
    # 取消预览时恢复

```

4. 染色剂物品

```

class DyeItem:
    var id: String
    var name: String
    var color: Color
    var slot: String      # "top" / "any"
    var rarity: String

```

```

func use_dye(item_id: String, slot: String):
    var dye = DyeDB.get(item_id)
    if dye.slot != "any" and dye.slot != slot:
        return false
    if not InventoryManager.remove(item_id, 1):
        return false
    _fashion_data.dyes[slot] = dye.color
    _update_appearance()
    return true

```


5. 最终方案模板

```
# DyeManager.gd (Autoload)
# 核心：每个外观部件独立 modulate

func set_color(slot: String, color: Color):
    _save.dyes[slot] = color
    _apply()

func reset_color(slot: String):
    _save.dyes.erase(slot)
    _apply()

func _apply():
    for slot in _save.dyes:
        var node = _get_sprite_node(slot)
        if node:
            node.modulate = _save.dyes[slot]

# 染色数据保存在 FashionManager 里：
# fashion_data.dyes = { "top": "#ff0000", "pants": "#0000ff" }
```

第73章 拖拽系统演化

Godot 拖拽系统：从点击到拖放

拖拽 = 按住物体移动, 松开放下。最早的操作是"点一下、再点一下"。后来有了按住拖动、拖到目标区域释放、排序拖拽、触摸拖拽。拖拽系统的进化 = "怎么让玩家可以直接'拿'东西"。

1. 原始：点击放置

```
# 点一下 A 位置, 再点一下 B 位置
# 物体直接跳到 B, 没有拖动过程

func on_click(pos: Vector2):
    $Item.position = pos
```

2. 鼠标跟随

```
func _input(event):
    if event is InputEventMouseButton and event.button_index ==
    MOUSE_BUTTON_LEFT:
        if event.pressed:
            # 捡起
            var item = _get_item_at(event.position)
            if item:
                _dragging = item
                item.z_index = 100 # 提到最前面
        else:
            # 放下
            if _dragging:
                _drop(_dragging)
```

```

        _dragging.z_index = 0
        _dragging = null

    if event is InputEventMouseMotion and _dragging:
        _dragging.position = event.position

```

3. 拖拽到目标区域

```

# 拖到特定区域触发放下事件
func _drop(item):
    # 检测是否在目标区域内
    for slot in $Slots.get_children():
        if slot.get_rect().has_point(item.position):
            slot.put(item)
            return
    # 没放到目标区域 → 回到原位
    item.position = item.original_position

```

4. 背包拖拽 (格子对齐)

```

# 拖到背包网格上，自动对齐到格子
class DragSlot:
    var grid_position: Vector2i # 格子坐标
    var world_rect: Rect2      # 世界坐标范围
    var item: Node = null

func snap_to_grid(pos: Vector2) -> Vector2i:
    var cell_size = 32
    return Vector2i(
        floor(pos.x / cell_size),
        floor(pos.y / cell_size)
    )

func _drop():
    var grid_pos = snap_to_grid(_dragging.position)
    var slot = _get_slot(grid_pos)
    if slot and slot.item == null:
        # 放到空位

```

```

        slot.item = _dragging
        _dragging.position = grid_pos * 32
    else:
        # 回到原位
        _dragging.position = _dragging.original_position

```

5. 触摸拖拽 + 滚动容器

```

# 在 ScrollContainer 内拖拽:
# 1. 开始拖拽时禁用滚动
# 2. 拖拽过程中不滚动
# 3. 放下后恢复滚动

func _on_item_gui_input(event):
    if event is InputEventScreenTouch or event is
    InputEventMouseButton:
        if event.pressed:
            # 禁用父级 ScrollContainer 滚动
            $ScrollContainer.scroll_horizontal = false
            $ScrollContainer.scroll_vertical = false
            _start_drag(item)
        else:
            $ScrollContainer.scroll_horizontal = true
            $ScrollContainer.scroll_vertical = true
            _end_drag()

```

6. 最终方案模板

```

# DragManager.gd (Autoload)
# 最简单的全局拖拽:

var dragging = null
var offset = Vector2.ZERO

func _input(event):
    if event is InputEventMouseButton and event.button_index ==
    MOUSE_BUTTON_LEFT:
        if event.pressed:

```

```

        var item = _get_draggable_at(event.position)
        if item:
            dragging = item
            offset = item.position - event.position
            item.z_index = 100
        else:
            if dragging:
                _drop(dragging)
                dragging.z_index = 0
                dragging = null

    if event is InputEventMouseMotion and dragging:
        dragging.position = event.position + offset

func _get_draggable_at(pos: Vector2) -> Control:
    # 从最上层到最下层检测
    var items = get_tree().get_nodes_in_group("draggable")
    for item in items:
        if item.get_global_rect().has_point(pos):
            return item
    return null

func _drop(item):
    item.drop_signal.emit(item)

```

第74章 Tooltip 系统演化

Godot Tooltip 系统: 从文字到悬浮卡片

Tooltip = 鼠标悬停时显示的信息。最早是浏览器风格的 title 属性。后来有了延迟出现、位置自适应、富文本、动态信息。Tooltip 的进化 = "怎么在不占空间的前提下展示信息"。

1. 原始: 静态文字

```
# 鼠标悬停时弹出系统提示
$itemButton.tooltip_text = "一把锋利的剑"
# Godot 内置 tooltip_text 属性
```

2. 自定义 Tooltip

```
# 创建 Tooltip 场景:
# TooltipPanel (Control)
#   └─ $Icon (TextureRect)
#   └─ $Name (Label)
#   └─ $Desc (Label)

func _make_custom_tooltip(for_text: String) -> Control:
    var tip = preload("res://ui/tooltip.tscn").instantiate()
    var data = ItemDB.get(for_text)
    tip.get_node("Icon").texture = data.icon
    tip.get_node("Name").text = data.name
    tip.get_node("Desc").text = data.description
    if data.rarity == "rare":
        tip.get_node("Name").modulate = Color.BLUE
    return tip
```


3. 延迟 + 跟随鼠标

```
# TooltipManager.gd
extends CanvasLayer

var _timer := 0.0
var _show_delay := 0.3
var _current_tip: Control = null
var _target: Control = null

func _process(delta):
    if _target:
        _timer += delta
        if _timer >= _show_delay and not _current_tip:
            _show_tip()
    if _current_tip:
        _current_tip.global_position =
get_global_mouse_position() + Vector2(10, 10)
        # 防止超出屏幕
        _clamp_to_screen(_current_tip)

func show_tooltip(target: Control):
    _target = target
    _timer = 0.0

func hide_tooltip():
    _target = null
    _timer = 0.0
    if _current_tip:
        _current_tip.queue_free()
        _current_tip = null

func _show_tip():
    if not _target: return
    _current_tip =
_target.make_custom_tooltip(_target.tooltip_text)
    if not _current_tip:
        _current_tip = _make_default_tip(_target.tooltip_text)
    add_child(_current_tip)
```

4. 最终方案模板

```
# 在物品/技能按钮上:
func _ready():
    mouse_entered.connect(func():
        TooltipManager.show_tooltip(self))
    mouse_exited.connect(func(): TooltipManager.hide_tooltip())

func _make_custom_tooltip(for_text: String) -> Control:
    var tip = preload("res://ui/
item_tooltip.tscn").instantiate()
    var item = ItemDB.get(for_text)
    tip.init(item)
    return tip
```

第75章 屏幕震动系统演化

Godot 屏幕震动系统：从静态到受击反馈

屏幕震动 = 受到伤害/爆炸时摄像机抖动。最早游戏没有震动, 所有动作都是静态的。后来有了位移衰减、多轴震动、Shake 曲线、Perlin 噪声。
屏幕震动的进化 = "怎么让受击感传达到玩家的手上"。

1. 原始：无震动

```
# 被打中了，但屏幕什么反应都没有  
# 玩家感觉不到自己受伤了
```

2. 简单位移

```
# 直接偏移 Camera2D  
func shake(intensity: float):  
    camera.offset = Vector2(randf_range(-1, 1), randf_range(-1, 1)) * intensity  
    await get_tree().create_timer(0.1).timeout  
    camera.offset = Vector2.ZERO  
  
# 问题：只偏移一瞬间，看起来很生硬
```

3. 衰减震动

```
# ShakeManager.gd  
extends Node
```

```

var _trauma: float = 0.0 # 0.0 ~ 1.0
var _camera: Camera2D
var _smoothing := {}

func _ready():
    _camera = get_viewport().get_camera_2d()

func add_trauma(amount: float):
    _trauma = min(_trauma + amount, 1.0)

func _process(delta):
    if _trauma <= 0.0:
        _camera.offset = _camera.offset.lerp(Vector2.ZERO, delta
* 10)
        return

    var shake = _trauma * _trauma # 平方衰减, 让轻微震动更自然
    var offset = Vector2(
        randf_range(-1, 1) * shake * 20,
        randf_range(-1, 1) * shake * 20,
    )
    _camera.offset = offset
    _trauma = max(_trauma - delta * 0.5, 0.0) # 随时间衰减

```

4. 多轴旋转震动

```

func _process(delta):
    if _trauma <= 0.0:
        _camera.offset = _camera.offset.lerp(Vector2.ZERO, delta
* 10)
        _camera.rotation = lerp(_camera.rotation, 0.0, delta *
10)
        return

    var shake = _trauma * _trauma
    _camera.offset = Vector2(
        randf_range(-1, 1) * shake * 15,
        randf_range(-1, 1) * shake * 10,
    )
    _camera.rotation = randf_range(-1, 1) * shake * 0.05 # 轻微
旋转

```

```
_trauma = max(_trauma - delta * _decay_rate, 0.0)
```

5. 最终方案模板

```
# ShakeManager.gd (Autoload)
# 核心: trauma 值 → 衰减 → 随机偏移 + 旋转

var _trauma := 0.0
var _max_offset := 20.0
var _max_rotation := 0.05
var _decay := 3.0 # 每秒衰减速度

func shake(intensity: float):
    _trauma = min(_trauma + intensity, 1.0)

func _process(delta):
    var camera = get_viewport().get_camera_2d()
    if not camera: return

    if _trauma <= 0.0:
        camera.offset = camera.offset.move_toward(Vector2.ZERO,
delta * 50)
        camera.rotation = move_toward(camera.rotation, 0.0,
delta * 5)
        return

    var s = _trauma * _trauma
    camera.offset = Vector2(
        randf_range(-1, 1) * s * _max_offset,
        randf_range(-1, 1) * s * _max_offset,
    )
    camera.rotation = randf_range(-1, 1) * s * _max_rotation
    _trauma = max(_trauma - delta * _decay, 0.0)
```

第76章 HUD 系统演化

Godot HUD 系统: 从调试文字到合成界面

HUD = 游戏中始终显示在屏幕上的信息。最早只有一行 HP 数字。后来有了血条、技能栏、小地图、状态图标、Buff/Debuff 追踪。HUD 的进化 = "怎么在不遮挡游戏画面的前提下展示关键信息"。

1. 原始: 数字

```
# 左上角显示数字
$HUD/Label.text = "HP: " + str(player.hp) + "/" +
str(player.max_hp)
```

2. 血条 + 图标

```
# 使用 TextureProgress 做血条
$HUD/HPBar.max_value = player.max_hp
$HUD/HPBar.value = player.hp

# 技能快捷栏
for i in range(skills.size()):
    $HUD/SkillSlot[i].icon = skills[i].icon
    $HUD/SkillSlot[i].cd_mask.value =
skills[i].cooldown_remaining
```


3. 分层 HUD

```
class HUDLayer:
    var canvas_layer: CanvasLayer
    var z_index: int
    var elements: Array[Control]

# 分层:
# Layer 0: 背景 (小地图框架)
# Layer 1: 核心 (血条/能量条)
# Layer 2: 动态 (通知/Damage数字)
# Layer 3: 交互 (技能栏/背包按钮)
# Layer 4: 模态 (弹窗/商店)

func show_layer(layer_idx: int, visible: bool):
    layers[layer_idx].canvas_layer.visible = visible

# 战斗时隐藏非核心层:
func on_combat_start():
    show_layer(2, false) # 隐藏通知
    show_layer(3, false) # 隐藏交互
```

4. 最终方案模板

```
# HUDManager.gd (Autoload)
# 核心: 订阅数据变化 → 自动更新 UI

var _hud_scene: Control

func init(hud_scene: Control):
    _hud_scene = hud_scene
    PlayerStats.hp_changed.connect(_update_hp)
    PlayerStats.mp_changed.connect(_update_mp)
    SkillManager.cooldown_tick.connect(_update_skills)
    BuffManager.buff_changed.connect(_update_buffs)

func _update_hp(hp: int, max_hp: int):
    _hud_scene.get_node("HPBar").value = hp
    _hud_scene.get_node("HPBar/Text").text = str(hp) + "/" +
```

```
str(max_hp)
    _hud_scene.get_node("HPBar").modulate = Color.RED if hp <
max_hp * 0.2 else Color.WHITE

func _update_skills(skills: Array):
    for i in skills.size():
        var slot = _hud_scene.get_node("SkillBar/Slot" + str(i))
        slot.icon = skills[i].icon
        slot.cd_remaining = skills[i].cooldown_remaining /
skills[i].cooldown
```

第77章 加载画面系统演化

Godot 加载画面：从黑屏到沉浸过渡

加载画面 = 场景切换/资源加载时的过渡界面。最早就是黑屏, 啥也没有。后来有了进度条、提示文字、背景图、异步加载。加载画面的进化 = "怎么让等待不无聊"。

1. 原始：黑屏

```
# 直接 change_scene
# 屏幕黑了, 等几秒, 突然亮了
func go_to_scene(path):
    get_tree().change_scene_to_file(path)
```

2. 简单的 Loading 界面

```
# 切换前显示一个 Label
func go_to_scene(path):
    $LoadingLabel.visible = true
    $LoadingLabel.text = "加载中..."
    await get_tree().process_frame
    get_tree().change_scene_to_file(path)
    $LoadingLabel.visible = false
```

3. 进度条加载

```
func load_scene(path: String):
    $Loading.visible = true
```

```

$Loading/ProgressBar.value = 0

var loader = ResourceLoader.load_threaded_request(path)
while true:
    var progress = []
    var status =
ResourceLoader.load_threaded_get_status(path, progress)
    $Loading/ProgressBar.value = progress[0] * 100
    $Loading/Tip.text = tips[randi() % tips.size()]
    if status == ResourceLoader.THREAD_LOAD_LOADED:
        break
    await get_tree().process_frame

var scene = ResourceLoader.load_threaded_get(path)
get_tree().change_scene_to_packed(scene)
$Loading.visible = false

```

4. 动画加载画面

```

# LoadingScreen.tscn
# 包含：进度条 + 旋转动画 + 背景图 + 小提示

func show():
    $AnimationPlayer.play("fade_in")
    $Spinner.rotation = 0
    await get_tree().create_timer(0.3).timeout

func update_progress(pct: float):
    $ProgressBar.value = pct * 100

func hide():
    await $AnimationPlayer.play("fade_out")
    queue_free()

# 使用：
var loading_screen = preload("res://ui/
loading_screen.tscn").instantiate()
add_child(loading_screen)
loading_screen.show()
loading_screen.update_progress(0.5)

```

5. 最终方案模板

```
# LoadingManager.gd (Autoload)
# 核心: 异步加载 + 进度回调 + 动画过渡

var _screen: Control = null

func load_scene(path: String):
    _screen = preload("res://ui/
loading_screen.tscn").instantiate()
    get_tree().root.add_child(_screen)

    var loader = ResourceLoader.load_threaded_request(path)
    while true:
        var p = []
        var status =
ResourceLoader.load_threaded_get_status(path, p)
        _screen.update(p[0])
        if status == ResourceLoader.THREAD_LOAD_LOADED:
            var scene = ResourceLoader.load_threaded_get(path)
            _screen.fade_out()
            await _screen.animation_finished
            get_tree().change_scene_to_packed(scene)
            _screen.queue_free()
            return
        await get_tree().process_frame
```

第78章 伤害数字系统演化

Godot 伤害数字系统: 从 print 到飘字带感

伤害数字 = 命中时飘出的数值。最早就是 print 在控制台。后来有了漂浮动画、颜色区分、暴击大字体、元素图标。伤害数字的进化 = "怎么让玩家一眼看出打了多少"。

1. 原始: print

```
func take_damage(amount):  
    hp -= amount  
    print("玩家受到 ", amount, " 点伤害")
```

2. Label 飞出

```
# 在被击目标位置创建一个 Label  
# 向上飘动 + 淡出  
  
func spawn_damage_number(position: Vector2, amount: int,  
is_crit: bool):  
    var label = Label.new()  
    label.text = str(amount)  
    label.global_position = position  
    label.modulate = Color.YELLOW if is_crit else Color.WHITE  
    label.scale = Vector2(1.5, 1.5) if is_crit else Vector2(1,  
1.0)  
    add_child(label)  
  
    var tween = create_tween()  
    tween.tween_property(label, "position:y", position.y - 50,
```



```

0.8)
    tween.tween_property(label, "modulate:a", 0.0,
0.5).set_delay(0.3)
    tween.tween_callback(label.queue_free)

```

3. 对象池 + 类型

```

class FloatingText extends Node2D:
    var label: Label
    var tween: Tween

    func show(text: String, color: Color, pos: Vector2):
        global_position = pos
        label.text = text
        label.modulate = color
        visible = true
        tween = create_tween()
        tween.tween_property(self, "position:y", pos.y - 60,
1.0)
            tween.tween_property(label, "modulate:a", 0.0,
0.4).set_delay(0.4)
            tween.tween_callback(func(): hide(); pool.release(self))

# 伤害类型:
# 正常伤害: 白色
# 暴击:      黄色 + 大字号
# 治疗:      绿色
# 免疫:      灰色 + "免疫" 文字
# 吸收:      蓝色 (护盾吸收量)
# 元素伤害: 对应元素颜色 (火=红, 冰=蓝, 毒=紫)

```

4. 最终方案模板

```

# FloatingDamage.gd (Autoload)
# 核心: 对象池 + 动画 + 类型颜色

var _pool := []

```

```

func show(pos: Vector2, amount: int, type: String = "normal"):
    var node = _get_or_create()
    var config = _get_config(type, amount)
    node.show(config.text, config.color, pos)

func _get_config(type: String, amount: int) -> Dictionary:
    match type:
        "normal": return { text = str(amount), color =
Color.WHITE }
        "crit":   return { text = str(amount) + "!", color =
Color.YELLOW }
        "heal":   return { text = "+" + str(amount), color =
Color.GREEN }
        "immune": return { text = "免疫", color = Color.GRAY }
        "absorb": return { text = str(amount), color =
Color.CYAN }
        return { text = str(amount), color = Color.WHITE }

func _get_or_create() -> Node:
    if _pool.size() > 0:
        return _pool.pop_back()
    var node = preload("res://ui/
floating_text.tscn").instantiate()
    get_tree().root.add_child(node)
    return node

```

第五卷 · 角色与场景

共计 24 章

第79章 NPC 系统演化

Godot NPC 系统: 从一句话到完整交互

NPC 不只是"站着不动的人"。他是给你任务的、卖你东西的、告诉你剧情的。从一句对话到多分支对话树, 从静态到日夜巡逻, NPC 系统的进化 = 怎么让游戏世界里的"人"越来越像人。

目录

1. [原始: 静态 NPC](#)
 2. [对话: 显示一句话](#)
 3. [对话树: 多分支选择](#)
 4. [NPC 商店: 买卖物品](#)
 5. [NPC 任务: 交互后触发任务](#)
 6. [NPC 日程: 白天晚上不同位置](#)
 7. [最终方案: 最简可用模板](#)
-

1. 原始: 静态 NPC

问题: "在游戏里放一个 NPC, 站在那里不动。"

```
# NPC.gd
extends CharacterBody2D
```

```
func _ready():  
    $Label.text = "村民"
```

交互方式: 靠 Area2D 检测玩家靠近。

```
func _on_detection_area_body_entered(body):  
    if body.is_in_group("player"):  
        $DialogueBubble.show() # 头上冒出一个对话气泡  
  
func _on_detection_area_body_exited(body):  
    if body.is_in_group("player"):  
        $DialogueBubble.hide()
```

问题: 1. **不会说话** – 气泡里是空的, 或者固定文字。 2. **不会动** – 站在那, 日复一日。 3. **交互单一** – 靠近显示, 离开隐藏。不能按 F 对话。

2. 对话: 显示一句话

问题: 靠近 NPC 后, 需要按交互键才能触发对话。

交互区域 + 对话 UI:

```
extends CharacterBody2D  
  
@export var npc_name: String = "村民"  
@export var dialogue_text: String = "你好, 旅行者."  
  
func _ready():  
    $Label.text = npc_name  
  
func on_interact():  
    # 玩家按了交互键 (F/E/空格)  
    DialogueUI.show(npc_name, dialogue_text)  
  
# DialogueUI.gd (Autoload, 或挂在场景里)  
extends CanvasLayer  
  
@onready var panel = $Panel
```

```

@onready var name_label = $Panel/Name
@onready var text_label = $Panel/Text

var current_npc = null

func show(npc_name: String, text: String):
    panel.show()
    name_label.text = npc_name
    text_label.text = text
    get_tree().paused = true # 对话时暂停游戏

func hide():
    panel.hide()
    get_tree().paused = false

func _input(event):
    if not panel.visible:
        return
    if event.is_action_pressed("interact") or
event.is_action_pressed("ui_accept"):
        hide()

```

每个 NPC 一句固定对话:

```

# 村民 A: "你好, 旅行者。"
# 村民 B: "今天天气不错。"
# 铁匠:    "需要武器吗?"

```

3. 对话树: 多分支选择

问题: 每个 NPC 只说一句话。玩家没有选择权。

对话树: 对话不是"说一句结束", 而是"根据玩家的选择走到不同的对话分支"。

```

# DialogueNode.gd
class_name DialogueNode
extends Resource

@export var npc_text: String = ""

```

```
@export var responses: Array[Response]

class Response:
    @export var player_text: String = ""      # 玩家的选项文字
    @export var next_node: DialogueNode      # 跳转到哪个节点
    @export var condition: String = ""       # 条件（如
"has_quest"）
    @export var action: String = ""         # 执行动作（如
"give_quest"）
```

对话树示例：

```
[问候]
NPC: "你好，旅行者。有什么需要?"
├─ "你是谁?" → [介绍]
│   NPC: "我是这个村的铁匠。"
│   └─ "我要买武器。" → [商店]（打开商店界面）
│   └─ "没事了。" → [结束]
├─ "我要买装备。" → [商店]
└─ "再见。" → [结束]
```

```
# DialogueManager.gd
extends Node

var current_node: DialogueNode
var dialogue_nodes := {}

func start(dialogue_id: String):
    current_node = dialogue_nodes[dialogue_id]
    _show_current()

func select_response(index: int):
    if index >= current_node.responses.size():
        return

    var response = current_node.responses[index]

    # 检查条件
    if response.condition != "" and not
_check_condition(response.condition):
        return # 条件不满足，不显示或不可选

    # 执行动作
```



```

if response.action != "":
    _execute_action(response.action)

# 跳转
if response.next_node:
    current_node = response.next_node
    _show_current()
else:
    # 没有下一个节点 → 结束对话
    DialogueUI.hide()

func _show_current():
    DialogueUI.show_dialogue(current_node)

func _check_condition(condition: String) -> bool:
    match condition:
        "has_quest":
            return QuestManager.has_active_quest("iron_sword")
        "level_5":
            return GameState.level >= 5
    return true

func _execute_action(action: String):
    match action:
        "give_quest":
            QuestManager.start_quest("iron_sword")
        "open_shop":
            ShopUI.open(ShopManager.get_shop("blacksmith"))

```

效果: 对话从"说一句结束"变成了"玩家选择 → NPC 回应 → 再选择 → ...".

4. NPC 商店: 买卖物品

问题: 铁匠说"需要武器吗?" 但你不能真的买。

商店系统: NPC 持有商品列表, 玩家可以购买/出售。

```

# ShopManager.gd
class ShopManager:
    var items: Array[ShopItem]

```

```

var buy_price_multiplier: float = 1.0
var sell_price_multiplier: float = 0.5

class ShopItem:
    var item: ItemResource
    var stock: int = -1    # -1 = 无限
    var price: int = 0     # 0 = 使用 item.buy_price

```

```

# NPC_Shopkeeper.gd
extends CharacterBody2D

@export var shop_id: String = "blacksmith"
@export var greeting: String = "欢迎光临!"

func on_interact():
    if QuestManager.has_active_quest("iron_sword"):
        DialogueUI.show("铁匠", "你要的铁剑打好了, 拿着吧。")
        QuestManager.complete_quest("iron_sword")
    else:
        ShopUI.open(shop_id)

```

商店 UI 交互流:

```

玩家点"我要买武器"
→ ShopUI 显示商店物品列表
→ 玩家选一把剑
→ 检查金币够不够
→ 够: 扣金币, 加物品到背包
→ 不够: 显示"金币不足"

```

5. NPC 任务: 交互后触发任务

问题: NPC 可以对话、可以交易。但 NPC 的核心功能是让玩家有事做。

任务系统 + NPC:

```

# NPC_QuestGiver.gd
extends CharacterBody2D

```

```

@export var quest_id: String = "kill_slimes"
@export var quest_name: String = "消灭史莱姆"
@export var quest_description: String = "击杀 5 只史莱姆。"

var quest_states = ["available", "active", "completed", "done"]

func on_interact():
    var state = QuestManager.get_state(quest_id)

    match state:
        "available":
            DialogueUI.show_choice(
                "村长",
                "年轻人，能帮我消灭村外的史莱姆吗？",
                ["接受", "改天吧"]
            )
            # 选"接受" → QuestManager.start_quest(quest_id)
            # 选"改天吧" → 结束对话

        "active":
            var progress = QuestManager.get_progress(quest_id)
            if progress >= 5:
                DialogueUI.show("村长", "太好了，你做到了！这是你的报酬。")
                QuestManager.complete_quest(quest_id)
            else:
                DialogueUI.show("村长", "史莱姆还有 %d 只，加油。" %
(5 - progress))

        "completed":
            DialogueUI.show("村长", "谢谢你的帮助。")

```

NPC 对话根据任务状态变化：

```

还没接任务： 村长问你要不要帮忙
任务进行中： 村长问进度
任务完成：   村长表示感谢
任务已经结： 没有更多了

```

6. NPC 日程: 白天晚上不同位置

问题: "NPC 24 小时站在同一个位置, 像一座雕像。"

日程系统: NPC 在不同时间去不同位置, 做不同事情。

```
# NPC_Schedule.gd
extends Node

class ScheduleEntry:
    var hour: int      # 几点开始
    var position: Vector2 # 去哪
    var animation: String # 播放啥动画
    var dialogue_id: String # 这个时段说什么话

var schedule = [
    ScheduleEntry.new(6, Vector2(100, 200), "walk", "morning"),
    ScheduleEntry.new(8, Vector2(300, 200), "idle", "work"),
    ScheduleEntry.new(12, Vector2(200, 300), "idle", "lunch"),
    ScheduleEntry.new(13, Vector2(300, 200), "idle", "work"),
    ScheduleEntry.new(18, Vector2(100, 100), "walk", "home"),
    ScheduleEntry.new(21, Vector2(100, 100), "sleep", "home"),
]

func get_current_entry(game_hour: int) -> ScheduleEntry:
    for i in schedule.size() - 1:
        if game_hour >= schedule[i].hour and game_hour <
            schedule[i + 1].hour:
            return schedule[i]
    return schedule[-1]

func _process(delta):
    var entry = get_current_entry(GameState.game_hour)
    # 移到目标位置
    var target = entry.position
    global_position = global_position.move_toward(target, speed
* delta)

    # 播对应动画
    if entry.animation == "walk":
        $AnimatedSprite2D.play("walk")
    elif entry.animation == "idle":
```

```
$AnimatedSprite2D.play("idle")
elif entry.animation == "sleep":
    $AnimatedSprite2D.play("sleep")
```

NPC 日程效果:

```
早上 6:00 - 铁匠从家里走到铁匠铺
早上 8:00 - 铁匠在铺子里打铁 (对话: "看看武器?")
中午 12:00 - 铁匠去吃饭 (对话: "午休时间, 待会再来。")
下午 13:00 - 回铁匠铺继续工作
晚上 18:00 - 收工回家
晚上 21:00 - 睡觉 (对话: "呼...Zzz...")
```

7. 最终方案: 最简可用模板

```
``gdscript
```

———— NPC.gd (通用 NPC, 复制即用) ————

```
extends CharacterBody2D
```

```
@export var npc_name: String = "村民" @export var dialogue: String =  
"你好。"
```

```
func on_interact(): DialogueUI.show(npc_name, dialogue)
```

DialogueUI.gd (Autoload)

```
extends CanvasLayer
```

```
@onready var panel = $Panel @onready var name_label = $Panel/  
Name @onready var text_label = $Panel/Text
```

```
func show(name: String, text: String): panel.show() name_label.text =  
name text_label.text = text get_tree().paused = true
```

```
func _input(event): if panel.visible and  
event.is_action_pressed("interact"): panel.hide() get_tree().paused =  
false
```

—— 玩家交互检测 (挂在玩家身上或场景里)

Area2D 检测最近的 NPC, 按 F 交互

```
@onready var interact_area = $InteractArea

func _ready(): interact_area.area_entered.connect(_on_enter)
interact_area.area_exited.connect(_on_exit)

var nearby_npcs := []

func _on_enter(area): if area.is_in_group("npc_interact"):
    nearby_npcs.append(area.get_parent())

func _on_exit(area): nearby_npcs.erase(area.get_parent())

func _input(event): if event.is_action_pressed("interact"): if
    nearby_npcs.size() > 0: nearby_npcs[0].on_interact() # 跟最近的 NPC 对
    话
```

第80章 对话系统演化

Godot 对话系统: 从气泡到分支树

对话 = 游戏世界的角色跟玩家说话。从头上冒气泡, 到字幕框, 到多分支对话树, 到条件对话。对话系统的进化 = "怎么让玩家觉得自己在跟一个活人说话"。

1. 原始: 头上气泡

```
# NPC 头上固定显示文字
$Label.text = "你好!"
```

2. 打字机效果

```
extends Label

var full_text = ""
var timer = 0.0
var char_index = 0

func show_text(text: String):
    full_text = text
    char_index = 0
    timer = 0

func _process(delta):
    timer += delta
    while timer > 0.05 and char_index < full_text.length():
        text += full_text[char_index]
```

```
char_index += 1
timer -= 0.05
```

3. 对话框 + 选项

```
# DialogueUI.gd
extends CanvasLayer

signal option_selected(index: int)

func show_dialogue(npc_name: String, text: String):
    $Name.text = npc_name
    $Text.text = text
    $Options.hide()

func show_options(options: Array[String]):
    $Options.show()
    for i in options.size():
        var btn = $Options.get_child(i)
        btn.text = options[i]
        btn.disabled = false
```

4. 对话树 (Resource)

```
class_name DialogueNode
extends Resource

@export var npc_text: String
@export var responses: Array[Response]

class Response:
    @export var player_text: String
    @export var next_node: DialogueNode
    @export var condition: String # 条件, 为空则无条件
    @export var action: String    # 触发动作
```

5. 最终方案模板

```
# 最简单的对话：
# 1. Area2D 检测玩家靠近
# 2. 按 F 显示对话
# 3. 打字机效果显示文字
# 4. 按 F 继续/关闭

func _input(event):
    if event.is_action_pressed("interact") and _player_nearby:
        if not DialogueUI.is_visible():
            DialogueUI.show(npc_name, dialogues[current_line])
            current_line += 1
        else:
            if current_line < dialogues.size():
                DialogueUI.show(npc_name,
dialogues[current_line])
                current_line += 1
            else:
                DialogueUI.hide()
                current_line = 0
```

第81章 好感度系统演化

Godot 好感度系统: 从对话到关系

好感度 = NPC 对玩家的态度数值。最早 NPC 对所有人都一样。后来有了送礼物加好感、对话选择影响好感、好感解锁新内容。好感度系统的进化 = "怎么让 NPC 从工具人变成有温度的人"。

1. 原始: NPC 固定态度

```
# 每次对说话一样的话
func talk():
    say("你好, 冒险者。")
```

2. 好感度数值

```
class Affinity:
    var npc_id: String
    var value: int = 0          # -100 ~ 100
    var gift_count: int = 0

const LEVELS = [
    { "min": -100, "title": "仇恨" },
    { "min": -50,  "title": "冷淡" },
    { "min": 0,    "title": "中立" },
    { "min": 50,   "title": "友好" },
    { "min": 80,   "title": "亲密" },
]

func get_level() -> int:
    for i in LEVELS.size():
```

```

        if value >= LEVELS[i].min:
            return i
    return 0

func add(value: int, reason: String = ""):
    self.value = clamp(self.value + value, -100, 100)

```

3. 礼物 + 每日上限

```

func give_gift(npc_id: String, item_id: String) -> bool:
    var affinity = _get_affinity(npc_id)
    var gift_data = GiftDB.get(item_id)

    if _today_gifts.has(npc_id) and _today_gifts[npc_id] >= 5:
        return false # 每天最多送 5 次

    if not InventoryManager.remove(item_id, 1):
        return false

    var gain = gift_data.affinity_gain
    if gift_data.is_favorite:
        gain *= 2 # 喜欢的礼物 2 倍

    affinity.add(gain, "gift")
    _today_gifts[npc_id] = _today_gifts.get(npc_id, 0) + 1
    return true

```

4. 好感度解锁

```

func get_available_dialogue(npc_id: String) -> Array:
    var affinity = _get_affinity(npc_id)
    var level = affinity.get_level()
    return DialogueDB.filter(func(d):
        return d.npc_id == npc_id and d.require_affinity <=
level
    )

func get_available quests(npc_id: String) -> Array:

```



```

    var affinity = _get_affinity(npc_id)
    return QuestDB.filter(func(q):
        return q.giver == npc_id and q.min_affinity <=
affinity.value
    )

```

5. 最终方案模板

```

# AffinityManager.gd (Autoload)
extends Node

var _data := {} # npc_id → { value, gifts_today, date }

func add(npc_id: String, amount: int):
    var d = _get(npc_id)
    d.value = clamp(d.value + amount, -100, 100)
    _save()

func get_level(npc_id: String) -> int:
    var v = _get(npc_id).value
    if v >= 80: return 4 # 亲密
    if v >= 50: return 3 # 友好
    if v >= 0: return 2 # 中立
    if v >= -50: return 1 # 冷淡
    return 0 # 仇恨

func give_gift(npc_id: String, item_id: String) -> bool:
    var d = _get(npc_id)
    var today = Time.get_unix_time_from_system() / 86400
    if d.date != today:
        d.date = today
        d.gifts_today = 0
    if d.gifts_today >= 5:
        return false
    if not InventoryManager.remove(item_id, 1):
        return false
    d.gifts_today += 1
    var gain = ItemDB.get(item_id).affinity_gain
    d.value = clamp(d.value + gain, -100, 100)
    _save()
    return true

```

第82章 捏脸系统演化

Godot 捏脸系统：从固定外观到自定义形象

捏脸 = 让玩家自定义角色的外观。最早角色外观是固定的，选个人物就完了。后来有了预设脸型、滑块调参数、颜色选择器、数据保存。捏脸系统的进化 = "怎么让每个玩家的角色都不一样"。

1. 原始：固定角色

```
# 只有一种外观  
# 没得选
```

2. 预设选择

```
class CharacterPreset:  
    var id: String  
    var name: String  
    var head_texture: Texture2D  
    var hair_texture: Texture2D  
    var eye_texture: Texture2D  
    var skin_color: Color  
    var hair_color: Color  
  
# 玩家从几个预设里选一个  
func apply_preset(preset_id: String):  
    var p = PresetDB.get(preset_id)  
    $Head.texture = p.head_texture  
    $Hair.texture = p.hair_texture  
    $Eyes.texture = p.eye_texture
```

```
$Body.modulate = p.skin_color  
$Hair.modulate = p.hair_color
```

3. 滑块参数化

```
class FaceParams:  
    # 连续值参数 (0.0~1.0)  
    var face_width: float = 0.5    # 脸宽  
    var jaw_width: float = 0.5     # 下巴宽  
    var eye_size: float = 0.5      # 眼睛大小  
    var eye_distance: float = 0.5  # 眼距  
    var nose_size: float = 0.5     # 鼻子大小  
    var mouth_width: float = 0.5   # 嘴宽  
    var brow_height: float = 0.5   # 眉毛高度  
    var skin_color: Color = Color(1, 0.8, 0.6)  
    var hair_color: Color = Color(0.2, 0.1, 0.05)  
    var eye_color: Color = Color(0.3, 0.5, 0.8)  
  
    func to_dict() -> Dictionary:  
        return {  
            "face_width": face_width,  
            "jaw_width": jaw_width,  
            "eye_size": eye_size,  
            "skin_color": skin_color.to_html(),  
            "hair_color": hair_color.to_html(),  
        }
```

```
func apply_params(params: FaceParams):  
    # 2D 实现: 用 Bone2D 或 shader 变形  
    # 或者预渲染不同脸型的贴图, 按参数 blend  
  
    # 方案A: 使用 Skeleton2D + Bone2D 变形  
    $Skeleton/BoneFace.scale.x = lerp(0.8, 1.2,  
params.face_width)  
    $Skeleton/BoneJaw.position.y = lerp(-5, 5, params.jaw_width)  
    $Skeleton/BoneEyes.scale = Vector2(  
        lerp(0.8, 1.4, params.eye_size),  
        lerp(0.8, 1.4, params.eye_size)  
    )  
  
    # 方案B: 颜色
```

```
$Body.material.set_shader_parameter("skin_color",
params.skin_color)
$Hair.material.set_shader_parameter("hair_color",
params.hair_color)
```

4. 捏脸数据存档

```
func save_face(params: FaceParams, slot: int):
    var file = FileAccess.open("user://face_%d.json" % slot,
FileAccess.WRITE)
    file.store_string(JSON.stringify(params.to_dict()))

func load_face(slot: int) -> FaceParams:
    if not FileAccess.file_exists("user://face_%d.json" % slot):
        return FaceParams.new()
    var file = FileAccess.open("user://face_%d.json" % slot,
FileAccess.READ)
    var data = JSON.parse_string(file.get_as_text())
    var params = FaceParams.new()
    for key in data:
        if params.has(key):
            params[key] = data[key]
    return params
```

5. 最终方案模板

```
# FaceCustomizer.gd (挂载到捏脸场景)
# 核心: 参数 → 实时预览 → 存档

var params := FaceParams.new()
var slot := 0

func _ready():
    params = load_face(slot)
    apply_params(params)

func update_param(name: String, value: float):
    params.set(name, value)
```

```
    apply_params(params)

func save():
    save_face(params, slot)

# UI 连接:
# $HSlider.value_changed.connect(func(v):
update_param("face_width", v))
```

第83章 场景切换演化

Godot 场景切换: 从一个场景到另一个场景

`change_scene_to_file("res://Level2.tscn")` – 一行代码就能切场景。但它让你: 黑屏 500ms、进度条没有、音乐断了、上一个场景的数据丢了。

场景切换不只是"加载新文件", 是"怎么让玩家感觉不到切换"。

目录

1. [直接切换: `change\_scene`](#)
 2. [淡入淡出: 过渡动画](#)
 3. [场景间传数据: 不被销毁的东西](#)
 4. [后台加载: 不卡主线程](#)
 5. [加载进度条: 告诉玩家没死机](#)
 6. [SceneManager: 统一切换管理](#)
 7. [场景预加载: 提前准备好](#)
 8. [最终方案: 最简可用模板](#)
-

1. 直接切换: `change_scene`

问题: "关卡通关了, 切换到下一关。"

最初的做法:


```
func on_level_complete():  
    get_tree().change_scene_to_file("res://Level2.tscn")
```

效果: 当前场景的所有节点被销毁, 新场景加载, 显示出来。

这条隐藏操作链:

```
change_scene_to_file("res://Level2.tscn") 内部:  
1. 卸载当前场景 (所有节点 queue_free)  
2. 从硬盘加载 Level2.tscn (同步加载, 阻塞主线程)  
3. 解析场景资源, 实例化所有节点  
4. 将新场景设置为 SceneTree 的 current_scene  
5. 调用新场景所有节点的 _ready()
```

问题:

1. **卡顿** – 加载是同步的。如果 Level2.tscn 有 500 个节点、贴图、声音 → 阻塞主线程 → 画面冻结 → 看起来像程序崩溃了。
2. **黑屏** – 没有过渡动画。上一个场景瞬间消失, 下一个瞬间出现。中间的几百毫秒是"黑屏或上一帧的残留"。
3. **音乐中断** – 新场景加载时, 上一个场景的所有节点被销毁。BGM 节点也被销毁了 → 音乐停。
4. **数据丢失** – 玩家在这一关收集了多少金币? 血量多少? 这些数据在 Level1 的脚本的变量里。Level1 被卸载 → 变量全丢了。
5. **没有进度** – 如果加载要 3 秒钟, 玩家盯着黑屏不知道还要等多久。

开发者开始逐个解决这些问题。

2. 淡入淡出: 过渡动画

问题: 场景切换时直接黑屏, 体验差。

解决方案: 在切换前后加过渡动画。

```
# 淡入淡出切换
```

```
@onready var transition = $ColorRect # 一个铺满全屏的黑色 ColorRect  
var is_transitioning = false
```

```
func go_to_scene(path: String):
```

```
    if is_transitioning:
```

```
        return
```

```
    is_transitioning = true
```

```
    # 淡出 (当前场景 → 黑)
```

```
    var tween = create_tween()
```

```
    tween.tween_property(transition, "modulate:a", 1.0, 0.3)
```

```
    await tween.finished
```

```
    # 切场景
```

```
    get_tree().change_scene_to_file(path)
```

```
    # 淡入 (黑 → 新场景)
```

```
    transition =
```

```
    get_tree().current_scene.find_child("ColorRect")
```

注意: change_scene_to_file 之后, 之前的 transition 节点已经被销毁了!

```
    # 需要新场景也有一个 ColorRect
```

```
    tween = create_tween()
```

```
    tween.tween_property(transition, "modulate:a", 0.0, 0.3)
```

```
    await tween.finished
```

```
    is_transitioning = false
```

存在一个严重问题: `change_scene_to_file` 会把当前场景的所有节点销毁。

如果 `$ColorRect` 在当前场景里, 它就没了。新场景里必须有另一个

`ColorRect`。

正确做法: 用一个不随场景切换而销毁的 `CanvasLayer`。

```
# Autoload: TransitionLayer.gd
```

```
extends CanvasLayer
```

```
@onready var color_rect = $ColorRect
```

```

func fade_out(duration: float = 0.3):
    color_rect.modulate = Color(0, 0, 0, 0)
    var tween = create_tween()
    tween.tween_property(color_rect, "modulate:a", 1.0,
duration)
    await tween.finished

func fade_in(duration: float = 0.3):
    color_rect.modulate = Color(0, 0, 0, 1)
    var tween = create_tween()
    tween.tween_property(color_rect, "modulate:a", 0.0,
duration)
    await tween.finished

```

```

# 使用:
TransitionLayer.fade_out()
await TransitionLayer.fade_out # 等淡出结束

get_tree().change_scene_to_file("res://Level2.tscn")

TransitionLayer.fade_in()

```

但 **autoload** 不受 **change_scene** 影响, 所以 **TransitionLayer** 会一直存在。

效果: 从"啪, 黑了" 变成 "慢慢变黑 → 加载 → 慢慢亮起"。体验好了一个档次。

3. 场景间传数据: 不被销毁的东西

问题: Level1 里玩家有 50 金币、3 条命。切换到 Level2, 这些数据全丢了。

原因: 变量存在场景的脚本里, 场景被卸载 → 变量消失。

方案 1: Autoload (单例)

```

# GameState.gd (设为 Autoload)
extends Node

var gold: int = 0
var lives: int = 3
var current_level: int = 1

```

```
# Level1.gd
func on_gold_collected(amount):
    GameState.gold += amount

func on_level_complete():
    GameState.current_level += 1
    get_tree().change_scene_to_file("res://Level2.tscn")

# Level2.gd
func _ready():
    $GoldLabel.text = "金币: %d" % GameState.gold
    $LivesLabel.text = "命: %d" % GameState.lives
```

Autoload 不会被 change_scene 销毁, 数据一直存活。

方案 2: Resource 文件

```
# PlayerData.gd
class_name PlayerData
extends Resource

@export var gold: int = 0
@export var lives: int = 3
@export var max_hp: int = 100
@export var current_hp: int = 100
```

```
# 在 Autoload 或场景脚本中:
var player_data = PlayerData.new()

# change_scene 不会销毁 Resource 实例
# 因为 Resource 不是场景树节点
```

方案 3: 跨场景引用 (不推荐)

```
# 切场景前把数据存到 Autoload
Global.pending_data = {"gold": 50, "hp": 80}

# 新场景 _ready 时读取
func _ready():
    gold = Global.pending_data.gold
```

Core 原则: 所有"跨场景需要保留的数据"必须放在 Autoload 或 Resource 里, 不能放在场景的脚本变量里。

4. 后台加载: 不卡主线程

问题: `change_scene_to_file` 是**同步加载**。加载期间主线程被阻塞, 画面冻结。如果场景很大 (3D 关卡, 几百 MB 资源), 冻结可能持续好几秒。

解决方案: `ResourceLoader.load_async`

```
# 后台加载场景, 不阻塞游戏

func load_scene_async(path: String):
    # 启动异步加载
    var loader = ResourceLoader.load_threaded_request(path)

    # 每帧检查加载进度
    while true:
        var status =
ResourceLoader.load_threaded_get_status(path)
        match status:
            0: # 正在加载
                await get_tree().process_frame # 等一帧, 让游戏继
续运行
            1: # 加载完成, 但还没实例化
                var scene =
ResourceLoader.load_threaded_get(path) # 获取场景
                get_tree().change_scene_to_packed(scene)
                return
            2: # 加载失败
                push_error("加载失败: " + path)
                return
```

更完整的写法:

```
# SceneLoader.gd (Autoload)

signal load_started(path: String)
```

```

signal load_progress(progress: float)
signal load_completed(path: String)

var _loader: ResourceLoader
var _loading_path: String

func load_scene(path: String, use_transition: bool = true):
    _loading_path = path

    # 1. 淡出 (可选)
    if use_transition:
        await TransitionLayer.fade_out()

    # 2. 通知旧场景清理

get_tree().root.propagate_notification(NOTIFICATION_WM_WINDOW_F
OCUS_OUT)
    # 或自定义: get_tree().call_group("level", "on_before_unload")

    # 3. 启动后台加载
    load_started.emit(path)
    ResourceLoader.load_threaded_request(path)

    # 4. 等待完成, 每帧更新进度
    while true:
        var progress := []
        var status =
ResourceLoader.load_threaded_get_status(path, progress)

        match status:
            ResourceLoader.THREAD_LOAD_IN_PROGRESS:
                if progress.size() > 0:
                    load_progress.emit(progress[0])
                    await get_tree().process_frame

            ResourceLoader.THREAD_LOAD_LOADED:
                var scene =
ResourceLoader.load_threaded_get(path)
                get_tree().change_scene_to_packed(scene)
                load_completed.emit(path)

                if use_transition:
                    await TransitionLayer.fade_in()
                return

```

```
ResourceLoader.THREAD_LOAD_FAILED:
    push_error("场景加载失败: " + path)
    return

ResourceLoader.THREAD_LOAD_INVALID_RESOURCE:
    push_error("无效资源路径: " + path)
    return
```

后台加载的核心价值:

```
change_scene_to_file:
    加载时: 主线程阻塞, 画面冻结, 无法交互
    加载后: 突然出现新场景

后台加载:
    加载时: 游戏继续运行, 可以显示进度条/动画/提示
    加载后: 通过过渡动画平滑切入新场景
```

5. 加载进度条: 告诉玩家没死机

问题: 后台加载时不卡了, 但玩家看着黑屏不知道要等多久。可能会以为游戏崩溃了。

解决方案: 显示加载进度。

```
# LoadingScreen.gd (Autoload, 或独立场景)
extends CanvasLayer

@onready var progress_bar = $ProgressBar
@onready var hint_label = $HintLabel
@onready var tip = $TipLabel

var tips = [
    "按 Shift 可以冲刺",
    "收集金币可以解锁新角色",
    "敌人从背后攻击伤害翻倍",
]
```

```

func show():
    show()
    progress_bar.value = 0.0
    tip.text = tips.pick_random()

func update_progress(progress: float):
    progress_bar.value = progress
    # 显示百分比
    progress_bar.get_node("Label").text = "%d%%" % (progress *
100)

func hide():
    hide()

```

整合到 SceneLoader:

```

func load_scene(path: String):
    LoadingScreen.show()
    load_started.emit(path)
    ResourceLoader.load_threaded_request(path)

    while true:
        var progress := []
        var status =
ResourceLoader.load_threaded_get_status(path, progress)

        match status:
            ResourceLoader.THREAD_LOAD_IN_PROGRESS:
                if progress.size() > 0:
                    LoadingScreen.update_progress(progress[0])
                    await get_tree().process_frame

            ResourceLoader.THREAD_LOAD_LOADED:
                var scene =
ResourceLoader.load_threaded_get(path)
                LoadingScreen.update_progress(1.0)
                await get_tree().create_timer(0.2).timeout # 让
玩家看到 100%

                get_tree().change_scene_to_packed(scene)
                LoadingScreen.hide()
                return

```


进度条的意义: 不是"真的告诉玩家还剩多少", 是让玩家知道游戏还在跑, 没死机。

6. SceneManager: 统一切换管理

问题: 每个关卡的切换都要写淡入/后台加载/进度条。重复代码散落在各个场景里。

统一封装:

```
# SceneManager.gd (Autoload)

extends Node

var current_scene_path: String
var _loading := false

signal before_scene_change(from: String, to: String)
signal after_scene_change(scene_path: String)

func switch_scene(path: String):
    if _loading:
        return
    _loading = true

    before_scene_change.emit(current_scene_path, path)

    # 1. 淡出
    await TransitionLayer.fade_out()

    # 2. 后台加载
    LoadingScreen.show()
    ResourceLoader.load_threaded_request(path)

    while true:
        var progress := []
        var status =
ResourceLoader.load_threaded_get_status(path, progress)
        match status:
```

```

        ResourceLoader.THREAD_LOAD_IN_PROGRESS:
            LoadingScreen.update_progress(progress[0])
            await get_tree().process_frame
        ResourceLoader.THREAD_LOAD_LOADED:
            var scene =
ResourceLoader.load_threaded_get(path)
            LoadingScreen.update_progress(1.0)
            await get_tree().create_timer(0.15).timeout

            get_tree().change_scene_to_packed(scene)
            current_scene_path = path

            LoadingScreen.hide()
            await TransitionLayer.fade_in()

            after_scene_change.emit(path)
            _loading = false
            return
        ResourceLoader.THREAD_LOAD_FAILED:
            LoadingScreen.hide()
            push_error("加载失败: " + path)
            _loading = false
            return

# 使用:
# 在任何脚本里:
SceneManager.switch_scene("res://Level2.tscn")

```

加上打包的场景 (PackedScene) 支持:

```

# 支持直接传 PackedScene (预加载好的)
func switch_to_packed(scene: PackedScene):
    if _loading:
        return
    _loading = true

    before_scene_change.emit(current_scene_path,
scene.resource_path)

    await TransitionLayer.fade_out()
    get_tree().change_scene_to_packed(scene)
    current_scene_path = scene.resource_path
    await TransitionLayer.fade_in()

```

```
after_scene_change.emit(scene.resource_path)
_loading = false
```

7. 场景预加载: 提前准备

问题: 即使后台加载, 玩家还是要等。如果能在玩家还在玩第一关时, 就把第二关在后台加载好, 切换时瞬间完成。

在玩家进入第一关时:

```
func _ready():
    # 第二关已经在后台开始加载了
    SceneManager.precache("res://Level2.tscn")
```

SceneManager 增加预缓存功能:

```
var _cache := {}

func precache(path: String):
    if _cache.has(path):
        return
    # 启动后台加载, 但加载完成后存在缓存里, 不切换
    ResourceLoader.load_threaded_request(path)
    _cache[path] = "loading" # 标记为正在加载

func _process(delta):
    # 检查预加载是否完成
    for path in _cache.keys():
        if _cache[path] != "loading":
            continue
        var progress := []
        var status =
            ResourceLoader.load_threaded_get_status(path, progress)
        if status == ResourceLoader.THREAD_LOAD_LOADED:
            _cache[path] =
                ResourceLoader.load_threaded_get(path)
        elif status == ResourceLoader.THREAD_LOAD_FAILED:
            _cache.erase(path)
```

```

func get_cached(path: String):
    if _cache.has(path) and _cache[path] is PackedScene:
        return _cache[path]
    return null

func switch_scene_with_cache(path: String):
    var cached = get_cached(path)
    if cached:
        # 预加载完成，瞬间切换
        var full_path = path
        await TransitionLayer.fade_out()
        get_tree().change_scene_to_packed(cached)
        current_scene_path = full_path
        await TransitionLayer.fade_in()
        _cache.erase(path)
    else:
        # 没预加载，走正常流程
        await switch_scene(path)

```

预加载策略：

```

# 游戏开始时预加载 "主菜单"（反正马上要用）
→ 玩家进第一关时预加载 "第二关"
→ 玩家进第二关时预加载 "第三关"
→ ...

```

8. 最终方案：最简可用模板

```

# 如果你不想搞复杂，这个模板够用：

# —— SceneManager.gd (Autoload) ——

extends Node

@onready var transition = $CanvasLayer/ColorRect

func go_to(path: String):
    # 淡出

```

```

var tween = create_tween()
tween.tween_property(transition, "modulate:a", 1.0, 0.3)
await tween.finished

# 后台加载（同时显示"加载中"）
ResourceLoader.load_threaded_request(path)
while true:
    var p := []
    var status =
ResourceLoader.load_threaded_get_status(path, p)
    if status == ResourceLoader.THREAD_LOAD_LOADED:
        break
    await get_tree().process_frame

# 切场景
var scene = ResourceLoader.load_threaded_get(path)
get_tree().change_scene_to_packed(scene)

# 淡入
tween = create_tween()
tween.tween_property(transition, "modulate:a", 0.0, 0.3)
await tween.finished

# 场景切换只需：
# SceneManager.go_to("res://Level2.tscn")

```

这个故事告诉我们：- `change_scene_to_file` 只是起点，不是终点 - 好切换 = 过渡动画 + 数据保持 + 后台加载 + 进度反馈 - Autoload 是场景切换中唯一不变的东西 - 过渡、数据、管理器都在 Autoload 里 - 预加载把"等"变成"提前准备好"，不是换场景时加载，而是加载好了再换

第84章 场景系统演化

Godot 场景系统: 从单场景到场景管理

一个游戏 = 标题画面 → 选人 → 关卡1 → 关卡2 → Boss → 结局。每个"画面"都是一个场景。场景系统 = 怎么把这些画面串起来, 以及串起来时数据不丢、不卡顿。

1. 原始: 一个场景做所有事

```
# Game.gd - 所有逻辑写在一个场景里
extends Node2D

var state = "title" # title / playing / boss / ending

func _process(delta):
    match state:
        "title":
            if Input.is_action_just_pressed("start"):
                state = "playing"
                $Title.hide()
                $Level1.show()
        "playing":
            # 关卡逻辑...
            if is_level_clear:
                state = "boss"
                $Level1.hide()
                $Boss.show()
```

问题: 一个场景文件上千节点, 加载慢, 改一处要重新导入整个场景, 团队协作困难。

2. 场景即关卡：每个关卡一个独立场景

把每个"画面"拆成独立场景文件：

```
res://scenes/  
├─ Title.tscn  
├─ CharacterSelect.tscn  
├─ Level1.tscn  
├─ Level2.tscn  
├─ BossLevel.tscn  
├─ Ending.tscn  
└─ GameOver.tscn
```

```
get_tree().change_scene_to_file("res://scenes/Level1.tscn")
```

每个场景独立开发、独立加载。问题只在于切换时的卡顿和数据传递。

3. 场景层级架构

```
# 游戏分为三层：  
#   1. 永久层 (Autoload)：Manager, Audio, Data  
#   2. 游戏层：当前场景（关卡/菜单）  
#   3. UI 层：HUD, 对话, 菜单 (CanvasLayer, 不随场景销毁)  
  
# SceneTree 结构：  
# SceneTree  
# └─ AudioManager (Autoload)  
# └─ GameState (Autoload)  
# └─ PoolManager (Autoload)  
# └─ 当前场景 (change_scene 时替换)  
# └─ UILayer (CanvasLayer, 常驻)
```

4. 场景堆叠：多个场景同时存在

问题：关卡里打开背包，需要暂停关卡但保留关卡状态。


```
# 不是切换场景，是在关卡之上叠加 UI 场景
func open_inventory():
    var inventory = preload("res://ui/
Inventory.tscn").instantiate()
    get_tree().root.add_child(inventory) # 挂在 root 上，不干扰当前
场景
    get_tree().paused = true # 暂停游戏
```

5. 最终方案模板

```
# SceneSystem.gd (Autoload)
extends Node

var _current_scene: Node
var _overlays := []

func switch(path: String):
    if _current_scene:
        _current_scene.queue_free()
    _current_scene = load(path).instantiate()
    get_tree().root.add_child(_current_scene)
    get_tree().current_scene = _current_scene

func push_overlay(path: String) -> Node:
    var overlay = load(path).instantiate()
    get_tree().root.add_child(overlay)
    _overlays.append(overlay)
    return overlay

func pop_overlay():
    if _overlays.size() > 0:
        var overlay = _overlays.pop_back()
        overlay.queue_free()

# 使用:
# SceneSystem.switch("res://scenes/Title.tscn")
# var inv = SceneSystem.push_overlay("res://ui/Inventory.tscn")
# SceneSystem.pop_overlay()
```

第85章 场景交互系统演化

Godot 场景交互系统：从碰触到可交互物

场景交互 = 玩家在场景中与物品互动。最早场景里的东西就只是背景贴图。后来有了可破坏的罐子、可推动的箱子、可攀爬的藤蔓。场景交互系统的进化 = "怎么让场景里的东西都能互动"。

1. 原始：纯装饰

```
# 场景里的罐子、箱子、草堆都是背景  
# 玩家走过去没有任何事发生
```

2. 可破坏物

```
class BreakableObject:  
    extends StaticBody2D  
  
    @export var hp: int = 1  
    @export var drop_item: String = ""  
    @export var drop_count: int = 1  
    @export var break_effect: PackedScene  
  
    func take_damage(amount: int):  
        hp -= amount  
        if hp <= 0:  
            _break()  
  
    func _break():  
        # 掉落物品  
        if drop_item:
```

```

        InventoryManager.add(drop_item, drop_count)
# 特效
if break_effect:
    var e = break_effect.instantiate()
    get_parent().add_child(e)
    e.global_position = global_position
# 碎片
$Sprite.texture = broken_texture
$CollisionShape2D.disabled = true
# 消失

create_tween().tween_interval(2.0).tween_callback(queue_free)

```

3. 可推动箱子

```

# PushBlock.gd
extends CharacterBody2D

var is_pushing := false

func push(direction: Vector2):
    if is_pushing: return
    is_pushing = true
    velocity = direction * 100
    move_and_slide()
    is_pushing = false
# 落在网格上
position = position.snapped(Vector2(32, 32))

```

```

# 玩家检测前方是否有可推动箱子
func _check_push():
    var space = get_world_2d().direct_space_state
    var query = PhysicsRayQueryParameters2D.new(global_position,
    global_position + Vector2.RIGHT * 32)
    query.collide_with_areas = false
    var result = space.intersect_ray(query)
    if result.collider and result.collider.has_method("push"):
        result.collider.push(Vector2.RIGHT)

```

4. 可攀爬

```
# ClimbableArea.gd
extends Area2D

@export var climb_speed := 100

func _on_body_entered(body):
    if body.is_in_group("player"):
        body.set_climbing(true, self)

func _on_body_exited(body):
    if body.is_in_group("player"):
        body.set_climbing(false, null)
```

```
# 玩家代码中:
var is_climbing := false

func _physics_process(delta):
    if is_climbing:
        gravity = 0
        velocity = Vector2.ZERO
        if Input.is_action_pressed("up"):
            velocity.y = -climb_speed
        elif Input.is_action_pressed("down"):
            velocity.y = climb_speed
    else:
        gravity = normal_gravity
        move_and_slide()
```

5. 最终方案模板

```
# 交互系统核心:
# 1. 可破坏: 受攻击 → HP 归零 → 掉落 + 特效 + 消失
# 2. 可推动: 玩家检测 + 沿方向移动 + 网格吸附
# 3. 可攀爬: 进入区域 → 切换移动模式 → 退出区域 → 恢复
# 4. 可拾取: Area2D 检测 + 按键拾取
# 5. 可开关: 拉杆/按钮 + Signal
```

```
func interact():  
    # 检测前方可交互物  
    var hit = _raycast_front()  
    if hit and hit.collider.has_method("on_interact"):  
        hit.collider.on_interact()
```

第86章 机关系系统演化

Godot 机关/陷阱系统: 从静态场景到交互机关

机关 = 场景里的可交互机制。最早背景就是背景, 什么都碰不了。后来有了压力板开门、尖刺陷阱、旋转刀刃、传送带。机关系统的进化 = "怎么让场景本身成为 gameplay 的一部分"。

1. 原始: 静态场景

```
# 场景里的东西只是贴图  
# 玩家走过没有反应
```

2. 压力板 + 门

```
# PressurePlate.gd  
extends Area2D  
  
signal activated  
signal deactivated  
  
var _activated = false  
  
func _on_body_entered(body):  
    if not _activated:  
        _activated = true  
        $Sprite.frame = 1 # 压下去  
        activated.emit()  
  
func _on_body_exited(body):  
    # 检查是否还有其他物体在上面
```



```

    if get_overlapping_bodies().is_empty():
        _activated = false
        $Sprite.frame = 0
        deactivated.emit()

```

```

# Door.gd
extends StaticBody2D

func open():
    $CollisionShape2D.disabled = true
    create_tween().tween_property(self, "modulate:a", 0, 0.3)

func close():
    $CollisionShape2D.disabled = false
    create_tween().tween_property(self, "modulate:a", 1, 0.3)

```

3. 陷阱类型

```

enum TrapType {
    SPIKE,          # 尖刺：踩到扣血
    ROTATING_BLADE, # 旋转刀：碰到扣血
    ARROW_SHOOTER,  # 射箭：定时触发
    TELEPORTER,     # 传送：到另一个位置
    CONVEYOR,       # 传送带：推动玩家
    BOUNCE_PAD,     # 弹跳板：弹飞玩家
}

```

```

# SpikeTrap.gd
extends Area2D

@export var damage := 30
@export var active_duration := 1.0
@export var interval := 3.0
var _timer := 0.0
var _active := false

func _process(delta):
    _timer += delta
    if _active and _timer >= active_duration:
        _retract()

```

```

        elif not _active and _timer >= interval:
            _extend()

func _extend():
    _active = true
    _timer = 0.0
    $AnimationPlayer.play("extend")
    $Hitbox.monitoring = true

func _retract():
    _active = false
    _timer = 0.0
    $AnimationPlayer.play("retract")
    $Hitbox.monitoring = false

func _on_hitbox_body_entered(body):
    if body.is_in_group("player"):
        body.take_damage(damage)

```

4. 最终方案模板

```

# 机关系统核心模式：Signal 连接

# 压力板.activated → 开门
# 压力板.deactivated → 关门
# 多个压力板需要同时激活 → AND 门
# 拉杆 → 切换门/桥/平台

# 通用机关接口：
# activate() / deactivate() / toggle()
# 通过 Signal 连接不需要写胶水代码

```

第87章 建筑系统演化

Godot 建筑系统：从固定场景到玩家建造

最早的游戏建筑是场景里摆好的素材，玩家不能改。后来玩家能建房子、能摆家具、能经营农场。建筑系统的进化 = "怎么让玩家在游戏世界里留下自己的痕迹"。

1. 固定建筑 (场景里手摆)

```
# 在场景编辑器里拖入 Sprite2D + CollisionShape2D
# 建筑是场景的一部分，不可交互、不可改变
```

2. 可交互建筑 (商店/传送点)

```
# 建筑本身是一个节点，带交互区域
extends Area2D

func _ready():
    body_entered.connect(_on_enter)

func _on_enter(body):
    if body.is_in_group("player"):
        $Prompt.show() # "按 F 进入"

func interact():
    SceneSystem.push_overlay("res://ui/Shop.tscn")
```

3. 玩家放置建筑

```
# 玩家选择"建造模式" → 点空地 → 放置建筑

func place_building(building_id: String, position: Vector2):
    var building =
    building_data[building_id].scene.instantiate()
    building.position = position
    building.building_id = building_id
    add_child(building)
    save_buildings() # 存档时保存所有建筑位置
```

建筑数据:

```
class BuildingData:
    var name: String
    var scene: PackedScene
    var size: Vector2          # 占几格
    var build_time: float      # 建造时间
    var cost: Dictionary       # {"wood": 50, "stone": 30}
    var produces: Dictionary   # {"gold": 10, "food": 5} (每
    tick)
```

格子系统 (建筑必须放在网格上):

```
func snap_to_grid(pos: Vector2) -> Vector2:
    var grid_size = 32
    return Vector2(
        round(pos.x / grid_size) * grid_size,
        round(pos.y / grid_size) * grid_size
    )
```

4. 最终方案模板

```
# 最简单的建筑放置:
# 1. 建筑数据用 Resource
# 2. 建造模式: 显示预览(绿色/红色) → 点击放置
# 3. 放置后保存到建筑列表
```

4. 读档时重新实例化所有建筑

```
func try_build(data: BuildingResource, pos: Vector2):
    if not has_resources(data.cost):
        return false
    spend_resources(data.cost)
    var b = data.scene.instantiate()
    b.position = snap_to_grid(pos)
    b.building_data = data
    add_child(b)
    saved_buildings.append({ "id": data.id, "pos": pos })
    return true
```

第88章 家园系统演化

Godot 家园系统：从功能建筑到个人空间

家园 = 玩家自己的空间, 可以装修布置。建筑系统是功能性建筑 (商店/传送门)。家园系统是装饰性空间 (摆放家具、种花、展示收藏)。家园系统的进化 = "怎么让玩家有自己的小天地"。

1. 原始: 无家园

只有公共场景, 没有私人空间

2. 固定家园场景

```
# 每个玩家有一个固定的家
# 场景固定, 不能装修
func enter_home():
    SceneManager.switch("player_home")
```

3. 家具摆放

```
class Furniture:
    var id: String
    var name: String
    var scene: PackedScene
    var grid_size: Vector2i      # 占几格 (2x1, 1x1)
    var category: String        # "table" / "chair" /
    "decoration" / "wall"
    var placeable_on: String    # "floor" / "wall"
```



```

class HomeData:
    var furniture: Array[PlacedFurniture]
    var wallpaper: String
    var floor: String

class PlacedFurniture:
    var furniture_id: String
    var position: Vector2          # 网格坐标
    var rotation: int             # 0/90/180/270

func place_furniture(furniture_id: String, grid_pos: Vector2) ->
bool:
    var f = FurnitureDB.get(furniture_id)
    if not f: return false
    if not InventoryManager.has(furniture_id, 1): return false

    # 检查位置是否被占用
    if _is_occupied(grid_pos, f.grid_size): return false

    InventoryManager.remove(furniture_id, 1)
    _home.furniture.append(PlacedFurniture.new(furniture_id,
grid_pos))
    _spawn_furniture(f, grid_pos)
    return true

func remove_furniture(index: int):
    var pf = _home.furniture[index]
    InventoryManager.add(pf.furniture_id, 1)
    _home.furniture.remove_at(index)
    _refresh()

```

4. 家园展示 + 访问

```

func visit_home(player_id: String):
    var data = HomeManager.get_home_data(player_id)
    # 加载该玩家的家园场景
    SceneManager.load_home_scene(data)

func get_home_score() -> int:
    # 根据家具数量/稀有度/摆放评分

```

```
var score = 0
for pf in _home.furniture:
    score += FurnitureDB.get(pf.furniture_id).score
return score
```

5. 最终方案模板

```
# HomeManager.gd (Autoload)
# 数据结构：家园 = 墙纸 + 地板 + 家具列表
# 核心：网格系统 + 碰撞检测 + 数据持久化

func enter():
    SceneManager.switch_to_instance("home", _build_home_scene())

func _build_home_scene() -> Node2D:
    var scene = preload("res://home_base.tscn").instantiate()
    scene.set_wallpaper(_data.wallpaper)
    scene.set_floor(_data.floor)
    for pf in _data.furniture:
        var node = FurnitureDB.instance(pf.furniture_id)
        node.position = _grid_to_world(pf.position)
        node.rotation = pf.rotation
        scene.add_furniture(node)
    return scene
```

第89章 昼夜系统演化

Godot 昼夜系统：从固定到实时时间

昼夜 = 游戏里的时间在走, 白天和晚上不一样。最早的游戏没有时间概念, 永远是白天。后来有了日夜交替、不同时段的 NPC 行为、夜晚刷怪、月光照明。昼夜系统的进化 = "怎么让同一个世界白天和晚上玩起来像两个游戏"。

1. 原始：固定时间

```
# 永远是白天
# 没有"时间"这个概念
```

2. 计时器 + 颜色变化

```
var time_of_day = 0.0 # 0~1, 0=午夜, 0.5=正午
var day_length = 600.0 # 一天现实 600 秒

func _process(delta):
    time_of_day += delta / day_length
    if time_of_day >= 1.0:
        time_of_day -= 1.0
    _update_light()

func _update_light():
    # 根据时间调亮度
    var brightness = sin(time_of_day * PI * 2 - PI / 2) * 0.5 + 0.5
```

```
$DirectionalLight2D.energy = brightness * 0.8 + 0.2  
modulate = Color(brightness, brightness, brightness)
```

3. 完整昼夜周期(2D)

```
# 用 GradientTexture1D 控制颜色  
func _update_light():  
    var t = time_of_day  
    # t=0: 午夜(黑)  t=0.25: 日出(橙)  t=0.5: 正午(白)  t=0.75: 日落  
    (红)  
    var color = $DayNightGradient.sample(t)  
    $WorldEnvironment.environment.background_color = color  
    $DirectionalLight2D.color = color  
    $DirectionalLight2D.energy = _brightness_curve(t)  
  
func _brightness_curve(t: float) -> float:  
    return clamp(sin(t * TAU - PI / 2) * 0.6 + 0.4, 0.05, 1.0)
```

4. 时间驱动游戏逻辑

```
func _process(delta):  
    time_of_day += delta / day_length  
  
    var hour = int(time_of_day * 24)  
  
    # NPC 根据时间去不同位置  
    for npc in npcs:  
        npc.go_to_schedule(hour)  
  
    # 夜晚刷怪  
    if _is_night():  
        if not _night_spawn_timer:  
            _night_spawn_timer = _start_night_spawn()  
        else:  
            if _night_spawn_timer:  
                _night_spawn_timer = null  
  
func _is_night() -> bool:
```

```
var t = time_of_day
return t < 0.2 or t > 0.75

func get_hour() -> int:
    return int(time_of_day * 24) % 24
```

5. 最终方案模板

```
# TimeManager.gd (Autoload)
extends Node

signal time_changed(hour: int, minute: int)
signal day_passed(day: int)

var day: int = 1
var time_of_day: float = 0.25 # 6:00 开始
var time_scale: float = 1.0

const DAY_LENGTH = 600.0 # 秒

func _process(delta):
    time_of_day += delta * time_scale / DAY_LENGTH
    if time_of_day >= 1.0:
        time_of_day -= 1.0
        day += 1
        day_passed.emit(day)
    time_changed.emit(get_hour(), get_minute())

func get_hour() -> int: return int(time_of_day * 24) % 24
func get_minute() -> int: return int(time_of_day * 24 * 60) % 60
func is_night() -> bool: return time_of_day < 0.2 or time_of_day > 0.75
func is_day() -> bool: return not is_night()
```

第90章 天气系统演化

Godot 天气系统：从晴天到动态天气

天气 = 刮风、下雨、下雪、大雾。最早的游戏永远是晴天。后来有了随机天气、天气影响 gameplay、季节系统。天气系统的进化 = "怎么让世界天气变化影响玩家决策"。

1. 原始：永远晴天

没有天气概念

2. 随机天气 + 视觉效果

```
enum Weather { CLEAR, RAIN, SNOW, FOG }

var current_weather = Weather.CLEAR

func _ready():
    _change_weather(Weather.RAIN)
    # Particle: 雨滴粒子系统
    # Audio: 雨声音效
    # Visual: 屏幕变暗 + 色调变化
```

```
func random_weather():
    var weights = {
        Weather.CLEAR: 0.5,
        Weather.RAIN: 0.3,
        Weather.SNOW: 0.1,
        Weather.FOG: 0.1,
    }
```



```
current_weather = _weighted_pick(weights)
_apply_weather(current_weather)
```

3. 天气影响 gameplay

```
func _apply_weather(w: Weather):
    match w:
        Weather.RAIN:
            # 视野缩小
            $Camera.zoom = Vector2(1.2, 1.2)
            # 火系伤害降低
            CombatSystem.element_modifier["fire"] = 0.7
            # 移动速度降低
            PlayerData.speed_modifier = 0.85
        Weather.SNOW:
            PlayerData.speed_modifier = 0.6
            # 脚印效果
            $FootprintSpawner.enabled = true
        Weather.FOG:
            $Camera.fog_enabled = true
            $Camera.fog_range = Vector2(50, 200)
        Weather.CLEAR:
            # 恢复正常
            $Camera.zoom = Vector2(1, 1)
            PlayerData.speed_modifier = 1.0
```

4. 最终方案模板

```
# WeatherManager.gd (Autoload)
extends Node

signal weather_changed(old: int, new: int)

enum Type { CLEAR, RAIN, SNOW, FOG, STORM }
var current := Type.CLEAR
var duration := 300.0 # 当前天气持续秒数
var timer := 0.0
```

```
func _process(delta):
    timer += delta
    if timer >= duration:
        _transition_to(_random_weather())
        timer = 0.0

func _transition_to(new_weather: Type):
    var old = current
    current = new_weather
    duration = randf_range(120, 600)
    # 渐变动画过渡
    var tween = create_tween()
    tween.tween_method(_blend_weather, 0.0, 1.0, 2.0)
    weather_changed.emit(old, new_weather)
```

第91章 采集系统演化

Godot 采集系统：从点击即得到指定资源点

采集 = 从场景中获取资源 (挖矿、采药、砍树)。最早是"走到矿石旁边, 点击, 得到矿石"。后来有了采集动画、采集进度条、采集工具要求、资源刷新。采集系统的进化 = "怎么让采集本身有手感"。

1. 原始：点击即得

```
func _on_ore_body_entered(body):
    if body.is_in_group("player"):
        InventoryManager.add("iron_ore", 1)
        queue_free() # 矿石消失
```

2. 采集进度 + 打断

```
extends StaticBody2D

var is_gathering = false
var gather_time = 2.0
var progress = 0.0

func interact():
    if is_gathering: return
    is_gathering = true
    progress = 0.0
    $AnimationPlayer.play("gather")
    $ProgressBar.show()

func _process(delta):
```

```

    if not is_gathering: return
    progress += delta
    $ProgressBar.value = progress / gather_time
    if progress >= gather_time:
        _finish()

func _finish():
    InventoryManager.add(resource_id, 1)
    is_gathering = false
    $ProgressBar.hide()
    $AnimationPlayer.stop()
    # 进入刷新计时
    hide()
    await get_tree().create_timer(respawn_time).timeout
    show()

func cancel():
    if is_gathering:
        is_gathering = false
        $ProgressBar.hide()
        $AnimationPlayer.stop()

```

3. 工具要求 + 品质

```

class GatherNode:
    var resource_id: String
    var min_tool_level: int = 1      # 需要至少铁镐
    var required_tool_type: String  # "pickaxe" / "axe" /
"sickle"
    var min_count: int = 1
    var max_count: int = 3
    var respawn_time: float = 60.0

func can_gather(player) -> bool:
    var tool = player.get_equipped_tool()
    if not tool: return false
    if tool.type != required_tool_type: return false
    return tool.level >= min_tool_level

func _finish():
    var count = randi_range(min_count, max_count)

```

```
if player_tool.quality == "rare":  
    count *= 2 # 高品质工具双倍产出  
InventoryManager.add(resource_id, count)
```

4. 最终方案模板

```
# GatheringNode.gd (挂载到场景资源点)  
extends StaticBody2D  
  
@export var resource_id: String = "iron_ore"  
@export var required_tool: String = "pickaxe"  
@export var gather_time: float = 2.0  
@export var respawn_time: float = 30.0  
  
# 玩家交互流程：  
# 1. 检测玩家是否有对应工具  
# 2. 播放采集动画 + 进度条  
# 3. 玩家不能移动（或减速）  
# 4. 完成 → 获得资源 → 开始刷新倒计时  
# 5. 中途移动 → 打断采集
```

第92章 宠物系统演化

Godot 宠物系统：从跟屁虫到独立战斗单位

最早的游戏里宠物只是"跟着玩家走的模型"。后来它能捡东西、能战斗、能有自己的装备、能进化。宠物系统的进化 = "怎么让一个跟随物变得有价值 and 情感连接"。

1. 原始：视觉跟随

```
func _process(delta):  
    position = player.position + Vector2(-30, 0) # 固定在玩家左边
```

2. 平滑跟随

```
func _process(delta):  
    var t = 1.0 - exp(-3.0 * delta)  
    position = position.lerp(player.position + offset, t)
```

3. 独立 AI 跟随 (不穿墙)

```
@onready var nav_agent = $NavigationAgent2D  
  
func _physics_process(delta):  
    nav_agent.target_position = player.global_position  
    if nav_agent.is_navigation_finished():  
        return  
    var next = nav_agent.get_next_path_position()
```



```
velocity = global_position.direction_to(next) * speed  
move_and_slide()
```

4. 宠物背包 + 属性

```
class Pet:  
    var name: String  
    var level: int  
    var exp: int  
    var attack: int  
    var defense: int  
    var inventory: Array # 宠物背包（可设置自动拾取）  
    var auto_pickup: bool = true
```

5. 最终方案模板

```
# Pet.gd  
extends CharacterBody2D  
  
@export var follow_distance: float = 50.0  
var target: Node2D  
var nav_agent: NavigationAgent2D  
  
func _physics_process(delta):  
    if not target:  
        return  
    var dist =  
global_position.distance_to(target.global_position)  
    if dist > follow_distance:  
        nav_agent.target_position = target.global_position  
        var next = nav_agent.get_next_path_position()  
        velocity = global_position.direction_to(next) * speed  
        move_and_slide()  
    else:  
        velocity = Vector2.ZERO  
        move_and_slide()
```

第93章 坐骑系统演化

Godot 坐骑系统：从换皮到独立移动模式

坐骑 = 跑得更快的你。最早坐骑只是"把玩家速度乘以 2"。后来坐骑有自己的模型、自己的移动方式(飞行/游泳)、自己的技能、自己的获取任务。坐骑系统的进化 = "怎么让赶路这件事变得有趣"。

1. 原始：加速模式

```
var is_mounted = false

func _physics_process(delta):
    var speed = 200 if is_mounted else 100
    velocity = direction * speed
    move_and_slide()

func mount():
    is_mounted = true
    $Sprite.texture = mount_texture # 换图

func dismount():
    is_mounted = false
    $Sprite.texture = player_texture
```

2. 独立坐骑场景

```
# 坐骑不是一个"状态", 是一个独立的场景
func mount():
    var mount_scene = preload("res://mounts/
Horse.tscn").instantiate()
```

```
mount_scene.initialize(self)    # 坐骑带着玩家
add_child(mount_scene)
hide()                          # 玩家隐藏
$AnimationPlayer.play("mount")
```

坐骑控制玩家而不是玩家控制自己:

```
# Horse.gd
extends CharacterBody2D

var rider: Node2D

func initialize(player):
    rider = player
    rider.hide()
    add_child(rider)
    rider.position = Vector2(0, -20) # 坐在马背上

func _physics_process(delta):
    var dir = Input.get_vector("left", "right", "up", "down")
    velocity = dir * mount_speed
    move_and_slide()

func dismount():
    rider.show()
    rider.global_position = global_position + Vector2(50, 0)
    rider.get_parent().add_child(rider) # 放回场景
    queue_free()
```

3. 坐骑技能 (冲刺/二段跳/飞行)

```
class MountSkill:
    var name: String
    var cooldown: float
    var is_flying: bool = false
```

4. 最终方案模板

- # 最简单的坐骑：
- # 1. 独立场景，挂在玩家父节点下
- # 2. 坐骑控制移动，玩家坐在上面
- # 3. 按 X 下马，坐骑销毁，玩家恢复

第94章 载具系统演化

Godot 载具系统：从坐骑到可驾驶机械

载具 = 可驾驶的交通工具 (车、船、飞机、机甲)。坐骑是跟随玩家移动，载具是玩家进入后控制。最早的载具就是"玩家移速变快"。后来有了独立物理、驾驶视角、载具武器、乘客位。载具系统的进化 = "怎么让开车和走路是两种体验"。

1. 原始：速度 Buff

```
func enter_vehicle():
    speed *= 2
    is_in_vehicle = true

func exit_vehicle():
    speed /= 2
    is_in_vehicle = false
```

2. 独立场景 + 驾驶控制

```
# Vehicle.gd
extends CharacterBody2D

@export var drive_speed := 400
@export var rotation_speed := 2.0
@export var acceleration := 500
@export var friction := 300

var current_speed := 0.0
var driver: Node2D = null
```

```

func enter(driver_node: Node2D):
    driver = driver_node
    driver.visible = false
    driver.global_position = $Seat.global_position
    driver.get_parent().remove_child(driver)
    add_child(driver)

func exit() -> Node2D:
    var p = driver
    remove_child(driver)
    get_tree().current_scene.add_child(driver)
    driver.global_position = global_position + Vector2(0, 50)
    driver.visible = true
    driver = null
    return p

func _physics_process(delta):
    if not driver: return
    var dir = Input.get_axis("left", "right")
    rotation += dir * rotation_speed * delta

    if Input.is_action_pressed("up"):
        current_speed = move_toward(current_speed, drive_speed,
acceleration * delta)
    elif Input.is_action_pressed("down"):
        current_speed = move_toward(current_speed, -drive_speed
* 0.5, acceleration * delta)
    else:
        current_speed = move_toward(current_speed, 0, friction *
delta)

    velocity = Vector2.RIGHT.rotated(rotation) * current_speed
    move_and_slide()

```

3. 载具类型

```

class VehicleConfig:
    var type: String          # "car" / "boat" / "plane" /
"mech"
    var scene: PackedScene

```



```

var speed: float
var handling: float      # 转向灵敏度
var hp: int
var passenger_count: int # 乘客位数量
var has_weapons: bool
var can_fly: bool

```

不同类型载具的物理差异:

```

func _physics_process(delta):
    match type:
        "car":
            # 地面摩擦, 不能横向移动
            pass
        "boat":
            # 浮力, 水流影响
            apply_force(_water_force())
        "plane":
            # 升力, 俯仰控制
            gravity = 0
            apply_force(Vector2.UP * lift)

```

4. 最终方案模板

```

# VehicleManager.gd (Autoload)
# 载具核心:
# 1. 进入: 角色隐藏, 附加到载具
# 2. 驾驶: 载具独立移动逻辑
# 3. 退出: 角色出现在载具旁边
# 4. 载具受伤害: 耐久度, 摧毁时弹出

```

```

func enter_vehicle(vehicle: Node2D, player: Node2D):
    vehicle.enter(player)

```

```

func exit_vehicle(vehicle: Node2D) -> Node2D:
    return vehicle.exit()

```

第95章 瓦片系统演化

Godot 瓦片系统: 从大图到 TileMap

瓦片 = 用小块拼出大地图。最早是一整张背景图, 改一点就得重画。后来有了 TileMap: 用小格子拼出关卡, 省内存、好修改、能复用。瓦片系统的进化 = "怎么用小图块高效构建大世界"。

1. 原始: 整张背景图

```
# 一张大图做背景
$Background.texture = preload("res://level1_bg.png")

# 问题: 改地图要改图, 不能复用, 占用内存大
```

2. Sprite 手摆

```
# 用多个 Sprite 拼地面
for x in range(20):
    for y in range(15):
        var tile = Sprite2D.new()
        tile.texture = preload("res://tile_grass.png")
        tile.position = Vector2(x * 32, y * 32)
        add_child(tile)

# 比整张图灵活, 但节点太多, 性能差
```

3. TileMap 基础

```
# TileMap 是一整层网格，一个节点管理所有瓦片
# 设置 TileSet 定义有哪些瓦片
# 用坐标画瓦片

func _ready():
    # 用 TileMap 画一个 10x10 的地面
    for x in range(10):
        for y in range(10):
            $TileMap.set_cell(0, Vector2i(x, y), 0, Vector2i(0,
0))
        # set_cell(layer, coords, source_id, atlas_coords)
```

优势：一个 TileMap 节点 = 之前数百个 Sprite 节点，渲染快，内存少。

4. 多层 TileMap

```
# 层 0：地面 (grass, dirt, water)
# 层 1：装饰 (flowers, rocks)
# 层 2：碰撞层 (walls, fences) – 用 StaticBody2D 碰撞

# 分层的意义：不同层可以有不同的碰撞、排序、滚动速度
func setup_layers():
    $GroundLayer.set_cells_from_array(...) # 地面
    $DecorationLayer.set_cells_from_array(...) # 装饰
    $CollisionLayer.set_cells_from_array(...) # 碰撞

# 在编辑器中配置 TileSet 的物理层：
# 选择某个瓦片 → 设置 Physics Layer → 画碰撞形状
```

5. 自动瓦片 / Terrain

```
# 地形瓦片：根据相邻瓦片自动选择合适的过渡
# 比如草地→沙漠的过渡：
```

```
# TileSet 中配置 Terrains:
# - terrain 0: 草地
# - terrain 1: 沙漠
# - terrain 2: 水

# 在地形集里配置匹配规则:
# 四周全是草地 → 草地中央
# 左边是沙漠 → 草地右过渡
# 左上右是草地，下是沙漠 → 草地右上过渡

func auto_place():
    # 用 Terrain 模式画图，TileMap 自动匹配
    $TileMap.set_cells_terrain_connect(0, cells, 0, 0)
    # 自动计算相邻关系并选择合适的瓦片
```

6. 随机瓦片 (Scatter)

```
# 同一位置多个变体，随机选择

# TileSet 配置 Random Tiles:
# - tile_grass_1 (权重 1)
# - tile_grass_2 (权重 1)
# - tile_grass_3 (权重 0.5, 稀有)

# TileMap 放瓦片时随机选一个
# 视觉上不重复
```

7. 动画瓦片

```
# 水/火焰/传送门 用动画瓦片
# TileSet 中为单个瓦片配置多帧动画

# Animation Frames: 4
# Frame Duration: 0.25s
# 不需要额外的 AnimationPlayer
```

8. 最终方案模板

瓦片系统最佳实践:

1. 一个 TileMap 节点 = 一类瓦片 (地面/装饰/碰撞)

2. 用 TileSet 的 Terrain 功能处理过渡

3. 物理碰撞直接在 TileSet 中画

4. 随机瓦片避免重复感

5. 动画瓦片处理动态元素

6. 大世界用分块加载, 不用一个 TileMap 撑满

从代码生成地图:

```
func generate_map(width: int, height: int):  
    var tilemap = TileMap.new()  
    tilemap.tile_set = preload("res://world.tres")  
    for x in width:  
        for y in height:  
            var t = _get_terrain_at(x, y)  
            tilemap.set_cell(0, Vector2i(x, y), 0, t)  
    add_child(tilemap)  
    # 生成碰撞  
    tilemap.add_child(_create_collision_from_tilemap(tilemap))
```

第96章 传送点系统演化

Godot 传送点系统: 从坐标到快速旅行

传送点 = 让玩家可以在已探索的位置之间快速移动。最早传送靠手动改坐标。后来有了锚点坐标、激活条件、传送列表、快捷传送。传送点的进化 = "怎么让跑图不浪费时间"。

1. 原始: 硬编码坐标

```
func teleport(x: float, y: float):  
    player.global_position = Vector2(x, y)
```

2. 锚点方式

```
# 场景中放置 TeleportPoint (Area2D)  
class TeleportPoint:  
    @export var point_id: String          # "village_square"  
    @export var point_name: String        # "村庄广场"  
    @export var target_scene: String      # "res://scenes/  
village.tscn"  
    @export var target_spawn: String      # "spawn_square"  
    @export var is_activated: bool = false  
  
# 玩家进入范围时激活  
func _on_body_entered(body):  
    if body.is_in_group("player"):  
        is_activated = true  
        TeleportSystem.activate(point_id)
```



```

# TeleportSystem.gd (Autoload)
extends Node

var activated_points := []

func activate(point_id: String):
    if not point_id in activated_points:
        activated_points.append(point_id)
        save_progress()

func teleport_to(point_id: String):
    var point = _find_point(point_id)
    if not point.is_activated: return # 未激活，不能传送
    get_tree().change_scene_to_file(point.target_scene)
    # 跨场景后设置玩家位置（需要 SceneManager 配合）

```

3. 传送列表 UI

```

# 打开地图按 M 显示已激活的传送点列表
class TeleportUI:
    var _points: Array[TeleportPoint]

    func show():
        _points = TeleportSystem.get_activated_points()
        # 按区域分组显示
        for point in _points:
            var btn = Button.new()
            btn.text = point.point_name
            btn.pressed.connect(_on_select.bind(point))
            $List.add_child(btn)

    func _on_select(point: TeleportPoint):
        # 消耗传送道具 / 金币
        if CurrencyManager.spend("gold", 100):
            TeleportSystem.teleport_to(point.point_id)

```

4. 最终方案模板

```
# TeleportPoint.gd (挂载到场景中的传送锚点)
extends Area2D

@export var id: String
@export var label: String
@export_enum("point", "scenic", "city", "dungeon") var type:
String
@export var locked: bool = false

func _ready():
    if SaveManager.get_teleport_state(id):
        locked = false

func _on_body_entered(body):
    if body.is_in_group("player"):
        SaveManager.unlock_teleport(id)
        locked = false
        Notification.show(tr("TELEPORT_UNLOCKED") % label)

# FastTravelPanel.gd
func _refresh():
    for child in $List.get_children(): child.queue_free()
    for point in TeleportSystem.get_unlocked():
        var btn = Button.new()
        btn.text = point.label
        btn.pressed.connect(_travel.bind(point.id))
        $List.add_child(btn)

func _travel(id: String):
    TeleportSystem.go(id) # change_scene + 设置出生点
```

第97章 大地图系统演化

Godot 大地图系统: 从一张图到分层世界

大地图 = 游戏世界的俯瞰图, 用于导航和快速旅行。最早就是游戏中铺一张大图。后来有了多层缩放、区域标记、迷雾探索、实时位置追踪。大地图的进化 = "怎么让玩家知道自己在哪里、要去哪里"。

1. 原始: 静态图片

```
# 整个地图是一张图片
# 玩家位置用一个小点表示
func show_map():
    $MapTexture.texture = preload("res://maps/world_map.png")
    $PlayerDot.position = player.global_position / world_scale
```

2. 分层缩放

```
class WorldMap:
    var zoom_level: int = 0
    var max_zoom: int = 4
    var map_layers: Dictionary = {
        0: preload("res://maps/world_overview.png"), # 整个世界
        1: preload("res://maps/region_map.png"),      # 区域
        2: preload("res://maps/local_area.png"),      # 本地
        3: preload("res://maps/town_map.png"),        # 城镇
        4: preload("res://maps/dungeon_map.png"),     # 副本
    }

    func zoom_in(pos: Vector2):
        zoom_level = min(zoom_level + 1, max_zoom)
```

```

        update()

func zoom_out():
    zoom_level = max(zoom_level - 1, 0)
    update()

```

3. 区域标记

```

class MapMarker:
    var id: String
    var position: Vector2
    var type: String # "town" / "dungeon" / "quest" /
    "player" / "friend"
    var label: String
    var discovered: bool = false
    var icon: Texture2D

func load_markers():
    for marker in MapDB.get_all():
        if not marker.discovered: continue
        var icon = Sprite2D.new()
        icon.texture = marker.icon
        icon.global_position = marker.position * map_scale
        $MarkerLayer.add_child(icon)

    if marker.type == "quest":
        # 任务标记闪烁
        var tween = create_tween().set_loops()
        tween.tween_property(icon, "modulate:a", 0.5, 0.5)
        tween.tween_property(icon, "modulate:a", 1.0, 0.5)

```

4. 最终方案模板

```

# WorldMapManager.gd
# 核心：地图渲染 + 缩放 + 标记 + 迷雾

var _texture_rect: TextureRect
var _markers: Node2D

```

```

var _fog: TileMap

func init(map_node: Control):
    _texture_rect = map_node.get_node("MapTexture")
    _markers = map_node.get_node("Markers")
    _fog = map_node.get_node("FogLayer")

func render_markers(markers: Array[MapMarker]):
    for child in _markers.get_children(): child.queue_free()
    for m in markers:
        if not m.discovered: continue
        var btn = TextureButton.new()
        btn.texture_normal = m.icon
        btn.position = m.position
        btn.pressed.connect(_on_marker_click.bind(m.id))
        _markers.add_child(btn)

func reveal_area(pos: Vector2, radius: int):
    # 清除迷雾 TileMap 中的对应区域
    var tiles = _fog.get_used_cells(0)
    for cell in tiles:
        if cell.distance_to(pos) < radius:
            _fog.set_cell(0, cell, -1) # 移除迷雾

```

第98章 游泳潜水系统演化

Godot 游泳/潜水系统：从走水路到水下世界

游泳/潜水 = 角色在水中时的特殊移动。最早角色碰到水就是"死"。后来有了浮力、游泳速度、氧气条、水下视野。游泳的进化 = "怎么让水成为可探索的区域而不是死亡边界"。

1. 原始：水 = 死亡

```
# WaterArea.gd
func _on_body_entered(body):
    if body.is_in_group("player"):
        body.die() # 碰水就死
```

2. 水面行走

```
# 角色碰到水只是变慢
func _on_body_entered(body):
    var player = body as Player
    if player:
        player.set_speed(player.base_speed * 0.3)
```

3. 游泳模式

```
# CharacterBody2D 检测水域：
func _ready():
    add_to_group("player")
    water_detector.body_entered.connect(_on_water_entered)
```



```

        water_detector.body_exited.connect(_on_water_exited)

var is_swimming := false
var oxygen := 100.0
var max_oxygen := 100.0

func _on_water_entered(area: Area2D):
    is_swimming = true
    gravity = 200          # 浮力 → 重力变小
    swim_speed = 150      # 比陆地慢
    if area.is_diving_zone:
        oxygen = max_oxygen
        is_diving = true

func _on_water_exited(_area):
    is_swimming = false
    gravity = 980          # 恢复重力
    is_diving = false

func _physics_process(delta):
    if is_swimming:
        # 游泳控制: 上下左右
        var dir = Vector2(
            Input.get_axis("left", "right"),
            Input.get_axis("up", "down"),
        )
        velocity = dir * swim_speed
        move_and_slide()

        # 水下氧气消耗
        if is_diving:
            oxygen -= 10 * delta
            if oxygen <= 0:
                take_damage(5) # 缺氧掉血

func _on_water_surface():
    # 浮出水面恢复氧气
    oxygen = min(oxygen + 20 * delta, max_oxygen)

```

4. 最终方案模板

```
# SwimmingComponent.gd
extends Node

@export var swim_speed := 150.0
@export var dive_speed := 100.0
@export var max_oxygen := 100.0
@export var oxygen_drain := 10.0
@export var oxygen_regen := 20.0

var is_swimming := false
var is_diving := false
var oxygen := 100.0

func enter_water():
    is_swimming = true
    body.gravity_scale = 0.2
    swim_speed = body.base_speed * 0.6

func exit_water():
    is_swimming = false
    is_diving = false
    body.gravity_scale = 1.0

func dive():
    if not is_swimming: return
    is_diving = true
    $SurfaceDetector.monitoring = false

func surface():
    is_diving = false
    $SurfaceDetector.monitoring = true

func _process(delta):
    if not is_swimming: return
    if is_diving:
        oxygen -= oxygen_drain * delta
        if oxygen <= 0:
            body.take_damage(5)
    else:
        oxygen = min(oxygen + oxygen_regen * delta, max_oxygen)
```

第99章 攀爬梯子系统演化

Godot 攀爬/梯子系统: 从跳跳乐到垂直探索

攀爬 = 角色在梯子/墙壁上垂直移动。最早梯子只是装饰, 或者用跳的上去。后来有了梯子碰撞、攀爬状态、壁挂、攀爬耐力。攀爬的进化 = "怎么让垂直移动像走路一样自然"。

1. 原始: 装饰

```
# 梯子是贴图, 不能交互  
# 玩家要用跳的上去
```

2. 梯子触发

```
# LadderArea.gd  
extends Area2D  
  
func _on_body_entered(body):  
    if body.is_in_group("player"):  
        body.set_ladder_mode(true, self)  
  
func _on_body_exited(body):  
    if body.is_in_group("player"):  
        body.set_ladder_mode(false, null)
```

```
# Player.gd  
var on_ladder := false  
var ladder_top: float  
  
func set_ladder_mode(on: bool, ladder):
```

```

on_ladder = on
if on:
    ladder_top = ladder.top_y
    gravity_scale = 0
    velocity = Vector2.ZERO
else:
    gravity_scale = 1

func _physics_process(delta):
    if on_ladder:
        var dir = Input.get_axis("up", "down")
        velocity.y = dir * climb_speed
        velocity.x = 0
        move_and_slide()

```

3. 攀爬耐力

```

var climb_stamina := 100.0
var max_climb_stamina := 100.0
var is_climbing := false

func start_climb():
    is_climbing = true
    gravity_scale = 0
    can_attack = false # 攀爬时不能攻击

func stop_climb():
    is_climbing = false
    gravity_scale = 1
    can_attack = true

func _process(delta):
    if is_climbing:
        climb_stamina -= stamina_drain * delta
        if climb_stamina <= 0:
            stop_climb() # 没体力掉下来
    else:
        climb_stamina = min(climb_stamina + stamina_regen *
delta, max_climb_stamina)

```

4. 墙面攀爬

```
# 不只是梯子，特定墙面也可以爬
func check_wall_climb():
    if not is_on_wall(): return
    if not Input.is_action_pressed("grab"): return

    var wall_normal = get_wall_normal()
    if abs(wall_normal.x) > 0.5: # 侧面墙
        start_climb()
        # 沿墙上下移动
        var dir = Input.get_axis("up", "down")
        velocity.y = dir * climb_speed
        velocity.x = 0
```

5. 最终方案模板

```
# ClimbComponent.gd
extends Node

@export var climb_speed := 100.0
@export var stamina_max := 100.0
@export var drain_rate := 15.0

var is_climbing := false
var stamina := 100.0

func enter():
    is_climbing = true
    body.gravity_scale = 0
    body.velocity.y = 0

func exit():
    is_climbing = false
    body.gravity_scale = 1

func _physics_process(delta):
    if not is_climbing: return
    var dir = Input.get_axis("up", "down")
```

```
body.velocity.y = dir * climb_speed
body.velocity.x = 0
body.move_and_slide()

stamina -= drain_rate * delta
if stamina <= 0:
    exit()
else:
    stamina = min(stamina + drain_rate * delta * 0.3,
stamina_max)
```

第100章 坠落伤害系统演化

Godot 坠落伤害系统：从跳不死到摔死

坠落伤害 = 从高处落下时受到的伤害。最早角色从任何高度落下都无伤。后来有了高度检测、速度检测、缓冲机制。坠落伤害的进化 = "怎么让地形高低差有意义"。

1. 原始：无坠落伤害

```
# 从多高跳下来都没事
```

2. 高度检测

```
var fall_start_y := 0.0
var was_in_air := false

func _process(delta):
    if not is_on_floor():
        if not was_in_air:
            fall_start_y = global_position.y
            was_in_air = true
        else:
            if was_in_air:
                var fall_distance = fall_start_y - global_position.y
                was_in_air = false
                if fall_distance > 500: # 5 米以上有伤害
                    var damage = int((fall_distance - 500) * 0.5)
                    take_damage(damage)
```

3. 速度检测 (更准确)

```
func _on_landed():  
    # 落地时的垂直速度决定伤害  
    var fall_speed = abs(velocity.y)  
    var damage_threshold = 500.0  
  
    if fall_speed > damage_threshold:  
        var damage = int((fall_speed - damage_threshold) * 0.1)  
        damage = min(damage, max_hp * 0.5) # 最多摔 50% HP  
        take_damage(damage)  
        ShakeManager.shake(0.3)  
  
    # 落地缓冲: 落在某些表面伤害减半  
    if _landed_on_soft_surface():  
        damage = int(damage * 0.5)
```

4. 缓冲机制

```
func _on_landed():  
    var fall_speed = abs(velocity.y)  
  
    # 翻滚缓冲  
    if Input.is_action_pressed("roll"):  
        fall_speed *= 0.3 # 翻滚减少坠落伤害  
  
    # 水面缓冲  
    if _is_over_water:  
        fall_speed *= 0.1 # 掉水里基本无伤  
  
    if fall_speed > FALL_THRESHOLD:  
        var damage = int((fall_speed - FALL_THRESHOLD) *  
DAMAGE_SCALE)  
        take_damage(damage)  
  
    # 摔伤后短暂减速  
    slow_effect(duration = 1.0, factor = 0.5)
```

5. 最终方案模板

```
# FallDamageComponent.gd
extends Node

@export var threshold := 600.0 # 超过此速度才受伤
@export var damage_scale := 0.08
@export var max_pct := 0.5 # 最多摔一半血

func on_landed(fall_speed: float, modifiers: Dictionary) -> int:
    if fall_speed <= threshold: return 0
    var dmg = int((fall_speed - threshold) * damage_scale)

    if modifiers.get("has_roll", false): dmg = int(dmg * 0.3)
    if modifiers.get("in_water", false): dmg = int(dmg * 0.1)
    if modifiers.get("soft_ground", false): dmg = int(dmg * 0.5)

    dmg = min(dmg, int(body.max_hp * max_pct))
    return dmg
```

第101章 种植畜牧系统演化

Godot 种植/畜牧系统: 从刷新到生长模拟

种植/畜牧 = 玩家种田养动物, 等待收获。最早资源点是定时刷新。后来有了种子/作物、生长阶段、浇水/施肥、动物喂养。种植的进化 = "怎么让等待收获的过程有参与感"。

1. 原始: 资源刷新

```
# 采集点到期自动刷新
func _ready():
    respawn_timer.start(respawn_time)

func _on_respawn():
    is_harvestable = true
    $Sprite.visible = true
```

2. 种植: 种子 → 作物

```
class Crop:
    var seed_id: String          # "carrot_seed"
    var growth_time: float       # 总生长时间
    var stages: Array[float]     # 每个阶段的时长占比 [0.3, 0.3, 0.4]
    var current_stage: int = 0
    var elapsed: float = 0.0
    var is_watered: bool = false
    var is_mature: bool = false

    func process(delta):
        elapsed += delta * (1.5 if is_watered else 1.0)
```

```

    var total = growth_time
    for i in stages.size():
        if elapsed < total * stages[i]:
            current_stage = i
            return
        total -= total * stages[i]
    current_stage = stages.size() - 1
    is_mature = true

func harvest() -> Dictionary:
    if not is_mature: return {}
    return { "item": "carrot", "quantity": randf_range(1,
3) }

```

3. 畜牧

```

class Livestock:
    var type: String          # "chicken" / "cow" / "sheep"
    var growth_time: float    # 幼崽 → 成年
    var produce_interval: float # 产蛋/产奶间隔
    var feed_required: float   # 每次喂食量
    var hunger: float = 0.0    # 饱腹度
    var age: float = 0.0
    var is_adult: bool = false
    var next_produce: float = 0.0

    func feed(amount: float):
        hunger = min(hunger + amount, max_hunger)

    func process(delta):
        if not is_adult:
            age += delta
            if age >= growth_time:
                is_adult = true

        if is_adult:
            hunger -= delta * hunger_rate
            if hunger <= 0: return # 饿肚子不生产

        next_produce -= delta * (1.0 if hunger > max_hunger
* 0.5 else 0.5)

```

```

        if next_produce <= 0:
            _produce()
            next_produce = produce_interval

func _produce():
    match type:
        "chicken": drop_item("egg", 1)
        "cow":     drop_item("milk", 1)
        "sheep":   drop_item("wool", 1)

```

4. 最终方案模板

```

# FarmingManager.gd
# 核心：地块 + 作物/动物 + 生长模拟

class FarmPlot:
    var crop: Crop = null
    var position: Vector2i

    def plant(seed_id: String):
        crop = Crop.new(seed_id)
        update_visual()

    def water():
        if crop: crop.is_watered = true

    def harvest() -> Dictionary:
        if not crop or not crop.is_mature: return {}
        var result = crop.harvest()
        crop = null
        update_visual()
        return result

    func update_visual():
        if not crop:
            $Sprite.texture = empty_plot
        else:
            $Sprite.texture = crop.get_stage_texture()

func _process(delta):
    for plot in plots:

```

```
if plot.crop:  
    plot.crop.process(delta)  
    plot.update_visual()
```


第102章 光源系统演化

Godot 光源系统：从全亮到动态光影

光源 = 游戏世界中的照明。最早整个屏幕都是亮的。后来有了点光源、聚光、阴影、昼夜照明变化。光源的进化 = "怎么用光照营造氛围"。

1. 原始：全亮

```
# 没有光照系统
# 所有物体颜色固定
```

2. PointLight2D

```
# 场景中放置 PointLight2D
# CanvasModulate 控制环境光

# CanvasModulate:
#   - 白天: Color.WHITE
#   - 夜晚: Color.DIM_GRAY

# PointLight2D:
#   - 火把: 橙色, 半径 300
#   - 魔法光: 蓝色, 半径 500

# LightOccluder2D:
#   - 放在墙壁上产生阴影
```

3. 昼夜联动

```
func _process(delta):
    time = fmod(time + delta, cycle_duration)
    var pct = time / cycle_duration

    var color: Color
    if pct < 0.25: color = Color.LIGHT_SKY_BLUE   # 白天
    elif pct < 0.35: color = Color.ORANGE        # 黄昏
    elif pct < 0.65: color = Color.DIM_GRAY      # 夜晚
    elif pct < 0.75: color = Color.ORANGE        # 黎明
    else: color = Color.LIGHT_SKY_BLUE

    $CanvasModulate.color = color

    for light in get_tree().get_nodes_in_group("night_lights"):
        light.energy = 1.0 if (pct > 0.3 and pct < 0.7) else 0.0
```

4. 最终方案模板

```
# LightingManager.gd (Autoload)
func set_ambient(color: Color):
    var m = get_tree().root.get_node("CanvasModulate")
    if m: m.color = color

func create_torch(pos: Vector2):
    var light = PointLight2D.new()
    light.texture = preload("res://lighting/torch.tres")
    light.color = Color.ORANGE
    light.energy = 1.0
    light.position = pos
    light.shadow_enabled = true
    get_tree().current_scene.add_child(light)

    var tween = create_tween().set_loops()
    tween.tween_property(light, "energy", 0.8, 0.1)
    tween.tween_property(light, "energy", 1.2, 0.15)
```

第六卷·架构与数据

共计 12 章

第103章 对象池演化

Godot 对象池: 从创建销毁到复用

每次 `instantiate()` + `queue_free()` 都是一次内存分配和释放。100 颗子弹每帧创建销毁 → GC 卡顿 → 帧率掉。对象池就是"别扔, 留着再用"。

目录

1. [原始: 随用随建, 用完就扔](#)
 2. [第一代: 手写数组池](#)
 3. [加速预热: 提前创建](#)
 4. [通用池: 一个池管所有类型](#)
 5. [自动扩展: 不够了自动加](#)
 6. [多类型池管理](#)
 7. [对象池的复位问题](#)
 8. [最终方案: 最简可用模板](#)
-

1. 原始: 随用随建, 用完就扔

问题: "玩家按射击, 生成一颗子弹, 撞到墙后消失。"

最直接的做法:

```
# 开枪时创建
func shoot():
```

```

var bullet = bullet_scene.instantiate()
bullet.position = $Muzzle.global_position
bullet.direction = aim_direction
add_child(bullet)

# 子弹撞墙后自毁
# Bullet.gd
func _on_hit_wall():
    queue_free()

```

这有什么问题? 单发测试没问题。但:

连续射击:

创建子弹 → 分配内存 → 添加到场景树
 子弹飞出 → 撞墙 → 释放内存 → 从场景树移除
 创建子弹 → 分配内存 → ...

每颗子弹:

一次内存分配 (instantiate)
 一次场景树插入 (add_child)
 一次场景树移除 (queue_free)
 一次内存释放 (free)

60 发/秒 × 4 次操作 = 240 次内存操作/秒

当子弹数量多时:

霰弹枪 1 发 → 8 颗子弹
 冲锋枪 10 发/秒 → 80 颗子弹/秒
 塔防 20 座炮塔 × 2 发/秒 → 40 颗子弹/秒
 弹幕 Boss → 500+ 颗子弹同时存在

内存反复分配/释放导致: - **GC 压力** (GDScript 有引用计数, 但场景树操作不轻)
 - **帧率抖动** (instantiate 有时会卡) - **CPU 缓存不友好** (新对象的内存地址不连续)

改进目标: 不创建也不销毁, 用完了藏起来, 下次拿出来再用。

2. 第一代: 手写数组池

想法: 用一个数组存"暂时不用的对象"。需要用的时候从数组取, 用完了放回去。

```
# BulletPool.gd
extends Node

var bullet_scene = preload("res://Bullet.tscn")
var pool := []

func get_bullet() -> Node:
    if pool.size() > 0:
        # 从池里拿一个
        var bullet = pool.pop_back()
        return bullet
    else:
        # 池空了, 新建一个
        return bullet_scene.instantiate()

func return_bullet(bullet: Node):
    # 重置并放回池
    bullet.hide()
    bullet.set_process(false)
    bullet.set_physics_process(false)
    pool.append(bullet)
```

```
# 使用:
var bullet = bullet_pool.get_bullet()
bullet.position = $Muzzle.global_position
bullet.direction = aim_direction
bullet.show()
bullet.set_process(true)
bullet.set_physics_process(true)
add_child(bullet)

# 子弹用完时:
# Bullet.gd
func _on_hit_wall():
    bullet_pool.return_bullet(self)
    # 注意: 不是 queue_free()
```


进步: 不再反复 instantiate/queue_free, 复用已有对象。

但引入新问题:

1. **需要手动管理父子关系** – 从池里取出来时要 add_child, 放回去时要不要 remove_child? 如果还挂在场景树上, 隐藏状态下的对象还在做碰撞检测。
2. **复位不完整** – 子弹的 position, rotation, velocity 可能还保留着上一发的值。忘记复位 → 奇怪的 Bug。
3. **Node 臃肿** – 如果池里的对象一直挂在场景树的某个隐藏节点下, 每帧虽然不渲染, 但 _process 如果没关, 还在跑代码。

改进: 增加复位和挂载点:

```
# BulletPool 改进版

var pool := []
var holder: Node # 隐藏的挂载点

func _ready():
    holder = Node.new()
    holder.name = "BulletPoolHolder"
    add_child(holder)

func get_bullet() -> Node:
    var bullet: Node
    if pool.size() > 0:
        bullet = pool.pop_back()
        holder.remove_child(bullet) # 从隐藏挂载点移除
    else:
        bullet = bullet_scene.instantiate()
    # 复位
    _reset_bullet(bullet)
    return bullet

func return_bullet(bullet: Node):
    bullet.get_parent().remove_child(bullet) # 从场景移除
    holder.add_child(bullet) # 放到隐藏挂载点
    bullet.hide()
    bullet.set_process(false)
    bullet.set_physics_process(false)
```

```

        pool.append(bullet)

func _reset_bullet(bullet: Node):
    bullet.position = Vector2.ZERO
    bullet.rotation = 0.0
    bullet.velocity = Vector2.ZERO
    bullet.show()
    bullet.set_process(true)
    bullet.set_physics_process(true)

```

3. 预热：提前创建

新问题：第一批发子弹时，池子是空的，还是要 instantiate。这导致第一批子弹发射时依然卡顿。

预热：游戏加载时，预先创建好一批对象放池里。

```

# BulletPool.gd

func _ready():
    holder = Node.new()
    holder.name = "BulletPoolHolder"
    add_child(holder)
    _warm_up(30) # 提前创建 30 颗子弹

func _warm_up(count: int):
    for i in range(count):
        var bullet = bullet_scene.instantiate()
        bullet.hide()
        bullet.set_process(false)
        bullet.set_physics_process(false)
        holder.add_child(bullet)
        pool.append(bullet)

```

预热数量怎么定？

```

# 预估最大同时活跃数量
# 冲锋枪：10 发/秒 × 2 秒飞行寿命 = 20 颗
# 预热 20 颗就够了

```

```
# 多预留 30% 防意外
```

```
var max_bullets = 20  
var warmup_count = ceili(max_bullets * 1.3)
```

效果: 预热完后, 游戏中的 get_bullet 永远不会 instantiate, 全部从池里拿。零分配, 零卡顿。

问题: 每个池都要写一套 get/return/warmup, 不同子弹类型要写不同的池, 重复代码太多。

4. 通用池: 一个脚本管所有

问题: 子弹要池, 敌人要池, 特效要池, 掉落物要池。每个类型写一个池? 代码爆炸。

泛型对象池:

```
# ObjectPool.gd  
# 通用对象池, 支持任何场景  
  
class_name ObjectPool  
extends Node  
  
@export var scene: PackedScene # 在编辑器里拖入场景  
@export var prewarm_count: int = 0  
@export var max_size: int = 100  
  
var _pool := []  
var _holder: Node  
  
func _ready():  
    _holder = Node.new()  
    _holder.name = "Pool_%s" % scene.resource_path.get_file()  
    add_child(_holder)  
    for i in range(prewarm_count):  
        var obj = _create()  
        _return_to_pool(obj)
```

```

func get() -> Node:
    var obj: Node
    if _pool.size() > 0:
        obj = _pool.pop_back()
        _holder.remove_child(obj)
    else:
        obj = _create()
    obj.show()
    obj.set_process(true)
    obj.set_physics_process(true)
    return obj

func return_to_pool(obj: Node):
    if _pool.size() >= max_size:
        obj.queue_free() # 超过上限就真的销毁
        return
    if obj.get_parent():
        obj.get_parent().remove_child(obj)
    _holder.add_child(obj)
    obj.hide()
    obj.set_process(false)
    obj.set_physics_process(false)
    _pool.append(obj)

func _create() -> Node:
    var obj = scene.instantiate()
    _call_reset(obj)
    return obj

func _call_reset(obj: Node):
    if obj.has_method("reset_for_pool"):
        obj.reset_for_pool()

```

池中对象只要实现 reset_for_pool 方法即可

使用：在编辑器里添加 ObjectPool 节点
拖入 Bullet.tscn, 设 prewarm = 20

取：

```

var bullet = $BulletPool.get()
add_child(bullet)
bullet.global_position = muzzle.global_position

```

```

bullet.direction = aim_dir

# 还:
# Bullet.gd
func _on_hit_wall():
    $/root/Game/BulletPool.return_to_pool(self)

# 或更优雅: 用信号
signal hit_wall
# 在 ObjectPool 里连接

```

但子弹的复位在哪里做?

5. 自动扩展: 不够了自动加

问题: 预热 20 颗, 但如果玩家疯狂射击, 同时活跃超过 20 颗, get_bullet 时池为空, 还是要 instantiate。

自动扩展: 池空时自动创建新对象, 同时发出警告。

```

func get() -> Node:
    var obj: Node
    if _pool.size() > 0:
        obj = _pool.pop_back()
        _holder.remove_child(obj)
    else:
        obj = _create()
        if _pool.size() + get_active_count() > prewarm_count *
2:
            push_warning("ObjectPool %s 频繁超限, 考虑增加预热量" %
scene.resource_path)
        obj.show()
        obj.set_process(true)
        obj.set_physics_process(true)
        return obj

func get_active_count() -> int:
    # 粗略估计活跃数量
    return _holder.get_child_count()

```

何时该增大预热量:

频繁看到 warning → 预热量不够 → 增大 `prewarm_count`
偶尔 warning → 可以接受, 自动扩展了
从不 warning → 预热量过大 → 减小 (省内存)

6. 多类型池管理

问题: 子弹、敌人、特效各一个 `ObjectPool` 节点。场景树上挂了一堆池。

```
# 用 PoolManager 统一管理

class_name PoolManager
extends Node

var _pools := {}

func register_pool(scene: PackedScene, prewarm: int = 0) -> ObjectPool:
    var pool = ObjectPool.new()
    pool.scene = scene
    pool.prewarm_count = prewarm
    add_child(pool)
    pool._ready() # 手动初始化预热
    _pools[scene.resource_path] = pool
    return pool

func get_pool(scene: PackedScene) -> ObjectPool:
    var path = scene.resource_path
    if not _pools.has(path):
        register_pool(scene)
    return _pools[path]

func spawn(scene: PackedScene) -> Node:
    return get_pool(scene).get()

func despawn(obj: Node, scene: PackedScene):
    get_pool(scene).return_to_pool(obj)
```

```

# 使用:
# PoolManager 设为 Autoload

var bullet = PoolManager.spawn(bullet_scene)
add_child(bullet)
bullet.global_position = pos

# 回收:
PoolManager.despawn(self, bullet_scene)

```

原理: 每个场景文件绑定一个池, 自动创建, 自动管理。

7. 对象池的复位问题

对象池最容易被忽视的 Bug: 借出去的对象还回来时, 没有"洗干净"。

```

# 子弹射击 → 撞墙 → 还回池 → 下次射击

# 如果还回来时:
#   bullet.position = (500, 300) # 上次撞墙的位置
#   bullet.rotation = 1.57       # 上次的朝向
#   bullet.direction = (0.7, 0.7) # 上次的方向
#   bullet.modulate = Color.RED  # 上次中了火焰效果

# 下次取出来:
#   position 没复位 → 子弹从 (500, 300) 出发, 而不是枪口
#   direction 没复位 → 子弹往奇怪的方向飞

```

解决方案: reset_for_pool 契约:

```

# 每个池化对象必须实现 reset_for_pool
# 在 "从池取出" 时调用

# Bullet.gd
func reset_for_pool():
    position = Vector2.ZERO
    rotation = 0.0
    velocity = Vector2.ZERO
    modulate = Color.WHITE

```

```

visible = true
scale = Vector2.ONE
# 关掉所有临时状态
is_on_fire = false
trail_emitter.emitting = false
hitbox.monitoring = false # 碰撞检测关闭

```

复位完整清单:

```

Transform (position/rotation/scale)    → 初始值
Visual (modulate/visible/self_modulate) → 默认值
Physics (velocity/collision mask)      → 默认值
状态变量 (自定义属性)                 → 初始值
子节点 (粒子/动画/计时器)             → 停止并重置
信号连接                               → 断开? (小心重复连接)

```

常见陷阱: 如果池化对象连接了外部信号, 还回池时没断开, 下次取出来时会重复连接。

```

func reset_for_pool():
    # 断开信号 (如果之前 connect 过)
    if hit_wall.is_connected(_on_hit_wall):
        hit_wall.disconnect(_on_hit_wall)

```

8. 最终方案: 最简可用模板

```

# 对象池模板: 复制即用

# —— ObjectPool.gd ——
# 挂载为场景的子节点, 或通过 PoolManager 自动管理

class_name ObjectPool
extends Node

@export var scene: PackedScene
@export var prewarm: int = 10
@export var max_pool: int = 50

```



```

var _pool: Array[Node] = []
var _holder: Node

func _ready():
    _holder = Node.new()
    _holder.name = "Holder"
    add_child(_holder)
    for i in range(prewarm):
        var obj = scene.instantiate()
        obj.reset_for_pool()
        _holder.add_child(obj)
        obj.hide()
        _pool.append(obj)

func get() -> Node:
    if _pool.is_empty():
        return scene.instantiate()
    var obj = _pool.pop_back()
    _holder.remove_child(obj)
    obj.reset_for_pool()
    obj.show()
    obj.set_process(true)
    obj.set_physics_process(true)
    return obj

func recycle(obj: Node):
    if _pool.size() >= max_pool:
        obj.queue_free()
        return
    if obj.get_parent():
        obj.get_parent().remove_child(obj)
    obj.hide()
    obj.set_process(false)
    obj.set_physics_process(false)
    _holder.add_child(obj)
    _pool.append(obj)

# —— 池化对象必须实现： ——

# Bullet.gd / Enemy.gd / Effect.gd
func reset_for_pool():
    position = Vector2.ZERO
    rotation = 0.0

```

```

    velocity = Vector2.ZERO
    modulate = Color.WHITE
    scale = Vector2.ONE
    # 恢复所有自定义属性到初始值

# —— 使用示例 ——

# 射击:
var b = $BulletPool.get()
add_child(b)
b.global_position = $Muzzle.global_position
b.direction = aim_dir

# 回收 (在子弹脚本里):
func _on_hit():
    $BulletPool.recycle(self)

```

这个故事告诉我们: - 对象池 = 不销毁, 藏起来, 下次用 - 预热 = 把卡顿提前到加载阶段, 而不是游玩阶段 - 复位 = 借出去的东西还回来时要恢复原样 - 通用池 = 一个脚本管所有类型场景 - 最容易被忘记的是复位, 最容易出 Bug 的也是复位

第104章 存档系统演化

Godot 存档系统: 从全局变量到序列化

存档 = 把游戏当前的状态"拍个照片", 下次打开时恢复。最早的游戏有密码续关 (输入一串字符回到上一关)。后来有了文件存档、云存档、自动存档。每个进化都在解决"怎么让玩家的进度不丢"。

1. 原始: 不存档

```
# 关掉游戏 = 一切重来  
# 没有存档能力
```

2. 全局变量硬存

```
func save_game():  
    var file = FileAccess.open("user://save.dat",  
FileAccess.WRITE)  
    file.store_var(level)  
    file.store_var(gold)  
    file.store_var(hp)  
    file.close()
```

问题: 所有变量手写。加了新变量忘了加 save/load → 读档数据丢失。

3. 字典存档

```
func save() -> Dictionary:
    return {
        "level": level,
        "gold": gold,
        "hp": hp,
        "position": [position.x, position.y],
        "inventory": _serialize_inventory(),
        "quests": _serialize_quests(),
    }

func load(data: Dictionary):
    level = data.get("level", 1)
    gold = data.get("gold", 0)
    hp = data.get("hp", 100)
    position = Vector2(data["position"][0], data["position"][1])
    _deserialize_inventory(data.get("inventory", []))
    _deserialize_quests(data.get("quests", []))
```

自动序列化: 每个 Manager 提供自己的 save/load 方法。

```
# Autoload 里汇总所有 Manager 的存档:
func save_game():
    var data = {
        "player": PlayerManager.save(),
        "inventory": InventoryManager.save(),
        "quests": QuestManager.save(),
        "game_state": GameState.save(),
        "timestamp": Time.get_unix_time_from_system(),
    }
    var json = JSON.stringify(data)
    var file = FileAccess.open("user://save.json",
FileAccess.WRITE)
    file.store_string(json)
    file.close()
```

4. 多存档槽

```
func save_slot(slot: int):
    var path = "user://save_%d.json" % slot
    # 同上，保存到不同文件

func load_slot(slot: int):
    var path = "user://save_%d.json" % slot
    var file = FileAccess.open(path, FileAccess.READ)
    if file:
        var json = file.get_as_text()
        var data = JSON.parse_string(json)
        _apply(data)

func get_save_preview(slot: int) -> Dictionary:
    # 读取存档文件名、等级、时间，用于显示
```

5. 自动存档

```
# 切场景时自动存
func _on_scene_changed():
    if GameState.current_level > 0:
        save_slot(0) # 自动存档在 slot 0

# 定期存档
func _ready():
    var timer = Timer.new()
    timer.timeout.connect(func(): save_slot(0))
    timer.start(300) # 每 5 分钟自动存
    add_child(timer)
```

6. 最终方案模板

```
# SaveManager.gd (Autoload)
extends Node

const SLOT_COUNT = 5
```

```

func save(slot: int):
    var data = {
        "level": GameState.level,
        "gold": CurrencyManager.get_amount("gold"),
        "position": [Player.position.x, Player.position.y],
        "inventory": InventoryManager.save(),
        "timestamp": Time.get_unix_time_from_system(),
    }
    var path = "user://slot_%d.save" % slot
    var file = FileAccess.open(path, FileAccess.WRITE)
    file.store_string(JSON.stringify(data))
    file.close()

func load(slot: int) -> bool:
    var path = "user://slot_%d.save" % slot
    if not FileAccess.file_exists(path):
        return false
    var file = FileAccess.open(path, FileAccess.READ)
    var data = JSON.parse_string(file.get_as_text())
    GameState.level = data.level
    InventoryManager.load(data.inventory)
    Player.position = Vector2(data.position[0],
data.position[1])
    return true

func delete(slot: int):
    DirAccess.remove_absolute("user://slot_%d.save" % slot)

```

第105章 设置系统演化

Godot 设置系统: 从硬编码到可配置

设置 = 音量、分辨率、语言、按键。最早写在代码最上面, 改设置在代码里改。后来有了设置界面、保存设置到文件、云同步。设置系统的进化 = "怎么让每个玩家都能调出适合自己的体验"。

1. 原始: 代码常量

```
const MASTER_VOLUME = 0.8
const SFX_VOLUME = 1.0
const SCREEN_WIDTH = 1920
const SCREEN_HEIGHT = 1080
```

2. 全局变量 (游戏内可改, 但不存档)

```
var settings = {
    "master_volume": 0.8,
    "sfx_volume": 1.0,
    "bgm_volume": 0.7,
    "fullscreen": false,
    "language": "zh",
}
```

3. 设置界面 + 存档

```
# SettingsManager.gd (Autoload)
extends Node
```

```

var data := {
    "master_volume": 0.8,
    "sfx_volume": 1.0,
    "bgm_volume": 0.7,
    "fullscreen": false,
    "language": "zh",
    "camera_sensitivity": 0.5,
    "show_damage_numbers": true,
}

func _ready():
    load_settings()

func set(key: String, value):
    data[key] = value
    _apply(key, value)
    save_settings()

func get(key: String):
    return data.get(key)

func _apply(key: String, value):
    match key:
        "master_volume":
            AudioServer.set_bus_volume_db(0,
linear_to_db(value))
        "sfx_volume":
            AudioServer.set_bus_volume_db(
                AudioServer.get_bus_index("SFX"),
linear_to_db(value))
        "fullscreen":
            if value:
                DisplayServer.window_set_mode(DisplayServer.WINDOW_MODE_FULLSCREEN)
            else:
                DisplayServer.window_set_mode(DisplayServer.WINDOW_MODE_WINDOWED)
        "language":
            TranslationServer.set_locale(value)

func save_settings():

```

```

    var file = FileAccess.open("user://settings.json",
FileAccess.WRITE)
    file.store_string(JSON.stringify(data))
    file.close()

func load_settings():
    if not FileAccess.file_exists("user://settings.json"):
        return
    var file = FileAccess.open("user://settings.json",
FileAccess.READ)
    var loaded = JSON.parse_string(file.get_as_text())
    if loaded:
        for key in loaded:
            if data.has(key):
                data[key] = loaded[key]
        for key in data:
            _apply(key, data[key])

```

4. 最终方案模板

```

# 使用:
# SettingsManager.set("master_volume", 0.5)
# SettingsManager.get("camera_sensitivity")

# 设置 UI:
# 滑块 → SettingsManager.set("master_volume", value)
# 开关 → SettingsManager.set("fullscreen", value)
# 下拉 → SettingsManager.set("language", value)

```

第106章 联网系统演化

Godot 联网系统：从本地到多人联机

联网 = 让多个玩家在同一个世界里互动。最早是本地同屏（一台电脑两个手柄）。后来有了局域网、专用服务器、P2P、房间匹配。联网系统的进化 = "怎么让千里之外的朋友一起玩"。

1. 原始：本地同屏

```
# 两个玩家在同一台电脑，用不同键盘按键
# Player1: WASD
# Player2: 方向键
# 不需要网络，节点都在同一个场景树里
```

2. 局域网

```
# Godot 内置 ENet 多人
# 一个玩家开服务器，其他玩家连接 IP

# Server:
func _ready():
    var peer = ENetMultiplayerPeer.new()
    peer.create_server(8888, 4)
    multiplayer.multiplayer_peer = peer

# Client:
func connect_to_server(ip: String):
    var peer = ENetMultiplayerPeer.new()
    peer.create_client(ip, 8888)
    multiplayer.multiplayer_peer = peer
```

3. RPC 同步

```
# 自动同步位置
@rpc("unreliable", "any_peer")
func sync_position(pos: Vector2):
    position = pos

func _process(delta):
    if is_multiplayer_authority():
        position += velocity * delta
        sync_position.rpc(position)

# 服务器验证
@rpc("authority")
func deal_damage(target_id: int, amount: int):
    # 只有服务器可以扣血，防止外挂
    var target = instance_from_id(target_id)
    target.hp -= amount
```

4. 最终方案模板

```
# 最简单的联机：
# 1. ENetMultiplayerPeer 创建连接
# 2. @rpc 注解标记需要同步的函数
# 3. is_multiplayer_authority() 区分谁控制
# 4. rpc() / rpc_id() 发送消息

# 关键包：
# multiplayer_peer = ENetMultiplayerPeer.new()
# multiplayer.multiplayer_peer = peer
# multiplayer.peer_connected.connect(_on_player_join)
# multiplayer.peer_disconnected.connect(_on_player_leave)
```

第107章 聊天系统演化

Godot 聊天系统: 从 print 到实时消息

聊天 = 玩家之间发消息。最早是本地同屏, 扭头就能说话。后来有了聊天框、频道 (世界/公会/私聊)、表情、敏感词过滤。聊天系统的进化 = "怎么让玩家之间高效沟通"。

1. 原始: 无聊天

单机游戏, 不需要聊天

2. 聊天框

```
# ChatManager.gd
extends Control

@onready var chat_log = $RichTextLabel
@onready var input = $LineEdit

func _ready():
    input.text_submitted.connect(_send)

func _send(text: String):
    if text.strip_edges().is_empty(): return
    var msg = "[%s] %s" % [PlayerData.name, text]
    _display(msg)
    # 网络: 发送到服务器
    ChatService.send.rpc(text)
    input.clear()
```



```
func _display(msg: String):
    chat_log.text += msg + "\n"
    # 自动滚动到底部
    $ScrollContainer.scroll_vertical =
    $ScrollContainer.get_v_scroll_bar().max_value
```

3. 频道 + 表情 + 敏感词

```
enum Channel { WORLD, GUILD, TEAM, PRIVATE, SYSTEM }

func send(channel: Channel, text: String, target_id: String =
    ""):
    if _filter.contains_bad_word(text):
        _display("消息包含敏感词")
        return

    match channel:
        Channel.WORLD:
            ChatService.broadcast.rpc(text)
        Channel.GUILD:
            ChatService.guild_chat.rpc(text)
        Channel.TEAM:
            ChatService.team_chat.rpc(text)
        Channel.PRIVATE:
            ChatService.private_chat.rpc(target_id, text)
        Channel.SYSTEM:
            pass # 只有服务器可以发

# 表情解析:
# ":" → 😊
# "/dance" → 角色播放跳舞动画
func _process_text(text: String) -> String:
    return text.replace(":", "😊").replace(":D", "😄")
```

4. 最终方案模板

```
# ChatManager.gd (Autoload)
# 单机简化版: 系统消息 + 调试命令
```

```
func system(text: String):
    _display("[系统] " + text)

func debug(text: String):
    if OS.is_debug_build():
        _display("[DEBUG] " + text)

# 网络版需要:
# - ChatService (RPC)
# - 频道管理
# - 敏感词过滤
# - 聊天记录缓存
# - 离线消息
```

第108章 音频系统演化

Godot 音频系统: 从单音效到混音总线

最早的游戏只有一个音效通道——同时只能播一个声音。后来有了多通道、音量控制、音效类别混音、3D 声音。音频系统的进化 = "怎么让玩家听着不吵、有层次、有沉浸感"。

1. 原始: 单音效播放

```
# 直接播, 打断上一个
func play_hit():
    $AudioStreamPlayer2D.play()
```

问题: 同时播放两个音效时, 第二个打断第一个。脚步声和攻击声不能共存。

2. 多 `AudioStreamPlayer`

```
# 每个音效一个 Player 节点
$BGM.play()
$SFX_Attack.play()
$SFX_Footstep.play()
$SFX_Hit.play()
$SFX_UI.play()
```

节点爆炸。10 种音效 = 10 个 `AudioStreamPlayer`。

3. AudioStreamPlayer2D/3D

```
# 2D/3D 声音：距离越远声音越小
# 敌人靠近时脚步声音变大，走远变小
$AudioStreamPlayer2D.max_distance = 500
$AudioStreamPlayer2D.attenuation = 0.5
```

4. 音频总线 (Audio Bus)

```
Master (主音量)
├─ BGM (背景音乐)
├─ SFX (音效)
│   └─ Combat (战斗音效)
│       └─ UI (界面音效)
└─ Voice (语音)
```

```
# 播到指定总线：
$AudioStreamPlayer.bus = "SFX/Combat"

# 代码控制音量：
AudioServer.set_bus_volume_db(
    AudioServer.get_bus_index("BGM"),
    linear_to_db(0.5) # 50% 音量
)

# 淡出 BGM：
var tween = create_tween()
tween.tween_method(func(v): AudioServer.set_bus_volume_db(
    AudioServer.get_bus_index("BGM"), linear_to_db(v)
), 1.0, 0.0, 2.0)
```

5. 最终方案模板

```
# AudioManager.gd (Autoload)
extends Node
```

```

func play_sfx(path: String, bus: String = "SFX"):
    var player = AudioStreamPlayer2D.new()
    player.stream = load(path)
    player.bus = bus
    player.finished.connect(player.queue_free)
    add_child(player)
    player.play()

func play_bgm(path: String):
    var current = get_node_or_null("BGM")
    if current:
        current.queue_free()
    var player = AudioStreamPlayer.new()
    player.stream = load(path)
    player.bus = "BGM"
    player.name = "BGM"
    add_child(player)
    player.play()

func set_volume(bus_name: String, volume: float):
    var idx = AudioServer.get_bus_index(bus_name)
    AudioServer.set_bus_volume_db(idx, linear_to_db(volume))

```

第109章 反作弊系统演化

Godot 反作弊系统：从信任到防御

反作弊 = 防止玩家修改游戏数据。最早游戏全部信任客户端。后来有了内存修改、变速齿轮、破解存档、外挂脚本。反作弊的进化 = "怎么让玩家没法改不该改的东西"。

1. 原始：完全信任

```
# 所有数据在客户端
# 玩家可以随意修改内存/存档
# 修改器直接改金币、攻击力
```

2. 服务端校验

```
# 核心数据在服务端
# 客户端只发操作，服务端验证

# 客户端：发送操作
func attack(enemy_id: String):
    ServerAPI.send("attack", { "target": enemy_id })

# 服务端：验证并计算
func _on_attack(player_id: String, data: Dictionary):
    var player = players[player_id]
    var enemy = enemies[data.target]
    if player.position.distance_to(enemy.position) <
player.attack_range:
        enemy.hp -= player.attack
```


3. 敏感数据加密

```
# 存档加密
func save_game(data: Dictionary):
    var json = JSON.stringify(data)
    var encrypted = _xor_encrypt(json, "secret_key")
    var file = FileAccess.open_encrypted_with_pass("user://
save.dat", FileAccess.WRITE, "password123")
    file.store_string(encrypted)

# 内存中关键数据不直接暴露
var _gold := 0
func get_gold() -> int: return _gold
func add_gold(amount: int): _gold += amount # 不走信号, 防止被
hook
```

4. 完整性校验

```
# 校验关键文件是否被修改
func check_integrity() -> bool:
    var checksums = {
        "res://scripts/player.gd": "a1b2c3d4...",
        "res://scripts/combat.gd": "e5f6g7h8...",
    }
    for path in checksums:
        var file = FileAccess.open(path, FileAccess.READ)
        var hash = file.get_as_text().md5_text()
        if hash != checksums[path]:
            return false # 文件被篡改
    return true
```

5. 行为检测

```
# 检测异常行为模式
class AntiCheatMonitor:
    var actions_per_second := []
    var last_attack_time := 0.0
```

```

func check_action(action: String) -> bool:
    var now = Time.get_ticks_msec() / 1000.0
    match action:
        "attack":
            # 攻击速度检测（防止变速）
            if now - last_attack_time < 0.1:
                return false # 太快了，疑似外挂
            last_attack_time = now
        "move":
            # 移动速度检测
            if speed > max_possible_speed * 1.5:
                return false # 瞬移

    # 操作频率检测
    actions_per_second.append(now)
    actions_per_second = actions_per_second.filter(func(t):
return now - t < 1.0)
    if actions_per_second.size() > 20:
        return false # 每秒操作超过上限

    return true

```

6. 最终方案模板

```

# AntiCheatManager.gd (Autoload)
# 核心：服务端校验 + 行为检测 + 数据加密

func validate_action(player, action: String, data: Dictionary) -
> bool:
    match action:
        "move":
            return _validate_move(player, data.pos,
data.timestamp)
        "attack":
            return _validate_attack(player, data.target_id)
        "use_item":
            return _validate_item(player, data.item_id)
    return false

func _validate_move(player, new_pos: Vector2, timestamp: float)

```

```
-> bool:
    var elapsed = Time.get_ticks_msec() / 1000.0 - timestamp
    var max_dist = player.speed * elapsed * 1.5 # 容忍 50% 误差
    return player.position.distance_to(new_pos) <= max_dist
```

第110章 外挂制作演化

Godot 外挂制作：从改内存到内核驱动

外挂 = 修改游戏来获得不公平优势。最早做外挂只需要改存档文件。后来有了内存扫描、封包截获、DLL 注入、图色脚本。外挂的进化 = "游戏防御升级后, 外挂怎么绕过去"。

1. 原始：改文件

```
# 存档文件是明文的 JSON
# 直接改 save.json 里的金币数值
# 重进入游戏就生效
{
  "gold": 999999,
  "level": 99
}
```

2. 内存修改

```
# 使用 Cheat Engine 扫描内存
# 1. 搜索当前金币值（精确搜索）
# 2. 花掉一些金币（改变搜索值）
# 3. 再次搜索，定位内存地址
# 4. 锁定/修改该地址的值

# 对付方法：
# 存档加密 + 关键数值不在内存中以明文存在
# 但还是能被"不断扫描"找到
```

3. 变速齿轮

```
# 通过修改游戏进程的时间相关 API
# 让游戏运行速度变快或变慢
# 加速：怪物不动，玩家十几倍速移动
# 减速：看清弹幕轨迹再躲避

# GameManager.gd
# 防变速：服务端校验时间戳
# 客户端发操作时附带服务端下发的时间戳
# 服务端检测两个操作之间的时间间隔是否合理
```

4. 封包截获

```
# 客户端和服务端之间传输的数据包
# 使用 Fiddler / Wireshark / Proxifier 截获
# 修改数据包内容再转发

# 原始（明文）：
# 客户端 → 服务端：{"action": "attack", "damage": 100}
# 外挂修改为：{"action": "attack", "damage": 999999}

# 对付方法：
# 协议加密 + 服务端校验（服务端自己算伤害）
# 最终方案：服务端权威，客户端只发操作不传结果
```

5. DLL 注入

```
// 外挂编写者用 C++ 写一个 DLL
// 通过 CreateRemoteThread 注入目标进程

// cheat.cpp - 注入到游戏进程后执行
#include <windows.h>
#include <stdint>

// 找到游戏中的金币地址（基址 + 偏移）
uintptr_t gold_addr = base_addr + 0x004A2F10;
```

```
int* gold = (int*)gold_addr;
*gold = 999999; // 直接改内存

// 或 Hook 函数：
// 拦截发包函数，修改数据后再调用原函数
// 让服务端收到修改过的内容
```

6. 图色脚本

```
# 不修改游戏内存，只模拟人操作
# 读取屏幕像素颜色识别位置
# 模拟鼠标/键盘输入

# auto_click.py
import pyautogui
import time

while True:
    # 在屏幕上找怪物血条位置
    location = pyautogui.locateOnScreen('monster_hp.png',
    confidence=0.8)
    if location:
        # 自动按技能键
        pyautogui.press('1')
        time.sleep(0.5)
    # 捡掉落物
    location = pyautogui.locateOnScreen('loot.png',
    confidence=0.7)
    if location:
        pyautogui.click(location)

# 对付方法：行为检测（每秒操作频率/鼠标轨迹分析）
# 服务端检测到异常精确的间隔 + 固定的点击模式 → 判定为脚本
```

7. 内核驱动

```
// 反外挂在内核层保护自己
// 外挂也进内核层绕过反外挂
```

```
// 驱动级外挂：
// 1. 绕过反外挂的进程扫描
// 2. 直接读写物理内存（绕过用户态 API）
// 3. 伪装进程名和签名

// 反制：内核回调 + 云检测
// 外挂作者通过在驱动中 Hook 内核回调来隐藏自己
// 反外挂作者通过检测驱动模块签名来发现异常驱动

// 这是一个永无止境的军备竞赛：
// 反外挂 → 外挂 → 更强的反外挂 → 更强的外挂 → ...
```

8. 最终方案：没有最终方案

反外挂和外挂是猫鼠游戏，不存在“最终方案”。

对于 Godot 游戏开发者：

1. 小型游戏：存档加密 + 服务端校验就够了
2. 中型游戏：行为检测 + 简单反内存修改
3. 大型游戏：接入商业反外挂 SDK（BattlEye/EAC）

无论如何，记住三条：

- 服务端永远不信任客户端
- 操作频率一定要校验
- 关键计算放服务端

第111章 Mod 制作系统演化

Godot Mod 制作系统: 从封闭到可扩展

Mod = 玩家修改游戏内容。最早游戏是封闭的, 改不了。后来有了开放文件格式、资源替换、脚本注入、Steam Workshop。Mod 系统的进化 = "怎么让玩家可以自己创造内容"。

1. 原始: 封闭

```
# 所有资源打包在 pck 里  
# 玩家改不了任何东西
```

2. 开放资源目录

```
# 允许玩家替换 res:// 下的文件  
# PCK 加载优先级  
  
func load_mods():  
    var dir = DirAccess.open("user://mods/")  
    if not dir: return  
    for file in dir.get_files():  
        if file.ends_with(".pck"):  
            ProjectSettings.load_resource_pack("user://mods/" +  
file)  
            print("加载 Mod: " + file)
```

```
# 资源覆盖优先级:  
# 1. user://mods/ 中的 PCK  
# 2. 游戏原始 PCK  
# 3. 同名文件 → 优先使用 mod 中的版本
```

```
# Mod 作者只需要放一个同名 pck 文件:  
# user://mods/retexture_pack.pck → 包含替换的贴图
```

3. Mod 接口脚本

```
# 游戏暴露一些可重写的钩子  
class_name ModAPI  
extends Node  
  
func on_player_init(player): pass  
func on_combat_start(attacker, defender): pass  
func on_damage_dealt(source, target, damage): return damage  
func on_item_used(player, item_id): pass
```

```
# Mod 作者继承 ModAPI:  
# mods/my_mod/mod.gd  
extends "res://core/ModAPI.gd"  
  
func on_damage_dealt(source, target, damage):  
    if source.is_in_group("player"):  
        return damage * 2 # 伤害翻倍  
    return damage
```

4. 资源热加载

```
# 开发期/运行时替换资源  
var _loaded_mods := []  
  
func load_mod_pck(path: String) -> bool:  
    if ProjectSettings.load_resource_pack(path):  
        _loaded_mods.append(path)  
        _apply_mod_scripts()  
        return true  
    return false  
  
func unload_mod(path: String):  
    # 移除 mod (需要重启场景)
```

```

        _loaded_mods.erase(path)
    # Godot 不支持卸载 PCK, 推荐重启
    get_tree().reload_current_scene()

func _apply_mod_scripts():
    for mod in _loaded_mods:
        var config = load("res://mod_config.tres")
        if config and config.has_method("init"):
            config.init(self)

```

5. 最终方案模板

```

# ModManager.gd (Autoload)
# 最简 Mod 支持:

func discover_mods() -> Array:
    var found = []
    var dir = DirAccess.open("user://mods/")
    if not dir: return found
    for f in dir.get_files():
        if f.ends_with(".pck"):
            found.push_back("user://mods/" + f)
    return found

func enable_mod(path: String) -> bool:
    if ProjectSettings.load_resource_pack(path):
        md = load("res://mod_manifest.tres")
        if md: print("Mod: " + md.name)
        return true
    return false

# 玩家需要放 .pck 文件到 user://mods/
# 游戏启动时自动扫描加载

```

第112章 本地化系统演化

Godot 本地化系统: 从硬编码到多语言

本地化 = 让游戏支持多种语言。最早文字写在代码里, 改语言要改代码。
后来有了 CSV 翻译表、自动换字体、RTL 支持。本地化的进化 = "怎么让全世界的玩家都看得懂"。

1. 原始: 硬编码

```
$Label.text = "你好, 冒险者!"  
# 换语言要改代码, 重新编译
```

2. CSV 翻译表

```
# translations.csv:  
# key, zh, en, ja  
# hello, 你好, Hello, こんにちは  
# start_game, 开始游戏, Start Game, ゲーム開始  
  
func load_translations(path: String):  
    var file = FileAccess.open(path, FileAccess.READ)  
    var lines = file.get_as_text().split("\n")  
    var headers = lines[0].split(",")  
    var lang_idx = headers.find(_current_lang)  
  
    for i in range(1, lines.size()):  
        var cols = lines[i].split(",")  
        var key = cols[0]  
        var text = cols[lang_idx] if lang_idx < cols.size() else  
cols[1]
```

```
TranslationServer.add_translation(_build_translation(key, text))
```

3. Godot TranslationServer

```
# Godot 内置本地化：
# 1. 创建 .csv / .po 翻译文件
# 2. 在 Project Settings → Localization 中导入
# 3. 代码中：tr("KEY")

# 使用：
$Label.text = tr("HELLO")
$StartButton.text = tr("START_GAME")

# 切换语言：
TranslationServer.set_locale("en")
TranslationServer.set_locale("zh")

# 自动重载所有 tr() 的文本
get_tree().call_group("auto_translate", "_update_text")
```

4. 字体切换

```
# 不同语言需要不同字体
# 中/日/韩需要支持汉字的字体
# 阿拉伯语需要 RTL 支持

var font_map = {
    "zh": preload("res://fonts/NotoSansSC-Regular.tres"),
    "ja": preload("res://fonts/NotoSansJP-Regular.tres"),
    "en": preload("res://fonts/NotoSans-Regular.tres"),
    "ar": preload("res://fonts/NotoSansArabic-Regular.tres"),
}

func apply_font(locale: String):
    var theme = Theme.new()
    theme.set_default_font(font_map.get(locale, font_map["en"]))
    get_tree().root.theme = theme
```

5. 最终方案模板

```
# I18nManager.gd (Autoload)
extends Node

signal locale_changed(locale: String)

const SUPPORTED = ["zh", "en", "ja", "ko"]

func set_locale(locale: String):
    if not locale in SUPPORTED: return
    TranslationServer.set_locale(locale)
    _apply_font(locale)
    locale_changed.emit(locale)
    # 刷新所有 tr() 文本
    for node in get_tree().get_nodes_in_group("auto_tr"):
        if node.has_method("_update_text"):
            node._update_text()

func t(key: String) -> String:
    return tr(key)
```


第113章 资源预加载系统演化

Godot 资源预加载系统: 从即用到预准备

预加载 = 在用之前就把资源加载好。最早场景切换时顿卡, 因为要现读硬盘。后来有了后台线程加载、进度条、预缓存池。预加载的进化 = "怎么让加载过程不被玩家注意到"。

1. 原始: 即用即加载

```
# 切换场景时:
func change_scene(path: String):
    get_tree().change_scene_to_file(path) # 卡住, 直到加载完
```

2. 后台线程加载

```
# Godot 的 ResourceLoader 支持后台加载
var loader: ResourceLoader

func start_loading(path: String):
    loader = ResourceLoader.load_threaded_request(path)

func _process(delta):
    if not loader: return
    var progress = []
    var status = ResourceLoader.load_threaded_get_status(path,
    progress)
    match status:
        ResourceLoader.THREAD_LOAD_IN_PROGRESS:
            $ProgressBar.value = progress[0] # 显示进度
        ResourceLoader.THREAD_LOAD_LOADED:
```

```
var scene = ResourceLoader.load_threaded_get(path)
get_tree().change_scene_to_packed(scene)
```

优势：加载过程中游戏仍然在运行，不卡顿

3. 预缓存管理器

```
# PreloadManager.gd (Autoload)
extends Node

var _cache := {}

func preload_scene(path: String):
    if _cache.has(path): return
    ResourceLoader.load_threaded_request(path)
    _cache[path] = null # 标记加载中

func get_scene(path: String) -> PackedScene:
    if _cache.get(path) == null:
        if ResourceLoader.load_threaded_get_status(path) ==
ResourceLoader.THREAD_LOAD_LOADED:
            _cache[path] =
ResourceLoader.load_threaded_get(path)
        return _cache.get(path)

func preload_multiple(paths: Array):
    for p in paths:
        preload_scene(p)
```

```
# 使用：在关卡选择界面就预加载下一关
func on_level_select(level_id: String):
    var scene_path = LevelDB.get(level_id).scene_path
    PreloadManager.preload_scene(scene_path)
    # 当玩家点击"开始"时，场景已加载完毕
```

4. 场景预实例化

```
# 隐藏实例，需要时显示
var _pools := {}

func pre_instantiate(scene: PackedScene, count: int):
    var pool = []
    for i in count:
        var instance = scene.instantiate()
        instance.visible = false
        add_child(instance)
        pool.append(instance)
    _pools[scene.resource_path] = pool

func get_pre_instantiated(path: String) -> Node:
    var pool = _pools.get(path)
    if pool and pool.size() > 0:
        var node = pool.pop_back()
        node.visible = true
        return node
    return null # 池子空了，现场 instantiate
```

5. 最终方案模板

```
# PreloadManager.gd (Autoload)
# 核心：后台加载 + 缓存 + 进度回调

var _loading := {} # path → status

func preload(path: String):
    if _loading.has(path): return
    _loading[path] = ResourceLoader.THREAD_LOAD_IN_PROGRESS
    ResourceLoader.load_threaded_request(path)

func get_progress(path: String) -> float:
    var p = []
    ResourceLoader.load_threaded_get_status(path, p)
    return p[0] if p.size() > 0 else 0.0
```

```
func is_loaded(path: String) -> bool:
    return ResourceLoader.load_threaded_get_status(path) ==
ResourceLoader.THREAD_LOAD_LOADED

func get(path: String):
    if is_loaded(path):
        return ResourceLoader.load_threaded_get(path)
    return null
```

第114章 调试控制台系统演化

Godot 调试控制台: 从 print 到飞行模式

调试控制台 = 开发期间用来测试/修改游戏状态的工具。最早用 print() 输出日志。后来有了 ~ 开命令行、调数值、瞬移、无敌、刷物品。调试控制台的进化 = "怎么让测试效率最高"。

1. 原始: print

```
print("player hp: ", player.hp)
print("enemy pos: ", enemy.global_position)
# 只能在输出面板看, 不能交互
```

2. In-Game Console

```
# 按 ~ 打开控制台, 输入命令
# Console.gd (CanvasLayer)

extends CanvasLayer

var _history := []
var _history_idx := -1
var _commands := {}

func _ready():
    register("help", _cmd_help, "显示帮助")
    register("god", _cmd_god, "无敌模式")
    register("hp", _cmd_hp, "设置血量: hp <数值>")
    register("spawn", _cmd_spawn, "生成物品: spawn <物品ID>")
    register("tp", _cmd_tp, "传送: tp <x> <y>")
```

```

    register("coins", _cmd_coins, "设置金币: coins <数值>")

func register(name: String, callback: Callable, desc: String =
    ""):
    _commands[name] = { "fn": callback, "desc": desc }

func execute(input: String):
    var parts = input.split(" ", false)
    if parts.is_empty(): return
    var cmd = _commands.get(parts[0])
    if cmd:
        cmd.fn.call(parts.slice(1))
        _history.push_back(input)
        _history_idx = _history.size()
    else:
        print("未知命令: " + parts[0])

# 命令示例:
func _cmd_god(args: Array):
    player.is_invincible = not player.is_invincible
    print("无敌: " + str(player.is_invincible))

func _cmd_hp(args: Array):
    if args.size() > 0:
        player.hp = int(args[0])
        print("HP 设为: " + args[0])

func _cmd_tp(args: Array):
    if args.size() >= 2:
        player.global_position = Vector2(float(args[0]),
            float(args[1]))

```

3. 参数化系统

```

# 自动暴露可调参数
# debug_config.gd
var params := {
    "player_speed": { "value": 200, "min": 0, "max": 1000 },
    "enemy_damage": { "value": 10, "min": 0, "max": 100 },
    "drop_rate":     { "value": 0.5, "min": 0.0, "max": 1.0 },
}

```



```
# 控制台自动生成命令
func _ready():
    for name in params.keys():
        var p = params[name]
        register("set_" + name, func(args):
            if args.size() > 0:
                p.value = float(args[0])
                print(name + " = " + str(p.value))
        )
```

4. 最终方案模板

```
# Console.gd (Autoload)
# 核心: 命令注册 + TEXTEDIT 输入框 + 执行回调

var _cmd := {}

func reg(name: String, fn: Callable, desc: String = ""):
    _cmd[name] = { fn = fn, desc = desc }

func exec(line: String):
    var parts = line.strip_edges().split(" ", false, 1)
    var c = _cmd.get(parts[0])
    if not c:
        print("? " + parts[0])
        return
    if parts.size() > 1:
        c.fn.call(parts[1].split(" ", false))
    else:
        c.fn.call([])

func _ready():
    reg("fly", _fly, "飞行/取消飞行")
    reg("pos", _pos, "显示当前坐标")
    reg("time", _time, "设置时间: time <0-24>")
    reg("clear", _clear, "清空背包")

func _fly(args):
    player.has_flight = not player.has_flight
    print("飞行: " + str(player.has_flight))
```

```
func _pos(args):  
    print("坐标: " + str(player.global_position))
```

第七卷·社交与内容

共计 18 章

第115章 新手引导系统演化

Godot 新手引导系统：从提示文字到流程控制

最早的新手引导 = 第一关简单点, 让玩家自己摸索。后来有了文字提示、箭头指引、强制步骤、跳过功能。引导系统的进化 = "怎么让玩家学会又不觉得被教做事"。

1. 原始：场景里放提示

```
# 在场景里放 Label: "按 WASD 移动"
# 玩家看到了就看到了, 没看到就算了

# 或在墙上贴字:
$HintLabel.text = "按 E 开门"
```

2. 定时弹出提示

```
# 玩家走到某个区域触发提示
func _on_trigger_area_body_entered(body):
    if body.is_in_group("player") and not _shown:
        _shown = true
        $TutorialPopup.show_text("按 空格 跳跃")
        await get_tree().create_timer(3.0).timeout
        $TutorialPopup.hide()
```

3. 分步引导

```
class TutorialStep:
    var target_path: String      # 高亮的 UI 元素
    var text: String            # 提示文字
    var required_action: String # "move" / "jump" / "attack" /
    "open_inventory"
    var arrow_position: Vector2 # 箭头指向位置
    var auto_continue: bool     # 完成动作后自动下一步
```

```
func start_tutorial():
    steps = [
        TutorialStep.new("", "按 WASD 移动", "move", Vector2(0,
-50), true),
        TutorialStep.new("", "按 空格 跳跃", "jump", Vector2(0,
-80), true),
        TutorialStep.new("$InventoryButton", "打开背包",
"open_inventory", Vector2(100, 100), true),
    ]
    current_step = 0
    show_step(current_step)

func show_step(index: int):
    var step = steps[index]
    $Arrow.position = step.arrow_position
    $TipLabel.text = step.text
    $HighlightTarget.target = get_node_or_null(step.target_path)

func _input(event):
    var step = steps[current_step]
    if step.auto_continue and
_check_action(step.required_action):
        current_step += 1
        if current_step < steps.size():
            show_step(current_step)
        else:
            finish_tutorial()
```

4. 最终方案模板

```
# 最简单的引导：
# 1. 用触发区域（Area2D）检测玩家位置
# 2. 进入区域弹提示
# 3. 玩家执行操作后提示消失
# 4. 用 GameState 记录引导是否已看过

func show_tutorial_once(id: String, text: String):
    if GameState.tutorials_seen.has(id):
        return
    GameState.tutorials_seen.append(id)
    $TutorialBubble.show_text(text)
    await get_tree().create_timer(4.0).timeout
    $TutorialBubble.hide()
```

第116章 任务系统演化

Godot 任务系统: 从 if-else 到链式任务

任务 = "去 A 杀 B 只怪, 回来领赏。" 最早的任务写在 NPC 对话的 if 里。后来有了任务数据表、任务链、日常任务。任务系统的进化 = "怎么让玩家一直有事做"。

1. 原始: NPC 对话里硬编码

```
# NPC 对话里直接 if:
func talk():
    if not has_killed_5_slimes:
        say("帮我杀 5 只史莱姆")
    else:
        say("干得好, 这是奖励")
        gold += 50
        has_killed_5_slimes = false # 重置
```

问题: 每加一个任务要改 NPC 脚本。任务状态散落在各个变量里。

2. 任务数据结构化

```
class Quest:
    var id: String
    var name: String
    var description: String
    var objectives: Array[Objective]
    var rewards: Dictionary # {"gold": 50, "exp": 100}
    var state: String # "unavailable" / "available" /
    "active" / "completed" / "done"
```

```

class Objective:
    var type: String          # "kill" / "collect" / "talk" /
    "reach"
    var target_id: String     # "slime" / "hp_potion" /
    "npc_01"
    var required: int = 1
    var current: int = 0

func is_completed() -> bool:
    for obj in objectives:
        if obj.current < obj.required:
            return false
    return true

```

3. 任务链

```

# 任务完成后自动触发下一个
class QuestChain:
    var quests: Array[String] # ["quest_01", "quest_02",
    "quest_03"]

    func on_quest_completed(quest_id: String):
        var next_index = quests.find(quest_id) + 1
        if next_index < quests.size():
            QuestManager.start(quests[next_index])

```

4. 最终方案模板

```

# QuestManager.gd (Autoload)
extends Node

signal quest_changed(quest_id: String, state: String)
signal quest_completed(quest_id: String)

var _quests := {}

func start(quest_id: String):

```

```

var quest = _load_quest_data(quest_id)
quest.state = "active"
_requests[quest_id] = quest
quest_changed.emit(quest_id, "active")

func update_objective(quest_id: String, type: String, target_id:
String, amount: int = 1):
    var quest = _quests.get(quest_id)
    if not quest or quest.state != "active":
        return
    for obj in quest.objectives:
        if obj.type == type and obj.target_id == target_id:
            obj.current = mini(obj.current + amount,
obj.required)
    if quest.is_completed():
        quest.state = "completed"
        _give_rewards(quest)
        quest_completed.emit(quest_id)
        quest_changed.emit(quest_id, "active")

func get_state(quest_id: String) -> String:
    var quest = _quests.get(quest_id)
    return quest.state if quest else "unavailable"

func _load_quest_data(id: String) -> Quest:
    return load("res://quests/%s.tres" % id)

```

第117章 成就系统演化

Godot 成就系统：从隐藏变量到解锁条件

成就 = "你做到了某件特别的事"。最早藏在代码里, 玩家不知道有成就。后来有了成就列表、成就点数、成就奖励。成就系统的进化 = "怎么让玩家去做你没教过他的事"。

1. 原始：隐藏变量

```
var has_killed_100_slimes = false

func _on_slime_killed():
    slime_kill_count += 1
    if slime_kill_count >= 100 and not has_killed_100_slimes:
        has_killed_100_slimes = true
        print("成就解锁：屠虫者")
```

2. 成就数据化

```
class Achievement:
    var id: String
    var name: String
    var description: String
    var icon: Texture2D
    var points: int
    var unlocked: bool = false
    var progress: float = 0.0
```

3. 成就管理器

```
# AchievementManager.gd (Autoload)
extends Node

signal unlocked(achievement_id: String)

var _achievements := {}

func _ready():
    _load_all()

func _load_all():
    var dir = DirAccess.open("res://achievements/")
    for file in dir.get_files():
        if file.ends_with(".tres"):
            var ach = load("res://achievements/" + file)
            _achievements[ach.id] = ach

func progress(id: String, value: float):
    var ach = _achievements.get(id)
    if not ach or ach.unlocked:
        return
    ach.progress = value
    if ach.progress >= 1.0:
        ach.unlocked = true
        unlocked.emit(id)

func is_unlocked(id: String) -> bool:
    var ach = _achievements.get(id)
    return ach.unlocked if ach else false

# 使用:
# 在任何地方:
# AchievementManager.progress("kill_100_slimes",
# slime_kill_count / 100.0)
```

4. 最终方案模板

```
# AchievementResource.gd
class_name AchievementResource
extends Resource

@export var id: String
@export var name: String
@export var description: String
@export var icon: Texture2D
@export var points: int = 10
@export var secret: bool = false

# 使用:
# 在编辑器里创建 .tres 文件
# 代码中: AchievementManager.progress("kill_100_slimes", count /
100.0)
```

第118章 称号系统演化

Godot 称号系统：从标签到身份

称号 = 显示在名字上面的标签。"屠龙勇士" "首充玩家" "万人斩"。最早没有称号。后来有了成就称号、活动称号、排行榜称号。称号系统的进化 = "怎么让玩家把成就顶在头上"。

1. 原始：无称号

只显示名字

2. 称号数据

```
class Title:
    var id: String
    var name: String
    var description: String
    var icon: Texture2D
    var rarity: String          # "common" / "rare" / "legendary"
    var stat_bonus: Dictionary # { "attack": 5, "defense": 3 }
    var source: String         # "achievement" / "event" /
    "shop" / "rank"
```

```
var unlocked_titles = []
var current_title = ""

func unlock(title_id: String):
    if not unlocked_titles.has(title_id):
        unlocked_titles.append(title_id)
```

```

func equip(title_id: String):
    if unlocked_titles.has(title_id):
        current_title = title_id
        _apply_stat_bonus(title_id)

func get_display_name(player_name: String) -> String:
    if current_title:
        return "[%s] %s" % [TitleDB.get(current_title).name,
player_name]
    return player_name

```

3. 最终方案模板

```

# TitleManager.gd (Autoload)

var unlocked := []
var equipped := ""

func unlock(id: String):
    if id in unlocked: return
    unlocked.append(id)

func equip(id: String):
    if id in unlocked: equipped = id

func get_display_prefix() -> String:
    if equipped == "": return ""
    return "[" + TitleDB.get(equipped).name + "]"

# 称号通常在以下时机解锁:
# AchievementManager.unlocked.connect(func(id):
TitleManager.unlock("achieve_" + id))

```

第119章 邮件系统演化

Godot 邮件系统：从收件箱到附件领取

邮件 = 游戏给你发东西, 或者玩家给玩家发东西。最早是系统邮件 (维护补偿、奖励发放)。后来有了附件 (领取物品)、过期删除、一键领取。邮件系统的进化 = "怎么让发奖励和收奖励都方便"。

1. 原始：直接塞背包

```
# 维护补偿直接发：
func give_maintenance_compensation():
    InventoryManager.add("gems", 100)
    # 玩家上线发现背包多了 100 钻，不知道哪来的
```

2. 邮件通知

```
class Mail:
    var id: String
    var title: String
    var sender: String
    var content: String
    var attachments: Array[MailAttachment]
    var sent_time: int
    var expire_time: int
    var is_read: bool = false
    var is_claimed: bool = false

class MailAttachment:
    var item_id: String
    var count: int
```

```

func send_system_mail(title: String, content: String,
attachments: Array):
    var mail = Mail.new()
    mail.title = title
    mail.content = content
    mail.sender = "系统"
    mail.attachments = attachments
    mail.sent_time = Time.get_unix_time_from_system()
    mail.expire_time = mail.sent_time + 86400 * 30 # 30 天后过期
    _mails.append(mail)

func claim_attachment(mail_id: String):
    var mail = _get_mail(mail_id)
    if not mail or mail.is_claimed: return
    for att in mail.attachments:
        InventoryManager.add(att.item_id, att.count)
    mail.is_claimed = true

```

3. 最终方案模板

```

# MailManager.gd (Autoload)
extends Node

signal new_mail()

var _mails := []

func send(title: String, content: String, items: Array = []):
    _mails.append({
        "title": title,
        "content": content,
        "sender": "系统",
        "items": items,
        "read": false,
        "claimed": false,
        "time": Time.get_unix_time_from_system(),
    })
    new_mail.emit()

func claim(index: int) -> bool:
    var mail = _mails[index]

```

```
    if not mail or mail.claimed: return false
    for item in mail.items:
        InventoryManager.add(item.id, item.count)
    mail.claimed = true
    return true

func claim_all():
    for i in _mails.size():
        claim(i)
```

第120章 活动系统演化

Godot 活动系统: 从固定到限时

活动 = 限时开放的玩法。"周末双倍经验" "节日限定关卡" "7 天签到活动"。最早的活动写在代码里, 到了日期手动开启。后来有了活动配置表、自动开始/结束、活动商店。活动系统的进化 = "怎么让游戏每天都有新东西玩"。

1. 原始: 硬编码活动

```
func _ready():
    var month = Time.get_date_dict_from_system().month
    if month == 12:
        # 圣诞活动
        EnemySpawner.set_drop_rate(2.0)
```

问题: 改活动时间要发版本更新。

2. 活动数据化

```
class GameEvent:
    var id: String
    var name: String
    var start_time: int        # unix timestamp
    var end_time: int
    var type: String           # "double_exp" / "limited_shop" /
    "boss_rush"
    var params: Dictionary    # { "rate": 2.0, "map": "snow" }
```



```
func is_event_active(event: GameEvent) -> bool:
    var now = Time.get_unix_time_from_system()
    return now >= event.start_time and now < event.end_time
```

3. 活动管理器

```
# EventManager.gd (Autoload)
extends Node

signal event_started(event_id: String)
signal event_ended(event_id: String)

var _events := {}
var _active_events := []

func _ready():
    _load_events()
    _check_events()

func _process(_delta):
    _check_events()

func _check_events():
    for event in _events.values():
        var was_active = _active_events.has(event.id)
        var now_active = is_event_active(event)

        if not was_active and now_active:
            _active_events.append(event.id)
            event_started.emit(event.id)
            _apply_event(event)

        elif was_active and not now_active:
            _active_events.erase(event.id)
            event_ended.emit(event.id)
            _remove_event(event)

func get_active_events() -> Array:
    return _active_events.duplicate()
```

```
func is_active(event_id: String) -> bool:  
    return _active_events.has(event_id)
```

4. 最终方案模板

```
# Event 的典型使用:  
# 1. 活动期间双倍掉落: EventManager.is_active("double_drop") → 掉落  
#    x2  
# 2. 活动期间限时商店: 判断活动活跃才显示  
# 3. 活动关卡: 活动期间场景里多一个入口
```

第121章 日常系统演化

Godot 日常/周常系统: 从一次性任务到每日刷新

日常 = 每天/每周可以重复完成的任务。签到是"登录就领", 日常是"做完任务领奖励"。最早所有任务都是一次性的。后来有了每日任务、每周任务、活跃度宝箱。日常系统的进化 = "怎么让玩家每天有明确的目标"。

1. 原始: 一次性任务

```
# 任务做完就没了  
# 玩家上线后不知道干什么
```

2. 每日任务列表

```
class DailyTask:  
    var id: String  
    var description: String  
    var condition: String      # "kill_10_slimes" /  
    "collect_5_herbs"  
    var required_count: int  
    var progress: int = 0  
    var reward: Dictionary    # { "gold": 500, "exp": 200 }  
    var completed: bool = false
```

```
func daily_reset():  
    for task in _daily_tasks:  
        task.progress = 0
```

```

        task.completed = false
    _save()

func progress(task_id: String, amount: int = 1):
    var task = _find(task_id)
    if not task or task.completed: return
    task.progress += amount
    if task.progress >= task.required_count:
        task.completed = true
        _claim_reward(task)

func claim_reward(task_id: String) -> bool:
    var task = _find(task_id)
    if not task or not task.completed: return false
    if task.claimed: return false
    for item_id in task.reward:
        InventoryManager.add(item_id, task.reward[item_id])
    task.claimed = true
    return true

```

3. 活跃度系统

```

class ActivitySystem:
    var daily_tasks: Array[DailyTask]
    var weekly_tasks: Array[WeeklyTask]
    var activity_points: int = 0
    var activity_rewards: Array[ActivityReward] # { points:
100, reward: {...} }

    func get_activity_progress() -> float:
        return float(activity_points) / 100.0

    func claim_activity_reward(threshold: int) -> bool:
        if activity_points < threshold: return false
        var reward = _rewards[threshold]
        if reward.claimed: return false
        reward.claimed = true
        for item_id in reward.items:
            InventoryManager.add(item_id, reward.items[item_id])
        return true

```

4. 最终方案模板

```
# DailyManager.gd (Autoload)
extends Node

signal daily_reset()
signal task_progressed(task_id: String, progress: int)

var tasks := []
var activity := 0
var date := 0

func _ready():
    var today = Time.get_unix_time_from_system() / 86400
    if _load().date != today:
        _reset()
        daily_reset.emit()

func progress(type: String, amount: int = 1):
    for t in tasks:
        if t.type == type and not t.done:
            t.progress = min(t.progress + amount, t.need)
            task_progressed.emit(t.id, t.progress)
            if t.progress >= t.need:
                t.done = true
                activity += t.activity_reward
    _save()

func _reset():
    date = Time.get_unix_time_from_system() / 86400
    for t in tasks:
        t.progress = 0
        t.done = false
    activity = 0
```

第122章 公告系统演化

Godot 公告系统：从弹窗到滚动通知

公告 = 游戏内的重要通知。"服务器维护" "新版本发布" "限时活动开启"。最早是弹窗, 玩家必须点掉。后来有了公告栏、滚动公告、公告列表、未读标记。公告系统的进化 = "怎么在不打断玩家的前提下通知消息"。

1. 原始：弹窗

```
func show_announcement(text: String):
    $AcceptDialog.dialog_text = text
    $AcceptDialog.popup_centered()
```

2. 公告列表

```
class Announcement:
    var id: String
    var title: String
    var content: String
    var type: String          # "maintenance" / "event" /
    "update" / "notice"
    var priority: int         # 0=普通, 1=重要, 2=紧急
    var start_time: int
    var end_time: int
    var read: bool = false
    var image_url: String    # 公告配图
```

```
func get_active_announcements() -> Array:
    var now = Time.get_unix_time_from_system()
    return _announcements.filter(func(a):
```



```

        return now >= a.start_time and now < a.end_time
    )

func get_unread_count() -> int:
    return get_active_announcements().filter(func(a): return not
a.read).size()

func mark_read(id: String):
    var a = _find(id)
    if a: a.read = true

```

3. 滚动公告 (Marquee)

```

# 屏幕顶部滚动文字
class Marquee:
    var text: String
    var speed: float = 50.0
    var interval: float = 5.0 # 每条间隔

var queue = []
var current = 0

func add_marquee(text: String):
    queue.append(text)
    if queue.size() == 1:
        _start_marquee()

func _start_marquee():
    if current >= queue.size():
        queue.clear()
        current = 0
        return
    $MarqueeLabel.text = queue[current]
    $MarqueeLabel.position.x = get_viewport().size.x
    var tween = create_tween()
    tween.tween_property($MarqueeLabel, "position:x",
        -$MarqueeLabel.size.x, 4.0).set_ease(Tween.EASE_LINEAR)
    tween.tween_callback(_next_marquee)

func _next_marquee():
    current += 1

```

```
await get_tree().create_timer(1.0).timeout
_start_marquee()
```

4. 最终方案模板

```
# AnnouncementManager.gd (Autoload)
# 公告来源：本地配置 / 服务器拉取

var announcements := []

func init(data: Array):
    announcements = data

func get_unread() -> Array:
    return announcements.filter(func(a): return not a.read and
is_active(a))

func mark_read(id: String):
    var a = announcements.filter(func(x): return x.id == id)
    if a.size() > 0: a[0].read = true
```

第123章 好友系统演化

Godot 好友系统: 从本地到联机

好友 = 和其他玩家建立社交关系。最早没有好友系统, 游戏是单人的。后来有了本地好友列表、在线状态、好友赠送体力。好友系统的进化 = "怎么让一个人的游戏变成一群人的游戏"。

1. 原始: 无好友

纯单机, 没有"好友"这个概念

2. 本地好友列表

```
var friends = [
    { "name": "小明", "level": 10, "last_login": "2024-01-01" },
    { "name": "小红", "level": 15, "last_login": "2024-01-02" },
]
```

3. 好友功能 (借角色/送体力)

```
func send_stamina(friend_id: String):
    # 每天最多送 10 次
    var sent_today = _get_sent_count()
    if sent_today >= 10:
        return false
    _send_stamina_to_server(friend_id)
    return true
```

```
func borrow_support(friend_id: String) -> Node:  
    # 在关卡中使用好友的角色  
    var data = _get_friend_data(friend_id)  
    return load_character(data.character_id)
```

4. 最终方案模板

```
# FriendManager.gd (Autoload)  
# 最简单的本地好友:  
# 1. 好友列表: 名字 + 等级 + 最后上线  
# 2. 赠送体力: 每天互送  
# 3. 助战借用: 用好友的角色打关卡  
  
var friends := []  
func add(name: String):  
    friends.append({ "name": name, "level": 1, "stamina_given":  
false })  
  
func give_stamina(index: int) -> bool:  
    var f = friends[index]  
    if f.stamina_given: return false  
    f.stamina_given = true  
    return true
```

第124章 组队系统演化

Godot 组队系统：从单人到队伍协作

组队 = 几个玩家一起玩。最早是"大家一起进同一个关卡"。后来有了经验分配、掉落分配、队长转移、队友状态显示。组队系统的进化 = "怎么让多人一起玩的体验比单人好"。

1. 原始：各自为战

```
# 每个玩家独立实例，互不干扰
# "组队"只存在于玩家的口头约定中
```

2. 队伍数据结构

```
class Party:
    var members: Array[PartyMember]
    var leader_id: String
    var loot_mode: String = "free"      # "free" / "order" /
"leader"
    var exp_share_range: float = 500.0

class PartyMember:
    var player_id: String
    var player_name: String
    var level: int
    var hp: int
    var max_hp: int
    var is_online: bool = true
```

3. 组队功能

```
# 邀请
func invite(player_id: String):
    # 发送邀请
    # 对方同意 → 加入队伍
    pass

# 踢出
func kick(player_id: String):
    if _party.leader_id == local_id:
        _party.members.erase(player_id)

# 退出
func leave():
    _party.members.erase(local_id)
    if _party.members.size() == 0:
        _party = null # 队伍解散

# 经验分配
func distribute_exp(total_exp: int):
    var share = total_exp / _party.members.size()
    for member in _party.members:
        member.add_exp(share)
```

4. 最终方案模板

```
# 最简单的组队（单机多角色）：
class Party:
    var members := []
    var leader := 0

    func add_member(node: Node2D):
        members.append(node)

    func get_avg_level() -> int:
        var total = 0
        for m in members:
            total += m.level
```



```
        return total / members.size()

func distribute_exp(total: int):
    var each = total / members.size()
    for m in members:
        m.add_exp(each)
```

第125章 公会系统演化

Godot 公会系统：从单人到组织

公会 = 玩家自发组成的群体。最早没有公会, 大家都是散人。后来有了公会列表、公会聊天、公会任务、公会战。公会系统的进化 = "怎么让一群玩家有归属感"。

1. 原始：无公会

只有单人或临时组队

2. 公会数据结构

```
class Guild:
    var id: String
    var name: String
    var leader_id: String
    var members: Array[GuildMember]
    var level: int = 1
    var exp: int = 0
    var notice: String = ""
    var created_time: int

class GuildMember:
    var player_id: String
    var role: String    # "leader" / "officer" / "member"
    var join_time: int
    var contribution: int
    var last_login: int
```

3. 公会功能

```
func create_guild(name: String, leader_id: String) -> bool:
    if _guilds.values().any(func(g): return g.name == name):
        return false # 名字已存在
    var guild = Guild.new()
    guild.id = _generate_id()
    guild.name = name
    guild.leader_id = leader_id
    guild.members.append(GuildMember.new(leader_id, "leader"))
    _guilds[guild.id] = guild
    return true

func join_request(guild_id: String, player_id: String):
    var guild = _guilds[guild_id]
    guild.pending_requests.append(player_id)
    # 通知会长/官员审批

func approve_join(guild_id: String, approver_id: String,
    player_id: String):
    var guild = _guilds[guild_id]
    if not _is_officer(guild, approver_id):
        return
    guild.pending_requests.erase(player_id)
    guild.members.append(GuildMember.new(player_id, "member"))
```

4. 公会贡献 + 商店

```
func add_contribution(player_id: String, amount: int):
    var member = _get_member(player_id)
    member.contribution += amount
    _get_guild_by_member(player_id).exp += amount

func guild_shop_buy(player_id: String, item_id: String) -> bool:
    var item = GuildShopDB.get(item_id)
    var member = _get_member(player_id)
    if member.contribution < item.cost:
        return false
    member.contribution -= item.cost
```

```
InventoryManager.add(item_id, 1)
return true
```

5. 最终方案模板

```
# GuildManager.gd (Autoload)
# 核心是 公会 + 成员 + 贡献 的数据结构

# 单机简化版 (AI 公会):
class AIGuild:
    var name: String
    var members: Array[AIMember]
    var level: int
    var shop: Array[GuildShopItem]

# 玩家可以加入一个 AI 公会:
# 1. 每天签到 → 公会贡献
# 2. 贡献换公会商店物品
# 3. 公会等级解锁更强物品
```

第126章 PVP 系统演化

Godot PVP 系统: 从打怪到打人

PVP = 玩家和玩家对战。最早没有 PVP, 玩家只能打怪物。后来有了决斗、竞技场、天梯排位、赛季结算。PVP 系统的进化 = "怎么让玩家之间的战斗公平又有趣"。

1. 原始: 无 PVP

```
# 所有伤害只对怪物有效  
# 玩家之间不能互相攻击
```

2. 自由 PVP (野外 PK)

```
var pvp_mode = false  
  
func toggle_pvp():  
    pvp_mode = not pvp_mode  
    $PvPIcon.visible = pvp_mode  
  
func _on_hitbox_body_entered(body):  
    if body.is_in_group("player") and pvp_mode and  
    body.pvp_mode:  
        # 双方都开了 PVP 模式才能互相伤害  
        _deal_damage(body)
```

3. 竞技场 (公平模式)

```
# 进入竞技场后，所有玩家属性拉平
class ArenaMatch:
    var player1: PlayerData
    var player2: PlayerData
    var state: String = "countdown" # countdown → fighting → finished

const BASE_STATS = {
    "hp": 1000, "attack": 100, "defense": 50, "speed": 200
}

func enter_arena(opponent_id: String):
    # 复制一份拉平属性的角色数据
    var p1 = _build_arena_player(PlayerData.id)
    var p2 = _build_arena_player(opponent_id)
    var match = ArenaMatch.new(p1, p2)
    _start_match(match)

func _build_arena_player(id: String) -> ArenaPlayer:
    var real = PlayerManager.get(id)
    var ap = ArenaPlayer.new()
    ap.id = id
    ap.hp = BASE_STATS.hp
    ap.max_hp = BASE_STATS.hp
    ap.attack = BASE_STATS.attack
    ap.defense = BASE_STATS.defense
    # 保留技能和天赋（战术差异）
    ap.skills = real.equipped_skills.duplicate()
    return ap
```

4. 天梯排位

```
class RankSystem:
    var tiers = ["青铜", "白银", "黄金", "白金", "钻石", "大师"]
    var stars_per_tier = [3, 3, 4, 4, 5, 0] # 大师无星

    var current_tier: int = 0 # 索引
```



```

var current_stars: int = 0

func win():
    current_stars += 1
    if current_stars >= stars_per_tier[current_tier]:
        current_stars = 0
        current_tier = min(current_tier + 1, tiers.size() -
1)
        return "promoted"
    return "star_gained"

func lose():
    current_stars -= 1
    if current_stars < 0:
        if current_tier > 0:
            current_tier -= 1
            current_stars = stars_per_tier[current_tier] - 1
            return "demoted"
        current_stars = 0
    return "star_lost"

```

5. 最终方案模板

```

# PvPManager.gd (Autoload)
# 核心:
# 1. 匹配: 找段位相近的对手
# 2. 公平化: 拉平装备属性, 保留技能/天赋
# 3. 对战: 实时或自动战斗
# 4. 结算: 加减星/段位

func start_match(opponent_id: String):
    var my_stats = _build_arena_stats(PlayerData)
    var opp_stats = _build_arena_stats(opponent_id)
    _load_arena_scene(my_stats, opp_stats)

func on_match_end(winner_id: String):
    var won = winner_id == PlayerData.id
    _update_rank(won)
    _give_rewards(won)

```

第127章 排行榜系统演化

Godot 排行榜系统：从本地分数到全球排名

排行榜 = 谁的分数最高。最早是"游戏结束, 输入名字, 记录前三名"。后来有了好友排名、全球排名、分段排名、赛季排名。排行榜的进化 = "怎么让玩家为了排名反复刷"。

1. 原始：本地记录

```
var high_scores = []

func save_score(score: int):
    high_scores.append(score)
    high_scores.sort()
    high_scores.reverse()
    if high_scores.size() > 10:
        high_scores.resize(10)
```

2. 名字 + 分数

```
class ScoreEntry:
    var name: String
    var score: int
    var date: String
```

```
func submit_score(name: String, score: int):
    var entry = ScoreEntry.new()
    entry.name = name
    entry.score = score
    entry.date = Time.get_date_string_from_system()
```

```
_scores.append(entry)
_scores.sort_custom(func(a, b): return a.score > b.score)
_save()
```

3. 分段排名 (好友/全球)

```
enum RankScope { LOCAL, FRIENDS, GLOBAL }

func get_rank(score: int, scope: RankScope) -> int:
    match scope:
        RankScope.LOCAL:
            return _local_rank(score)
        RankScope.FRIENDS:
            return _friends_rank(score)
        RankScope.GLOBAL:
            return _global_rank(score)
    return 0

func _local_rank(score: int) -> int:
    var rank = 1
    for s in _scores:
        if score < s.score:
            rank += 1
    return rank
```

4. 最终方案模板

```
# 最简单的本地排行榜:
# 1. 存到 user://leaderboard.json
# 2. 取 top N 显示
# 3. 网络版接入: Steam / GameCenter / 自建服务器

func submit(score: int):
    var entry = {
        "name": PlayerData.name,
        "score": score,
        "time": Time.get_unix_time_from_system(),
        "level": PlayerData.level,
```

```

    }
    _list.append(entry)
    _list.sort_custom(func(a, b): return a.score > b.score)
    _list = _list.slice(0, 100)
    _save()

func get_top(n: int) -> Array:
    return _list.slice(0, n)

func get_player_rank() -> int:
    for i in _list.size():
        if _list[i].name == PlayerData.name:
            return i + 1
    return -1

```

第128章 表情系统演化

Godot 表情/动作系统: 从文字到角色表演

表情 = 角色做出特定动作/表情来表达情绪。最早只有文字聊天, "/
laugh"。后来有了预设动作 (跳舞/挥手/打坐)、表情气泡、互动动作。
表情系统的进化 = "怎么让角色替代文字表达情绪"。

1. 原始: 文字表情

```
# 聊天框输入 /dance  
# 只有文字, 角色没反应
```

2. 预设动画

```
class Emote:  
    var id: String  
    var name: String  
    var animation: String      # AnimationPlayer 动画名  
    var duration: float  
    var chat_text: String      # 触发时发的文字  
    var icon: Texture2D  
  
var emotes = {  
    "wave":    Emote.new("wave",    "挥手", "emote_wave",    2.0,  
    "你好!"),  
    "dance":   Emote.new("dance",    "跳舞", "emote_dance",    4.0,  
    "来跳舞吧!"),  
    "sit":     Emote.new("sit",      "打坐", "emote_sit",      0.0,  
    "" ),  
    "bow":     Emote.new("bow",      "鞠躬", "emote_bow",      1.5,
```

```

"请多关照"),
    "angry": Emote.new("angry", "生气", "emote_angry", 2.0,
"!"),
    "cry": Emote.new("cry", "哭泣", "emote_cry", 3.0,
"呜呜..."),
}

```

```

func play_emote(emote_id: String):
    var emote = emotes.get(emote_id)
    if not emote: return

    # 打断当前动作
    $AnimationPlayer.play(emote.animation)

    # 动作期间不能移动（可选）
    is_emoting = true
    await get_tree().create_timer(emote.duration).timeout
    is_emoting = false

    # 广播给其他人（联网）
    if multiplayer.is_server():
        _broadcast_emote.rpc(emote_id)

```

3. 表情气泡

```

# EmoteBubble.gd
extends Node2D

var bubble_scene = preload("res://ui/emote_bubble.tscn")

func show_emote(emote_id: String):
    var bubble = bubble_scene.instantiate()
    add_child(bubble)
    bubble.play(emote_id)
    # 3 秒后自动消失
    await get_tree().create_timer(3.0).timeout
    bubble.queue_free()

```


4. 互动动作

```
# 两个玩家之间的互动
class InteractionEmote:
    var initiator_emote: String # 发起者动作
    var target_emote: String    # 目标反应
    var distance_required: float # 需要靠近

# "击掌": 两人靠近, 同时做击掌动画
# "拥抱": 两人靠近, 拥抱动画
# "干杯": 两人举杯
```

5. 最终方案模板

```
# EmoteManager.gd (Autoload 或挂载到角色)
# 核心: Press B → 打开表情轮盘 → 选择 → 播放动画

var emotes := {}
var is_emoting := false

func play(id: String) -> bool:
    if is_emoting: return false
    var e = emotes.get(id)
    if not e: return false
    is_emoting = true
    $AnimationPlayer.play(e.anim)
    if e.duration > 0:
        await get_tree().create_timer(e.duration).timeout
        is_emoting = false
    return true
```

第129章 举报屏蔽系统演化

Godot 举报/屏蔽系统: 从不管到社区自治

举报/屏蔽 = 玩家举报违规行为、屏蔽不想看到的玩家。最早游戏不管, 全靠玩家自己忍。后来有了举报表单、屏蔽列表、关键字过滤、自动禁言。举报的进化 = "怎么让社区环境不需要官方 24 小时盯着"。

1. 原始: 不管

```
# 谩骂、刷屏、外挂 - 没人管
# 玩家只能自己忍受或离开游戏
```

2. 屏蔽

```
var blocked_players := []

func add_to_blocklist(player_id: String):
    if not player_id in blocked_players:
        blocked_players.append(player_id)

func is_blocked(player_id: String) -> bool:
    return player_id in blocked_players

# 聊天时过滤:
func on_chat_message(sender_id: String, content: String):
    if is_blocked(sender_id):
        return # 屏蔽玩家的消息不显示
    ChatDisplay.show(sender_id, content)
```

3. 举报

```
# 右键玩家 → 举报
# 提交到服务器给管理员审核

func report_player(target_id: String, reason: String, details: Dictionary):
    ServerAPI.send("report", {
        target = target_id,
        reason = reason,
        details = details,
        time = Time.get_unix_time(),
    })

# 举报类型:
# "cheating" - 外挂
# "harassment" - 辱骂/骚扰
# "spam" - 刷屏/广告
# "scam" - 欺诈
# "inappropriate_name" - 名字不当
```

4. 关键字 + 自动禁言

```
class ChatFilter:
    var banned_words := ["敏感词1", "敏感词2", "脏话"]

    func filter(content: String) -> String:
        for word in banned_words:
            if word in content:
                content = content.replace(word, "***")
        return content

    func is_reportable(content: String) -> bool:
        for word in banned_words:
            if content.find(word) >= 0:
                return true
        return false

# 自动禁言:
```

- # 1. 发送的消息包含关键字 → 替换为 ***
- # 2. 短时间发送大量消息 → 临时禁言 5 分钟
- # 3. 累计举报到达阈值 → 自动封禁 + 人工审核

5. 最终方案模板

```
# ReportManager.gd (联网用)
# 核心: 客户端举报 → 服务端记录 → 自动/人工处理

func report(target: String, reason: String, screenshot: Image = null):
    var data = {
        reporter_id = player.id,
        target_id = target,
        reason = reason,
        time = Time.get_unix_time(),
        scene = get_tree().current_scene.name,
    }
    if screenshot:
        ServerAPI.upload_image(screenshot, func(url):
            data.evidence = url
            ServerAPI.send("report", data)
        )
    else:
        ServerAPI.send("report", data)
    Notification.show(tr("REPORT_SENT"))

func block(target: String):
    player.blocklist.append(target)
    SaveManager.save()
    Chat.hide_messages_from(target)
```

第130章 师徒结婚系统演化

Godot 师徒/结婚系统：从单打独斗到社交绑定

师徒/结婚 = 玩家之间的特殊社交关系, 有额外奖励。最早玩家之间没有任何绑定关系。后来有了师徒、结婚、好感度、额外加成。师徒系统的进化 = "怎么让老玩家愿意带新玩家"。

1. 原始：无绑定

```
# 玩家之间只是"好友"  
# 没有任何特殊关系
```

2. 师徒系统

```
class Mentorship:  
    var mentor_id: String  
    var apprentice_id: String  
    var start_time: int  
    var status: String # "active" / "graduated" / "cancelled"  
  
    func graduate():  
        status = "graduated"  
        # 师傅获得奖励：名师点 + 称号  
        MentorRewards.give(mentor_id, "mentor_points", 100)  
        MentorRewards.give(mentor_id, "title", "名师")  
        # 徒弟获得奖励：出师礼包  
        ApprenticeRewards.give(apprentice_id, "graduation_gift")  
  
    func check_apprentice_progress():  
        # 徒弟每升 10 级，师傅获得一次奖励
```

```

var level = PlayerDB.get(apprentice_id).level
var milestones = [10, 20, 30, 40, 50]
for m in milestones:
    if level >= m and not reward_claimed(m):
        claim_reward(mentor_id, m)

```

3. 结婚系统

```

class Marriage:
    var player_a: String
    var player_b: String
    var married_at: int
    var intimacy: int = 0 # 亲密度

    func get_partner(my_id: String) -> String:
        return player_b if my_id == player_a else player_a

    func increase_intimacy(amount: int):
        intimacy += amount
        # 亲密度解锁奖励:
        # 100: 双人表情
        # 500: 夫妻技能
        # 1000: 专属称号
        # 2000: 双人坐骑

    func get_bonus() -> Dictionary:
        return {
            "exp_bonus": 0.1 + intimacy * 0.0001,      # 经验加成
            "atk_bonus": 0.05 + intimacy * 0.00005,    # 攻击加成
        }

# 求婚流程:
# 1. A 向 B 送求婚戒指
# 2. B 同意 → 举办婚礼 (可选)
# 3. 系统公告: "玩家A 和 玩家B 喜结连理"
# 4. 夫妻称号 + 特殊技能

```


4. 最终方案模板

```
# SocialRelationManager.gd (联网用)
# 核心: 师徒 / 结婚 / 亲密度

func apply_mentor(mentor: String, apprentice: String):
    ServerAPI.send("mentor_apply", {
        mentor = mentor,
        apprentice = apprentice,
    })

func apply_marriage(proposer: String, target: String, ring_id:
String):
    ServerAPI.send("marriage_propose", {
        proposer = proposer,
        target = target,
        ring = ring_id,
    })

func get_relation_bonus(player_id: String) -> Dictionary:
    var bonus = { "exp": 1.0, "atk": 1.0 }
    if has_mentor: bonus.exp += 0.15
    if has_apprentice: bonus.exp += 0.10
    if has_spouse:
        var intimacy = get_intimacy(player_id)
        bonus.exp += 0.10 + intimacy * 0.0001
        bonus.atk += 0.05 + intimacy * 0.00005
    return bonus
```

第131章 回放录像系统演化

Godot 回放/录像系统: 从录屏到动作回放

回放/录像 = 记录玩家操作/战斗过程, 之后重新播放。最早靠录屏软件。后来有了操作序列记录、关键帧回放、逐帧观看。回放的进化 = "怎么用最少的数据重现完整过程"。

1. 原始: 录屏

- # 玩家用 OBS 等软件录制屏幕
- # 游戏没有任何内置回放功能

2. 操作序列

```
# 记录每一帧的输入 + 时间戳
class ReplayRecorder:
    var frames: Array[Dictionary] = []

    func record_frame():
        frames.append({
            "time": Time.get_ticks_msec(),
            "input": {
                "left": Input.is_action_pressed("left"),
                "right": Input.is_action_pressed("right"),
                "up": Input.is_action_pressed("up"),
                "down": Input.is_action_pressed("down"),
                "attack":
Input.is_action_just_pressed("attack"),
                "skill1":
Input.is_action_just_pressed("skill_1"),
```

```

        },
        "pos": player.global_position,
        "hp": player.hp,
        "mp": player.mp,
        "action": player.current_action, # "idle" / "run" /
"attack"
    })

func save_replay(filename: String):
    var file = FileAccess.open("user://replays/" + filename +
".json", FileAccess.WRITE)
    file.store_string(JSON.stringify(frames))

```

3. 回放系统

```

class ReplayPlayer:
    var frames: Array[Dictionary]
    var frame_idx: int = 0
    var start_time: int
    var unit: Node2D # 回放用的傀儡单位

    func load_replay(path: String):
        var file = FileAccess.open(path, FileAccess.READ)
        frames = JSON.parse_string(file.get_as_text())
        # 创建傀儡角色, 隐藏真实玩家
        unit = preload("res://player_replay.tscn").instantiate()
        get_tree().root.add_child(unit)

    func play():
        start_time = Time.get_ticks_msec()

    func _process(delta):
        if frame_idx >= frames.size(): return
        var f = frames[frame_idx]
        var now = Time.get_ticks_msec()

        while frame_idx < frames.size() and now - start_time >=
frames[frame_idx].time:
            f = frames[frame_idx]
            unit.global_position = f.pos
            unit.hp = f.hp

```

```

        unit.play_animation(f.action)
        frame_idx += 1

func toggle_pause():
    is_paused = not is_paused

func set_speed(speed: float):
    time_scale = speed # 0.5 = 慢放, 2.0 = 快进

```

4. 最终方案模板

```

# ReplayManager.gd (Autoload)
# 核心: 记录 → 存储 → 回放

const RECORD_INTERVAL := 0.033 # 30 FPS

var _frames := []
var _timer := 0.0
var _dummy: Node2D = null

func start_recording():
    _frames.clear()
    _timer = 0.0

func stop_recording() -> Dictionary:
    return { "frames": _frames, "length": _frames.size() *
RECORD_INTERVAL }

func _process(delta):
    if not recording: return
    _timer += delta
    if _timer >= RECORD_INTERVAL:
        _timer = 0
        _frames.append(_snapshot())

func _snapshot() -> Dictionary:
    return {
        pos = player.global_position,
        hp = player.hp,
        anim = player.current_animation,

```

```
        timestamp = Time.get_ticks_msec(),  
    }
```

第132章 观战系统演化

Godot 观战系统：从看不到到上帝视角

观战 = 玩家死亡后/主动选择观看其他玩家的战斗。最早死了就是黑白屏，啥也看不到。后来有了跟随观看、自由视角、多玩家切换。观战的进化 = "怎么让死亡后的时间不无聊"。

1. 原始：黑白屏

```
func on_player_die():
    $ScreenOverlay.color = Color.BLACK
    # 玩家只能看自己尸体，不能看别人
```

2. 跟随视角

```
func enter_spectate(target_id: String):
    # 把相机切换到目标玩家
    var target = PlayerDB.get_node(target_id)
    camera.reparent(target)
    camera.global_position = target.global_position
    # 显示目标玩家的血条/状态
    $SpectateHUD.show_target_info(target)
```

3. 观战模式

```
class SpectateMode:
    var available_targets: Array[Node2D]
    var current_target_idx: int = 0
```



```

var is_free_cam: bool = false

func enter():
    # 隐藏自己的 UI
    player.hide()
    camera.make_current()

    # 按 Tab 切换观战目标
    InputManager.register("tab", _next_target)
    # 按 F 切换自由视角
    InputManager.register("f", _toggle_free_cam)

func _next_target():
    if available_targets.is_empty(): return
    current_target_idx = (current_target_idx + 1) %
available_targets.size()
    var target = available_targets[current_target_idx]
    camera.reparent(target)
    camera.position = Vector2.ZERO

func _toggle_free_cam():
    is_free_cam = not is_free_cam
    if is_free_cam:
        camera.reparent(get_tree().root)
        # 鼠标控制视角移动
    else:
        _next_target()

func exit():
    camera.reparent(player)
    player.show()
    player.respawn()

```

4. 最终方案模板

```

# SpectateManager.gd (Autoload)
# 核心: 玩家列表 → 切换目标 → 自由视角

var _targets := []
var _idx := 0
var _active := false

```

```

var _cam: Camera2D

func start():
    _active = true
    _cam = player.camera
    _targets = get_tree().get_nodes_in_group("player")
    _targets.erase(player)
    _cam.reparent(_targets[0])

func _unhandled_input(event):
    if not _active: return
    if event.is_action_pressed("ui_focus_next"):
        _idx = (_idx + 1) % _targets.size()
        _cam.reparent(_targets[_idx])
        _cam.position = Vector2.ZERO
        _show_hud(_targets[_idx])

func stop():
    _active = false
    _cam.reparent(player)
    player.show()
    $SpectateHUD.hide()

```